

U S E R ' S G U I D E

S I E S T A 5.4.0

May 28, 2025

<https://siesta-project.org>

SIESTA Steering Committee:

Emilio Artacho	<i>CIC-Nanogune and University of Cambridge</i>
José María Cela	<i>Barcelona Supercomputing Center</i>
Julian D. Gale	<i>Curtin University of Technology, Perth</i>
Alberto García	<i>Institut de Ciència de Materials, CSIC, Barcelona</i>
Javier Junquera	<i>Universidad de Cantabria, Santander</i>
Richard M. Martin	<i>University of Illinois at Urbana-Champaign</i>
Pablo Ordejón	<i>Centre de Investigació en Nanociència i Nanotecnologia, (CSIC-ICN), Barcelona</i>
Nick Rübner Papior	<i>Technical University of Denmark</i>
Daniel Sánchez-Portal	<i>Unidad de Física de Materiales, Centro Mixto CSIC-UPV/EHU, San Sebastián</i>
José M. Soler	<i>Universidad Autónoma de Madrid</i>

SIESTA is Copyright © 1996-2025 by The Siesta Group

Contributors to SIESTA

The SIESTA project was initiated by Pablo Ordejon (then at the Univ. de Oviedo), and Jose M. Soler and Emilio Artacho (Univ. Autonoma de Madrid, UAM). The development team was then joined by Alberto Garcia (then at Univ. del Pais Vasco, Bilbao), Daniel Sanchez-Portal (UAM), and Javier Junquera (Univ. de Oviedo and later UAM), and sometime later by Julian Gale (then at Imperial College, London). In 2007 Jose M. Cella (Barcelona Supercomputing Center, BSC) became a core developer and member of the Steering Committee.

The original TRANSIESTA module was developed by Pablo Ordejon and Jose L. Mozos (then at ICMAB-CSIC), and Mads Brandbyge, Kurt Stokbro, and Jeremy Taylor (Technical Univ. of Denmark).

The current TRANSIESTA module within SIESTA is developed by Nick R. Papior and Mads Brandbyge. Nick R. Papior became a core developer and member of the Steering Committee in 2015.

Other contributors (we apologize for any omissions):

Eduardo Anglada, Thomas Archer, Luis C. Balbas, Xavier Blase, Jorge I. Cerdá, Ramón Cuadrado, Michele Ceriotti, Fabiano Corsetti, Raul de la Cruz, Gabriel Fabricius, Marivi Fernandez-Serra, Jaime Ferrer, Chu-Chun Fu, Sandra Garcia, Victor M. Garcia-Suarez, Rogeli Grima, Rainer Hoft, Georg Huhs, Jorge Kohanoff, Richard Korytar, In-Ho Lee, Lin Lin, Nicolas Lorente, Miquel Llunell, Eduardo Machado, Maider Machado, Jose Luis Martins, Volodymyr Maslyuk, Juana Moreno, Frederico Dutilh Novaes, Micael Oliveira, Magnus Paulsson, Oscar Paz, Federico Pedron, Andrei Postnikov, Roberto Robles, Tristana Sondon, Rafi Ullah, Andrew Walker, Andrew Walkingshaw, Toby White, Francois Willaime, Chao Yang.

O.F. Sankey, D.J. Niklewski and D.A. Drabold made the FIREBALL code available to P. Ordejon. Although we no longer use the routines in that code, it was essential in the initial development of SIESTA, which still uses many of the algorithms developed by them.

Contents

Contributors to SIESTA	2
1 INTRODUCTION	9
2 COMPILATION	11
2.1 Notes on compiler flags	12
2.1.1 Debug options	12
2.2 Parallel operation	12
2.2.1 MPI	12
2.2.2 OpenMP	13
2.3 Library dependencies	14
2.4 Known Issues	17
2.5 Installing git-enabled versions	17
3 EXECUTION OF THE PROGRAM	17
3.1 Specific execution options	18
4 THE FLEXIBLE DATA FORMAT (FDF)	19
5 PROGRAM OUTPUT	20
5.1 Standard output	20
5.2 Output to dedicated files	21
6 DETAILED DESCRIPTION OF PROGRAM OPTIONS	21
6.1 General system descriptors	22
6.2 Pseudopotentials	24
6.3 Basis set and KB projectors	27
6.3.1 Overview of atomic-orbital bases implemented in SIESTA	27
6.3.2 Type of basis sets	30
6.3.3 Size of the basis set	31
6.3.4 Range of the orbitals	32
6.3.5 Generation of multiple-zeta orbitals	32
6.3.6 Polarization-orbital options	34
6.3.7 Soft-confinement options	35
6.3.8 Kleinman-Bylander projectors	35

6.3.9	The PAO.Basis block	37
6.3.10	Filtering	40
6.3.11	Saving and reading basis-set information	41
6.3.12	Tools to inspect the orbitals and KB projectors	41
6.3.13	Basis optimization	41
6.3.14	Low-level options regarding the radial grid	42
6.3.15	Summary of options and defaults enabling automatic basis-set generation . .	43
6.3.16	Notes on backward compatibility in regard to new program defaults	44
6.4	Structural information	44
6.4.1	Traditional structure input in the fdf file	45
6.4.2	Z-matrix format and constraints	47
6.4.3	Output of structural information	51
6.4.4	Input of structural information from external files	52
6.4.5	Input from a FIFO file	53
6.4.6	Precedence issues in structural input	53
6.4.7	Interatomic distances	53
6.5	k-point sampling	53
6.5.1	Output of k-point information	55
6.6	Exchange-correlation functionals	55
6.7	Spin polarization	58
6.8	Spin-Orbit coupling	59
6.8.1	On-site approximation	60
6.9	The self-consistent-field loop	61
6.9.1	Harris functional	62
6.9.2	Mixing options	62
6.9.3	Mixing of the Charge Density	69
6.9.4	Initialization of the density-matrix	71
6.9.5	Initialization of the SCF cycle with charge densities	74
6.9.6	Output of density matrix and Hamiltonian	74
6.9.7	Convergence criteria	76
6.10	The real-space grid and the eggbox-effect	78
6.11	Matrix elements of the Hamiltonian and overlap	81
6.11.1	The auxiliary supercell	82
6.12	Calculation of the electronic structure	83

6.12.1	Diagonalization options	84
6.12.2	Output of eigenvalues and wavefunctions	88
6.12.3	Occupation of electronic states and Fermi level	88
6.12.4	Orbital minimization method (OMM)	89
6.12.5	Order(N) calculations	91
6.13	The ELSI solver family	93
6.13.1	Input parameters	93
6.14	The CheSS solver	96
6.14.1	Input parameters	96
6.15	The PEXSI solver (native interface)	97
6.15.1	Pole handling	97
6.15.2	Parallel environment and control options	98
6.15.3	Electron tolerance and the PEXSI solver	99
6.15.4	Inertia-counting	100
6.15.5	Re-use of μ information accross iterations	101
6.15.6	Calculation of the density of states by inertia-counting	102
6.15.7	Calculation of the LDOS by selected-inversion	103
6.16	Band-structure analysis	103
6.16.1	Format of the .bands file	104
6.16.2	Output of wavefunctions associated to bands	105
6.17	Output of selected wavefunctions	105
6.18	Density of states	106
6.18.1	Total density of states	106
6.18.2	Partial (projected) density of states	107
6.18.3	Local density of states	108
6.19	Options for chemical analysis	109
6.19.1	Charge populations	109
6.19.2	Deprecated population flags	112
6.19.3	Crystal-Orbital overlap and hamilton populations (COOP/COHP)	113
6.20	Optical properties	114
6.21	Macroscopic polarization	115
6.22	Wannier Functions. Interface with WANNIER90	117
6.22.1	wannier90 as a postprocessing tool	117
6.22.2	wannier90 called on-the-fly within siesta	119

6.23	Systems with net charge or dipole, and electric fields	122
6.23.1	Bulk current	127
6.24	Output of charge densities and potentials on the grid	129
6.25	Auxiliary Force field	131
6.26	Grimme's DFT-D3 dispersion model	132
6.26.1	DFTD3 and periodicity	134
6.26.2	A note on LIBXC functionals	134
6.27	Parallel options	134
6.27.1	Parallel decompositions for O(N)	135
6.28	Efficiency options	135
6.29	Memory, CPU-time, and Wall time accounting options	136
6.30	The catch-all option UseSaveData	137
6.31	Output of information for Denchar	137
6.32	NetCDF (CDF4) output file	137
7	STRUCTURAL RELAXATION, PHONONS, AND MOLECULAR DYNAMICS	138
7.1	Compatibility with pre-v4 versions	140
7.2	Structural relaxation	141
7.2.1	Conjugate-gradients optimization	142
7.2.2	Broyden optimization	142
7.2.3	FIRE relaxation	143
7.3	Target stress options	143
7.4	Molecular dynamics	144
7.5	Output options for dynamics	146
7.6	Restarting geometry optimizations and MD runs	147
7.7	Use of general constraints	147
7.8	Phonon calculations	150
8	DFT+U	151
9	RT-TDDFT	153
9.1	Brief description	154
9.2	Partial Occupations	154
9.3	Input options for RT-TDDFT	154

10 External control of SIESTA	156
10.1 Examples of Lua programs	158
10.2 External MD/relaxation methods	159
11 TRANSIESTA	159
11.1 Source code structure	159
11.2 Compilation	159
11.3 Brief description	159
11.4 Electrodes	162
11.4.1 Matching coordinates	162
11.4.2 Principal layer interactions	164
11.5 Convergence of electrodes and scattering regions	164
11.6 NEGF equations	165
11.7 TRANSIESTA Options	166
11.7.1 Quick and dirty	166
11.7.2 General options	167
11.8 k -point sampling	173
11.8.1 Algorithm specific options	173
11.8.2 Poisson solution for fixed boundary conditions	175
11.8.3 Electrode description options	176
11.8.4 Chemical potentials	181
11.8.5 Complex contour integration options	182
11.8.6 Bias contour integration options	185
11.9 Output	185
11.10 Utilities for analysis: TBTRANS	186
12 QM/MM with SIESTA	186
12.1 Using external MM driver	188
12.2 The in-house QM/MM driver	188
12.2.1 Running the driver	188
12.2.2 The MM parameter file	189
12.2.3 QM/MM driver-specific options	189
13 ANALYSIS TOOLS	191
13.1 2D Lindhard function calculator	191

14 SCRIPTING	192
15 PROBLEM HANDLING	192
15.1 Error and warning messages	192
16 REPORTING BUGS	193
17 ACKNOWLEDGMENTS	193
18 APPENDIX: Physical unit names recognized by FDF	194
19 APPENDIX: XML Output	195
19.1 Controlling XML output	195
19.2 Converting XML to XHTML	195
20 APPENDIX: Selection of precision for storage	196
21 APPENDIX: Data structures and reference counting	197
Bibliography	198
Index	200

1 INTRODUCTION

This Reference Manual contains descriptions of all the input, output and execution features of SIESTA, but is not really a tutorial introduction to the program. Interested users can find tutorial material prepared for SIESTA schools and workshops at the web page <https://docs.siesta-project.org>

SIESTA (Spanish Initiative for Electronic Simulations with Thousands of Atoms) is both a method and its computer program implementation, to perform electronic structure calculations and *ab initio* molecular dynamics simulations of molecules and solids. Its main characteristics are:

- It uses the standard Kohn-Sham selfconsistent density functional method in the local density (LDA-LSD) and generalized gradient (GGA) approximations, as well as in a non local functional that includes van der Waals interactions (VDW-DF).
- It uses norm-conserving pseudopotentials in their fully nonlocal (Kleinman-Bylander) form.
- It uses atomic orbitals as a basis set, allowing unlimited multiple-zeta and angular momenta, polarization and off-site orbitals. The radial shape of every orbital is numerical and any shape can be used and provided by the user, with the only condition that it has to be of finite support, i.e., it has to be strictly zero beyond a user-provided distance from the corresponding nucleus. Finite-support basis sets are the key for calculating the Hamiltonian and overlap matrices in $O(N)$ operations.
- Projects the electron wavefunctions and density onto a real-space grid in order to calculate the Hartree and exchange-correlation potentials and their matrix elements.
- Besides the standard Rayleigh-Ritz eigenstate method, it allows the use of localized linear combinations of the occupied orbitals (valence-bond or Wannier-like functions), making the computer time and memory scale linearly with the number of atoms. Simulations with several hundred atoms are feasible with modest workstations.
- It is written in Fortran 2003 and memory is allocated dynamically.
- It may be compiled for serial or parallel execution (under MPI).

It routinely provides:

- Total and partial energies.
- Atomic forces.
- Stress tensor.
- Electric dipole moment.
- Atomic, orbital and bond populations (Mulliken).
- Electron density.

And also (though not all options are compatible):

- Geometry relaxation, fixed or variable cell.
- Constant-temperature molecular dynamics (Nose thermostat).
- Variable cell dynamics (Parrinello-Rahman).
- Spin polarized calculations (colinear or not).
- k-sampling of the Brillouin zone.
- Local and orbital-projected density of states.
- COOP and COHP curves for chemical bonding analysis.
- Dielectric polarization.
- Vibrations (phonons).
- Band structure.
- Ballistic electron transport under non-equilibrium (through TRANSIESTA)

Starting from version 3.0, SIESTA includes the TRANSIESTA module. TRANSIESTA provides the ability to model open-boundary systems where ballistic electron transport is taking place. Using TRANSIESTA one can compute electronic transport properties, such as the zero bias conductance and the I-V characteristic, of a nanoscale system in contact with two electrodes at different electrochemical potentials. The method is based on using non equilibrium Greens functions (NEGF), that are constructed using the density functional theory Hamiltonian obtained from a given electron density. A new density is computed using the NEGF formalism, which closes the DFT-NEGF self consistent cycle.

Starting from version 4.1, TRANSIESTA is an intrinsic part of the SIESTA code. I.e. a separate executable is not necessary anymore. See Sec. 11 for details.

For more details on the formalism, see the main TRANSIESTA reference cited below. A section has been added to this User's Guide, that describes the necessary steps involved in doing transport calculations, together with the currently implemented input options.

References:

- “Unconstrained minimization approach for electronic computations that scales linearly with system size” P. Ordejón, D. A. Drabold, M. P. Grumbach and R. M. Martin, Phys. Rev. B **48**, 14646 (1993); “Linear system-size methods for electronic-structure calculations” Phys. Rev. B **51** 1456 (1995), and references therein.

Description of the order- N eigensolvers implemented in this code.

- “Self-consistent order- N density-functional calculations for very large systems” P. Ordejón, E. Artacho and J. M. Soler, Phys. Rev. B **53**, 10441, (1996).

Description of a previous version of this methodology.

- “Density functional method for very large systems with LCAO basis sets” D. Sánchez-Portal, P. Ordejón, E. Artacho and J. M. Soler, *Int. J. Quantum Chem.*, **65**, 453 (1997).
Description of the present method and code.
- “Linear-scaling ab-initio calculations for large and complex systems” E. Artacho, D. Sánchez-Portal, P. Ordejón, A. García and J. M. Soler, *Phys. Stat. Sol. (b)* **215**, 809 (1999).
Description of the numerical atomic orbitals (NAOs) most commonly used in the code, and brief review of applications as of March 1999.
- “Numerical atomic orbitals for linear-scaling calculations” J. Junquera, O. Paz, D. Sánchez-Portal, and E. Artacho, *Phys. Rev. B* **64**, 235111, (2001).
Improved, soft-confined NAOs.
- “The SIESTA method for ab initio order- N materials simulation” J. M. Soler, E. Artacho, J.D. Gale, A. García, J. Junquera, P. Ordejón, and D. Sánchez-Portal, *J. Phys.: Condens. Matter* **14**, 2745-2779 (2002)
Extensive description of the SIESTA method.
- “Computing the properties of materials from first principles with SIESTA”, D. Sánchez-Portal, P. Ordejón, and E. Canadell, *Structure and Bonding* **113**, 103-170 (2004).
Extensive review of applications as of summer 2003.
- “Improvements on non-equilibrium and transport Green function techniques: The next-generation TranSIESTA”, Nick Papior, Nicolas Lorente, Thomas Frederiksen, Alberto García and Mads Brandbyge, *Computer Physics Communications*, **212**, 8–24 (2017).
Description of the TRANSIESTA method.
- “Density-functional method for nonequilibrium electron transport”, Mads Brandbyge, Jose-Luis Mozos, Pablo Ordejón, Jeremy Taylor, and Kurt Stokbro, *Phys. Rev. B* **65**, 165401 (2002).
Description of the original TRANSIESTA method (prior to 4.1).
- “Siesta: Recent developments and applications”, Alberto García, *et al.*, *J. Chem. Phys.* **152**, 204108 (2020).
Extensive review of applications and developments as of 2020.

For more information you can visit the web page <https://siesta-project.org>.

2 COMPILATION

Please see the file `INSTALL.md` in the top directory of the SIESTA distribution for basic instructions on how to use CMake (≥ 3.20) to build SIESTA and the utility programs.

A set of Spack recipes is also available to handle some dependencies and multiple build configurations automatically.

2.1 Notes on compiler flags

NOTE: Intel compilers default to high optimizations which tend to break SIESTA. We advice to use `-fp-model source` flag and to avoid optimizations higher than `-O2`.

NOTE: Since gfortran version 10.x the interfaces are strictly checked. Currently one has to add `-fallow-argument-mismatch` to the compiler flags to turn errors into warnings. These warnings are safe to ignore and will look something like:

```
.../siesta/Src/fsiesta_mpi.F90:441:18:
440 |   call MPI_Bcast( n, 1, MPI_Integer, 0, MPI_Comm_Siesta, error )
      |               2
441 |   call MPI_Bcast( x, 3*na, MPI_Double_Precision, 0, MPI_Comm_Siesta, error )
      |               1
Warning: Type mismatch between actual argument at (1) and actual argument at (2) (REAL(8)/INTEGER(4)).
```

The CMake system takes care of adding the extra flag automatically. Note that compilations with `-pedantic` flag are no longer possible.

2.1.1 Debug options

Being able to build SIESTA in debug mode is crucial for finding bugs and debugging builds.

For GFortran, use the following flags:

```
FFLAGS = -Og -g -Wall -fcheck=all -fbacktrace -Warray-bounds -Wunused -Wuninitialized
```

For Intel, use the following flags:

```
FFLAGS = -Og -g -check bounds -traceback -fp-model strict
```

This will make SIESTA run significantly slower. Please report any crashes to the developer team at <https://gitlab.com/siesta-project/siesta/-/issues>.

2.2 Parallel operation

To achieve a parallel build of SIESTA one should first determine which type of parallelism one requires. It is advised to use MPI for calculations with moderate number of cores. If one requires extra parallelism SIESTA provides hybrid parallelism using both MPI and OpenMP.

2.2.1 MPI

MPI is a message-passing interface which enables communication between equivalently executed binaries. This library will thus duplicate all non-distributed data such as local variables etc.

MPI is compiled-in by default in SIESTA, as long as the appropriate libraries can be found. If MPI is not desired, simply set

```
-DSIESTA_WITH_MPI=OFF
```

in the CMake invocation.

Subsequently one may run SIESTA using the `mpirun/mpiexec` commands:

```
mpirun -np <> siesta RUN.fdf
```

where `<>` is the number of cores used. Note that the actual commands and syntax are system-dependent.

2.2.2 OpenMP

OpenMP is shared memory parallelism. It typically does not incur any memory overhead and may be used if memory is scarce and the regular MPI compilation is crashing due to insufficient memory, or is not efficient due to the communication overhead.

To enable OpenMP, simply add this to your CMake invocation

```
-DSIESTA_WITH_OPENMP=ON
```

The above will yield the most basic parallelism using OpenMP. However, the BLAS/LAPACK libraries, which are the most time-consuming part of SIESTA are also required to be threaded, please see Sec. 2.3 for correct linking.

The minimum required version of OpenMP is 3.0 (internally identified by the YYYYMM date string 200805).

Subsequently one may run SIESTA using OpenMP through the environment variable `OMP_NUM_THREADS` which determine the number of threads/cores used in the execution.

```
OMP_NUM_THREADS=<> siesta RUN.fdf
# or (bash)
export OMP_NUM_THREADS=<>
siesta RUN.fdf
# or (csh)
setenv OMP_NUM_THREADS <>
siesta RUN.fdf
```

where `<>` is the number of threads/cores used.

If SIESTA is also compiled using MPI it is more difficult to obtain a good performance. Please refer to your local cluster documentation for how to correctly call MPI with hybrid parallelism. An example for running SIESTA with good performance using OpenMPI > 1.8.2 *and* OpenMP on a machine with 2 sockets and 8 cores per socket, one may do:

```
# MPI = 2 cores, OpenMP = 8 threads per core (total=16)
mpirun --map-by ppr:1:socket:pe=8 \
  -x OMP_NUM_THREADS=8 \
  -x OMP_PROC_BIND=true siesta RUN.fdf
```

```
# MPI = 4 cores, OpenMP = 4 threads per core (total=16)
mpirun --map-by ppr:2:socket:pe=4 \
  -x OMP_NUM_THREADS=4 \
  -x OMP_PROC_BIND=true siesta RUN.fdf

# MPI = 8 cores, OpenMP = 2 threads per core (total=16)
mpirun --map-by ppr:4:socket:pe=2 \
  -x OMP_NUM_THREADS=2 \
  -x OMP_PROC_BIND=true siesta RUN.fdf
```

If using only 1 thread per MPI core it is advised to compile SIESTA without OpenMP. As such it may be advantageous to compile SIESTA in 3 variants; OpenMP-only (small systems), MPI-only (medium to large systems) and MPI+OpenMP (large > systems).

The variable `OMP_PROC_BIND` may heavily influence the performance of the executable! Please perform tests for the architecture used.

2.3 Library dependencies

NOTE: The *required* libraries: `xmlf90`, `libPSML`, `libfdf`, and `libGridXC`, can be installed automatically on-the-fly during the SIESTA compilation process.

They can also be pre-installed, using their own CMake-based building systems. In that case their installation paths can be added to `CMAKE_PREFIX_PATH` for the SIESTA compilation.

XMLF90 is required as a prerequisite for `libPSML`, and to produce XML and CML output. (<https://gitlab.com/siesta-project/libraries/xmlf90>).

libPSML is required to use pseudopotentials in PSML format (<https://gitlab.com/siesta-project/libraries/libpsml>)

libfdf is required to parse fdf files and handle the options in them. (<https://gitlab.com/siesta-project/libraries/libfdf>)

libGridXC is required. (<https://gitlab.com/siesta-project/libraries/libgridxc>)

libXC is optional. `libGridXC` can use it if present. (<https://gitlab.com/libxc/libxc>)

BLAS it is recommended to use a high-performance library (OpenBLAS or MKL library from Intel, or BLIS)

LAPACK it is recommended to use a high-performance library (OpenBLAS¹ or MKL library from Intel)

NOTE: If you use your *nix distribution package manager to install BLAS/LAPACK you are bound to have a poor performance. Please try and use performance libraries, whenever possible!

The CMake building system will search for the BLAS/LAPACK libraries (maybe with help, by setting the `BLAS_LIBRARY` or `LAPACK_LIBRARY` CMake variables) and set the appropriate linking options.

¹OpenBLAS enables the inclusion of the LAPACK routines. This is advised.

ScaLAPACK *Only required for MPI compilation.*

Here one may rely on the NetLIB² version of ScaLAPACK.

Additionally SIESTA may be compiled with support for several other libraries

fdict This library is shipped with SIESTA and compiled automatically when needed.

NetCDF It is advised to compile NetCDF in CDF4 compliant mode (thus also linking with HDF5) as this enables more advanced IO. If you only link against a CDF3 compliant library you will not get the complete feature set of SIESTA.

NetCDF (both the C and Fortran interfaces) are typically already installed in supercomputer centers and can be installed in most systems using package managers. As a temporary convenience, SIESTA is shipped with the installation script `Docs/install_netcdf4.bash`, which installs NetCDF with full CDF4 support. Thus it installs `zlib`, `hdf5` and NetCDF C and Fortran.

ncdf This library is shipped with SIESTA and is compiled automatically if NetCDF (v4) is enabled, unless the user sets `-DSIESTA_WITH_NCDF=OFF`.

If the NetCDF library is compiled with parallel support one may take advantage of parallel IO by `-DSIESTA_WITH_NCDF_PARALLEL=ON`

ELPA The ELPA^[1;11] library provides faster diagonalization routines.

The version of ELPA *must* be 2017.05.003 or later, since the new ELPA API is used.

ELPA is used by default if found during the configuration phase of the CMake run.

NOTE: ELPA can only be used in the parallel version of SIESTA.

Metis The Metis library may be used with TRANSIESTA.

Currently there is no full support in CMake to build the Metis library. It has to be pre-compiled, and the options passed to CMake in the form:

```
cmake [.....] -DFortran_FLAGS="-DSIESTA__METIS -L/opt/metis/lib -lmetis"
```

MUMPS The MUMPS library may currently be used with TRANSIESTA.

Currently there is no full support in CMake to build the MUMPS library. It has to be pre-compiled, and the options passed to CMake in the form:

```
cmake [.....] \
-DFortran_FLAGS="-DSIESTA__MUMPS -L/opt/mumps/lib -lmumps -lmumps_common <>"
```

where `<>` are any libraries that MUMPS depends on.

PEXSI The PEXSI library may be used with this version of SIESTA for massively-parallel calculations, see Sec. 6.15.

There are two PEXSI interfaces in this version. The first is the original native interface, using the heuristics developed for SIESTA in collaboration with the PEXSI developers, with the following features and limitations:

²ScaLAPACK's performance is mainly governed by BLAS and LAPACK.

- It works only for the Gamma point (i.e. real matrices H and S). This is not really a major limitation, since the PEXSI method is typically used for large systems.
- It works for (collinear) spin-polarized systems.
- It can compute the DOS through inertia counting, and the local DOS using selected inversion.
- It determines the Fermi level using a Newton algorithm.
- It offers two levels of parallelization: over poles, and over orbitals.

The second is the PEXSI interface provided by the ELSI library, which offers

- Support for (collinear) spin-polarized systems.
- Determination of the Fermi level using a parallel interpolation procedure.
- Three levels of parallelization: over poles, over orbitals, and over interpolation points.
- Arbitrary sampling of the Brillouin Zone (both real and complex H,S)

The PEXSI library routines used by the native interface in SIESTA are offered both by the PEXSI library itself (versions 2.0 and higher), and by the ELSI library (if compiled with PEXSI support). A special fortran interface file has been used to allow the compilation of SIESTA with any one of these libraries.

ELSI The ELSI library may be used with SIESTA for access to a range of solvers, see Sec. 6.13. Currently the interface is implemented (tested) for ELSI 2.2.0 to 2.10. If newer versions retain the same interface they may also be used.

To enable ELSI support, add `-DSIESTA_WITH_ELSI=ON` in the CMake configuration step. More details in `INSTALL.md` and in the documentation in `External/ELSI`.

CheSS SIESTA allows calculation of the electronic structure through the use of the Order-N method **CheSS**³. To enable this solver (see **SolutionMethod**) one needs to first compile the **CheSS**-suite and subsequently use:

```
cmake [.....] -DSIESTA_WITH_CHESS=ON
```

NOTE: This feature is experimental, and the developers are working on streamlining the integration of this library into the CMake build system.

flook SIESTA allows external control via the LUA scripting language. Using this library one may do advanced MD simulations and much more *without* changing any code in SIESTA.

This library is compiled automatically if found during the configuration process.

See `Tests/Dependency_Tests/h2o_lua` for an example on the LUA interface.

DFT-D3 This library is required in order to add Grimme's D3 dispersion corrections to SIESTA. It is compiled on the fly (controlled by `-DSIESTA_WITH_DFTD3=ON/OFF`) if the directories under `External/DFTD3` are populated (through the use of git submodules or otherwise).

³See <https://launchpad.net/chess>.

2.4 Known Issues

Cray There are few known issues when compiling SIESTA with Cray compilers.

- Compilation with debug information "-g" fails for Cray Compiler versions lower than 14.0.3.
- For Cray versions 15.0 or higher available on certain systems, it might be mandatory to manually add a compiler flag for OpenMP compilations. For example:

```
cmake -B _build ...[Your Options Here]... -DFortran_FLAGS="-fopenmp"
```

ScaLAPACK In some Linux-native versions of the ScaLAPACK distribution, CMake detection might fail with the following message:

```
CMake Warning at CMakeLists.txt:63 (message):
MPI is found, but ScaLAPACK library cannot be found (or compiled against).

If parallel support is required please supply the ScaLAPACK library with
appropriate flags:

-DSCALAPACK_LIBRARY=<lib>
```

In order to fix this, the cmake variable `-DSCALAPACK_LIBRARY` must be explicitly set to `-DSCALAPACK_LIBRARY=-lscalapack-openmpi` (or the appropriate ScaLAPACK version in your system).

2.5 Installing git-enabled versions

When installing versions of SIESTA via `git clone` or similar approaches, one can take advantage of git to automatically setup all of SIESTA's internal dependencies, using:

```
git submodule update --init --recursive
```

NOTE: Note that this requires git versions of **2.13 and above**.

3 EXECUTION OF THE PROGRAM

A fast way to test your installation of SIESTA and get a feeling for the workings of the program is implemented in directory `Tests`. Assuming that you have built SIESTA in `_build`, you can do

```
cd _build
ctest -L simple
```

to test the execution of an assortment of tests. Executing `ctest` with no other options will run all possible tests. Output verification is also available via the `VERIFY_TESTS` environment variable:

```
VERIFY_TESTS=1 ctest -L simple
```

Other examples are provided in the **Examples** directory.

Further information about the running of SIESTA with tutorials and how-to's on various topics, including the generation of pseudopotentials with the **ATOM** code, can be found in the documentation site <https://docs.siesta-project.org>.

3.1 Specific execution options

SIESTA may be executed in different forms. The basic execution form is

```
siesta < RUN.fdf > RUN.out
```

which uses a *pipe* statement. SIESTA 4.1 and later does not require one to pipe in the input file and the input file may instead be specified on the command line:

```
siesta RUN.fdf > RUN.out
```

SIESTA 4.1 and later also accepts special flags and options described in what follows:

- All flags must start by one or more dashes (-). The number of leading dashes is irrelevant, as long as there is at least one of them.
- Some flags (e.g., [-L]) must be followed by a properly formed option string. Other flags (e.g., [-elec]) are logical toggles and they are not followed by option strings.
- Flags and option strings must all be separated by spaces (and only spaces are valid separators for this).
- Option strings may be quoted. Option strings that contain spaces need to either be quoted or have the spaces replaced by a colon (:) or by an equal sign (=).
- If the input file is not piped in, it can be given as an argument:

```
siesta -L Hello -V 0.25:eV RUN.fdf > RUN.out
siesta -L Hello RUN.fdf -V 0.25:eV > RUN.out
```

The list of available flags and options is:

-help|-h Print a help instruction and quit.

-version|-v Print version information and quit.

-out|-o Specify the output file (instead of printing to the terminal). Example:

```
siesta --out RUN.out
```

-L Override, temporarily, the **SystemLabel** flag. Example:

```
siesta -L Hello
```

-electrode|-elec

overwrites: **Save.HS**, **TS.HS.Save**, **TS.DE.Save**

Denote this as an electrode calculation which forces the `SystemLabel.TS.HSX`, `SystemLabel.TSHS` and `SystemLabel.TSDE` files to be saved.

NOTE: This is equivalent to specifying **Save.HS true**, **TS.HS.Save true** and **TS.DE.Save true** in the input file.

-V

overwrites: **TS.Voltage**

Specify the bias for the current TRANSIESTA run. If no units are specified, eV are assumed. Example: any of the following three commands set the applied bias to 0.25 eV:

```
siesta -V 0.25:eV
siesta -V "0.25 eV"
siesta -V 0.25
```

NOTE: This is equivalent to specifying **TS.Voltage** in the input file.

-fdf Specify any FDF option string. For example, another way to specify the bias of the example of the previous option would be:

```
siesta --fdf TS.Voltage=0.25:eV
```

4 THE FLEXIBLE DATA FORMAT (FDF)

The main input file, typically with extension `.fdf`, contains the physical data of the system and the parameters of the simulation to be performed. This file is written in a special format called FDF, developed by Alberto García and José M. Soler. This format allows data to be given in any order, or to be omitted in favor of default values. Refer to documentation of `libfdf` for details. Here we offer a glimpse of it through the following rules:

- The FDF syntax is a “data label” followed by its value. Values that are not specified in the datafile are assigned a default value.
- FDF labels are case insensitive, and characters - `_` . in a data label are ignored. Thus, **LatticeConstant** and **lattice_constant** represent the same label.
- All text following the `#` character is taken as comment.
- Logical values can be specified as T, true, .true., yes, F, false, .false., no. Blank is also equivalent to true.
- Character strings should **not** be in apostrophes.
- Real values which represent a physical magnitude must be followed by its units. Look at function `fdf_convfac` in file `~/siesta/Src/fdf/fdf.f` for the units that are currently supported. It is important to include a decimal point in a real number to distinguish it from an integer, in order to prevent ambiguities when mixing the types on the same input line.
- Complex data structures are called blocks and are placed between “%block label” and a “%end-block label” (without the quotes).

- You may “include” other FDF files and redirect the search for a particular data label to another file. If a data label appears more than once, its first appearance is used.
- If the same label is specified twice, the first one takes precedence.
- If a label is misspelled it will not be recognized (there is no internal list of “accepted” tags in the program). You can check the actual value used by SIESTA by looking for the label in the output `fdf.log` file.

These are some examples:

```

SystemName      Water molecule # This is a comment
SystemLabel     h2o
Spin polarized
SaveRho
NumberOfAtoms   64
LatticeConstant 5.42 Ang
%block LatticeVectors
    1.000  0.000  0.000
    0.000  1.000  0.000
    0.000  0.000  1.000
%endblock LatticeVectors
KgridCutoff < BZ_sampling.fdf

# Reading the coordinates from a file
%block AtomicCoordinatesAndAtomicSpecies < coordinates.data

# Even reading more FDF information from somewhere else
%include mydefaults.fdf

```

The file `fdf-XXXXX.log` contains all the parameters used by SIESTA in a given run, both those specified in the input fdf file and those taken by default. They are written in fdf format, so that you may reuse them as input directly. Input data blocks are copied to the `fdf.log` file only if you specify the *dump* option for them. In practice, the name of a FDF log file contains a sequence of digits (e.g., `fdf-12345.log`) chosen on-the-fly in order to have a reduced chance of overwriting other FDF log files that may be present in the same directory.

5 PROGRAM OUTPUT

5.1 Standard output

SIESTA writes a log of its workings to standard output (unit 6), which is usually redirected to an “output file”.

A brief description follows. See the example cases in the `siesta/Tests` directory for illustration.

The program starts writing the version of the code which is used. Then, the input FDF file is dumped into the output file as is (except for empty lines). The program does part of the reading

and digesting of the data at the beginning within the `redata` subroutine. It prints some of the information it digests. It is important to note that it is only part of it, some other information being accessed by the different subroutines when they need it during the run (in the spirit of FDF input). A complete list of the input used by the code can be found at the end in the file `fdf.log`, including defaults used by the code in the run.

After that, the program reads the pseudopotentials, factorizes them into Kleinman-Bylander form, and generates (or reads) the atomic basis set to be used in the simulation. These stages are documented in the output file.

The simulation begins after that, the output showing information of the MD (or CG) steps and the SCF cycles within. Basic descriptions of the process and results are presented. The user has the option to customize it, however, by defining different options that control the printing of informations like coordinates, forces, $\vec{k}$ points, etc. The options are discussed in the appropriate sections, but take into account the behavior of the legacy **LongOutput** option, as in the current implementation might silently activate output to the main `.out` file at the expense of auxiliary files.

LongOutput `false` *(logical)*

SIESTA can write to standard output different data sets depending on the values for output options described below. By default SIESTA will not write most of them. They can be large for large systems (coordinates, eigenvalues, forces, etc.) and, if written to standard output, they accumulate for all the steps of the dynamics. SIESTA writes the information in other files (see Output Files) in addition to the standard output, and these can be cumulative or not.

Setting **LongOutput** to **true** changes the default of some options, obtaining more information in the output (verbose). In particular, it redefines the defaults for the following:

- **WriteKpoints**
- **WriteKbands**
- **WriteCoorStep**
- **WriteForces**
- **WriteEigenvalues**
- **WriteWaveFunctions**
- **WriteMullikenPop**(it sets it to 1)

The specific changing of any of these options has precedence.

5.2 Output to dedicated files

SIESTA can produce a wealth of information in dedicated files, with specific formats, that can be used for further analysis. See the appropriate sections, and the appendix on file formats. Please take into account the behavior of **LongOutput**, as in the current implementation might silently activate output to the main `.out` file at the expense of auxiliary files.

6 DETAILED DESCRIPTION OF PROGRAM OPTIONS

Here follows a description of the variables that you can define in your SIESTA input file, with their data types and default values. For historical reasons the names of the tags do not have an uniform

structure, and can be confusing at times.

Almost all of the tags are optional: SIESTA will assign a default if a given tag is not found when needed (see `fdf.log`).

6.1 General system descriptors

SystemLabel `siesta` (string)

A *single* word (max. 20 characters *without blanks*) containing a nickname of the system, used to name output files.

SystemName `<None>` (string)

A string of one or several words containing a descriptive name of the system (max. 150 characters).

NumberOfSpecies `<lines in ChemicalSpeciesLabel>` (integer)

Number of different atomic species in the simulation. Atoms of the same species, but with a different pseudopotential or basis set are counted as different species.

NOTE: This is not required to be set.

NumberOfAtoms `<lines in AtomicCoordinatesAndAtomicSpecies>` (integer)

Number of atoms in the simulation.

NOTE: This is not required to be set.

%block ChemicalSpeciesLabel `<None>` (block)

It specifies the different chemical species that are present, assigning them a number for further identification. SIESTA recognizes the different atoms by the given atomic number.

```
%block ChemicalSpecieslabel
  1   6   C   pbe/C.psml
  2  14  Si
  3  14  Si_surface Si.psf
%endblock ChemicalSpecieslabel
```

The first number in a line is the species number, it is followed by the atomic number, and then by the desired unique label. This label will be used to identify each species. For example the label is the equivalent label name that should be found in the **PAO.Basis** block.

This construction allows you to have atoms of the same species but with different basis or pseudopotential, for example.

Optionally, a string *ps-file-spec* after the species name determines the pseudopotential file to be used. In the example above, the **C** atoms will use the pseudopotential file `pbe/C.psml` (with reference to the current directory), and both the **Si** and the **Si_surface** species will use a pseudopotential file named `Si.psf`. See section 6.2 for a full discussion of options.

Negative atomic numbers are used for *ghost* atoms (see **PAO.Basis**).

For atomic numbers over 200 or below -200 you should read **SyntheticAtoms**.

NOTE: This block is mandatory.

%block SyntheticAtoms `<None>` (block)

This block provides information about the ground-state valence configuration of a species. Its

main use is to complement the information in **ChemicalSpeciesLabel** for *synthetic* (alchemical) species, which are represented by atomic numbers over 200 in **ChemicalSpeciesLabel**. These species are created for example as a “mixture” of two real ones for a “virtual crystal” (VCA) calculation. In this special case a new **SyntheticAtoms** block must be present to give SIESTA information about the “ground state” of the synthetic atom.

```
%block ChemicalSpeciesLabel
1 201 ON-0.50000
%endblock ChemicalSpeciesLabel
%block SyntheticAtoms
1 # Species index
2 2 3 4 # n numbers for valence states with l=0,1,2,3
2.0 3.5 0.0 0.0 # occupations of valence states with l=0,1,2,3
%endblock SyntheticAtoms
```

Pseudopotentials for synthetic atoms can be created using the `mixps` and `fractional` programs in the `Util/VCA` directory.

Atomic numbers below -200 represent *ghost synthetic atoms*.

Note that the procedure used in the automatic handling of semicore states does not work for synthetic atoms. If semicore states are present, the species must be put in the **PAO.Basis** block. Otherwise the program will assume that there are *no* semicore states.

This block can also be used to provide an alternate ground state valence configuration for real atoms in some special cases. For example, the nominal valence configuration for Pd in the Siesta internal tables is 5s1 5p0 4d9 4f0, but in some tables it appears as 5s0 5p0 4d10 4f0. In this case, the alternate configuration can be specified by the block:

```
%block ChemicalSpeciesLabel
1 46 Pd
%endblock ChemicalSpeciesLabel
%block synthetic-atoms
1
5 5 4 4
0.0 0.0 10.0 0.0
%endblock synthetic-atoms
```

As another example, the nominal valence for Cu in Siesta is 4s1 4p0 3d10 4f0, but in some cases a pseudopotential might be generated by considering the 3d shell as frozen in the core. In this case the proper valence configuration is:

```
%block ChemicalSpeciesLabel
1 29 Cu_3d_in_core
%endblock ChemicalSpeciesLabel
%block synthetic-atoms
1
4 4 4 4
1.0 0.0 0.0 0.0
%endblock synthetic-atoms
```

As a final example, the nominal valence configuration for Ce in Siesta is 6s2 6p0 5d0 4f2, but on some tables it appears as [Xe] 6s2 4f1 5d1. In addition, the pseudo-dojo pseudopotential (in the NC SR+3 table) has the 4f shell frozen in the core. This case can be handled by the block:

```
%block ChemicalSpeciesLabel
1 58 Ce_4f_in_core
```

```

%endblock ChemicalSpeciesLabel
%block synthetic-atoms
1
  6 6 5 5
  2.0 0.0 1.0 0.0
%endblock synthetic-atoms

```

Note that the change in the atomic ground-state configuration might change the choice of polarization orbitals, and possibly other Siesta heuristic decisions, so the results should be checked carefully.

%block AtomicMass *<None>* *(block)*

It allows the user to introduce the atomic masses of the different species used in the calculation, useful for the dynamics with isotopes, for example. If a species index is not found within the block, the natural mass for the corresponding atomic number is assumed. If the block is absent all masses are the natural ones. One line per species with the species index (integer) and the desired mass (real). The order is not important. If there is no integer and/or no real numbers within the line, the line is disregarded.

```

%block AtomicMass
  3 21.5
  1 3.2
%endblock AtomicMass

```

The default atomic mass are the natural masses. For *ghost* atoms (i.e. floating orbitals) the mass is 10^{30} a.u.

6.2 Pseudopotentials

SIESTA uses pseudopotentials to represent the electron-ion interaction (as do most plane-wave codes and in contrast to so-called “all-electron” programs). In particular, the pseudopotentials are of the “norm-conserving” kind.

The pseudopotentials will be read by SIESTA from different files, according to the species information in the block **ChemicalSpeciesLabel**). Recall that an optional *ps-file-spec* can be present for each species. If absent, *ps-file-spec* defaults to the species’ label (*Chemical_label*).

The name of the files can be:

- *ps-file-spec.vps* (unformatted) or
- *ps-file-spec.psf* (ASCII) or
- *ps-file-spec.psml* (PSML format)

Files are searched by default in the current directory. In addition, the environment variable `SIESTA_PS_PATH` can be used to provide a set of alternate paths in which to search for files.

The rules for pseudopotential file discovery, given *ps-file-spec*, are:

- If *ps-file-spec* does not have an extension, the following rules are applied in turn adding each of *.vps*, *.psf*, *.psml* to *ps-file-spec*, in that order of preference. The search ends with the first finding.

- If *ps-file-spec* is not an absolute path, (e.g. `Si` or `C.psf`, or `pbe/C.psm1`), the search is done on the implied path (i.e. with reference to the current directory), and in each of the sections in `SIESTA_PS_PATH` with *ps-file-spec* (and possibly an extension) appended.
- If *ps-file-spec* is an absolute path, (e.g. `/home/user/Si` or `/data/ps/C.psm1`), the search is done *only* in the path, possibly with an extension appended.

Pseudopotential files in the `.psf` format can be generated by the `ATOM` program, (see `Pseudo/README.ATOM`) and by a number of other codes such as `APE`. The `.vps` format is a binary version of the `.psf` format, and is deprecated.

Pseudopotential files in the PSML format (see García et al.^[7]) can be produced by the combination of `ATOM` and `psop` (see directory `Pseudo/vnl-operator`) in a form fully compatible with the SIESTA procedures to generate the non-local pseudopotential operator. Notably, they can also be produced by suitably patched versions of D.R. Hamann’s `oncvpsp` program (see directory `Pseudo/Third-Party-Tools/ONCVPSP`). The `oncvpsp` code can generate several projectors per *l* channel, leading to pseudopotentials that are more transferable.

For more information on the format itself and the PSML ecosystem of generators and client ab-initio codes, please see <http://esl.cecam.org/PSML>.

Note that curated databases of high-quality PSML files are available. In particular, the Pseudo-Dojo project <https://www.pseudo-dojo.org> offers PSML files for almost the whole periodic table, together with a report of the tests carried out during the generation procedure.

In this connection, it should be stressed that **all pseudopotentials should be thoroughly tested** before using them. We refer you to the standard literature on pseudopotentials, to the `ATOM` manual, and to the Pseudo-Dojo site for more information.

Please take into account the following when using PSML files:

- If present in the execution directory, `.psf` files take precedence over `.psml` files. That is, if both `Chemical_label.psf` and `Chemical_label.psm1` are present, SIESTA will process the former.
- PSML files typically contain semilocal potentials, a local potential, and non-local projectors. By default, SIESTA will use the local potential and non-local projectors from the PSML file, unless the respective options `PSML.Vlocal` and `PSML.KB.projectors` are set to **false**. These options are **true** by default. Several combinations are possible with these options:
 - The recommended (and default) is to use the local potential and projectors from the PSML file.
 - One could use only the semilocal potentials from the PSML file, and proceed to generate a local potential and KB projectors with the traditional SIESTA algorithm.
 - One could use the semilocal potentials and the local potential from the PSML file, and generate a set of KB projectors from them.
- In order to generate its basis set of pseudo-atomic orbitals (PAOs), SIESTA still needs the semilocal parts of the pseudopotential. Currently all available PSML files (generated by `ATOM+psop` or `ONCVPSP`) contain semilocal potentials, but this might change in the future (for example, when a PSML file is obtained from a projectors-only UPF file). This restriction will

be lifted in a later version: SIESTA will then be able to use the full pseudopotential operator to generate the PAOs.

- For the *full* (default) version of spin-orbit-coupling (SOC), SIESTA uses fully relativistic (*lj*) projectors. These are available in PSML files generated by **ONCVSP** in fully-relativistic mode, if the **psfile** option **upf** or **both** is used in the appropriate place in the input file. To obtain appropriate PSML files with the **ATOM+psop** chain (see the directory **Pseudo/vnl-operator**), the projector generation with **psop** must use the **-r** option. Note that *lj* projectors can still be directly generated by SIESTA from relativistic semilocal potentials.
- Fully-relativistic PSML files with only *lj* non-local projectors cannot be used directly in calculations not involving “full” SOC. For this, SIESTA needs the “scalar-relativistic” projectors. An algorithm for direct generation of SR projectors from an *lj* set already exists as part of the **oncvpsp** code, and it will be integrated in a forthcoming version. In the meantime, while in principle it is possible to read only the semilocal potentials from the file and proceed to generate the appropriate projectors, it is better to use PSML files which contain both (actually three) sets of non-local projectors: “sr”, “so”, and *lj*. These can be obtained with **ONCVSP** with the **both** option. (For the **ATOM+psop** chain, it is currently necessary to run **psop** twice (once with the **-r** option) and generate two different PSML files, and then “graft” the “sr” set into the file containing the *lj* set.)
- A large number of PSML files obtained from the Pseudo-Dojo database are generated with (several) semicore shells. Dealing with them has uncovered a few weaknesses in the standard heuristics used traditionally in SIESTA to generate basis sets:
 - Sometimes it was not possible to execute successfully the legacy split-norm algorithm. Now, the default is to use **PAO.SplitTailNorm true**, with a simpler, more robust algorithm. See the section on split-norm for full details.
 - The default perturbative scheme for polarization orbitals can fail in very specific cases. When the polarization orbital has to have a node due to the presence of a lower-lying orbital with the same *l*, the program can (if enabled by the **PAO.Polarization.NonPerturbative.Fallback** option, which is **true** by default) automatically switch to using a non-perturbative scheme. In other cases, include the *Chemical_label* in the **PAO.Polarization.Scheme** block to request a non-perturbative scheme:

```
%block PAO.Polarization.Scheme
  Mg non-perturbative
%endblock PAO.Polarization.Scheme
```

Please see the relevant section for a fuller explanation.

- A number of improvements to the PAO generation code have been made while implementing support for PSML pseudopotentials. In particular, SIESTA can now automatically detect and generate basis sets for atoms with semicore shells without the explicit use of a **PAO.Basis** block.

6.3 Basis set and KB projectors

6.3.1 Overview of atomic-orbital bases implemented in SIESTA

The main advantage of atomic orbitals is their efficiency (fewer orbitals needed per electron for similar precision) and their main disadvantage is the lack of systematics for optimal convergence, an issue that quantum chemists have been working on for many years. They have also clearly shown that there is no limitation on precision intrinsic to LCAO. This section provides some information about how basis sets can be generated for SIESTA.

It is important to stress at this point that neither the SIESTA method nor the program are bound to the use of any particular kind of atomic orbitals. The user can feed into SIESTA the atomic basis set he/she chooses by means of radial tables (see **User.Basis** below), the only limitations being: (i) the functions have to be atomic-like (radial functions multiplied by spherical harmonics), and (ii) they have to be of finite support, i.e., each orbital becomes strictly zero beyond some cutoff radius chosen by the user.

Most users, however, do not have their own basis sets. For these users we have devised some schemes to generate basis sets within the program with a minimum input from the user. If nothing is specified in the input file, SIESTA generates a default basis set of a reasonable quality that might constitute a good starting point. Of course, depending on the accuracy required in the particular problem, the user has the degree of freedom to tune several parameters that can be important for quality and efficiency. A description of these basis sets and some performance tests can be found in the references quoted below.

“Numerical atomic orbitals for linear-scaling calculations”, J. Junquera, O. Paz, D. Sánchez-Portal, and E. Artacho, *Phys. Rev. B* **64**, 235111, (2001)

An important point here is that the basis set selection is a variational problem and, therefore, minimizing the energy with respect to any parameters defining the basis is an “ab initio” way to define them.

We have also devised a quite simple and systematic way of generating basis sets based on specifying only one main parameter (the energy shift) besides the basis size. It does not offer the best NAO results one can get for a given basis size but it has the important advantages mentioned above. More about it in:

“Linear-scaling ab-initio calculations for large and complex systems”, E. Artacho, D. Sánchez-Portal, P. Ordejón, A. García and J. M. Soler, *Phys. Stat. Sol. (b)* **215**, 809 (1999).

In addition to SIESTA we provide the program **Gen-basis**, which reads SIESTA’s input and generates basis files for later use. **Gen-basis** can be found in **Util/Gen-basis**. It should be run from the **Tutorials/Bases** directory, using the **gen-basis.sh** script. It is limited to a single species.

Of course, as it happens for the pseudopotential, it is the responsibility of the user to check that the physical results obtained are converged with respect to the basis set used before starting any production run.

In the following we give some clues on the basics of the basis sets that SIESTA generates. The starting point is always the solution of Kohn-Sham’s Hamiltonian for the isolated pseudo-atoms, solved in a radial grid, with the same approximations as for the solid or molecule (the same exchange-correlation functional and pseudopotential), plus some way of confinement (see below). We describe in the following three main features of a basis set of atomic orbitals: size, range, and radial shape.

Size: number of orbitals per atom

Following the nomenclature of Quantum Chemistry, we establish a hierarchy of basis sets, from single- ζ to multiple- ζ with polarization and diffuse orbitals, covering from quick calculations of low quality to high precision, as high as the finest obtained in Quantum Chemistry. A single- ζ (also called minimal) basis set (SZ in the following) has one single radial function per angular momentum channel, and only for those angular momenta with substantial electronic population in the valence of the free atom. It offers quick calculations and some insight on qualitative trends in the chemical bonding and other properties. It remains too rigid, however, for more quantitative calculations requiring both radial and angular flexibilization.

Starting by the radial flexibilization of SZ, a better basis is obtained by adding a second function per channel: double- ζ (DZ). In Quantum Chemistry, the *split valence* scheme is widely used: starting from the expansion in Gaussians of one atomic orbital, the most contracted Gaussians are used to define the first orbital of the double- ζ and the most extended ones for the second. For strictly localized functions there was a first proposal of using the excited states of the confined atoms, but it would work only for tight confinement (see **PAO.BasisType** nodes below). This construction was proposed and tested in D. Sánchez-Portal *et al.*, J. Phys.: Condens. Matter **8**, 3859-3880 (1996).

We found that the basis set convergence is slow, requiring high levels of multiple- ζ to achieve what other schemes do at the double- ζ level. This scheme is related with the basis sets used in the OpenMX project [see T. Ozaki, Phys. Rev. B **67**, 155108 (2003); T. Ozaki and H. Kino, Phys. Rev. B **69**, 195113 (2004)].

We then proposed an extension of the split valence idea of Quantum Chemistry to strictly localized NAO which has become the standard and has been used quite successfully in many systems (see **PAO.BasisType** **split** below). It is based on the idea of supplementing the first ζ with, instead of a gaussian, a numerical orbital that reproduces the tail of the original PAO outside a matching radius r_m , and continues smoothly towards the origin as $r^l(a - br^2)$, with a and b ensuring continuity and differentiability at r_m . Within exactly the same Hilbert space, the second orbital can be chosen to be the difference between the smooth one and the original PAO, which gives a basis orbital strictly confined within the matching radius r_m (smaller than the original PAO!) continuously differentiable throughout.

Extra parameters have thus appeared: one r_m per orbital to be doubled. The user can again introduce them by hand (see **PAO.Basis** below). Alternatively, all the r_m 's can be defined at once by specifying the value of the tail of the original PAO beyond r_m , the so-called split norm. Variational optimization of this split norm performed on different systems shows a very general and stable performance for values around 15% (except for the $\sim 50\%$ for hydrogen). It generalizes to multiple- ζ trivially by adding an additional matching radius per new zeta.

Note: In previous versions of the program what was actually used as split-valence reference was the norm of the tail *plus* the norm of the parabola-like inner function.

Angular flexibility is obtained by adding shells of higher angular momentum. Ways to generate these so-called polarization orbitals have been described in the literature for Gaussians. For NAOs there are two ways for SIESTA and **Gen-basis** to generate them: (i) Use atomic PAO's of higher angular momentum with suitable confinement, and (ii) solve the pseudoatom in the presence of an electric field and obtain the $l + 1$ orbitals from the perturbation of the l orbitals by the field. Experience shows that method (i) tends to give better results.

So-called diffuse orbitals, that might be important in the description of open systems such as surfaces,

can be simply added by specifying extra “n” shells. [See S. Garcia-Gil, A. Garcia, N. Lorente, P. Ordejón, Phys. Rev. B **79**, 075441 (2009)]

Finally, the method allows the inclusion of off-site (ghost) orbitals (not centered around any specific atom), useful for example in the calculation of the counterpoise correction for basis-set superposition errors. Bessel functions for any radius and any excitation level can also be added anywhere to the basis set.

Range: cutoff radii of orbitals.

Strictly localized orbitals (zero beyond a cutoff radius) are used in order to obtain sparse Hamiltonian and overlap matrices for linear scaling. One cutoff radius per angular momentum channel has to be given for each species.

A balanced and systematic starting point for defining all the different radii is achieved by giving one single parameter, the energy shift, i.e., the energy increase experienced by the orbital when confined. Allowing for system and physical-quantity variability, as a rule of thumb $\Delta E_{\text{PAO}} \approx 100$ meV gives typical precisions within the accuracy of current GGA functionals. The user can, nevertheless, change the cutoff radii at will.

Shape

Within the pseudopotential framework it is important to keep the consistency between the pseudopotential and the form of the pseudoatomic orbitals in the core region. The shape of the orbitals at larger radii depends on the cutoff radius (see above) and on the way the localization is enforced.

The first proposal (and quite a standard among SIESTA users) uses an infinite square-well potential. It was originally proposed and has been widely and successfully used by Otto Sankey and collaborators, for minimal bases within the ab initio tight-binding scheme, using the **Fireball** program, but also for more flexible bases using the methodology of SIESTA. This scheme has the disadvantage, however, of generating orbitals with a discontinuous derivative at r_c . This discontinuity is more pronounced for smaller r_c ’s and tends to disappear for long enough values of this cutoff. It does remain, however, appreciable for sensible values of r_c for those orbitals that would be very wide in the free atom. It is surprising how small an effect such a kink produces in the total energy of condensed systems. It is, on the other hand, a problem for forces and stresses, especially if they are calculated using a (coarse) finite three-dimensional grid.

Another problem of this scheme is related to its defining the basis starting from the free atoms. Free atoms can present extremely extended orbitals, their extension being, besides problematic, of no practical use for the calculation in condensed systems: the electrons far away from the atom can be described by the basis functions of other atoms.

A traditional scheme to deal with this is one based on the radial scaling of the orbitals by suitable scale factors. In addition to very basic bonding arguments, it is soundly based on restoring the virial’s theorem for finite bases, in the case of Coulombic potentials (all-electron calculations). The use of pseudopotentials limits its applicability, allowing only for extremely small deviations from unity ($\sim 1\%$) in the scale factors obtained variationally (with the exception of hydrogen that can contract up to 25%). This possibility is available to the user.

Another way of dealing with the above problem and that of the kink at the same time is adding a soft confinement potential to the atomic Hamiltonian used to generate the basis orbitals: it smoothens the kink and contracts the orbital as suited. Two additional parameters are introduced for the purpose, which can be defined again variationally. The confining potential is flat (zero) in the core

region, starts off at some internal radius r_i with all derivatives continuous and diverges at r_c ensuring the strict localization there. It is

$$V(r) = V_o \frac{e^{-\frac{r_c - r_i}{r - r_i}}}{r_c - r} \quad (1)$$

and both r_i and V_o can be given to SIESTA together with r_c in the input (see **PAO.Basis** below). The kink is normally well smoothened with the default values for soft confinement by default (**PAO.SoftDefault** true), which are $r_i = 0.9r_c$ and $V_o = 40$ Ry.

When explicitly introducing orbitals in the basis that would be empty in the atom (e.g. polarisation orbitals) these tend to be extremely extended if not completely unbound. The above procedure produces orbitals that bulge as far away from the nucleus as possible, to plunge abruptly at r_c . Soft confinement can be used to try to force a more reasonable shape, but it is not ideal (for orbitals peaking in the right region the tails tend to be far too short). *Charge confinement* produces very good shapes for empty orbitals. Essentially a Z/r potential is added to the soft confined potential above. For flexibility the charge confinement option in SIESTA is defined as

$$V_Q(r) = \frac{Ze^{-\lambda r}}{\sqrt{r^2 + \delta^2}} \quad (2)$$

where δ is there to avoid the singularity (default $\delta = 0.01$ Bohr), and λ allows to screen the potential if longer tails are needed. The description on how to introduce this option can be found in the **PAO.Basis** entry below.

Finally, the shape of an orbital is also changed by the ionic character of the atom. Orbitals in cations tend to shrink, and they swell in anions. Introducing a δQ in the basis-generating free-atom calculations gives orbitals better adapted to ionic situations in the condensed systems.

More information about basis sets can be found in the proposed literature.

There are quite a number of options for the input of the basis-set and KB projector specification, and they are all optional! By default, SIESTA will use a DZP basis set with appropriate choices for the determination of the range, etc. Of course, the more you experiment with the different options, the better your basis set can get. To aid in this process we offer an auxiliary program for optimization which can be used in particular to obtain variationally optimal basis sets (within a chosen basis size). See **Util/Optimizer** for general information, and **Util/Optimizer/Examples/Basis_Optim** for an example. The directory **Tutorials/Bases** in the main SIESTA distribution contains some tutorial material for the generation of basis sets and KB projectors.

Finally, some optimized basis sets for particular elements are available at the SIESTA web page. Again, it is the responsibility of the users to test the transferability of the basis set to their problem under consideration.

6.3.2 Type of basis sets

PAO.BasisType split

(string)

The kind of basis to be generated is chosen. All are based on finite-range pseudo-atomic orbitals [PAO's of Sankey and Niklewsky, PRB 40, 3979 (1989)]. The original PAO's were described only for minimal bases. SIESTA generates extended bases (multiple- ζ , polarization, and diffuse orbitals) applying different schemes of choice:

- Generalization of the PAO's: uses the excited orbitals of the finite-range pseudo-atomic problem, both for multiple- ζ and for polarization [see Sánchez-Portal, Artacho, and Soler, JPCM **8**, 3859 (1996)]. Adequate for short-range orbitals.
- Multiple- ζ in the spirit of split valence, decomposing the original PAO in several pieces of different range, either defining more (and smaller) confining radii, or introducing Gaussians from known bases (Huzinaga's book).

All the remaining options give the same minimal basis. The different options and their FDF descriptors are the following:

split Split-valence scheme for multiple-zeta. The split is based on different radii.

splitgauss Same as **split** but using gaussian functions $e^{-(x/\alpha_i)^2}$. The gaussian widths α_i are read instead of the scale factors (see below). There is no cutting algorithm, so that a large enough r_c should be defined for the gaussian to have decayed sufficiently.

nodes Generalized PAO's.

nonodes The original PAO's are used, multiple-zeta is generated by changing the scale-factors, instead of using the excited orbitals.

filteret Use the filterets as a systematic basis set. The size of the basis set is controlled by the filter cut-off for the orbitals.

Note that, for the **split** and **nodes** cases the whole basis can be generated by SIESTA with no further information required. SIESTA will use default values as defined in the following (**PAO.BasisSize**, **PAO.EnergyShift**, and **PAO.SplitNorm**, see below).

6.3.3 Size of the basis set

PAO.BasisSize **DZP** (string)

It defines usual basis sizes. It has effect only if there is no block **PAO.Basis** present.

SZ|minimal Use single- ζ basis.

DZ Double zeta basis, in the scheme defined by **PAO.BasisType**.

SZP Single-zeta basis plus polarization orbitals.

DZP|standard Like **DZ** plus polarization orbitals.

NOTE: The ground-state atomic configuration used internally by SIESTA is defined in the source file `Src/periodic_table.f`. For some elements (e.g., Pd), the configuration might not be the standard one.

NOTE: By default, polarization orbitals are constructed from perturbation theory, and they are defined so they have the minimum angular momentum l such that there are no occupied orbitals with the same l in the valence shell of the ground-state atomic configuration. They polarize the corresponding $l - 1$ shell.

See **PAO.Polarization.NonPerturbative** and **PAO.Polarization.Scheme** in Sec. 6.3.6 for options to generate polarization orbitals non-perturbatively.

%block PAO.BasisSizes **<None>** (block)

Block which allows to specify a different value of the variable **PAO.BasisSize** for each species. For example,

```
%block PAO.BasisSizes
  Si      DZ
  H       DZP
  O       SZP
%endblock PAO.BasisSizes
```

6.3.4 Range of the orbitals

PAO.EnergyShift 0.01 Ry *(energy)*

A standard for orbital-confining cutoff radii. It is the excitation energy of the PAO's due to the confinement to a finite-range. It offers a general procedure for defining the confining radii of the original (first-zeta) PAO's for all the species guaranteeing the compensation of the basis. It only has an effect when the block **PAO.Basis** is not present or when the radii specified in that block are zero for the first zeta.

Write.Graphviz none|atom|orbital|atom+orbital *(string)*

Write out the sparsity pattern after having determined the basis size overlaps. This will generate `SystemLabel.ATOM.gv` or `SystemLabel.ORB.gv` which both may be converted to a graph using Graphviz's program `neato`:

```
neato -x -Tpng siesta.ATOM.gv -o siesta_ATOM.png
```

The resulting graph will list each atom as $i(j)$ where i is the atomic index and j is the number of other atoms it is connected to.

6.3.5 Generation of multiple-zeta orbitals

PAO.SplitNorm 0.15 *(real)*

depends on: **PAO.SplitTailNorm**

A standard to define sensible default radii for the split-valence type of basis. It gives the amount of norm that the second- ζ split-off piece has to carry. The split radius is defined accordingly. If multiple- ζ is used, the corresponding radii are obtained by imposing smaller fractions of the SplitNorm (1/2, 1/4, 1/6 ...) value as norm carried by the higher zetas. It only has an effect when the block **PAO.Basis** is not present or when the radii specified in that block are zero for zetas higher than one.

NOTE: When using **PAO.SplitTailNorm true** (the default as of SIESTA V5) the mapping of split-norm parameters to radial matching points changes. Legacy values might have to be revised. See the longer note under **PAO.SplitTailNorm**.

PAO.SplitNormH <PAO.SplitNorm> *(real)*

This option is as per **PAO.SplitNorm** but allows a separate default to be specified for hydrogen which typically needs larger values than those for other elements.

PAO.SplitTailNorm true *(logical)*

Use the norm of the tail instead of the "tail+parabola" norm to compute the "split" contribution. This is the behavior described in the JPC paper, but for numerical reasons the square root of

the tail norm is used in the algorithm. This approach guarantees a match for any value of the **PAO.SplitNorm** parameter, and is the preferred mode of operation.

NOTE: For a given value of **PAO.SplitNorm** one can find these two cases, depending on the setting of this option:

true the cutoff lengths of the higher ζ orbitals will be shorter

The split-norm curve (square root of the tail norm) approaches 0 faster in this case, hence the matching radius is found at a shorter distance.

false the cutoff lengths of the higher ζ orbitals will be longer

The (tail+parabola) norm is used as the split-norm reference. This curve *likely* approaches 0 more slowly (or not at all), hence the matching radius is found at a longer distance.

PAO.SplitValence.Legacy **false** (logical)

Recovers the behavior and default settings of the legacy split-valence options. In addition to PAO!SplitTailNorm, which would be **false** by default, users can set the option PAO!FixSplitTable (see below).

PAO.FixSplitTable **false** (logical)

After the scan of the (tail+parabola) norm table (curve), apply a damping function to the tail to make sure that the table goes to zero at the radius of the first-zeta orbital. **PAO.FixSplitTable** **true** guarantees that the program finds a solution, even when using the tail+parabola split-norm heuristic. The result might not be optimal (in the sense of producing a second- ζ r_c very close to the first- ζ one).

This option is not accessible with PAO!SplitValence!Legacy:false.

PAO.EnergyCutoff 20 Ry (energy)

If the multiple zetas are generated using filterets then only the filterets with an energy lower than this cutoff are included. Increasing this value leads to a richer basis set (provided the cutoff is raised above the energy of any filteret that was previously not included) but a more expensive calculation. It only has an effect when the option **PAO.BasisType** is set to **filteret**.

PAO.EnergyPolCutoff 20 Ry (energy)

If the multiple zetas are generated using filterets then only the filterets with an energy lower than this cutoff are included for the polarisation functions. Increasing this value leads to a richer basis set (provided the cutoff is raised above the energy of any filteret that was previously not included) but a more expensive calculation. It only has an effect when the option **PAO.BasisType** is set to **filteret**.

PAO.ContractionCutoff 0|0 – 1 (real)

If the multiple zetas are generated using filterets then any filterets that have a coefficient less than this threshold within the original PAO will be contracted together to form a single filteret. Increasing this value leads to a smaller basis set but allows the underlying basis to have a higher kinetic energy cut-off for filtering. It only has an effect when the option **PAO.BasisType** is set to **filteret**.

6.3.6 Polarization-orbital options

Polarization orbitals can be requested through an automatic basis-size specification such as DZP, or TZP, etc, or through the use of the 'P' shell option in the **PAO.Basis** block.

In these cases, by default, polarization orbitals are generated perturbatively, by formally applying an electric field to the orbital being polarized.

Polarization shells can also be put explicitly in the **PAO.Basis** block. In this case, the orbitals are generated in the standard way, using the appropriate confinement and split-norm options.

If the perturbative method is not wanted, even when using the standard basis specifications, the following global option can be used:

PAO.Polarization.NonPerturbative `false` *(logical)*

If enabled, it will promote any polarization shells to the status of explicit shells, thus using the standard generation options.

Also, this setting can be controlled species by species, by using a block

%block PAO.Polarization.Scheme `<None>` *(block)*

Block which allows to specify a different polarization scheme for each species. For example,

```
%block PAO.Polarization.Scheme
  Si      non-perturbative [ optional Q options]
  H       perturbative
%endblock PAO.Polarization.Scheme
```

The presence of 'perturbative' for a species in the block has the effect of *forcing* the use of the perturbative option.

If a species does not appear in the block, the setting of **PAO.Polarization.NonPerturbative** applies. The default scheme is perturbative.

An optional charge-confinement specification can follow, starting with a 'Q', in exactly the same way as in the **PAO.Basis** block.

The perturbative method does not require any extra information regarding confinement, since the r_c value for the polarization shell is the same as the one for the polarized shell. If the perturbative method is turned off, the new explicit shell created for the polarization orbital will be assigned an r_c equal to the one actually used for the shell to be polarized (for the 1st zeta). The only extra control offered at this point is a possible expansion of this value through the (global) option:

PAO.Polarization.Rc-Expansion-Factor `1.0` *(real)*

When turning off the perturbative method for the generation of polarization orbitals, assign to the 1st zeta of the explicit polarization shell the r_c of the polarized shell multiplied by this factor.

Note that, empirically, the perturbative method seems to give better results (in the variational sense), so the alternative should only be used when the default fails for some reason, for full basis-set optimization, or for experimentation purposes. In particular, non-perturbatively generated polarization orbitals tend to bulge outwards. To correct this, the charge-confinement options in the **PAO.Basis** block (or in the **PAO.Polarization.Scheme** block) might be helpful.

There is one case, however, which tends to exhibit problems in the perturbative algorithm: when a polarization orbital has to have a node due to the presence of a lower-lying orbital of the same l (this will happen, for example, for Ge if the $3d$ orbital is considered part of the valence). In this case, the program can automatically switch to using the non-perturbative scheme. To enable this automatic switch, the option **PAO.Polarization.NonPerturbative.Fallback** must be enabled (it is by default). Note that if the 'perturbative' option is explicitly set in the block above, the fallback is overridden.

A proper basis-set optimization should be carried out using a **PAO.Basis** block, which allows a full set of options.

6.3.7 Soft-confinement options

A brief description of the soft-confinement options is given below. This is the default way of generating basis orbitals, and disabling it is not recommended unless going for backwards compatibility. The default potential and inner radius options should not be changed either, except when needed for an specific application.

PAO.SoftDefault `true` *(logical)*

This option causes soft confinement to be the default form of potential during orbital generation. Disabling this will cause a fallback to the original hard-confined orbitals.

PAO.SoftInnerRadius `0.9` *(real)*

For default soft confinement, the inner radius is set at a fraction of the outer confinement radius determined by the energy shift. This option controls the fraction of the confinement radius to be used.

PAO.SoftPotential `40 Ry` *(energy)*

For default soft confinement, this option controls the value of the potential used for all orbitals.

NOTE: Soft-confinement options (inner radius, prefactor) have been traditionally used to optimize the basis set, even though formally they are just a technical necessity to soften the decay of the orbitals at r_c . To achieve this, it might be enough to use the above global options.

6.3.8 Kleinman-Bylander projectors

NOTE: SIESTA is now able to read directly the non-local projectors from a PSML file. For this, the options **PSML.Vlocal** and **PSML.KB.projectors** must be set to **true** (they are by default), and a **Chemical_label.psml** file must be present. The rest of the options discussed in this section will have no effect in that case.

%block PS.lmax `<None>` *(block)*

Block with the maximum angular momentum of the Kleinman-Bylander projectors, **l_{mxkb}**. This information is optional. If the block is absent, or for a species which is not mentioned inside it, SIESTA will take $l_{mxkb}(is) = l_{mxo}(is) + 1$, where $l_{mxo}(is)$ is the maximum angular momentum of the basis orbitals of species *is*. However, the value of **l_{mxkb}** is actually limited by the highest- l channel in the pseudopotential file.

%block Ps.lmax

```

        Al_adatom    3
        H            1
        O            2
%endblock Ps.lmax

```

By default `lmax` is the maximum angular momentum plus one, limited by the highest- l channel in the pseudopotential file.

%block PS.KBprojectors *<None>* *(block)*

This block provides information about the number of Kleinman-Bylander projectors per angular momentum that will be used in the calculation. This block is optional. If it is absent, or for species not mentioned in it, only one projector will be used for each angular momentum (except for l -shells with semicore states, for which two projectors will be constructed). The projectors will be constructed using the eigenfunctions of the respective pseudopotentials.

This block allows to specify also the reference energies of the wavefunctions used to build them. The specification of the reference energies is optional. If these energies are not given, the program will use the eigenfunctions with an increasing number of nodes (if there is not bound state with the corresponding number of nodes, the “eigenstates” are taken to be just functions which are made zero at very long distance of the nucleus). The units for the energy can be optionally specified; if not, the program will assume that they are given in Rydbergs. The data provided in this block must be consistent with those read from the block **PS.lmax**. For example,

```

%block PS.KBprojectors
  Si  3
    2  1
    -0.9    eV
    0  2
    -0.5  -1.0d4 Hartree
    1  2
  Ga  1
    1  3
    -1.0  1.0d5 -6.0
%endblock PS.KBprojectors

```

The reading is done this way (those variables in brackets are optional, therefore they are only read if present):

```

From is = 1 to nspecies
  read: label(is), l_shells(is)
  From lsh=1 to l_shells(is)
    read: l, nkbl(l,is)
    read: {erefKB(izeta,il,is)}, from ikb = 1 to nkbl(l,is), {units}

```

All angular momentum shells should be specified. Default values are assigned to missing shells with l below `lmax`, where `lmax` is the highest angular momentum present in the block for that particular species. High- l shells (beyond `lmax`) not specified in the block will also be assigned default values.

Care should be taken for l -shells with semicore states. For them, two KB projectors should be generated. This is not checked while processing this block.

When a very high energy, higher than 1000 Ry, is specified, the default is taken instead. On the other hand, very low (negative) energies, lower than -1000 Ry, are used to indicate that the energy derivative of the last state must be used. For example, in the block given above, two projectors will be used for the *s* pseudopotential of Si. One generated using a reference energy of -0.5 Hartree, and the second one using the energy derivative of this state. For the *p* pseudopotential of Ga, three projectors will be used. The second one will be constructed from an automatically generated wavefunction with one node, and the other projectors from states at -1.0 and -6.0 Rydberg.

The analysis looking for possible *ghost* states is only performed when a single projector is used. Using several projectors some attention should be paid to the “KB cosine” (*kbcos*), given in the output of the program. The KB cosine gives the value of the overlap between the reference state and the projector generated from it. If these numbers are very small (< 0.01 , for example) for **all** the projectors of some angular momentum, one can have problems related with the presence of ghost states.

The default is *one* KB projector from each angular momentum, constructed from the nodeless eigenfunction, used for each angular momentum, except for *l*-shells with semicore states, for which two projectors will be constructed. Note that the value of *lmaxkb* is actually limited by the highest-*l* channel in the pseudopotential file.

For full spin-orbit calculations, the program generates *lj* projectors using the $l+1/2$ and $l-1/2$ components of the (relativistic) pseudopotentials. In this case the specification of the reference energies for projectors is not changed: only *l* is relevant. Fully relativistic projectors can also be read from a suitably generated PSML file.

KB.New.Reference.Orbitals `false` (*logical*)

If **true**, the routine to generate KB projectors will use slightly different parameters for the construction of the reference orbitals involved (*Rmax*=60 Bohr both for integration and normalization).

6.3.9 The PAO.Basis block

%block PAO.Basis `<None>` (*block*)

Block with data to define explicitly the basis to be used. It allows the definition by hand of all the parameters that are used to construct the atomic basis. There is no need to enter information for all the species present in the calculation. The basis for the species not mentioned in this block will be generated automatically using the parameters **PAO.BasisSize**, **PAO.BasisType**, **PAO.EnergyShift**, **PAO.SplitNorm** (or **PAO.SplitNormH**), and the soft-confinement defaults (see **PAO.SoftDefault**).

NOTE: This block gives full control to the user, and with that comes the user’s responsibility of making sure that the contents of the block are appropriate. This is particularly important for the specification of the PAO shells to be included in the basis set. Some pseudopotentials are generated with semicore states, which should be included in the basis set. Conversely, a **PAO.Basis** block prepared for a species with semicore states will not work for the same chemical element if employing a pseudopotential without them. As a suggestion, users might want to try first a run in “automatic mode” (i.e. without using a **PAO.Basis** block) just to check which basis-set shells are identified by the built-in heuristics in the program. Then the skeleton **PAO.Basis** produced might be fully fleshed out according to the user’s needs.

Some parameters can be set to zero, or left out completely. In these cases the values will be generated from the magnitudes defined above, or from the appropriate default values. For example, the radii will be obtained from **PAO.EnergyShift** or from **PAO.SplitNorm** if they are zero; the scale factors will be put to 1 if they are zero or not given in the input. An example block for a two-species calculation (H and O) is the following (*opt* means optional):

```
%block PA0.Basis      # Define Basis set
O      2 nodes 1.0    # Label, l_shells, type (opt), ionic_charge (opt)
n=2 0 2 E 50.0 2.5    # n (opt if not using semicore levels), l, Nzeta, Softconf(opt)
      3.50 3.50      # rc(izeta=1,Nzeta)(Bohr)
      0.95 1.00      # scaleFactor(izeta=1,Nzeta) (opt)
      1 1 P 2        # l, Nzeta, PolOrb (opt), NzetaPol (opt)
      3.50           # rc(izeta=1,Nzeta)(Bohr)
H      2             # Label, l_shells, type (opt), ionic_charge (opt)
      0 2 S 0.2      # l, Nzeta, Per-shell split norm parameter
      5.00 0.00      # rc(izeta=1,Nzeta)(Bohr)
      1 1 Q 3. 0.2   # l, Nzeta, Charge conf (opt): Z and screening
      5.00           # rc(izeta=1,Nzeta)(Bohr)
%endblock PA0.Basis
```

The reading is done this way (those variables in brackets are optional, therefore they are only read if present) (See the routines in `Src/basis_specs.f` for detailed information):

```
From js = 1 to nspecies
  read: label(is), l_shells(is), { type(is) }, { ionic_charge(is) }
  From lsh=1 to l_shells(is)
    read:
      { n }, l(lsh), nzls(lsh,is), { PolOrb(l+1) }, { NzetaPol(l+1) },
      { SplitNormFlag(lsh,is) }, { SplitNormValue(lsh,is) }
      { SoftConfFlag(lsh,is) }, { PrefactorSoft(lsh,is) }, { InnerRadSoft(lsh,is) },
      { FilteretFlag(lsh,is) }, { FilteretCutoff(lsh,is) }
      { ChargeConfFlag(lsh,is) }, { Z(lsh,is) }, { Screen(lsh,is) }, { delta(lsh,is) }
      read: rcls(izeta,lsh,is), from izeta = 1 to nzls(l,is)
      read: { contrf(izeta,il,is) }, from izeta = 1 to nzls(l,is)
```

And here is the variable description:

- **Label**: Species label, this label determines the species index *is* according to the block **ChemicalSpeciesLabel**
- **l_shells(is)**: Number of shells of orbitals with different angular momentum for species *is*
- **type(is)**: *Optional input*. Kind of basis set generation procedure for species *is*. Same options as **PAO.BasisType**
- **ionic_charge(is)**: *Optional input*. Net charge of species *is*. This is only used for basis set generation purposes. *Default value*: 0.0 (neutral atom). Note that if the pseudopotential was generated in an ionic configuration, and no charge is specified in PAO.Basis, the ionic charge setting will be that of pseudopotential generation.
- **n**: Principal quantum number of the shell. This is an optional input for normal atoms, however it must be specified when there are *semicore* states (i.e. when states that usually are not considered to belong to the valence shell have been included in the calculation)
- **l**: Angular momentum of basis orbitals of this shell

- **nzls(lsh,is)**: Number of “zetas” for this shell. For a filteret basis this number is ignored since the number is controlled by the cutoff. For bessel-floating orbitals, the different ‘zetas’ map to increasingly excited states with the same angular momentum (with increasing number of nodes).
- **Pol0rb(l+1)**: *Optional input*. If set equal to P, a shell of polarization functions (with angular momentum $l+1$) will be constructed from the first-zeta orbital of angular momentum l . *Default value*: ‘ ’ (blank = No polarization orbitals).
- **NzetaPol(l+1)**: *Optional input*. Number of “zetas” for the polarization shell (generated automatically in a split-valence fashion). For a filteret basis this number is ignored since the number is controlled by the cutoff. Only active if **Pol0rb** = P. *Default value*: 1
- **SplitNormFlag(lsh,is)**: *Optional input*. If set equal to S, the following number sets the split-norm parameter for that shell.
- **SoftConfFlag(l,is)**: *Optional input*. If set equal to E, the soft confinement potential proposed in equation (1) of the paper by J. Junquera *et al.*, Phys. Rev. B **64**, 235111 (2001), is used instead of the Sankey hard-well potential.
- **PrefactorSoft(l,is)**: *Optional input*. Prefactor of the soft confinement potential (V_0 in the formula). Units in Ry. *Default value*: 0 Ry.
- **InnerRadSoft(l,is)**: *Optional input*. Inner radius where the soft confinement potential starts off (r_i in the formula). If negative, the inner radius will be computed as the given fraction of the PAO cutoff radius. Units in bohrs. *Default value*: 0 bohrs.
- **FilteretFlag(l,is)**: *Optional input*. If set equal to F, then an individual filter cut-off can be specified for the shell.
- **FilteretCutoff(l,is)**: *Optional input*. Shell-specific value for the filteret basis cutoff. Units in Ry. *Default value*: The same as the value given by **FilterCutoff**.
- **ChargeConfFlag(lsh,is)**: *Optional input*. If set equal to Q, the charge confinement potential in equation (2) above is added to the confining potential. If present it requires at least one number after it (Z), but it can be followed by two or three numbers.
- **Z(lhs,is)**: *Optional input, needed if Q is set*. Z charge in equation (2) above for charge confinement (units of e).
- **Screen(lhs,is)**: *Optional input*. Yukawa screening parameter λ in equation (2) above for charge confinement (in Bohr⁻¹).
- **delta(lhs,is)**: *Optional input*. Singularity regularisation parameter δ in equation (2) above for charge confinement (in Bohr).
- **rcls(izeta,l,is)**: Cutoff radius (Bohr) of each ‘zeta’ for this shell. For the second zeta onwards, if this value is negative, the actual rc used will be the given fraction of the first zeta’s rc. If the number of rc’s for a given shell is less than the number of ‘zetas’, the program will assign the last rc value to the remaining zetas, rather than stopping with an error. This is particularly useful for Bessel suites of orbitals.
- **contrf(izeta,l,is)**: *Optional input*. Contraction factor of each “zeta” for this shell. If the number of entries for a given shell is less than the number of ‘zetas’, the program will assign the last contraction value to the remaining zetas, rather than stopping with an error. *Default value*: 1.0

Polarization orbitals are generated by solving the atomic problem in the presence of a polarizing electric field. The orbitals are generated applying perturbation theory to the first-zeta orbital of lower angular momentum. They have the same cutoff radius as the orbitals from which they are constructed.

Note: The perturbative method has traditionally used the 'l' component of the pseudopotential, but it can be argued that it should use the 'l+1' component. The variable **PAO.OldStylePolOrbs** can be set to **true** in order to enable the former method, but this is discouraged unless testing for backwards compatibility.

There is a different possibility for generating polarization orbitals: by introducing them explicitly in the **PAO.Basis** block (see Sec. 6.3.6 for full details). It has to be remembered, however, that they sometimes correspond to unbound states of the atom, their shape depending very much on the cutoff radius, not converging by increasing it, similarly to the multiple-zeta orbitals generated with the **nodes** option. Using **PAO.EnergyShift** makes no sense, and a cutoff radius different from zero must be explicitly given (the same cutoff radius as the orbitals they polarize is usually a sensible choice).

A species with atomic number = -100 will be considered by SIESTA as a constant-pseudopotential atom, *i.e.*, the basis functions generated will be spherical Bessel functions with the specified r_c . In this case, r_c has to be given, as **PAO.EnergyShift** will not calculate it.

Other negative atomic numbers will be interpreted by SIESTA as *ghosts* of the corresponding positive value: the orbitals are generated and put in position as determined by the coordinates, but neither pseudopotential nor electrons are considered for that ghost atom. Useful for BSSE correction.

Use: This block is optional, except when Bessel functions are present.

Default: Basis characteristics defined by global definitions given above.

6.3.10 Filtering

FilterCutoff 0 eV (energy)

Kinetic energy cutoff of plane waves used to filter all the atomic basis functions, the pseudo-core densities for partial core corrections, and the neutral-atom potentials. The basis functions (which must be squared to obtain the valence density) are really filtered with a cutoff reduced by an empirical factor $0.7^2 \simeq 0.5$. The **FilterCutoff** should be similar or lower than the **Mesh.Cutoff** to avoid the *eggbox effect* on the atomic forces. However, one should not try to converge **Mesh.Cutoff** while simultaneously changing **FilterCutoff**, since the latter in fact changes the used basis functions. Rather, fix a sufficiently large **FilterCutoff** and converge only **Mesh.Cutoff**. If **FilterCutoff** is not explicitly set, its value is calculated from **FilterTol**.

FilterTol 0 eV (energy)

Residual kinetic-energy leaked by filtering each basis function. While **FilterCutoff** sets a common reciprocal-space cutoff for all the basis functions, **FilterTol** sets a specific cutoff for each basis function, much as the **PAO.EnergyShift** sets their real-space cutoff. Therefore, it is reasonable to use similar values for both parameters. The maximum cutoff required to meet the **FilterTol**, among all the basis functions, is used (multiplied by the empirical factor $1/0.7^2 \simeq 2$) to filter the pseudo-core densities and the neutral-atom potentials. **FilterTol** is ignored if **FilterCutoff** is present in the input file. If neither **FilterCutoff** nor **FilterTol** are present, no filtering is performed. See Soler and Anglada^[18], for details of the filtering

procedure.

Warning: If the value of **FilterCutoff** is made too small (or **FilterTol** too large) some of the filtered basis orbitals may be meaningless, leading to incorrect results or even a program crash. To be implemented: If **Mesh.Cutoff** is not present in the input file, it can be set using the maximum filtering cutoff used for the given **FilterTol** (for the time being, you can use **AtomSetupOnly true** to stop the program after basis generation, look at the maximum filtering cutoff used, and set the mesh-cutoff manually in a later run.)

6.3.11 Saving and reading basis-set information

SIESTA (and the standalone program GEN-BASIS) always generate the files *Atomlabel.ion*, where *Atomlabel* is the atomic label specified in block **ChemicalSpeciesLabel**. Optionally, if NetCDF support is compiled in, the programs generate NetCDF files *Atomlabel.ion.nc* (except for ghost atoms). See an Appendix for information on the optional NetCDF package.

These files can be used to read back information into SIESTA.

User.Basis `false` (logical)

If true, the basis, KB projector, and other information is read from files *Atomlabel.ion*, where *Atomlabel* is the atomic species label specified in block **ChemicalSpeciesLabel**. These files can be generated by a previous SIESTA run or (one by one) by the standalone program **Gen-basis**. No pseudopotential files are necessary.

User.Basis.NetCDF `false` (logical)

If true, the basis, KB projector, and other information is read from NetCDF files *Atomlabel.ion.nc*, where *Atomlabel* is the atomic label specified in block **ChemicalSpeciesLabel**. These files can be generated by a previous SIESTA run or by the standalone program **Gen-basis**. No pseudopotential files are necessary. NetCDF support is needed. Note that ghost atoms cannot yet be adequately treated with this option.

6.3.12 Tools to inspect the orbitals and KB projectors

The program **ioncat** in **Util/Gen-basis** can be used to extract orbital, KB projector, and other information contained in the *.ion* files. The output can be easily plotted with a graphics program. If the option **WriteIonPlotFiles** is enabled, SIESTA will generate an extra set of files that can be plotted with the **gnuplot** scripts in **Tutorials/Bases**. The stand-alone program **gen-basis** sets that option by default, and the script **Tutorials/Bases/gen-basis.sh** can be used to automate the process. See also the NetCDF-based utilities in **Util/PyAtom**.

6.3.13 Basis optimization

There are quite a number of options for the input of the basis-set and KB projector specification, and they are all optional! By default, SIESTA will use a DZP basis set with appropriate choices for the determination of the range, etc. Of course, the more you experiment with the different options, the better your basis set can get. To aid in this process we offer an auxiliary program for optimization which can be used in particular to obtain variationally optimal basis sets (within a chosen basis

size). See `Util/Optimizer` for general information, and `Util/Optimizer/Examples/Basis_Optim` for an example.

BasisPressure 0.2 GPa

(pressure)

depends on: **BasisPressure.Specs**

SIESTA will compute and print the value of the “effective basis enthalpy” constructed by adding a term of the form $p_{basis}V_{orbs}$ to the total energy. Here p_{basis} is a fictitious basis pressure and V_{orbs} is the volume of the system’s orbitals. This is a useful quantity for basis optimization (See Anglada *et al.*). The total basis enthalpy is also written to the ASCII file `SystemLabel.BASIS_ENTHALPY`.

If, instead, it’s interesting to add a different basis pressure for each atomic species, it’s possible to do so via the **BasisPressure.Specs** option.

%block BasisPressure.Specs `<None>`

(block)

depends on: **BasisPressure**

This will function similarly to **BasisPressure**, but using a different pressure for each atomic species. Within the block, there are three different ways to specify the pressure for each species:

```
Z from <A> to <B> <pressure> <units>
Z <A> <pressure> <units>
<Species> <pressure> <units>
```

The first two lines rely on atomic numbers. The first line will set the pressure for all species with atomic numbers from `<A>` to `<B>` (inclusive), while the second line will set the pressure for a single species with atomic number `<A>`. Note the "Z" indicator for both lines. The third line is similar to the second, but it only uses the species label, which should coincide with a species defined in the **ChemicalSpeciesLabel** block.

The lines within the block can be in any order, and they can be repeated as much as needed. If any species are not specified in this way, they will default to the value of **BasisPressure**.

The following example will apply a pressure of 0.1 GPa to species with atomic numbers 1 to 10, 0.01 GPa to atomic number 14, and 0.3 GPa to species Cgga.

```
%block BasisPressure.Specs
  Z from 1 to 10    0.1  GPa
  Z 14              0.01 GPa
  Cgga              0.3  GPa
%endblock BasisPressure.Specs
```

6.3.14 Low-level options regarding the radial grid

For historical reasons, the basis-set and KB projector code in SIESTA uses a logarithmic radial grid, which is taken from the pseudopotential file. Any “interesting” radii have to fall on a grid point, which introduces a certain degree of coarseness that can limit the accuracy of the results and the faithfulness of the mapping of input parameters to actual operating parameters. For example, the same orbital will be produced by a finite range of **PAO.EnergyShift** values, and any user-defined cutoffs will not be exactly reflected in the actual cutoffs. This is particularly troublesome for automatic optimization procedures (such as those implemented in `Util/Optimizer`), as the engine might be confused by the extra level of indirection. The following options can be used to fine-tune the mapping. Note that grid reparametrization is now (at version 5) enabled by default, and it

might change the numerical results appreciably (in effect, it leads to slightly different basis orbitals and projectors).

Reparametrize.Pseudos `true` *(logical)*

By changing the a and b parameters of the logarithmic grid, a new one with a more adequate grid-point separation can be used for the generation of basis sets and projectors. For example, by using $a = 0.001$ and $b = 0.01$, the grid point separations at $r = 0$ and 10 bohrs are 0.00001 and 0.01 bohrs, respectively. More points are needed to reach r 's of the order of a hundred bohrs, but the extra computational effort is negligible. The net effect of this option (notably when coupled to **Restricted.Radial.Grid**) is a closer mapping of any user-specified cutoff radii and of the radii implicitly resulting from other input parameters to the actual values used by the program. (The small grid-point separation near $r=0$ is still needed to avoid instabilities for s channels that occurred with the previous -reparametrized- default spacing of 0.005 bohr. This effect is not yet completely understood.)

New.A.Parameter `0.001` *(real)*

New setting for the pseudopotential grid's a parameter

New.B.Parameter `0.01` *(real)*

New setting for the pseudopotential grid's b parameter

Rmax.Radial.Grid `50.0` *(real)*

New setting for the maximum value of the radial coordinate for integration of the atomic Schrodinger equation.

If **Reparametrize.Pseudos** is **false** this will be the maximum radius in the pseudopotential file.

Restricted.Radial.Grid `true` *(logical)*

In normal operation of the basis-set and projector generation code the various cutoff radii are restricted to falling on an odd-numbered grid point, shifting then accordingly. This restriction can be lifted by setting this parameter to **false**.

6.3.15 Summary of options and defaults enabling automatic basis-set generation

In this section we review settings that can allow a fully automatic basis-set generation, with minimal user input. Philosophically this goes against the SIESTA mindset of careful basis-set optimization, but we must acknowledge the need for fully automatic, unattended, operation for high-throughput computing.

Partly in response to this, some program defaults have changed in version 5 of SIESTA.

- Multiple-zeta orbitals are generated with a new algorithm with better convergence properties.
- The generation of polarization orbitals can fall back to a non-perturbative scheme in some known problematic cases.
- Orbitals that would be unbound in the atom are now given a default r_c (equal to the range of the largest orbital). This value that can be controlled by the option

PAO.rc.unbound.state 0.0 Bohr (*length*)

A value of 0.0 will recover the old behavior (stopping if the orbital is unbound in the atom).

- The radial grid used in PAO and KB construction, which is inherited from that of the pseudopotential tables, is re-parametrized by default, to make it finer in the range where most relevant cutoffs are located. This is important for basis-optimization runs, since there is a more faithful mapping of the rc's requested by the optimization algorithm and those actually used.

In addition, some defaults have been changed to provide a better overall quality for the automatic basis sets. Note that full optimization is still preferred for most work:

- The default energy-shift has been lowered to 0.01 Ry.
- The soft-confinement potential is enabled by default.
- The default split-norm parameter for Hydrogen has been increased to 0.45.
- A change in the way perturbative polarization orbitals are generated: the $l+1$ pseudopotential channel is used, instead of the l channel.

6.3.16 Notes on backward compatibility in regard to new program defaults

If users want to reuse basis sets generated with previous versions of SIESTA, it is useful to keep in mind the old values of the defaults. In most cases, setting them back as in the following fdf stanza will recover the old behavior, *except* if any of the global options were given a different value in the original input. Recall also that options in the PAO.Basis block take precedence over any global options, but only if they are explicitly given.

```
PAO.EnergyShift 0.02 Ry
PAO.SoftDefault  F
PAO.SplitNormH -1.0
PAO.OldStylePolOrbs T
PAO.SplitValence.Legacy T
ReparametrizePseudos F
PAO.Polarization.NonPerturbative.Fallback F
PAO.rc.unbound.state 0.0 Bohr
```

If users keep the `.ion` or `.ion.nc` files from a run with a previous version of SIESTA, they can be reused with the **User.Basis** or **User.Basis.Netcdf** options. In this case there is no need to change any flags. This is the most convenient option to maintain compatibility with older results.

6.4 Structural information

There are many ways to give SIESTA structural information.

- Directly from the fdf file in traditional format.

- Directly from the fdf file in the newer Z-Matrix format, using a **Zmatrix** block.
- From an external data file

Note that, regardless of the way in which the structure is described, the **ChemicalSpeciesLabel** block is mandatory.

In the following sections we document the different structure input methods, and provide a guide to their precedence.

6.4.1 Traditional structure input in the fdf file

Firstly, the size of the cell itself should be specified, using some combination of the options **LatticeConstant**, **LatticeParameters**, and **LatticeVectors**, and **SuperCell**. If nothing is specified, SIESTA will construct a cubic cell in which the atoms will reside as a cluster (a molecule).

Secondly, the positions of the atoms within the cells must be specified, using either the traditional SIESTA input format (a modified xyz format) which must be described within a **AtomicCoordinatesAndAtomicSpecies** block.

LatticeConstant $\langle \text{None} \rangle$ *(length)*
depends on: **LatticeParameters**, **LatticeVectors**

Lattice constant. This is just to define the scale of the lattice vectors.

NOTE: This defaults to 1 Ang when used in combination with **LatticeParameters** or **LatticeVectors**. Otherwise it is not used.

%block LatticeParameters $\langle \text{None} \rangle$ *(block)*
depends on: **LatticeConstant**

Crystallographic way of specifying the lattice vectors, by giving six real numbers: the three vector modules, a , b , and c , and the three angles α (angle between $\vec{b}$ and $\vec{c}$), β , and γ . The three modules are in units of **LatticeConstant**, the three angles are in degrees.

For example a square cell with side-lengths equal to **LatticeConstant**.

```
1.0  1.0  1.0  90.  90.  90.
```

%block LatticeVectors $\langle \text{None} \rangle$ *(block)*
depends on: **LatticeConstant**

The cell vectors are read in units of the lattice constant, **LatticeConstant** which defaults to 1 Ang.

They are read as a matrix with each vector being one line.

For example a square cell with side-lengths equal to **LatticeConstant**.

```
1.0  0.0  0.0
0.0  1.0  0.0
0.0  0.0  1.0
```

%block SuperCell $\langle \text{None} \rangle$ *(block)*

Integer 3x3 matrix defining a supercell in terms of the unit cell. Any values larger than 1 will expand the unitcell (plus atoms) along that lattice vector direction (if possible).

```

%block SuperCell
  M(1,1) M(2,1) M(3,1)
  M(1,2) M(2,2) M(3,2)
  M(1,3) M(2,3) M(3,3)
%endblock SuperCell

```

and the supercell is defined as $\text{SuperCell}(ix, i) = \sum_j \text{CELL}(ix, j) * M(j, i)$. Notice that the matrix indexes are inverted: each input line specifies one supercell vector.

Warning: **SuperCell** is disregarded if the geometry is read from the XV file, which can happen inadvertently.

Use: The atomic positions must be given only for the unit cell, and they are 'cloned' automatically in the rest of the supercell. The **NumberOfAtoms** given must also be that in a single unit cell. However, all values in the output are given for the entire supercell. In fact, **CELL** is immediately redefined as the whole supercell and the program no longer knows the existence of an underlying unit cell. All other input (apart from **NumberOfAtoms** and atomic positions), including **kgrid.MonkhorstPack** must refer to the supercell (this is a change over previous versions). Therefore, to avoid confusions, we recommend to use **SuperCell** only to generate atomic positions, and then to copy them from the output to a new input file with all the atoms specified explicitly and with the supercell given as a normal unit cell.

AtomicCoordinatesFormat **Bohr** (string)

Character string to specify the format of the atomic positions in input. These can be expressed in four forms:

Bohr|NotScaledCartesianBohr atomic positions are given directly in Bohr, in Cartesian coordinates

Ang|NotScaledCartesianAng atomic positions are given directly in Ångström, in Cartesian coordinates

LatticeConstant|ScaledCartesian atomic positions are given in Cartesian coordinates, in units of the lattice constant

Fractional|ScaledByLatticeVectors atomic positions are given referred to the lattice vectors

AtomCoorFormatOut $\langle \text{AtomicCoordinatesFormat} \rangle$ (string)

Character string to specify the format of the atomic positions in output.

Same possibilities as for input **AtomicCoordinatesFormat**.

AtomicCoordinatesOrigin $\langle \text{None} \rangle$ (block/string)

The user can request a rigid shift of the coordinates, for example to place a molecule near the center of the cell. This shift can be specified in two ways:

- By an explicit vector, given in the same format and units as the coordinates. Notice that the atomic positions (shifted or not) need not be within the cell formed by **LatticeVectors**, since periodic boundary conditions are always assumed.

This defaults to the origin:

```
0.0  0.0  0.0
```

- By a string that indicates an automatic shift that places the “center” of the system at the center of the unit cell, or that places the system near the borders of the cell. In this case, the contents of the block, or the values associated directly to the label (see below) can be:

COP Place the center of coordinates in the middle of the unit-cell.

COM Place the center of mass in the middle of the unit-cell.

MIN Shift the coordinates so that the minimum value along each cartesian axis is 0.

NOTE: Ghost atoms are not taken into account for the above “centering” calculations (but their coordinates are indeed shifted).

All string options may be given an optional value. For instance, **COP-XZ** which limits the **COP** option to only affect *x* and *z* Cartesian coordinates.

The accepted suffixes are: **-X**, **-Y**, **-Z**, **-XY/-YX**, **-YZ/-YZ**, **-XZ/-ZX** and anything else will be regarded as all directions.

```
AtomicCoordinatesOrigin COP-X ! COP only for x-direction
AtomicCoordinatesOrigin COM-ZY ! COM only for y- and z-directions
AtomicCoordinatesOrigin MIN-Z ! MIN only for z-direction
AtomicCoordinatesOrigin MIN-XYZ ! MIN for all directions
AtomicCoordinatesOrigin MIN ! MIN for all directions
```

%block AtomicCoordinatesAndAtomicSpecies *<None>* *(block)*

Block specifying the position and species of each atom. One line per atom, the reading is done this way:

```
From ia = 1 to natoms
  read: xa(ix,ia), isa(ia)
```

where *xa(ix,ia)* is the *ix* coordinate of atom *ia* in the format (units) specified by **AtomicCoordinatesFormat**, and *isa(ia)* is the species index of atom *ia*.

NOTE: This block *must* be present in the fdf file. If **NumberOfAtoms** is not specified, **NumberOfAtoms** will be defaulted to the number of atoms in this block.

NOTE: **Zmatrix** has precedence if specified.

6.4.2 Z-matrix format and constraints

The advantage of the traditional format is that it is much easier to set up a system. However, when working on systems with constraints, there are only a limited number of (very simple) constraints that may be expressed within this format, and recompilation is needed for each new constraint.

For any more involved set of constraints, a full **Zmatrix** formulation should be used - this offers much more control, and may be specified fully at run time (thus not requiring recompilation) - but it is more work to generate the input files for this form.

%block Zmatrix *<None>* *(block)*

This block provides a means for inputting the system geometry using a Z-matrix format, as well as controlling the optimization variables. This is particularly useful when working with molecular systems or restricted optimizations (such as locating transition states or rigid unit movements). The format also allows for hybrid use of Z-matrices and Cartesian or fractional blocks, as is convenient for the study of a molecule on a surface. As is always the case for a Z-matrix, the responsibility falls to the user to chose a sensible relationship between the variables to avoid triads of atoms that become linear.

Below is an example of a Z-matrix input for a water molecule:


```

%block Zmatrix
molecule fractional
  1 0 0 0   0.0 0.0 0.0 0 0 0
  2 1 0 0   H01 90.0 37.743919 1 0 0
  2 1 2 0   H02 HOH 90.0 1 1 0
variables
  H01 0.956997
  H02 0.956997
  HOH 104.4
%endblock Zmatrix

```

The sections that can be used within the Zmatrix block are as follows:

Firstly, all atomic positions must be specified within either a “**molecule**” block or a “**cartesian**” block. Any atoms subject to constraints more complicated than “do not change this coordinate of this atom” must be specified within a “**molecule**” block.

molecule There must be one of these blocks for each independent set of constrained atoms within the simulation.

This specifies the atoms that make up each molecule and their geometry. In addition, an option of “**fractional**” or “**scaled**” may be passed, which indicates that distances are specified in scaled or fractional units. In the absence of such an option, the distance units are taken to be the value of “**ZM.UnitsLength**”.

A line is needed for each atom in the molecule; the format of each line should be:

```
Nspecies i j k r a t ifr ifa ift
```

Here the values **Nspecies**, **i**, **j**, **k**, **ifr**, **ifa**, and **ift** are integers and **r**, **a**, and **t** are double precision reals.

For most atoms, **Nspecies** is the species number of the atom, **r** is distance to atom number **i**, **a** is the angle made by the present atom with atoms **j** and **i**, while **t** is the torsional angle made by the present atom with atoms **k**, **j**, and **i**. The values **ifr**, **ifa** and **ift** are integer flags that indicate whether **r**, **a**, and **t**, respectively, should be varied; 0 for fixed, 1 for varying.

The first three atoms in a molecule are a special case. Because there are insufficient atoms defined to specify a distance/angle/torsion, the values are set differently. For atom 1, **r**, **a**, and **t**, are the Cartesian coordinates of the atom. For the second atom, **r**, **a**, and **t** are the coordinates in spherical form of the second atom relative to the first: first the radius, then the polar angle (angle between the *z*-axis and the displacement vector) and then the azimuthal angle (angle between the *x*-axis and the projection of the displacement vector on the *x-y* plane). Finally, for the third atom, the numbers take their normal form, but the torsional angle is defined relative to a notional atom 1 unit in the *z*-direction above the atom **j**.

Secondly. blocks of atoms all of which are subject to the simplest of constraints may be specified in one of the following three ways, according to the units used to specify their coordinates:

cartesian This section specifies a block of atoms whose coordinates are to be specified in Cartesian coordinates. Again, an option of “**fractional**” or “**scaled**” may be added, to specify the units used; and again, in their absence, the value of “**ZM.UnitsLength**” is taken.

The format of each atom in the block will look like:

```
Nspecies x y z ix iy iz
```


Here **Nspecies**, **ix**, **iy**, and **iz** are integers and **x**, **y**, **z** are reals. **Nspecies** is the species number of the atom being specified, while **x**, **y**, and **z** are the Cartesian coordinates of the atom in whichever units are being used. The values **ix**, **iy** and **iz** are integer flags that indicate whether the **x**, **y**, and **z** coordinates, respectively, should be varied or not. A value of 0 implies that the coordinate is fixed, while 1 implies that it should be varied. **NOTE:** When performing “variable cell” optimization while using a Zmatrix format for input, the algorithm will not work if some of the coordinates of an atom in a **cartesian** block are variables and others are not (i.e., **ix iy iz** above must all be 0 or 1). This will be fixed in future versions of the program.

A Zmatrix block may also contain the following, additional, sections, which are designed to make it easier to read.

constants Instead of specifying a numerical value, it is possible to specify a symbol within the above geometry definitions. This section allows the user to define the value of the symbol as a constant. The format is just a symbol followed by the value:

```
HOH 104.4
```

variables Instead of specifying a numerical value, it is possible to specify a symbol within the above geometry definitions. This section allows the user to define the value of the symbol as a variable. The format is just a symbol followed by the value:

```
H01 0.956997
```

Finally, constraints must be specified in a **constraints** block.

constraint This sub-section allows the user to create constraints between symbols used in a Z-matrix:

```
constraint
var1 var2 A B
```

Here **var1** and **var2** are text symbols for two quantities in the Z-matrix definition, and **AandB** are real numbers. The variables are related by $\text{var1} = A * \text{var2} + B$.

An example of a Z-matrix input for a benzene molecule over a metal surface is:

```
%block Zmatrix
molecule
 2 0 0 0 xm1 ym1 zm1 0 0 0
 2 1 0 0 CC 90.0 60.0 0 0 0
 2 2 1 0 CC CCC 90.0 0 0 0
 2 3 2 1 CC CCC 0.0 0 0 0
 2 4 3 2 CC CCC 0.0 0 0 0
 2 5 4 3 CC CCC 0.0 0 0 0
 1 1 2 3 CH CCH 180.0 0 0 0
 1 2 1 7 CH CCH 0.0 0 0 0
 1 3 2 8 CH CCH 0.0 0 0 0
 1 4 3 9 CH CCH 0.0 0 0 0
 1 5 4 10 CH CCH 0.0 0 0 0
 1 6 5 11 CH CCH 0.0 0 0 0
fractional
 3 0.000000 0.000000 0.000000 0 0 0
 3 0.333333 0.000000 0.000000 0 0 0
 3 0.666666 0.000000 0.000000 0 0 0
 3 0.000000 0.500000 0.000000 0 0 0
```

```

3 0.333333 0.500000 0.000000 0 0 0
3 0.666666 0.500000 0.000000 0 0 0
3 0.166667 0.250000 0.050000 0 0 0
3 0.500000 0.250000 0.050000 0 0 0
3 0.833333 0.250000 0.050000 0 0 0
3 0.166667 0.750000 0.050000 0 0 0
3 0.500000 0.750000 0.050000 0 0 0
3 0.833333 0.750000 0.050000 0 0 0
3 0.000000 0.000000 0.100000 0 0 0
3 0.333333 0.000000 0.100000 0 0 0
3 0.666666 0.000000 0.100000 0 0 0
3 0.000000 0.500000 0.100000 0 0 0
3 0.333333 0.500000 0.100000 0 0 0
3 0.666666 0.500000 0.100000 0 0 0
3 0.166667 0.250000 0.150000 0 0 0
3 0.500000 0.250000 0.150000 0 0 0
3 0.833333 0.250000 0.150000 0 0 0
3 0.166667 0.750000 0.150000 0 0 0
3 0.500000 0.750000 0.150000 0 0 0
3 0.833333 0.750000 0.150000 0 0 0
constants
  ym1 3.68
variables
  zm1 6.9032294
  CC 1.417
  CH 1.112
  CCH 120.0
  CCC 120.0
constraints
  xm1 CC -1.0 3.903229
%endblock Zmatrix

```

Here the species 1, 2 and 3 represent H, C, and the metal of the surface, respectively.

(Note: the above example shows the usefulness of symbolic names for the relevant coordinates, in particular for those which are allowed to vary. The current output options for Zmatrix information work best when this approach is taken. By using a “fixed” symbolic Zmatrix block and specifying the actual coordinates in a “variables” section, one can monitor the progress of the optimization and easily reconstruct the coordinates of intermediate steps in the original format.)

ZM.UnitsLength Bohr

(string)

Parameter that specifies the units of length used during Z-matrix input.

Specify **Bohr** or **Ang** for the corresponding unit of length.

ZM.UnitsAngle rad

(string)

Parameter that specifies the units of angles used during Z-matrix input.

Specify **rad** or **deg** for the corresponding unit of angle.

6.4.3 Output of structural information

SIESTA is able to generate several kinds of files containing structural information (maybe too many).

- **SystemLabel.STRUCT_OUT**: SIESTA always produces a **.STRUCT_OUT** file with cell vectors in Å and atomic positions in fractional coordinates. This file, renamed to **.STRUCT_IN** can be used for crystal-structure input. Note that the geometry reported is the last one for which forces and stresses were computed. See **UseStructFile**
- **SystemLabel.STRUCT_NEXT_ITER**: This file is always written, in the same format as **.STRUCT_OUT** file. The only difference is that it contains the structural information *after* it has been updated by the relaxation or the molecular-dynamics algorithms, and thus it could be used as input (renamed as **.STRUCT_IN**) for a continuation run, in the same way as the **.XV** file.

See **UseStructFile**

- **SystemLabel.XV**: The coordinates are always written in the **.XV** file, and overridden at every step.
- **OUT.UCELL.ZMATRIX**: This file is produced if the Zmatrix format is being used for input. (Please note that **SystemLabel** is not used as a prefix.) It contains the structural information in fdf form, with blocks for unit-cell vectors and for Zmatrix coordinates. The Zmatrix block is in a “canonical” form with the following characteristics:
 1. No symbolic variables or constants are used.
 2. The position coordinates of the first atom in each molecule are absolute Cartesian coordinates.
 3. Any coordinates in “cartesian” blocks are also absolute Cartesians.
 4. There is no provision for output of constraints.
 5. The units used are those initially specified by the user, and are noted also in fdf form.

Note that the geometry reported is the last one for which forces and stresses were computed.

- **NEXT_ITER.UCELL.ZMATRIX**: A file with the same format as **OUT.UCELL.ZMATRIX** but with a possibly updated geometry.
- The coordinates can be also accumulated in the **SystemLabel.MD** or **SystemLabel.MDX** files depending on **WriteMDHistory**.
- Additionally, several optional formats are supported:

WriteCoorXmol false (logical)

If **true** it originates the writing of an extra file named **SystemLabel.xyz** containing the final atomic coordinates in a format directly readable by XMOL.⁴ Coordinates come out in Ångström independently of what specified in **AtomicCoordinatesFormat** and in **AtomCoorFormatOut**. There is a present JAVA implementation of XMOL called JMOL.

⁴XMOL is under © copyright of Research Equipment Inc., dba Minnesota Supercomputer Center Inc.

WriteCoorCeri `false` *(logical)*

If **true** it originates the writing of an extra file named `SystemLabel.xtl` containing the final atomic coordinates in a format directly readable by CERIU⁵. Coordinates come out in **Fractional** format (the same as **ScaledByLatticeVectors**) independently of what specified in **AtomicCoordinatesFormat** and in **AtomCoorFormatOut**. If negative coordinates are to be avoided, it has to be done from the start by shifting all the coordinates rigidly to have them positive, by using **AtomicCoordinatesOrigin**. See the `Sies2arc` utility in the `Util/` directory for generating `.arc` files for CERIU animation.

WriteMDXmol `false` *(logical)*

If **true** it causes the writing of an extra file named `SystemLabel.ANI` containing all the atomic coordinates of the simulation in a format directly readable by XMOL for animation. Coordinates come out in Ångström independently of what is specified in **AtomicCoordinatesFormat** and in **AtomCoorFormatOut**. This file is accumulative even for different runs.

There is an alternative for animation by generating a `.arc` file for CERIU. It is through the `SIES2ARC` postprocessing utility in the `Util/` directory, and it requires the coordinates to be accumulated in the output file, i.e., **WriteCoorStep** **true**.

6.4.4 Input of structural information from external files

The structural information can be also read from external files. Note that **ChemicalSpeciesLabel** is mandatory in the `fdf` file.

MD.UseSaveXV `false` *(logical)*

Logical variable which instructs SIESTA to read the atomic positions and velocities stored in file `SystemLabel.XV` by a previous run.

If the file does not exist, a warning is printed but the program does not stop. Overrides **UseSaveData**, but can be implicitly set by it.

UseStructFile `false` *(logical)*

Controls whether the structural information is read from an external file of name `SystemLabel.STRUCT_IN`. If **true**, all other structural information in the `fdf` file will be ignored.

The format of the file is implied by the following code:

```
read(*,*) ((cell(ixyz,ivec),ixyz=1,3),ivec=1,3) ! Cell vectors, in Angstroms
read(*,*) na
do ia = 1,na
  read(iu,*) isa(ia), dummy, xfrac(1:3,ia) ! Species number
                                           ! Dummy numerical column
                                           ! Fractional coordinates
enddo
```

Warning: Note that the resulting geometry could be clobbered if an `.XV` file is read after this file. It is up to the user to remove any `.XV` files.

⁵CERIU is under © copyright of Molecular Simulations Inc.

MD.UseSaveZM false (logical)

Instructs to read the Zmatrix information stored in file `.ZM` by a previous run.

If the required file does not exist, a warning is printed but the program does not stop. Overrides **UseSaveData**, but can be implicitly set by it.

Warning: Note that the resulting geometry could be clobbered if an `.XV` file is read after this file. It is up to the user to remove any `.XV` files.

6.4.5 Input from a FIFO file

See the “Forces” option in **MD.TypeOfRun**. Note that **ChemicalSpeciesLabel** is still mandatory in the fdf file.

6.4.6 Precedence issues in structural input

- If the “Forces” option is active, it takes precedence over everything (it will overwrite all other input with the information it gets from the FIFO file).
- If **MD.UseSaveXV** is active, it takes precedence over the options below.
- If **MD.UseStructFile** (or **UseStructFile**) is active, it takes precedence over the options below.
- For atomic coordinates, the traditional and Zmatrix formats in the fdf file are mutually exclusive. If **MD.UseSaveZM** is active, the contents of the ZM file, if found, take precedence over the Zmatrix information in the fdf file.

6.4.7 Interatomic distances

WarningMinimumAtomicDistance 1 Bohr (length)

Fixes a threshold interatomic distance below which a warning message is printed.

MaxBondDistance 6 Bohr (length)

SIESTA prints the interatomic distances, up to a range of **MaxBondDistance**, to file `SystemLabel.BONDS` upon first reading the structural information, and to file `SystemLabel.BONDS_FINAL` after the last geometry iteration. The reference atoms are all the atoms in the unit cell. The routine now prints the real location of the neighbor atoms in space, and not, as in earlier versions, the location of the equivalent representative in the unit cell.

6.5 *k*-point sampling

These are options for the *k*-point grid used in the SCF cycle. For other specialized grids, see Secs. 6.21 and 6.18. The order of the following keywords is equivalent to their precedence.

kgrid.MonkhorstPack Γ -point (block/list)

Real-space supercell, whose reciprocal unit cell is that of the *k*-sampling grid, and grid displacement for each grid coordinate. Specified as an integer matrix and a real vector:

```

%block kgrid.MonkhorstPack
  Mk(1,1)  Mk(2,1)  Mk(3,1)  dk(1)
  Mk(1,2)  Mk(2,2)  Mk(3,2)  dk(2)
  Mk(1,3)  Mk(2,3)  Mk(3,3)  dk(3)
%endblock
kgrid.MonkhorstPack [Mk(1,1) Mk(2,2) Mk(3,3)]

```

where $Mk(j,i)$ are integers and $dk(i)$ are usually either 0.0 or 0.5 (the program will warn the user if the displacements chosen are not optimal). The k-grid supercell is defined from **Mk** as in block **SuperCell** above, i.e.: $KgridSuperCell(ix,i) = \sum_j CELL(ix,j) * Mk(j,i)$. Note again that the matrix indexes are inverted: each input line gives the decomposition of a supercell vector in terms of the unit cell vectors.

Use: Used only if **SolutionMethod** **diagon**. The k-grid supercell is compatible and unrelated (except for the default value, see below) with the **SuperCell** specifier. Both supercells are given in terms of the **CELL** specified by the **LatticeVectors** block. If **Mk** is the identity matrix and **dk** is zero, only the Γ point of the **unit** cell is used. Overrides **kgrid.Cutoff**.

One may also use the *list* input (last line in above example), in that case the block input must not be present and in this case the displacement vector cannot be selected.

kgrid.Cutoff 0. Bohr (length)

Parameter which determines the fineness of the k -grid used for Brillouin zone sampling. It is half the length of the smallest lattice vector of the supercell required to obtain the same sampling precision with a single k point. Ref: Moreno and Soler, PRB 45, 13891 (1992).

Use: If it is zero, only the gamma point is used. The resulting k-grid is chosen in an optimal way, according to the method of Moreno and Soler (using an effective supercell which is as spherical as possible, thus minimizing the number of k -points for a given precision). The grid is displaced for even numbers of effective mesh divisions. This parameter is not used if **kgrid.MonkhorstPack** is specified. If the unit cell changes during the calculation (for example, in a cell-optimization run, the k -point grid will change accordingly (see **ChangeKgridInMD** for the case of variable-cell molecular-dynamics runs, such as Parrinello-Rahman). This is analogous to the changes in the real-space grid, whose fineness is specified by an energy cutoff. If sudden changes in the number of k -points are not desired, then the Monkhorst-Pack data block should be used instead. In this case there will be an implicit change in the quality of the sampling as the cell changes. Both methods should be equivalent for a well-converged sampling.

kgrid.File none (string)

Specify a file from where the k -points are read in. The format of the file is identical to the **SystemLabel.KP** file with the exception that the k -points are given in units of the reciprocal lattice vectors. I.e. in the range of $]-1/2; 1/2]$.

An example input may be (not physically justified in any sense):

```

4
1 0.0 0.0 0.0 0.25
2 0.5 0.5 0.5 0.25
3 0.2 0.2 0.2 0.25
4 0.3 0.3 0.3 0.25

```

The first integer specifies the total number of k -points in the file. The first column is an index; the next 3 columns are the k -point specification for each of the reciprocal lattice vectors while the fifth column is the weight for the k -point.

SIESTA checks whether the sum of weights equals 1. If not, SIESTA will die.

ChangeKgridInMD **true** (logical)

If **true**, the k -point grid is recomputed at every iteration during MD runs that potentially change the unit cell: Parrinello-Rahman, Nose-Parrinello-Rahman, and Anneal. Regardless of the setting of this flag, the k -point grid is always updated at every iteration of a variable-cell optimization and after each step in a “siesta-as-server” run.

The only reason to set it to **false** would be to avoid sudden jumps in some properties when the sampling changes; but if the calculation is well-converged there should be no problems when the update is enabled.

TimeReversalSymmetryForKpoints **true** (logical)

depends on: **Spin**, **Spin.Spiral**

If **true**, the k -points in the BZ generated by the methods above are paired as $(k, -k)$ and only one member of the pair is retained. This symmetry is valid in the absence of external magnetic fields or non-collinear/spin-orbit interaction.

This flag is only honored for spinless or collinear-spin calculations, as the code will produce wrong results if there is no support for the appropriate symmetrization.

The default value is **true** unless: a) the option **Spin.Spiral** is used. In this case time-reversal-symmetry is broken explicitly. b) non-collinear/spin-orbit calculations. This case is less clear cut, but the time-reversal symmetry is not used to avoid possible breakings due to subtle implementation details, and to make the set of wavefunctions compatible with spin-orbit case in analysis tools.

6.5.1 Output of k -point information

The coordinates of the $\vec{k}$ points used in the sampling are always stored in the file **SystemLabel.KP**.

WriteKpoints **false** (logical)

If **true** it writes the coordinates of the $\vec{k}$ vectors used in the grid for k -sampling, into the main output file.

Default depends on **LongOutput**.

6.6 Exchange-correlation functionals

(Apart from the built-in functionals, SIESTA can use the functionals provided by the LibXC library, if support for it is compiled-in in the libGridXC library. See the description of the **XC.mix** block below for the appropriate syntax.)

XC.Functional **LDA** (string)

Exchange-correlation functional type. May be **LDA** (local density approximation, equivalent to **LSD**), **GGA** (Generalized Gradient Approximation), or **VDW** (van der Waals).

XC.Authors **PZ** (string)

Particular parametrization of the exchange-correlation functional. Options are:

- **CA** (equivalent to **PZ**): (Spin) local density approximation (LDA/LSD). Quantum Monte

- Carlo calculation of the homogeneous electron gas by D. M. Ceperley and B. J. Alder, Phys. Rev. Lett. **45**, 566 (1980), as parametrized by J. P. Perdew and A. Zunger, Phys. Rev B **23**, 5075 (1981)
- **PW92**: LDA/LSD, as parametrized by J. P. Perdew and Y. Wang, Phys. Rev B, **45**, 13244 (1992)
 - **PW91**: Generalized gradients approximation (GGA) of Perdew and Wang. Ref: P&W, J. Chem. Phys., **100**, 1290 (1994)
 - **PBE**: GGA of J. P. Perdew, K. Burke and M. Ernzerhof, Phys. Rev. Lett. **77**, 3865 (1996)
 - **revPBE**: Modified GGA-PBE functional of Y. Zhang and W. Yang, Phys. Rev. Lett. **80**, 890 (1998)
 - **RPBE**: Modified GGA-PBE functional of B. Hammer, L. B. Hansen and J. K. Norskov Phys. Rev. B **59**, 7413 (1999)
 - **WC**: Modified GGA-PBE functional of Z. Wu and R. E. Cohen, Phys. Rev. B **73**, 235116 (2006)
 - **AM05**: Modified GGA-PBE functional of R. Armiento and A. E. Mattsson, Phys. Rev. B **72**, 085108 (2005)
 - **PBEsol**: Modified GGA-PBE functional of J. P. Perdew et al, Phys. Rev. Lett. **100**, 136406 (2008)
 - **PBEJsJrLO**: GGA-PBE functional with parameters β , μ , and κ fixed by the jellium surface (Js), jellium response (Jr), and Lieb-Oxford bound (LO) criteria, respectively, as described by L. S. Pedroza, A. J. R. da Silva, and K. Capelle, Phys. Rev. B **79**, 201106(R) (2009), and by M. M. Odashima, K. Capelle, and S. B. Trickey, J. Chem. Theory Comput. **5**, 798 (2009)
 - **PBEJsJrHEG**: Same as PBEJsJrLO, with parameter κ fixed by the Lieb-Oxford bound for the low density limit of the homogeneous electron gas (HEG)
 - **PBEGcGxLO**: Same as PBEJsJrLO, with parameters β and μ fixed by the gradient expansion of correlation (Gc) and exchange (Gx), respectively
 - **PBEGcGxHEG**: Same as previous ones, with parameters β , μ , and κ fixed by the Gc, Gx, and HEG criteria, respectively.
 - **BLYP** (equivalent to **LYP**): GGA with Becke exchange (A. D. Becke, Phys. Rev. A **38**, 3098 (1988)) and Lee-Yang-Parr correlation (C. Lee, W. Yang, R. G. Parr, Phys. Rev. B **37**, 785 (1988)), as modified by B. Miehlich, A. Savin, H. Stoll, and H. Preuss, Chem. Phys. Lett. **157**, 200 (1989). See also B. G. Johnson, P. M. W. Gill and J. A. Pople, J. Chem. Phys. **98**, 5612 (1993). (Some errors were detected in this last paper, so not all of their expressions correspond exactly to those implemented in SIESTA)
 - **DRSLL** (equivalent to **DF1**): van der Waals density functional (vdW-DF) of M. Dion, H. Rydberg, E. Schröder, D. C. Langreth, and B. I. Lundqvist, Phys. Rev. Lett. **92**, 246401 (2004), with the efficient implementation of G. Román-Pérez and J. M. Soler, Phys. Rev. Lett. **103**, 096102 (2009)
 - **LMKLL** (equivalent to **DF2**): vdW-DF functional of Dion *et al* (same as DRSLL) reparametrized by K. Lee, E. Murray, L. Kong, B. I. Lundqvist and D. C. Langreth, Phys. Rev. B **82**, 081101 (2010)

- **KBM**: vdW-DF functional of Dion *et al* (same as DRSLL) with exchange modified by J. Klimes, D. R. Bowler, and A. Michaelides, J. Phys.: Condens. Matter **22**, 022201 (2010) (optB88-vdW version)
- **C09**: vdW-DF functional of Dion *et al* (same as DRSLL) with exchange modified by V. R. Cooper, Phys. Rev. B **81**, 161104 (2010)
- **BH**: vdW-DF functional of Dion *et al* (same as DRSLL) with exchange modified by K. Berland and P. Hyldgaard, Phys. Rev. B **89**, 035412 (2014)
- **VV**: vdW-DF functional of O. A. Vydrov and T. Van Voorhis, J. Chem. Phys. **133**, 244103 (2010)

`%block XC.Mix` `<None>` *(block)*

This data block allows the user to create a “cocktail” functional by mixing the desired amounts of exchange and correlation from each of the functionals described under XC.authors.

The first line of the block must contain the number of functionals to be mixed. On the subsequent lines the values of XC.functl and XC.authors must be given and then the weights for the exchange and correlation, in that order. If only one number is given then the same weight is applied to both exchange and correlation.

The following is an example in which a 75:25 mixture of Ceperley-Alder and PBE correlation is made, with an equal split of the exchange energy:

```
%block XC.mix
2
LDA CA 0.5 0.75
GGA PBE 0.5 0.25
%endblock XC.mix
```

These blocks can also be used to request the use of LibXC functionals (if the version of libGridXC in use is 0.7 or later and was compiled with LibXC support). For example:

```
%block XC.mix
2
GGA LIBXC-00-GGA_X_PBE 1.0 0.0
GGA LIBXC-00-GGA_C_PBE 0.0 1.0
%endblock XC.mix
```

The weights reflect the “exchange” or “correlation” character of each individual functional. In the above example we use mnemonic symbols for the functionals and leave the numerical functional id field as zero. It is also possible to use only the numerical id:

```
%block XC.mix
2
GGA LIBXC-101 1.0 0.0
GGA LIBXC-130 0.0 1.0
%endblock XC.mix
```

If both fields are used the information must be compatible. Also, the “family” field (GGA, LDA) must be compatible with the functional specified.

NOTE: In previous versions of the program this block was named, confusingly, **XC.Hybrid**, and in some other versions, **XC.Cocktail**. Those names are still allowed, but are deprecated.

Default value: If the block is not present, the XC information is read from the fdf variables above.

XC.Use.BSC.CellXC `false`

(*logical*)

If **true**, the version of `cellXC` from the BSC's mesh suite is used instead of the default `SiestaXC` version. BSC's version might be slightly better for GGA operations. `SiestaXC`'s version is mandatory when dealing with van der Waals functionals.

6.7 Spin polarization

Spin `non-polarized`

(*string*)

deprecates: **SpinPolarized**, **NonCollinearSpin**, **SpinOrbit**

Choose the spin-components in the simulation.

NOTE: This flag has precedence over **SpinOrbit**, **NonCollinearSpin** and **SpinPolarized** while these deprecated flags may still be used.

non-polarized Perform a calculation with spin-degeneracy (only one component).

polarized Perform a calculation with colinear spin (two spin components).

non-collinear Perform a calculation with non-collinear spin (4 spin components), up-down and angles.

Refs: T. Oda et al, PRL, **80**, 3622 (1998); V. M. García-Suárez et al, Eur. Phys. Jour. B **40**, 371 (2004); V. M. García-Suárez et al, Journal of Phys: Cond. Matt **16**, 5453 (2004).

spin-orbit Performs calculations including the spin-orbit coupling. By default the full SO option is set. To perform an on-site SO calculation (see 6.8.1) this option has to be **spin-orbit+onsite**. This requires the pseudopotentials to be relativistic.

See Sect. 6.8 for further specific spin-orbit options.

SIESTA can read a `.DM` with different spin structure by adapting the information to the currently selected spin multiplicity, averaging or splitting the spin components equally, as needed. This may be used to greatly increase convergence.

Certain options may not be used together with specific parallelization routines.

Spin.Fix `false`

(*logical*)

If **true**, the calculation is done with a fixed value of the spin of the system, defined by variable **Spin.Total**. This option can only be used for colinear spin polarized calculations.

Spin.Total `0`

(*real*)

Value of the imposed total spin polarization of the system (in units of the electron spin, 1/2). It is only used if **Spin.Fix true**.

%block Spin.Spiral `<None>`

(*block*)

depends on: **Spin**

Specify the spiral q vector for the non-collinear spin.

```
Spin.Spiral.Scale ReciprocalLatticeVectors
%block Spin.Spiral
  0. 0. 0.5
%endblock
```

NOTE: this option only applies for non-collinear spin (not for spin-orbit).

NOTE: this part of the code has not been tested, we would welcome any person who could assert its correctness and provide tests. Use with *extreme* care.

Spin.Spiral.Scale `<None>` (string)
depends on: **Spin.Spiral**

Specifies the scale of the spiral vector q vectors given in **Spin.Spiral**. The options are:

pi/a vector is given in Cartesian coordinates, in units of π/a , where a is the lattice constant
(LatticeConstant)

ReciprocalLatticeVectors vector is given in reciprocal-lattice-vector coordinates

SingleExcitation `false` (logical)

If **true**, SIESTA calculates a very rough approximation to the lowest excited state by swapping the populations of the HOMO and the LUMO. If there is no spin polarisation, it is half swap only. It is done for the first spin component (up) and first k vector.

6.8 Spin-Orbit coupling

SIESTA includes the option of including in the total Hamiltonian not only the Darwin and velocity correction terms (Scalar-Relativistic calculations), but also the spin-orbit (SO) contribution. See **Spin** on how to enable spin-orbit coupling.

The SO functionality has been implemented by Dr. Ramón Cuadrado and Dr. Jorge I. Cerdá based on their initial work (R. Cuadrado and J. I. Cerdá “Fully relativistic pseudopotential formalism under an atomic orbital basis: spin-orbit splittings and magnetic anisotropies”, J. Phys.: Condens. Matter **24**, 086005 (2012); “In-plane/out-of-plane disorder influence on the magnetic anisotropy of Fe_{1-y}Mn_yPt-L1(0) bulk alloy”, R. Cuadrado, Kai Liu, Timothy J. Klemmer and R. W. Chantrell, Applied Physics Letters, **108**, 123102 (2016)).

The inclusion of the SO term in the Hamiltonian (and in the Density Matrix) causes an increase in the number of non-zero elements in their off-diagonal parts, i.e., for some (μ, ν) pair of basis orbitals, $\mathbf{H}_{\mu\nu}^{\sigma\sigma'} (\mathbf{DM}_{\mu\nu}^{\sigma\sigma'})$ $[\sigma, \sigma' = \uparrow, \downarrow]$ will be $\neq 0$. This is mainly due to the fact that the $\mathbf{L} \cdot \mathbf{S}$ operator will promote the mixing between different spin-up/down components. In addition, these $\mathbf{H}_{\mu\nu}^{\sigma\sigma'}$ (and $\mathbf{DM}_{\mu\nu}^{\sigma\sigma'}$) elements will be complex, in contrast with typical polarized/non-polarized calculations where these matrices are purely real. Since the spin-up and spin-down manifolds are essentially mixed, the solver has to deal with matrices whose dimensions are twice as large as for the collinear (unmixed) spin problem. Due to this, we advise to take special attention to the memory needed to perform a spin-orbit calculation.

Apart from the study of effects of the spin-orbit interaction in the band structure, a feature enabled by a SO formalism is the computation of the Magnetic Anisotropy Energy (MAE): it can be obtained as the difference in the total selfconsistent energy in two different spin orientations, usually along the easy axis and the hard axis. In SIESTA it is possible to perform calculations for different magnetization orientations using the block **DM.InitSpin** in the fdf file. In doing so one will be able to include the initial orientation angles of the magnetization for each atom, as well as an initial value of their net magnetic moments. See also the recent review^[8].

Note: Due to the small contribution of the spin-orbit interaction to the total energy, the level of precision required is quite high. The following parameters should be carefully checked for each

specific system to assure that the results are converged and accurate enough: **SCF.H.Tolerance** during the selfconsistency (typically $<10^{-4}\text{eV}$), **ElectronicTemperature**, k-point sampling, and **Mesh.Cutoff** (specifically for extended solids). In general, one can say that a good calculation will have a high number of k-points, low **ElectronicTemperature**, very small **SCF.H.Tolerance** and high values of **Mesh.Cutoff**. We encourage the user to test carefully these options for each system.

An additional point to take into account when the spin-orbit contribution is included is the mixing scheme to use. You are encouraged to use the option to mix the Hamiltonian (**SCF.Mix hamiltonian**) instead of the density matrix to speed up convergence. In addition, the pseudopotentials have to be well tested for each specific system. They have to be generated in their fully relativistic form, and should use non-linear core corrections. Finally it is worth to mention that the selfconsistent convergence for some non-highly symmetric magnetizations directions with respect to the physical symmetry axis could still be difficult.

Spin.OrbitStrength 1.0 *(real)*

It allows to vary the strength of the spin-orbit interaction from zero to any positive value. It should be used only for debugging and testing purposes, as the only physical value is 1.0. Note that this feature is currently implemented by modifying the SO parts of the semilocal potentials read from a **.psf** file. It will not work when reading the *lj* projectors directly from a PSML file (or from a previous run's **.ion** file). Care must be taken when re-using any **.ion** files produced.

WriteOrbMom false *(logical)*

If **true**, a table is provided in the output file that includes an estimation of the vector orbital magnetic moments, in units of the Bohr magneton, projected onto each orbital and also onto each atom. The estimation for the orbital moments is based on a two-center approximation, and makes use of the Mulliken population analysis.

If **MullikenInScf** is **true**, this information is printed at every scf step.

SOC.Split.SR.SO true *(logical)*

In calculations with spin-orbit-coupling (SOC) the program carries out a splitting of the contributions to the Hamiltonian and energies into scalar-relativistic (SR) and spin-orbit (SO) parts. The splitting procedure when *lj* projectors are involved can sometimes be ill-defined, and in those cases the program relies on a heuristic to compute the two contributions. A warning is printed.

If this option is set to **false**, it will prevent the program from attempting the splitting (but it still will be able to detect a possible problem and report an informational message).

When the SO contribution is not split, the relevant energy contributions in the output file are tagged **Enl(+so)** and **Eso(nil)**.

The CML file is not thus changed (but there is a new parameter **Split-SR-SO**).

Note that this is only a cosmetic change affecting the reporting of some components of the energy. All the other results should be unchanged.

6.8.1 On-site approximation

Within the so-called “on-site” approximation only the intra-atomic SO contribution is taken into account, neglecting three-center SO matrix elements.

The on-site spin-orbit scheme in this version of SIESTA has been implemented by Dr. Ramón

Cuadrado based on the original formalism and implementation developed by Prof. Jaime Ferrer and his collaborators (L Fernández-Seivane, M Oliveira, S Sanvito, and J Ferrer, *Journal of Physics: Condensed Matter*, **18**, 7999 (2006); L Fernández-Seivane and Jaime Ferrer, *Phys. Rev. Lett.* **99**, 183401 (2007)). 183401).

It should be noted that this approximation, while based on the physically reasonable idea of the short-range of the SO interaction, is susceptible to some inaccuracies^[6]. Since the construction of the full SOC Hamiltonian represents a small fraction of the computational effort, the performance gains in using the on-site approximation are negligible and do not justify its use. Hence, the full SO formalism is used by default, being necessary to change the **Spin** flag in the input file if the on-site approximation is desired.

6.9 The self-consistent-field loop

IMPORTANT NOTE: Convergence of the Kohn-Sham energy and forces

In versions prior to 4.0 of the program, the Kohn-Sham energy was computed using the “in” DM. The typical DM used as input for the calculation of H was not directly computed from a set of wave-functions (it was either the product of mixing or of the initialization from atomic values). In this case, the “kinetic energy” term in the total energy computed in the way stated in the SIESTA paper had an error which decreased with the approach to self-consistency, but was non-zero. The net result was that the Kohn-Sham energy converged more slowly than the “Harris” energy (which is correctly computed).

When mixing H (see below under “Mixing Options”), the KS energy is in effect computed from DM(out), so this error vanishes.

As a related issue, the forces and stress computed after SCF convergence were calculated using the DM coming out of the cycle, which by default was the product of a final mixing. This also introduced errors which grew with the degree of non-selfconsistency.

The current version introduces several changes:

- When mixing the DM, the Kohn-Sham energy may be corrected to make it variational. This involves an extra call to **dhscf** (although with neither forces nor matrix elements being calculated, i.e. only calls to **rhoofd**, **poison**, and **cellxc**), and is turned on by the option **SCF.Want.Variational.EKS**.
- The program now prints a new column labeled “dHmax” for the self-consistent cycle. The value represents the maximum absolute value of the changes in the entries of H, but its actual meaning depends on whether DM or H mixing is in effect: if mixing the DM, dHmax refers to the change in H(in) with respect to the previous step; if mixing H, dHmax refers to H(out)-H(in) in the current step.
- When achieving convergence, the loop might be exited without a further mixing of the DM, thus preserving DM(out) for further processing (including the calculation of forces and the analysis of the electronic structure) (see the **SCF.Mix.AfterConvergence** option).
- It remains to be seen whether the forces, being computed “right” on the basis of DM(out), exhibit somehow better convergence as a function of the scf step. In order to gain some more data and heuristics on this we have implemented a force-monitoring option, activated by

setting to **true** the variable **SCF.MonitorForces**. The program will then print the maximum absolute value of the change in forces from one step to the next. Other statistics could be implemented.

- While the (mixed) DM is saved at every SCF step, as was standard practice, the final DM(out) overwrites the **SystemLabel.DM** file at the end of the SCF cycle. Thus it is still possible to use a “mixed” DM for restarting an interrupted loop, but a “good” DM will be used for any other post-processing.

MinSCFIterations 0 *(integer)*

Minimum number of SCF iterations per time step. In MD simulations this can with benefit be set to 3.

MaxSCFIterations 1000 *(integer)*

Maximum number of SCF iterations per time step.

SCF.MustConverge true *(logical)*

Defines the behaviour if convergence is not reached in the maximum number of SCF iterations. The default is to stop on the first SCF convergence failure. Increasing **MaxSCFIterations** to a large number may be advantageous when this is **true**.

6.9.1 Harris functional

Harris.Functional false *(logical)*

Logical variable to choose between self-consistent Kohn-Sham functional or non self-consistent Harris functional to calculate energies and forces.

- **false**: Fully self-consistent Kohn-Sham functional.
- **true**: Non self consistent Harris functional. Cheap but pretty crude for some systems. The forces are computed within the Harris functional in the first SCF step. Only implemented for LDA in the Perdew-Zunger parametrization. It really only applies to starting densities which are superpositions of atomic charge densities.

When this option is chosen, the values of **DM.UseSaveDM**, **SCF.MustConverge** and **SCF.Mix.First** are automatically set **false** and **MaxSCFIterations** is set to 1, no matter whatever other specification are in the INPUT file.

6.9.2 Mixing options

Whether a calculation reaches self-consistency in a moderate number of steps depends strongly on the mixing parameters used. The available mixing options should be carefully tested for a given calculation type. This search for optimal parameters can repay itself handsomely by potentially saving many self-consistency steps in production runs.

SCF.Mix Hamiltonian|density|charge *(string)*

Control what physical quantity to mix in the self-consistent cycle.

The default is mixing the Hamiltonian, which may typically perform better than density matrix mixing.

Hamiltonian Mix the Hamiltonian matrix (default).

density Mix the density matrix.

charge Mix the real-space charge density. Note this is an experimental feature.

NOTE: Real-space charge density does not follow the regular options that adhere to density-matrix or Hamiltonian mixing. Also it is not recommended to use real-space charge density mixing with TRANSIESTA.

SCF.Mix.Spin all|spinor|sum|sum+diff (string)

Controls how the mixing is performed when carrying out spin-polarized calculations.

all Use all spin-components in the mixing

spinor Estimate mixing coefficients using the spinor components

sum Estimate mixing coefficients using the sum of the spinor components

sum+diff Estimate mixing coefficients using the sum *and* the difference between the spinor components

NOTE: This option only influences density-matrix (ρ) or Hamiltonian (H) mixing when using anything but the **linear** mixing scheme. And it does not influence not charge (ρ) mixing.

SCF.Mix.First true (logical)

deprecates: **DM.MixSCF1**

depends on: **SCF.Mix.First.Force**

This flag is used to decide whether mixing (of the DM or H) should be done in the first SCF step. If mixing is not performed the output DM or H generated in the first SCF step is used as input in the next SCF step. When mixing the DM, this “reset” has the effect of avoiding potentially undesirable memory effects: for example, a DM read from file which corresponds to a different structure might not satisfy the correct symmetry, and mixing will not fix it. On the other hand, when reusing a DM for a restart of an interrupted calculation, a full reset might not be advised.

The value of this flag is one of the ingredients used by SIESTA to decide what to do. If **true** (the default), mixing will be performed in all cases, except when a DM has been read from file and the sparsity pattern of the DM on file is different from the current one. To ensure that a first-step mixing is done even in this case, **SCF.Mix.First.Force** should be set to **true**.

If the flag is **false**, no mixing in the first step will be performed, except if overridden by **SCF.Mix.First.Force**.

NOTE: that the default value for this flag has changed from the old (pre-version 4) setting in SIESTA. The new setting is most appropriate for the case of restarting calculations. On the other hand, it means that mixing in the first SCF step will also be performed for the standard case in which the initial DM is built as a (diagonal) superposition of atomic orbital occupation values. In some cases (e.g. spin-orbit calculations) better results might be obtained by avoiding this mixing.

SCF.Mix.First.Force false (logical)

Force the mixing (of DM or H) in the first SCF step, regardless of what SIESTA may heuristically decide.

This overrules **SCF.Mix.First**.

In the following the density matrix (ρ) will be used in the equations, while for Hamiltonian mixing, ρ , should be replaced by the Hamiltonian matrix. Also we define $R[i] = \rho_{\text{out}}^i - \rho_{\text{in}}^i$ and $\Delta R[i] = R[i] - R[i-1]$.

SCF.Mixer.Method Pulay|Broyden|Linear (string)

Choose the mixing algorithm between different methods. Each method may have different variants, see **SCF.Mixer.Variant**.

Linear A simple linear extrapolation of the input matrix as

$$\rho_{\text{in}}^{n+1} = \rho_{\text{in}}^n + w R[n]. \quad (3)$$

Pulay Using the Pulay mixing method corresponds using the Kresse and Furthmüller^[9] variant. It relies on the previous N steps and uses those for estimating an optimal input ρ_{in}^{n+1} for the following iteration. The equation can be written as

$$\rho_{\text{in}}^{n+1} = \rho_{\text{in}}^n + G R[n] + \sum_{i=n-N+1}^{N-1} \alpha_i (R[i] + G \Delta R[i]), \quad (4)$$

where G is the damping factor of the Pulay mixing (also known as the mixing weight). The values α_i are calculated using this formula

$$\alpha_i = - \sum_{j=1}^{N-1} \mathbf{A}_{ji}^{-1} \langle \Delta R[j] | R[N] \rangle, \quad (5)$$

with $\mathbf{A}_{ji} = \langle \Delta R[j] | \Delta R[i] \rangle$.

In SIESTA G is a constant, and not a matrix.

NOTE: Pulay mixing is a special case of Broyden mixing, see the Broyden method.

Broyden The Broyden mixing is mixing method relying on the previous N steps in the history for calculating an optimum input ρ_{in}^{n+1} for the following iteration. The equation can be written as

$$\rho_{\text{in}}^{n+1} = \rho_{\text{in}}^n + G R[n] - \sum_{i=n-N+1}^{N-1} \sum_{j=n-N+1}^{N-1} w_i w_j c_j \beta_{ij} (R[i] + G \Delta R[i]), \quad (6)$$

where G is the damping factor (also known as the mixing weight). The values weights may be expressed by

$$w_i = 1 \quad , \text{ for } i > 0 \quad (7)$$

$$c_i = \langle \Delta R[i] | R[n] \rangle, \quad (8)$$

$$\beta_{ij} = \left[(w_0^2 \mathbf{I} + \mathbf{A})^{-1} \right]_{ij} \quad (9)$$

$$A_{ij} = w_i w_j \langle \Delta R[i] | \Delta R[j] \rangle. \quad (10)$$

It should be noted that w_i for $i > 0$ may be chosen arbitrarily. Comparing with the Pulay mixing scheme it is obvious that Broyden and Pulay are equivalent for a suitable set of parameters.

SCF.Mixer.Variant `original` (string)

Choose the variant of the mixing method.

Pulay This is implemented in two variants:

original|kresse The original⁶ Pulay mixing scheme, as implemented in Kresse and Furthmüller^[9].

GR The “guaranteed-reduction” variant of Pulay^[4]. This variant has a special convergence path. It interchanges between linear and Pulay mixing thus using the exact gradient at each ρ_{in}^n . For relatively simple systems this may be advantageous to use. However, for complex systems it may be worse until it reaches a convergence basin.

To obtain the original guaranteed-reduction variant one should set **SCF.Mixer.<>.weight.linear** to 1.

SCF.Mixer.Weight `0.25` (real)
deprecates: DM.MixingWeight

The mixing weight used to mix the quantity. In the linear mixing case this refers to

$$\rho_{\text{in}}^{n+1} = \rho_{\text{in}}^n + w R[n]. \quad (11)$$

For details regarding the other methods please see **SCF.Mixer.Method**.

Convergence of a system heavily depends on:

SCF.Mixer.Weight A high value retains much of the output solution, which may result in leaving the convergence basin. However, when close to the solution a high value might decrease needed SCF steps.

A low value only uses very little of the output solution. This may result in high number of SCF steps but is more likely to converge since it becomes harder for the solution to leave the convergence basin.

This value is heavily system dependent.

Since the convergence depends on the input vs. the output, this value can practically be arbitrarily low. For very low numbers (e.g. $< 10^{-6}$) the convergence will just take a large number of SCF iterations because so little is added on every step.

SCF.Mixer.Method The linear mixing is the only method that does not make use of prior steps, for hard to converge systems it should only be tried with very low mixing weights.

The choice of method may result in some reduction of SCF steps, but experimentation with the mixing weight is preferred as a first resort.

SCF.Mixer.History Number of previous steps to use for the mixing. A too low value (say 2 – 6) might change the convergence properties a lot. While two different high values might not change the convergence properties significantly, if at all.

NOTE: the older keyword **DM.MixingWeight** is used if this key is not found in the input.

SCF.Mixer.History `2` (integer)
deprecates: DM.NumberPulay, DM.NumberBroyden

⁶As such the “original” version is a variant it-self. But this is more stable in the far majority of cases.

Number of previous SCF steps used in estimating the following input. Increasing this number, typically, increases stability and a number of around 6 or above may be advised.

NOTE: the older keyword **DM.NumberPulay/DM.NumberBroyden** is used if this key is not found in the input.

SCF.Mixer.Kick 0 (integer)

After every N SCF steps a linear mix is inserted to *kick* the SCF cycle out of a possible local minimum.

The mixing weight for this linear kick is determined by **SCF.Mixer.Kick.Weight**.

SCF.Mixer.Kick.Weight $\langle$ SCF.Mixer.Weight $\rangle$ (real)

The mixing weight for the linear kick (if used).

SCF.Mixer.Restart 0 (integer)

When using advanced mixers (Pulay/Broyden) the mixing scheme may periodically restart the history. This may greatly improve the convergence path as local constraints in the minimization process are periodically removed. This method has similarity to the method proposed in Banerjee et al.^[2] and is a special case of the **SCF.Mixer.Kick** method.

Please see **SCF.Mixer.Restart.Save** which is advised to be set simultaneously.

SCF.Mixer.Restart.Save 1 (integer)

When restarting the history of saved SCF steps one may choose to save a subset of the latest history steps. When using **SCF.Mixer.Restart** it is encouraged to also save a couple of previous history steps.

SCF.Mixer.Linear.After -1 (integer)

After reaching convergence one may run additional SCF cycles using a linear mixing scheme. If this has a value ≥ 0 SIESTA will perform linear mixing after it has converged using the regular mixing method (**SCF.Mixer.Method**).

The mixing weight for this linear mixing is controlled by **SCF.Mixer.Linear.After.Weight**.

SCF.Mixer.Linear.After.Weight $\langle$ SCF.Mixer.Weight $\rangle$ (real)

After reaching convergence one may run additional SCF cycles using a linear mixing scheme. If this has a value ≥ 0 SIESTA will perform linear mixing after it has converged using the regular mixing method (**SCF.Mixer.Method**).

The mixing weight for this linear mixing is controlled by **SCF.Mixer.Linear.After.Weight**.

In conjunction with the above simple settings controlling the SCF cycle SIESTA employs a very configurable mixing scheme. In essence one may switch mixing methods, arbitrarily, during the SCF cycle via control commands. This can greatly speed up convergence.

%block SCF.Mixers $\langle$ None $\rangle$ (block)

Each line in this block defines a separate mixer that is defined in a subsequent **SCF.Mixer.<>** block.

The first line is the initial mixer used.

See the following options for controlling individual mixing methods.

NOTE: If this block is defined you *must* define all mixing parameters individually.

%block SCF.Mixer.<> **<None>** (block)

This block controls the mixer named <>.

method Define the method for the mixer, see **SCF.Mixer.Method** for possible values.

variant Define the variant of the method, see **SCF.Mixer.Variant** for possible values.

weight|w Define the mixing weight for the mixing scheme, see **SCF.Mixer.Weight**.

history Define number of previous history steps used in the minimization process, see **SCF.Mixer.History**.

weight.linear|w.linear Define the linear mixing weight for the mixing scheme. This only has meaning for Pulay or Broyden mixing. It defines the initial linear mixing weight.

To obtain the original Pulay Guaranteed-Reduction variant one should set this to 1.

restart Define the periodic restart of the saved history, see **SCF.Mixer.Restart**.

restart.save Define number of latest history steps retained when restarting the history, see **SCF.Mixer.Restart.Save**.

iterations Define the maximum number of iterations this mixer should run before changing to another mixing method.

NOTE: This *must* be used in conjunction with the **next** setting.

next <> Specify the name of the next mixing scheme after having conducted **iterations** SCF cycles using this mixing method.

next.conv <> If SCF convergence is reached using this mixer, switch to the mixing scheme via <>. Then proceed with the SCF cycle.

next.p If the relative difference between the latest two residuals is below this quantity, the mixer will switch to the method given in **next**. Thus if

$$\frac{\langle R[i] | R[i] \rangle - \langle R[i-1] | R[i-1] \rangle}{\langle R[i-1] | R[i-1] \rangle} < \mathbf{next.p} \quad (12)$$

is fulfilled it will skip to the next mixer.

restart.p If the relative difference between the latest two residuals is below this quantity, the mixer will restart the history. Thus if

$$\frac{\langle R[i] | R[i] \rangle - \langle R[i-1] | R[i-1] \rangle}{\langle R[i-1] | R[i-1] \rangle} < \mathbf{restart.p} \quad (13)$$

is fulfilled it will reset the history.

The options covered now may be exemplified in these examples. If the input file contains:

```
SCF.Mixer.Method pulay
SCF.Mixer.Weight 0.05
SCF.Mixer.History 10
SCF.Mixer.Restart 25
SCF.Mixer.Restart.Save 4
SCF.Mixer.Linear.After 0
SCF.Mixer.Linear.After.Weight 0.1
```

This may be equivalently setup using the more advanced input blocks:

```
%block SCF.Mixers
  init
  final
%endblock

%block SCF.Mixer.init
  method pulay
  weight 0.05
  history 10
  restart 25
  restart.save 4
  next.conv final
%endblock

%block SCF.Mixer.final
  method linear
  weight 0.1
%endblock
```

This advanced setup may be used to change mixers during the SCF to change certain parameters of the mixing method, or fully change the method for mixing. For instance it may be advantageous to increase the mixing weight once a certain degree of self-consistency has been reached. In the following example we change the mixing method to a different scheme by increasing the weight and decreasing the history steps:

```
%block SCF.Mixers
  init
  final
%endblock

%block SCF.Mixer.init
  method pulay
  weight 0.05
  history 10
  next final
  # Switch when the relative residual goes below 5%
  next.p 0.05
%endblock

%block SCF.Mixer.final
  method pulay
  weight 0.1
  history 6
%endblock
```

In essence, very complicated schemes of convergence may be created using the block's input.

The following options refer to the global treatment of how/when mixing should be performed.

Compat.Pre-v4-DM-H false

(logical)

This

controls the default values of **SCF.Mix.AfterConvergence**, **SCF.RecomputeHAfterScf** and **SCF.Mix.First**.

In versions prior to v4 the two former options were defaulted to **true** while the latter option was defaulted to **false**.

SCF.Mix.AfterConvergence `false` *(logical)*

Indicate whether mixing is done in the last SCF cycle (after convergence has been achieved) or not. Not mixing after convergence improves the quality of the final Kohn-Sham energy and of the forces when mixing the DM.

NOTE: See **Compat.Pre-v4-DM-H**.

SCF.RecomputeHAfterSCF `false` *(logical)*

Indicate whether the Hamiltonian is updated after the scf cycle, while computing the final energy, forces, and stresses. Not recomputing H makes further analysis tasks (such as the computation of band structures) more consistent, as they will be able to use the same H used to generate the last density matrix.

NOTE: See **Compat.Pre-v4-DM-H**.

6.9.3 Mixing of the Charge Density

See **SCF.Mix** on how to enable charge density mixing. If charge density mixing is enabled the fourier components of the charge density are mixed, as done in some plane-wave codes. (See for example Kresse and Furthmüller, Comp. Mat. Sci. 6, 15-50 (1996), KF in what follows.)

The charge mixing is implemented roughly as follows:

- The charge density computed in dhsf is fourier-transformed and stored in a new module. This is done both for “ $\rho(\mathbf{G})(\text{in})$ ” and “ $\rho(\mathbf{G})(\text{out})$ ” (the “out” charge is computed during the extra call to dhsf for correction of the variational character of the Kohn-Sham energy)
- The “in” and “out” charges are mixed (see below), and the resulting “in” fourier components are used by dhsf in successive iterations to reconstruct the charge density.
- The new arrays needed and the processing of most new options is done in the new module `m_rhog.F90`. The fourier-transforms are carried out by code in `rhoft.F`.
- Following standard practice, two options for mixing are offered:
 - A simple Kerker mixing, with an optional Thomas-Fermi wavevector to damp the contributions for small G’s. The overall mixing weight is the same as for other kinds of mixing, read from **DM.MixingWeight**.
 - A DIIS (Pulay) procedure that takes into account a sub-set of the G vectors (those within a smaller cutoff). Optionally, the scalar product used for the construction of the DIIS matrix from the residuals uses a weight factor.

The DIIS extrapolation is followed by a Kerker mixing step.

The code is `m_diis.F90`. The DIIS history is kept in a circular stack, implemented using the new framework for reference-counted types. This might be overkill for this particular use, and there are a few rough edges, but it works well.

The default convergence criteria remains based on the differences in the density matrix, but in this case the differences are from step to step, not the more fundamental **DM_out**-**DM_in**. Perhaps some other criterion should be made the default ($\max |\Delta\rho(\mathbf{G})|$, convergence of the free-energy...)

Note that with charge mixing the Harris energy as it is currently computed in SIESTA loses its meaning, since there is no **DM_in**. The program prints zeroes in the Harris energy field.

Note that the KS energy is correctly computed throughout the scf cycle, as there is an extra step for the calculation of the charge stemming from **DM_out**, which also updates the energies. Forces and final energies are correctly computed with the final **DM_out**, regardless of the setting of the option for mixing after scf convergence.

Initial tests suggest that charge mixing has some desirable properties and could be a drop-in replacement for density-matrix mixing, but many more tests are needed to calibrate its efficiency for different kinds of systems, and the heuristics for the (perhaps too many) parameters:

SCF.Kerker.q0sq 0 Ry *(energy)*

Determines the parameter q_0^2 featuring in the Kerker preconditioning, which is always performed on all components of $\rho(\mathbf{G})$, even those treated with the DIIS scheme.

SCF.RhoGMixingCutoff 9 Ry *(energy)*

Determines the sub-set of $\mathbf{G}$ vectors which will undergo the DIIS procedure. Only those with kinetic energies below this cutoff will be considered. The optimal extrapolation of the $\rho(\mathbf{G})$ elements will be replaced in the fourier series before performing the Kerker mixing.

SCF.RhoG.DIIS.Depth 0 *(integer)*

Determines the maximum number of previous steps considered in the DIIS procedure.

NOTE: The information from the first scf step is not included in the DIIS history. There is no provision yet for any other kind of “kick-starting” procedure. The logic is in `m_rhog` (`rhog_mixing` routine).

SCF.RhoG.Metric.Preconditioner.Cutoff $\langle \text{None} \rangle$ *(energy)*

Determines the value of q_1^2 in the weighing of the different $\mathbf{G}$ components in the scalar products among residuals in the DIIS procedure. Following the KF ansatz, this parameter is chosen so that the smallest (non-zero) $\mathbf{G}$ has a weight 20 times larger than that of the smallest $\mathbf{G}$ vector in the DIIS set.

The default is the result of the KF prescription.

SCF.DebugRhoGMixing false *(logical)*

Controls the level of debugging output in the mixing procedure (basically whether the first few stars worth of Fourier components are printed). Note that this feature will only display the components in the master node.

Debug.DIIS false *(logical)*

Controls the level of debugging output in the DIIS procedure. If set, the program prints the DIIS matrix and the extrapolation coefficients.

SCF.MixCharge.SCF1 false *(logical)*

Logical variable to indicate whether or not the charge is mixed in the first SCF cycle. Anecdotal

evidence indicates that it might be advantageous, at least for calculations started from scratch, to avoid that first mixing, and retain the “out” charge density as “in” for the next step.

6.9.4 Initialization of the density-matrix

NOTE: The conditions and options for density-matrix re-use are quite varied and not completely orthogonal at this point. For further information, see routine `Src/m_new_dm.F`. What follows is a summary.

The Density matrix can be:

1. Synthesized directly from atomic occupations.
(See the options below for spin considerations)
2. Read from a .DM file (if the appropriate options are set)
3. Extrapolated from previous geometry steps
(this includes as a special case the re-use of the DM
of the previous geometry iteration)

In cases 2 and 3, the structure of the read or extrapolated DM is automatically adjusted to the current sparsity pattern.

In what follows, "Initialization" of the DM means that the DM is either read from file (if available) or synthesized from atomic data. This is confusing, and better terminology should be used.

Special cases:

Harris functional: The matrix is always initialized

Force calculation: The DM should be written to disk
at the time of the "no displacement"
calculation and read from file at
every subsequent step.

Variable-cell calculation:

If the auxiliary cell changes, the DM is forced to be synthesized (conceivably one could rescue some important information from an old DM, but it is too much trouble for now). NOTE that this is a change in policy with respect to previous versions of the program, in which a (blind?) re-use was allowed, except if 'ReInitialiseDM' was 'true'. Now 'ReInitialiseDM' is 'true' by default. Setting it to 'false' is not recommended.

In all other cases (including "server operation"), the

default is to allow DM re-use (with possible extrapolation) from previous geometry steps.

For "CG" calculations, the default is not to extrapolate the DM (unless requested by setting 'DM.AllowExtrapolation' to "true"). The previous step's DM is reused.

The fdf variables 'DM.AllowReuse' and 'DM.AllowExtrapolation' can be used to turn off DM re-use and extrapolation.

DM.UseSaveDM `true` *(logical)*

Instructs to read the density matrix stored in file `SystemLabel.DM` by a previous run. SIESTA will continue even if `.DM` is not found.

NOTE: That if the spin settings has changed SIESTA allows reading a `.DM` from a similar calculation with different **Spin** option. This may be advantageous when going from non-polarized calculations to polarized, and beyond, see **Spin** for details.

DM.Init.Unfold `true` *(logical)*
depends on: DM.UseSaveDM

When reading the DM from a previous calculation there may be inconsistencies in the auxiliary supercell. E.g. if the previous calculation did not use an auxiliary supercell and the current calculation does (adding *k*-point sampling). SIESTA will automatically *unfold* the Γ -only DM to the auxiliary supercell elements (if `true`).

For `false` the DM elements are assumed to originate from an auxiliary supercell calculation and the sparse elements are not unfolded but directly copied.

NOTE: Generally this shouldn't not be touched, however, if the initial DM is generated using SISL^[16] and only on-site DM elements are set, this should be set to `false`.

DM.FormattedFiles `false` *(logical)*

Setting this alters the default for **DM.FormattedInput** and **DM.FormattedOutput**. Instructs to use formatted files for reading and writing the density matrix. In this case, the files are labelled `SystemLabel.DMF`.

Only usable if one has problems transferring files from one computer to another.

DM.FormattedInput `false` *(logical)*

Instructs to use formatted files for reading the density matrix.

DM.FormattedOutput `false` *(logical)*

Instructs to use formatted files for writing the density matrix.

DM.Init `atomic`

Specify the initial density matrix composition. Methods are compatible with a possible specification of **DM.InitSpin.AF**. Only a single option is available now, but more could be implemented. See also **DM.Init.RandomStates**.

atomic Only initialize the diagonal (on-site) elements of the density matrix according to the atomic ground-state populations of the atomic orbitals.

DM.InitSpin.AF `false` *(logical)*

It defines the initial spin density for a spin polarized calculation. The spin density is initially constructed with the maximum possible spin polarization for each atom in its atomic configuration. This variable defines the relative orientation of the atomic spins:

If **false** the initial spin-configuration is a ferromagnetic order (all spins up). If **true** all odd atoms are initialized to spin-up, all even atoms are initialized to spin-down.

%block DM.InitSpin `<None>` *(block)*

Define the initial spin density for a spin polarized calculation atom by atom. In the block there is one line per atom to be spin-polarized, containing the atom index (integer, ordinal in the block **AtomicCoordinatesAndAtomicSpecies**) and the desired initial spin-polarization (real, positive for spin up, negative for spin down). The spin polarization is actually meant to be the net spin magnetic moment, in units of the Bohr magneton, and thus numerically equal to the charge imbalance (in electrons) between spin up and spin down channels (as the g-factor for the electron spin is very close to 2, and the spin angular momentum is 1/2). A value larger than possible will be reduced to the maximum possible polarization, keeping its sign. Maximum polarization can also be given by introducing the symbol + or - instead of the polarization value. There is no need to include a line for every atom, only for those to be polarized. The atoms not contemplated in the block will be given non-polarized initialization.

For non-collinear spin, the spin direction may be specified for each atom by the polar angle θ and the azimuthal angle ϕ (using the physics ISO convention), given as the last two arguments in degrees. If not specified, $\theta = 0$ is assumed (z -polarized). **Spin** must be set to use non-collinear or spin-orbit for the directions to have effect.

Example:

```
%block DM.InitSpin
  5  -1.   90.   0.   # Atom index, spin, theta, phi (deg)
  3   +    45. -90.
  7   -
%endblock DM.InitSpin
```

In the above example, atom 5 is polarized in the x -direction.

If this block is defined, but empty, all atoms are not polarized. This block has precedence over **DM.InitSpin.AF**.

DM.Init.RandomStates `0` *(integer)*

The program will 'remove' N electrons from the initial density matrix and add N electrons in randomized 'states' (i.e., N random vectors which are normalized according to the S metric are used as "synthetic states"). These extra states are not orthogonal to the occupied manifold. The orbital coefficients of these states are scaled with the atomic charges, to avoid populating high-lying shells.

This procedure is wholly experimental and meant to provide a kick to the DM. It is inspired by the "random-wavefunction" initialization used in some plane-wave codes. It is turned off by default.

This option only has an effect if the density matrix is initialized from an atomic density and/or when using **DM.InitSpin**.

In case it is used together with **DM.InitSpin** it also randomizes the spin-configuration, which may be undesirable.

NOTE: This option is currently experimental since the randomized states are not ensured to be orthogonal. This flag may be removed in later revisions or superseded by other options. If testing this, start with a value of 1 to see if it has an effect; any higher numbers will probably be worse.

DM.AllowReuse `true` *(logical)*

Controls whether density matrix information from previous geometry iterations is re-used to start the new geometry's SCF cycle.

DM.AllowExtrapolation `true` *(logical)*

Controls whether the density matrix information from several previous geometry iterations is extrapolated to start the new geometry's SCF cycle. This feature is useful for molecular dynamics simulations and possibly also for geometry relaxations. The number of geometry steps saved is controlled by the variable **DM.History.Depth**.

This is default **true** for molecular-dynamics simulations, but **false**, for now, for geometry-relaxations (pending further tests which users are kindly requested to perform).

DM.History.Depth `1` *(integer)*

Sets the number of geometry steps for which density-matrix information is saved for extrapolation.

6.9.5 Initialization of the SCF cycle with charge densities

SCF.Read.Charge.NetCDF `false` *(logical)*

Instructs SIESTA to read the charge density stored in the netCDF file `Rho.IN.grid.nc`. This feature allows the easier re-use of electronic-structure information from a previous run. It is not necessary that the basis sets are “similar” (a requirement if density-matrices are to be read in).

NOTE: This is an experimental feature. Until robust checks are implemented, care must be taken to make sure that the FFT grids in the `.grid.nc` file and in SIESTA are the same.

SCF.Read.Deformation.Charge.NetCDF `false` *(logical)*

Instructs SIESTA to read the deformation charge density stored in the netCDF file `DeltaRho.IN.grid.nc`. This feature allows the easier re-use of electronic-structure information from a previous run. It is not necessary that the basis sets are “similar” (a requirement if density-matrices are to be read in). The deformation charge is particularly useful to give a good starting point for slightly different geometries.

NOTE: This is an experimental feature. Until robust checks are implemented, care must be taken to make sure that the FFT grids in the `.grid.nc` file and in SIESTA are the same.

6.9.6 Output of density matrix and Hamiltonian

Performance Note: For large-scale calculations, writing the DM at every scf step can have a severe impact on performance. The sparse-matrix I/O is undergoing a re-design, to facilitate the analysis of data and to increase the efficiency.

Use.Blocked.WriteMat `false` *(logical)*

By using blocks of orbitals (according to the underlying default block-cyclic distribution), the

sparse-matrix I/O can be speeded-up significantly, both by saving MPI communication and by reducing the number of file accesses. This is essential for large systems, for which the I/O could take a significant fraction of the total computation time.

To enable this “blocked format” (recommended for large-scale calculations) use the option **Use.Blocked.WriteMat true**. Note that it is off by default.

The new format is not backwards compatible. A converter program (`Util/DensityMatrix/dmUnblock.F90`) has been written to post-process those files intended for further analysis or re-use in SIESTA. This is the best option for now, since it allows liberal checkpointing with a much smaller time consumption, and only incurs costs when re-using or analyzing files.

Note that TRANSIESTA will continue to produce `SystemLabel.DM` files, in the old format (See `save_density_matrix.F`)

To test the new features, the option **S.Only true** can be used. It will produce three files: a standard one, another one with optimized MPI communications, and a third, blocked one.

Write.DM true *(logical)*

Control the creation of the current iterations density matrix to a file for restart purposes and post-processing. If **false** nothing will be written.

If **Use.Blocked.WriteMat** is **false** the `SystemLabel.DM` file will be written. Otherwise these density matrix files will be created; `DM_MIXED.blocked` and `DM_OUT.blocked` which are the mixed and the diagonalization output, respectively.

Write.DM.end.of.cycle `<Write.DM>` *(logical)*

Equivalent to **Write.DM**, but will only write at the end of each SCF loop.

NOTE: The file generated depends on **SCF.Mix.AfterConvergence**.

Write.H false *(logical)*

Whether restart Hamiltonians should be written (not intrinsically supported in 4.1).

If **true** these files will be created; `H_MIXED` or `H_DMGEN` which is the mixed or the generated Hamiltonian from the current density matrix, respectively. If **Use.Blocked.WriteMat** the just mentioned files will have the additional suffix **.blocked**.

Write.H.end.of.cycle `<Write.H>` *(logical)*

Equivalent to **Write.H**, but will only write at the end of each SCF loop.

NOTE: The file generated depends on **SCF.Mix.AfterConvergence**.

The following options control the creation of netCDF files. The relevant routines have not been optimized yet for large-scale calculations, so in this case the options should not be turned on (they are off by default).

Write.DM.NetCDF true *(logical)*

It determines whether the density matrix (after the mixing step) is output as a `DM.nc` netCDF file or not.

The file is overwritten at every SCF step. Use the **Write.DM.History.NetCDF** option if a complete history is desired.

The `DM.nc` and standard DM file formats can be converted at will with the programs in `Util/DensityMatrix` directory. Note that the DM values in the `DM.nc` file are in single preci-

sion.

Write.DMHS.NetCDF `true` *(logical)*

If **true**, the input density matrix, Hamiltonian, and output density matrix, are stored in a netCDF file named **DMHS.nc**. The file also contains the overlap matrix **S**.

The file is overwritten at every SCF step. Use the **Write.DMHS.History.NetCDF** option if a complete history is desired.

Write.DM.History.NetCDF `false` *(logical)*

If **true**, a series of netCDF files with names of the form **DM-NNNN.nc** is created to hold the complete history of the density matrix (after mixing). (See also **Write.DM.NetCDF**). Each file corresponds to a geometry step.

Write.DMHS.History.NetCDF `false` *(logical)*

If **true**, a series of netCDF files with names of the form **DMHS-NNNN.nc** is created to hold the complete history of the input and output density matrix, and the Hamiltonian. (See also **Write.DMHS.NetCDF**). Each file corresponds to a geometry step. The overlap matrix is stored only once per SCF cycle.

Write.HS.History `false` *(logical)*

If true, a series of **SystemLabel.HSX** files with names of the form **SystemLabel.N.HSX** is created to hold the complete history of the Hamiltonian and overlap matrix. Each file corresponds to a geometry step.

If **MD.TypeOfRun FC** the **SystemLabel.HSX** will be named **SystemLabel.ATOM-N.HSX** where **ATOM** corresponds to the displaced atom, and **N** is between 1 and 6 describing the 6 Cartesian displacements.

Write.TSHS.History `false` *(logical)*

If true, a series of **SystemLabel.TSHS** files with names of the form **SystemLabel.N.TSHS** is created to hold the complete history of the Hamiltonian and overlap matrix. Each file corresponds to a geometry step. This option only works with **TRANSIESTA**.

NOTE: This will default to be true for **MD.TypeOfRun FC** if **TS.HS.Save** is true as well.

NOTE: This will be deprecated in favour of **Write.HS.History**, e.g. writing **HSX** files.

6.9.7 Convergence criteria

NOTE: The older options with a **DM** prefix is still working for backwards compatibility. However, the following flags has precedence.

Note that all convergence criteria are additive and may thus be used simultaneously for complete control.

SCF.DM.Converge `true` *(logical)*

Logical variable to use the density matrix elements as monitor of self-consistency.

SCF.DM.Tolerance `10-4` *(real)*

depends on: **SCF.DM.Converge**

Tolerance of Density Matrix. When the maximum difference between the output and the input

on each element of the DM in a SCF cycle is smaller than **SCF.DM.Tolerance**, the self-consistency has been achieved.

NOTE: DM.Tolerance is the actual default for this flag.

DM.Normalization.Tolerance 10^{-5} *(real)*

Tolerance for unnormalized density matrices (typically the product of solvers such as PEXSI which have a built-in electron-count tolerance). If this tolerance is exceeded, the program stops. It is understood as a fractional tolerance. For example, the default will allow an excess or shortfall of 0.01 electrons in a 1000-electron system.

SCF.H.Converge **true** *(logical)*

Logical variable to use the Hamiltonian matrix elements as monitor of self-consistency: this is considered achieved when the maximum absolute change (dHmax) in the H matrix elements is below **SCF.H.Tolerance**. The actual meaning of dHmax depends on whether DM or H mixing is in effect: if mixing the DM, dHmax refers to the change in H(in) with respect to the previous step; if mixing H, dHmax refers to H(out)-H(in) in the previous(?) step.

SCF.H.Tolerance 10^{-3} eV *(energy)*
depends on: **SCF.H.Converge**

If **SCF.H.Converge** is **true**, then self-consistency is achieved when the maximum absolute change in the Hamiltonian matrix elements is below this value.

SCF.EDM.Converge **true** *(logical)*

Logical variable to use the energy density matrix elements as monitor of self-consistency: this is considered achieved when the maximum absolute change (dEmax) in the energy density matrix elements is below **SCF.EDM.Tolerance**. The meaning of dEmax is equivalent to that of **SCF.DM.Tolerance**.

SCF.EDM.Tolerance 10^{-3} eV *(energy)*
depends on: **SCF.EDM.Converge**

If **SCF.EDM.Converge** is **true**, then self-consistency is achieved when the maximum absolute change in the energy density matrix elements is below this value.

SCF.FreeE.Converge **false** *(logical)*

Logical variable to request an additional requirement for self-consistency: it is considered achieved when the change in the total (free) energy between cycles of the SCF procedure is below **SCF.FreeE.Tolerance** and the density matrix change criterion is also satisfied.

SCF.FreeE.Tolerance 10^{-4} eV *(energy)*
depends on: **SCF.FreeE.Converge**

If **SCF.FreeE.Converge** is **true**, then self-consistency is achieved when the change in the total (free) energy between cycles of the SCF procedure is below this value and the density matrix change criterion is also satisfied.

SCF.Harris.Converge **false** *(logical)*

Logical variable to use the Harris energy as monitor of self-consistency: this is considered achieved when the change in the Harris energy between cycles of the SCF procedure is below **SCF.Harris.Tolerance**. This is useful if only energies are needed, as the Harris energy tends

to converge faster than the Kohn-Sham energy. The user is responsible for using the correct energies in further processing, e.g., the Harris energy if the Harris criterion is used.

NOTE: Setting this to **true** makes **SCF.DM.Converge** **SCF.H.Converge** default to **false**.

SCF.Harris.Tolerance 10^{-4} eV *(energy)*
depends on: **SCF.Harris.Converge**

If **SCF.Harris.Converge** is **true**, then self-consistency is achieved when the change in the Harris energy between cycles of the SCF procedure is below this value. This is useful if only energies are needed, as the Harris energy tends to converge faster than the Kohn-Sham energy.

6.10 The real-space grid and the eggbox-effect

SIESTA uses a finite 3D grid for the calculation of some integrals and the representation of charge densities and potentials. Its fineness is determined by its plane-wave cutoff, as given by the **Mesh.Cutoff** option. It means that all periodic plane waves with kinetic energy lower than this cutoff can be represented in the grid without aliasing. In turn, this implies that if a function (e.g. the density or the effective potential) is an expansion of only these plane waves, it can be Fourier transformed back and forth without any approximation.

The existence of the grid causes the breaking of translational symmetry (the egg-box effect, due to the fact that the density and potential *do have* plane wave components above the mesh cutoff). This symmetry breaking is clear when moving one single atom in an otherwise empty simulation cell. The total energy and the forces oscillate with the grid periodicity when the atom is moved, as if the atom were moving on an eggbox. In the limit of infinitely fine grid (infinite mesh cutoff) this effect disappears.

For reasonable values of the mesh cutoff, the effect of the eggbox on the total energy or on the relaxed structure is normally unimportant. However, it can affect substantially the process of relaxation, by increasing the number of steps considerably, and can also spoil the calculation of vibrations, usually much more demanding than relaxations.

The `Util/Scripting/eggbox_checker.py` script can be used to diagnose the eggbox effect to be expected for a particular pseudopotential/basis-set combination.

Apart from increasing the mesh cutoff (see the **Mesh.Cutoff** option), the following options might help in lessening a given eggbox problem. But note also that a filtering of the orbitals and the relevant parts of the pseudopotential and the pseudocore charge might be enough to solve the issue (see Sec. 6.3.10).

Mesh.Cutoff 300 Ry *(energy)*
 Defines the plane wave cutoff for the grid.

Mesh.Sizes $\langle$ Mesh.Cutoff $\rangle$ *(list)*
 Manual definition of grid size along each lattice vector. The value must be divisible by **Mesh.SubDivisions**, otherwise the program will die. The numbers should also be divisible with 2, 3 and 5 due to the FFT algorithms.

This option may be specified as a block, or a list:

```
%block Mesh.Sizes
```

```

100 202 210
%endblock
# Or equivalently:
Mesh.Sizes [100 202 210]

```

By default the grid size is determined via **Mesh.Cutoff**. This option has precedence if both are specified.

Mesh.SubDivisions 2 *(integer)*

Defines the number of sub-mesh points in each direction used to save index storage on the mesh. It affects the memory requirements and the CPU time, but not the results.

NOTE: The default value might be a bit conservative. Users might experiment with higher values, 4 or 6, to lower the memory and cputime usage.

%block Grid.CellSampling *(None)* *(block)*

It specifies points within the grid cell for a symmetrization sampling.

For a given grid the grid-cutoff convergence can be improved (and the eggbox lessened) by recovering the lost symmetry: by symmetrizing the sensitive quantities. The full symmetrization implies an integration (averaging) over the grid cell. Instead, a finite sampling can be performed.

It is a sampling of rigid displacements of the system with respect to the grid. The original grid-system setup (one point of the grid at the origin) is always calculated. It is the (0,0,0) displacement. The block **Grid.CellSampling** gives the additional displacements wanted for the sampling. They are given relative to the grid-cell vectors, i.e., (1,1,1) would displace to the next grid point across the body diagonal, giving an equivalent grid-system situation (a useless displacement for a sampling).

Examples: Assume a cubic cell, and therefore a (smaller) cubic grid cell. If there is no block or the block is empty, then the original (0,0,0) will be used only. The block:

```

%block Grid.CellSampling
0.5 0.5 0.5
%endblock Grid.CellSampling

```

would use the body center as a second point in the sampling. Or:

```

%block Grid.CellSampling
0.5 0.5 0.0
0.5 0.0 0.5
0.0 0.5 0.5
%endblock Grid.CellSampling

```

gives an fcc kind of sampling, and

```

%block Grid.CellSampling
0.5 0.0 0.0
0.0 0.5 0.0
0.0 0.0 0.5
0.0 0.5 0.5
0.5 0.0 0.5
0.5 0.5 0.0
0.5 0.5 0.5
%endblock Grid.CellSampling

```

gives again a cubic sampling with half the original side length. It is not trivial to choose a

right set of displacements so as to maximize the new 'effective' cutoff. It depends on the kind of cell. It may be automatized in the future, but it is now left to the user, who introduces the displacements manually through this block.

The quantities which are symmetrized are: (i) energy terms that depend on the grid, (ii) forces, (iii) stress tensor, and (iv) electric dipole.

The symmetrization is performed at the end of every SCF cycle. The whole cycle is done for the (0,0,0) displacement, and, when the density matrix is converged, the same (now fixed) density matrix is used to obtain the desired quantities at the other displacements (the density matrix itself is *not* symmetrized as it gives a much smaller egg-box effect). The CPU time needed for each displacement in the **Grid.CellSampling** block is of the order of one extra SCF iteration. This may be required in systems where very precise forces are needed, and/or if partial cores are used. It is advantageous to test whether the forces are sampled sufficiently by sampling one point.

Additionally this may be given as a list of 3 integers which corresponds to a "Monkhorst-Pack" like grid sampling. I.e.

```
Grid.CellSampling [2 2 2]
```

is equivalent to

```
%block Grid.CellSampling
    0.5    0.0    0.0
    0.0    0.5    0.0
    0.5    0.5    0.0
    0.0    0.0    0.5
    0.5    0.0    0.5
    0.0    0.5    0.5
    0.5    0.5    0.5
%endblock Grid.CellSampling
```

This is an easy method to see if the flag is important for your system or not.

```
%block EggboxRemove <None> (block)
```

For recovering translational invariance in an approximate way.

It works by subtracting from Kohn-Sham's total energy (and forces) an approximation to the eggbox energy, sum of atomic contributions. Each atom has a predefined eggbox energy depending on where it sits on the cell. This atomic contribution is species dependent and is obviously invariant under grid-cell translations. Each species contribution is thus expanded in the appropriate Fourier series. It is important to have a smooth eggbox, for it to be represented by a few Fourier components. A jagged egg-box (unless very small, which is then unimportant) is often an indication of a problem with the pseudo.

In the block there is one line per Fourier component. The first integer is for the atomic species it is associated with. The other three represent the reciprocal lattice vector of the grid cell (in units of the basis vectors of the reciprocal cell). The real number is the Fourier coefficient in units of the energy scale given in **EggboxScale** (see below), normally 1 eV.

The number and choice of Fourier components is free, as well as their order in the block. One can choose to correct only some species and not others if, for instance, there is a substantial difference in hardness of the cores. The 0 0 0 components will add a species-dependent constant energy per atom. It is thus irrelevant except if comparing total energies of different calculations,

in which case they have to be considered with care (for instance by putting them all to zero, i.e. by not introducing them in the list). The other components average to zero representing no bias in the total energy comparisons.

If the total energies of the free atoms are put as 0 0 0 coefficients (with spin polarisation if adequate etc.) the corrected total energy will be the cohesive energy of the system (per unit cell).

Example: For a two species system, this example would give a quite sufficient set in many instances (the actual values of the Fourier coefficients are not realistic).

```
%block EggBoxRemove
  1  0  0  0 -143.86904
  1  0  0  1   0.00031
  1  0  1  0   0.00016
  1  0  1  1  -0.00015
  1  1  0  0   0.00035
  1  1  0  1  -0.00017
  2  0  0  0 -270.81903
  2  0  0  1   0.00015
  2  0  1  0   0.00024
  2  1  0  0   0.00035
  2  1  0  1  -0.00077
  2  1  1  0  -0.00075
  2  1  1  1  -0.00002
%endblock EggBoxRemove
```

It represents an alternative to grid-cell sampling (above). It is only approximate, but once the Fourier components for each species are given, it does not represent any computational effort (neither memory nor time), while the grid-cell sampling requires CPU time (roughly one extra SCF step per point every MD step).

It will be particularly helpful in atoms with substantial partial core or semicore electrons.

NOTE: This should only be used for fixed cell calculations, i.e. not with **MD.VariableCell**.

For the time being, it is up to the user to obtain the Fourier components to be introduced. They can be obtained by moving one isolated atom through the cell to be used in the calculation (for a give cell size, shape and mesh), once for each species. The `Util/Scripting/eggbox_checker.py` script can be used as a starting point for this.

EggboxScale 1 eV (energy)

Defines the scale in which the Fourier components of the egg-box energy are given in the **EggboxRemove** block.

6.11 Matrix elements of the Hamiltonian and overlap

NeglNonOverlapInt false (logical)

Logical variable to neglect or compute interactions between orbitals which do not overlap. These come from the KB projectors. Neglecting them makes the Hamiltonian more sparse, and the calculation faster.

NOTE: Use with care!

SCF.Write.Extra false (logical)

Instructs SIESTA to write out a variety of files with the Hamiltonian and density matrix. The output depends on whether a Hamiltonian mixing or density matrix mixing is performed (see **SCF.Mixing**).

These files are created

- **H_MIXED**; the Hamiltonian after mixing
- **DM_OUT**; the density matrix as calculated by the current iteration
- **H_DMGEN**; the Hamiltonian used to calculate the density matrix
- **DM_MIXED**; the density matrix after mixing

Save.HS `true` *(logical)*

Instructs to write the Hamiltonian and overlap matrices, as well as other data required to generate bands and density of states, in file **SystemLabel.HSX**.

The **SystemLabel.HSX** file format contains all relevant information to construct the Brillouin zone Hamiltonian and can thus be used for subsequent density of states calculations, etc.

The program **hsx2hs** in **Util/HSX** can be used to generate an old-style **.HS** file if needed (prefer to update utilities to support the newer formats).

SIESTA produces also an **.HSX** file if the **COOP.Write** option is active.

SIESTA produces two types of **SystemLabel.HSX** when **TRANSIESTA** is used as a **Solution-Method**:

SystemLabel.HSX The Hamiltonian after the initial SIESTA diagonalization method. I.e. this contains only the Hamiltonian without any bias, and with any NEGF effects.

SystemLabel.TS.HSX The Hamiltonian produced *after* any self-consistent cycles with NEGF effects. I.e. one should always use the **SystemLabel.TS.HSX** file for subsequent **TRANSIESTA**/**TBTRANS** calculations.

For **MD.TypeOfRun FC** SIESTA produces separate **SystemLabel.HSX** files for each displacement. The displaced atom and the direction of the displacement are indicated in the filename.

NOTE: Since 5.0 the **SystemLabel.HSX** file format has changed to reduce disk-space and store data in double precision. This means that the file is not backward compatible and any external utilities should adapt their **SystemLabel.HSX** file reading. See e.g. **Util/HSX** for details on the new implementation.

See also the **Write.DMHS.NetCDF** and **Write.DMHS.History.NetCDF** options.

6.11.1 The auxiliary supercell

When using k-points, this auxiliary supercell is needed to compute properly the matrix elements involving orbitals in different unit cells. It is computed automatically by the program at every geometry step.

Note that for gamma-point-only calculations there is an implicit “folding” of matrix elements corresponding to the images of orbitals outside the unit cell. If information about the specific values of these matrix elements is needed (as for **COOP/COHP** analysis), one has to make sure that the unit cell is large enough, or force the use of an auxiliary supercell.

ForceAuxCell `false` *(logical)*

If **true**, the program uses an auxiliary cell even for gamma-point-only calculations. This might be needed for COOP/COHP calculations, as noted above, or in degenerate cases, such as when the cell is so small that a given orbital “self-interacts” with its own images (via direct overlap or through a KB projector). In this case, the diagonal value of the overlap matrix *S* for this orbital is different from 1, and an initialization of the DM via atomic data would be faulty. The program corrects the problem to zeroth-order by dividing the DM value by the corresponding overlap matrix entry, but the initial charge density would exhibit distortions from a true atomic superposition (See routine `m_new_dm.F`). The distortion of the charge density is a serious problem for Harris functional calculations, so this option must be enabled for them if self-folding is present. (Note that this should not happen in any serious calculation...)

6.12 Calculation of the electronic structure

SIESTA can use three qualitatively different methods to determine the electronic structure of the system. The first is standard diagonalization, which works for all systems and has a cubic scaling with the size. The second is based on the direct minimization of a special functional over a set of trial orbitals. These orbitals can either extend over the entire system, resulting in a cubic scaling algorithm, or be constrained within a localization radius, resulting in a linear scaling algorithm. The former is a recent implementation (described in 6.12.4), that can be viewed as an equivalent approach to diagonalization in terms of the accuracy of the solution; the latter is the historical $O(N)$ method used by SIESTA (described in 6.12.5); it scales in principle linearly with the size of the system (only if the size is larger than the radial cutoff for the local solution wave-functions), but is quite fragile and substantially more difficult to use, and only works for systems with clearly separated occupied and empty states. The default is to use diagonalization. The third method (PEXSI) is based on the pole expansion of the Fermi-Dirac function and the direct computation of the density matrix via an efficient scheme of selected inversion (see Sec 6.15).

Additionally, SIESTA can use the ELSI library of solvers (see Sec. 6.13), which provides its own versions of OMM and PEXSI, in addition to diagonalization and other solvers, and the new linear-scaling CheSS library (see Sec. 6.14)

The calculation of the *H* and *S* matrix elements is always done with an $O(N)$ method. The actual scaling is not linear for small systems, but it becomes $O(N)$ when the system dimensions are larger than the scale of orbital r_c ’s.

The relative importance of both parts of the computation (matrix elements and solution) depends on the size and quality of the calculation. The mesh cutoff affects only the matrix-element calculation; orbital cutoff radii affect the matrix elements and all solvers except diagonalization; the need for **k**-point sampling affects the solvers only, and the number of basis orbitals affects them all.

In practice, the vast majority of users employ diagonalization (or the OMM method) for the calculation of the electronic structure. This is so because the vast majority of calculations (done for intermediate system sizes) would not benefit from the $O(N)$ or PEXSI solvers.

SolutionMethod **diagon**

(*string*)

Character string to choose among diagonalization (**diagon**), cubic-scaling minimization (OMM), Order-*N* (**OrderN**) solution of the Kohn-Sham Hamiltonian, **transiesta** or **NEGF**, the PEXSI method (**PEXSI**), ELSI family of solvers (**ELSI**) or the **CheSS** solver. In addition, the **Dummy** solver will just return a slightly perturbed density-matrix without actually solving

for the electronic structure. This is useful for timing other routines.

6.12.1 Diagonalization options

NumberOfEigenStates `<all orbitals>` (integer)
depends on: **Diag.Algorithm**

This parameter allows the user to reduce the number of eigenstates that are calculated from the maximum possible. The benefit is that, for any calculation, the cost of the diagonalization is reduced by finding fewer eigenvalues/eigenvectors. For example, during a geometry optimisation, only the occupied states are required rather than the full set of virtual orbitals. Note, that if the electronic temperature is greater than zero then the number of partially occupied states increases, depending on the band gap. The value specified must be greater than the number of occupied states and less than the number of basis functions.

If a *negative* number is passed it corresponds to the number of orbitals above the total charge of the system. In effect it corresponds to the number of orbitals above the Fermi level for zero temperature. I.e. if -2 is specified for a system with 20 orbitals and 10 electrons it is equivalent to 12.

Using this option can *greatly* speed up your calculations if used correctly.

NOTE: If experiencing PDORMTR errors in Γ calculations with **MRRR** algorithm, it is because of a buggy ScaLAPACK implementation, simply use another algorithm.

NOTE: This only affects the **MRRR**, **ELPA** and **Expert** diagonalization routines.

Diag.WFS.Cache `none|cdf` (string)
deprecates: **UseNewDiagk**

Specify whether SIESTA should cache wavefunctions in the diagonalization routine. Without a cache, a standard two-pass procedure is used. First eigenvalues are obtained to determine the Fermi level, and then the wavefunctions are computed to build the density matrix.

Using a cache one can do everything in one go. However, this requires substantial IO and performance may vary.

none The wavefunctions will not be cached and the standard two-pass diagonalization method is used.

cdf The wavefunctions are stored in **WFS.nc** (NetCDF format) and created from a single root node. This requires NetCDF support, see Sec. 2.3.

NOTE: This is an experimental feature.

NOTE: It is not compatible with the **Diag.ParallelOverK** option.

Diag.Use2D `true` (logical)

Determine whether a 1D or 2D data decomposition should be used when calling ScaLAPACK. The use of 2D leads to superior scaling on large numbers of processors and is therefore the default. This option only influences the parallel performance.

If **Diag.BlockSize** is different from **BlockSize** this flag defaults to **true**, else if **Diag.ProcessorY** is 1 or the total number of processors, then this flag will default to **false**.

Diag.ProcessorY $\sim \sqrt{N}$ (integer)
depends on: **Diag.Use2D**

Set the number of processors in the 2D distribution along the rows. Its default is equal to the lowest multiple of N (number of MPI cores) below $\sqrt{N}$ such that, ideally, the distribution will be a square grid.

The input is required to be a multiple of the total number of MPI cores but SIESTA will reduce the input value such that it coincides with this.

Once the lowest multiple closest to $\sqrt{N}$, or the input, is determined the 2D distribution will be $\text{ProcessorY} \times N/\text{ProcessorY}$, rows $\times$ columns.

NOTE: If the automatic correction (lowest multiple of MPI cores) is 1 the default of **Diag.Use2D** will be **false**.

Diag.BlockSize $\langle \text{BlockSize} \rangle$ (integer)
depends on: **Diag.Use2D**

The block-size used for the 2D distribution in the ScaLAPACK calls. This number greatly affects the performance of ScaLAPACK.

If the ScaLAPACK library is threaded this parameter should not be too small. In any case it may be advantageous to run a few tests to find a suitable value.

NOTE: If **Diag.Use2D** is set to **false** this flag is not used.

Diag.Algorithm **Divide-and-Conquer|...** (string)
deprecates: **Diag.DivideAndConquer**, **Diag.MRRR**, **Diag.ELPA**, **Diag.NoExpert**

Select the algorithm when calculating the eigenvalues and/or eigenvectors.

The fastest routines are typically MRRR or ELPA which may be significantly faster by specifying a suitable **NumberOfEigenStates** value.

Currently the implemented solvers are:

divide-and-Conquer Use the divide-and-conquer algorithm.

divide-and-Conquer-2stage Use the divide-and-conquer 2stage algorithm (fall-back to the divide-and-conquer if not available).

MRRR *depends on:* **NumberOfEigenStates**

Use the multiple relatively robust algorithm.

NOTE: The MRRR method is defaulted not to be compiled in, however, if your ScaLAPACK library does contain the relevant sources one may add this pre-processor flag **-DSIESTA__MRRR**.

MRRR-2stage *depends on:* **NumberOfEigenStates**

Use the 2-stage multiple relatively robust algorithm.

expert *depends on:* **NumberOfEigenStates**

Use the expert algorithm which allows calculating a subset of the eigenvalues/eigenvectors.

expert-2stage *depends on:* **NumberOfEigenStates**

Use the 2-stage expert algorithm which allows calculating a subset of the eigenvalues/eigenvectors.

noexpert|QR Use the QR algorithm.

noexpert-2stage|QR-2stage Use the 2-stage QR algorithm.

ELPA-1stage *depends on:* **NumberOfEigenStates**

Use the ELPA^[1;11] 1-stage solver. Requires compilation of SIESTA with ELPA, see Sec. 2.3. This option is not compatible with **Diag.ParallelOverK**. In addition, when using a GPU-enabled version of ELPA it is important to verify that **Diag.BlockSize** is a power of 2; if not, ELPA will only run on CPU.

ELPA|ELPA-2stage

depends on: NumberOfEigenStates

Use the ELPA^[1;11] 2-stage solver. Requires compilation of SIESTA with ELPA, see Sec. 2.3. This option is not compatible with **Diag.ParallelOverK**. In addition, when using a GPU-enabled version of ELPA it is important to verify that **Diag.BlockSize** is a power of 2; if not, ELPA will only run on CPU.

NOTE: All the 2-stage solvers are (as of July 2017) only implemented in the LAPACK library, so they will only be usable in serial or when using **Diag.ParallelOverK**.

If found by CMake in the LAPACK library, 2-stage solvers will be enabled automatically, by setting the preprocessor variable

```
-DSIESTA__DIAG_2STAGE
```

Previous versions of SIESTA shipped a copy of the relevant LAPACK files, including the 2-stage solvers. That might no longer be the case, and there is no direct support for compiling those files with CMake.

NOTE: This flag has precedence over the deprecated flags: **Diag.DivideAndConquer**, **Diag.MRRR**, **Diag.ELPA** and **Diag.NoExpert**. However, the default is taken from the deprecated flags.

Diag.ELPA.GPU false

(logical)

Newer versions of the ELPA library have optional support for GPUs. This flag will request that GPU-specific code be used by the library.

To use this feature, GPU support has to be explicitly enabled during compilation of the ELPA library. At present, detection of GPU support in the code is not fool-proof, so this flag should only be enabled if GPU support is indeed available.

Diag.ELPA.GPU.String nvidia-gpu

(string)

Newer versions of the ELPA library have optional support for GPUs. This string will be used as the key to set the GPU feature in the ELPA interface.

Traditionally it was just “gpu”, but recent versions use “nvidia-gpu”, or “amd-gpu”, etc. This setting can still be overridden by the value of the environment variable **SIESTA_ELPA_GPU_STRING**. Its default value can be set at build time to match the characteristics of the ELPA library and the host architecture, using the CMake variable **SIESTA_ELPA_GPU_STRING**.

Diag.ParallelOverK false

(logical)

For the diagonalization there is a choice in strategy about whether to parallelise over the **k** points (**true**) or over the orbitals (**false**). **k** point diagonalization is close to perfectly parallel but is only useful where the number of **k** points is much larger than the number of processors and therefore orbital parallelisation is generally preferred. The exception is for metals where the unit cell is small, but the number of **k** points to be sampled is very large. In this last case it is recommend that this option be used.

NOTE: This scheme is not used for the diagonalizations involved in the generation of the band-

structure (as specified with **BandLines** or **BandPoints**) or in the generation of wave-function information (as specified with **WaveFuncKPoints**). In these cases the program falls back to using parallelization over orbitals.

Diag.AbsTol 10^{-16} (real)

The absolute tolerance for the orthogonality of the eigenvectors. This tolerance is only applicable for the solvers:

expert for both the serial and parallel solvers.

mrrr for the serial solver.

Diag.OrFac 10^{-3} (real)

Re-orthogonalization factor to determine when the eigenvectors should be re-orthogonalized.

Only applicable for the **expert** serial and parallel solvers.

Diag.Memory 1 (real)

Whether the parallel diagonalization of a matrix is successful or not can depend on how much workspace is available to the routine when there are clusters of eigenvalues. **Diag.Memory** allows the user to increase the memory available, when necessary, to achieve successful diagonalization and is a scale factor relative to the minimum amount of memory that ScaLAPACK might need.

Diag.UpperLower lower|upper (string)

Which part of the symmetric triangular part should be used in the solvers.

NOTE: Do not change this variable unless you are performing benchmarks. It should be fastest with the **lower** part.

Deprecated diagonalization options

Diag.MRRR false (logical)

depends on: **NumberOfEigenStates**

Use the MRRR method in ScaLAPACK for diagonalization. Specifying a number of eigenvectors to store is possible through the symbol **NumberOfEigenStates** (see above).

NOTE: The MRRR method is defaulted not to be compiled in, however, if your ScaLAPACK library does contain the relevant sources one may add this pre-processor flag **-DSIESTA__MRRR**.

NOTE: Use **Diag.Algorithm** instead.

Diag.DivideAndConquer true (logical)

Logical to select whether the normal or Divide and Conquer algorithms are used within the ScaLAPACK/LAPACK diagonalization routines.

NOTE: Use **Diag.Algorithm** instead.

Diag.ELPA false (logical)

depends on: **NumberOfEigenStates**

See the ELPA articles^[1;11] for additional information.

NOTE: It is not compatible with the **Diag.ParallelOverK** option.

NOTE: Use **Diag.Algorithm** instead.

Diag.NoExpert false

(logical)

Logical to select whether the simple or expert versions of the ScaLAPACK/LAPACK routines are used. Usually the expert routines are faster, but may require slightly more memory.

NOTE: Use **Diag.Algorithm** instead.

6.12.2 Output of eigenvalues and wavefunctions

This section focuses on the output of eigenvalues and wavefunctions produced during the (last) iteration of the self-consistent cycle, and associated to the appropriate k-point sampling.

For band-structure calculations (which typically use a different set of k-points) and specific requests for wavefunctions, see Secs. 6.16 and 6.17, respectively.

The complete set of wavefunctions obtained during the last iteration of the SCF loop will be written to a NetCDF file **WFS.nc** if the **Diag.WFS.Cache cdf** option is in effect.

The complete set of wavefunctions obtained during the last iteration of the SCF loop will be written to **SystemLabel.fullBZ.WFSX** if the **COOP.Write** option is in effect.

WriteEigenvalues false

(logical)

If **true** it writes the Hamiltonian eigenvalues for the sampling $\vec{k}$ points, in the main output file. If **false**, it writes them in the file **SystemLabel.EIG**, which can be used by the **Eig2DOS** postprocessing utility (in the **Util/Eig2DOS** directory) for obtaining the density of states.

NOTE: this option only works for **SolutionMethod** which calculates the eigenvalues.

6.12.3 Occupation of electronic states and Fermi level

OccupationFunction FD

(string)

String variable to select the function that determines the occupation of the electronic states. These options are available:

FD The usual Fermi-Dirac occupation function is used.

MP The occupation function proposed by Methfessel and Paxton (Phys. Rev. B, **40**, 3616 (1989)), is used.

Cold The occupation function proposed by Marzari, Vanderbilt et. al (PRL, **82**, 16 (1999)), is used, this is commonly referred to as *cold smearing*.

The smearing of the electronic occupations is done, in all cases, using an energy width defined by the **ElectronicTemperature** variable. Note that, while in the case of Fermi-Dirac, the occupations correspond to the physical ones if the electronic temperature is set to the physical temperature of the system, this is not the case in the Methfessel-Paxton function. In this case, the temperature is just a mathematical artifact to obtain a more accurate integration of the physical quantities at a lower cost. In particular, the Methfessel-Paxton scheme has the advantage that, even for quite large smearing temperatures, the obtained energy is very close to the physical energy at $T = 0$. Also, it allows a much faster convergence with respect to k -points, specially for metals. Finally, the convergence to selfconsistency is very much improved (allowing the use of larger mixing coefficients).

For the Methfessel-Paxton case, and similarly for cold smearing, one can use relatively large values for the **ElectronicTemperature** parameter. How large depends on the specific system. A guide can be found in the article by J. Kresse and J. Furthmüller, *Comp. Mat. Sci.* **6**, 15 (1996).

If Methfessel-Paxton smearing is used, the order of the corresponding Hermite polynomial expansion must also be chosen (see description of variable **OccupationMPOrder**).

We finally note that, in both cases (FD and MP), once a finite temperature has been chosen, the relevant energy is not the Kohn-Sham energy, but the Free energy. In particular, the atomic forces are derivatives of the Free energy, not the KS energy. See R. Wentzcovitch *et al.*, *Phys. Rev. B* **45**, 11372 (1992); S. de Gironcoli, *Phys. Rev. B* **51**, 6773 (1995); J. Kresse and J. Furthmüller, *Comp. Mat. Sci.* **6**, 15 (1996), for details.

OccupationMPOrder 1 (integer)

Order of the Hermite-Gauss polynomial expansion for the electronic occupation functions in the Methfessel-Paxton scheme (see *Phys. Rev. B* **40**, 3616 (1989)). Specially for metals, higher order expansions provide better convergence to the ground state result, even with larger smearing temperatures, and provide also better convergence with k-points.

NOTE: only used if **OccupationFunction** is **MP**.

ElectronicTemperature 300 K (temperature/energy)

Temperature for occupation function. Useful specially for metals, and to accelerate selfconsistency in some cases.

6.12.4 Orbital minimization method (OMM)

The OMM is an alternative cubic-scaling solver that uses a minimization algorithm instead of direct diagonalization to find the occupied subspace. The main advantage over diagonalization is the possibility of iteratively reusing the solution from each SCF/MD step as the starting guess of the following one, thus greatly reducing the time to solution. Typically, therefore, the first few SCF cycles of the first MD step of a simulation will be slower than diagonalization, but the rest will be faster. The main disadvantages are that individual Kohn-Sham eigenvalues are not computed, and that only a fixed, integer number of electrons at each k point/spin is allowed. Therefore, only spin-polarized calculations with **Spin.Fix** are allowed, and **Spin.Total** must be chosen appropriately. For non- Γ point calculations, the number of electrons is set to be equal at all k points. Non-collinear calculations (see **Spin**) are not supported at present. The OMM implementation was initially developed by Fabiano Corsetti.

It is important to note that the OMM requires all occupied Kohn-Sham eigenvalues to be negative; this can be achieved by applying a shift to the eigenspectrum, controlled by **ON.eta** (in this case, **ON.eta** simply needs to be higher than the HOMO level). If the OMM exhibits a pathologically slow or unstable convergence, this is almost certainly due to the fact that the default value of **ON.eta** (**0.0 eV**) is too low, and should be raised by a few eV.

OMM.UseCholesky true (logical)

Select whether to perform a Cholesky factorization of the generalized eigenvalue problem; this removes the overlap matrix from the problem but also destroys the sparsity of the Hamiltonian matrix.

- OMM.Use2D** `true` (logical)
 Select whether to use a 2D data decomposition of the matrices for parallel calculations. This generally leads to superior scaling for large numbers of MPI processes.
- OMM.UseSparse** `false` (logical)
 Select whether to make use of the sparsity of the Hamiltonian and overlap matrices where possible when performing matrix-matrix multiplications (these operations are thus reduced from $O(N^3)$ to $O(N^2)$ without loss of accuracy).
NOTE: not compatible with **OMM.UseCholesky**, **OMM.Use2D**, or non- Γ point calculations
- OMM.Precon** `-1` (integer)
 Number of SCF steps for *all* MD steps for which to apply a preconditioning scheme based on the overlap and kinetic energy matrices; for negative values the preconditioning is always applied. Preconditioning is usually essential for fast and accurate convergence (note, however, that it is not needed if a Cholesky factorization is performed; in such cases this variable will have no effect on the calculation).
NOTE: cannot be used with **OMM.UseCholesky**.
- OMM.PreconFirstStep** `<OMM.Precon>` (integer)
 Number of SCF steps in the *first* MD step for which to apply the preconditioning scheme; if present, this will overwrite the value given in **OMM.Precon** for the first MD step only.
- OMM.Diagon** `0` (integer)
 Number of SCF steps for *all* MD steps for which to use a standard diagonalization before switching to the OMM; for negative values diagonalization is always used, and so the calculation is effectively equivalent to **SolutionMethod** `diagon`. In general, selecting the first few SCF steps can speed up the calculation by removing the costly initial minimization (at present this works best for Γ point calculations).
- OMM.DiagonFirstStep** `<OMM.Diagon>` (integer)
 Number of SCF steps in the *first* MD step for which to use a standard diagonalization before switching to the OMM; if present, this will overwrite the value given in **OMM.Diagon** for the first MD step only.
- OMM.BlockSize** `<BlockSize>` (integer)
 Blocksize used for distributing the elements of the matrix over MPI processes. Specifically, this variable controls the dimension relating to the trial orbitals used in the minimization (equal to the number of occupied states at each k point/spin); the equivalent variable for the dimension relating to the underlying basis orbitals is controlled by **BlockSize**.
- OMM.TPreconScale** `10 Ry` (energy)
 Scale of the kinetic energy preconditioning (see C. K. Gan *et al.*, Comput. Phys. Commun. **134**, 33 (2001)). A smaller value indicates more aggressive kinetic energy preconditioning, while an infinite value indicates no kinetic energy preconditioning. In general, the kinetic energy preconditioning is much less important than the tensorial correction brought about by the overlap matrix, and so this value will have fairly little impact on the overall performance of the preconditioner; however, too aggressive kinetic energy preconditioning can have a detrimental

effect on performance and accuracy.

OMM.RelTol 10^{-9} *(real)*
Relative tolerance in the conjugate gradients minimization of the Kohn-Sham band energy (see **ON.Etol**).

OMM.Eigenvalues `false` *(logical)*
Select whether to perform a diagonalization at the end of each MD step to obtain the Kohn-Sham eigenvalues.

OMM.WriteCoeffs `false` *(logical)*
Select whether to write the coefficients of the solution orbitals to file at the end of each MD step.

OMM.ReadCoeffs `false` *(logical)*
Select whether to read the coefficients of the solution orbitals from file at the beginning of a new calculation. Useful for restarting an interrupted calculation, especially when used in conjunction with **DM.UseSaveDM**. Note that the same number of MPI processes and values of **OMM.Use2D**, **OMM.BlockSize**, and **BlockSize** must be used when restarting.

OMM.LongOutput `false` *(logical)*
Select whether to output detailed information of the conjugate gradients minimization for each SCF step.

6.12.5 Order(N) calculations

The Order(N) subsystem is quite fragile and only works for systems with clearly separated occupied and empty states. Note also that the option to compute the chemical potential automatically does not yet work in parallel.

NOTE: Since it is used less often, bugs creeping into the O(N) solver have been more resilient than in more popular bits of the code. Work is ongoing to clean and automate the O(N) process, to make the solver more user-friendly and robust.

ON.functional `Kim` *(string)*
Choice of order-N minimization functionals:

Kim Functional of Kim, Mauri and Galli, PRB 52, 1640 (1995).

Ordejon-Mauri Functional of Ordejón et al, or Mauri et al, see PRB 51, 1456 (1995). The number of localized wave functions (LWFs) used must coincide with $N_{el}/2$ (unless spin polarized). For the initial assignment of LWF centers to atoms, atoms with even number of electrons, n , get $n/2$ LWFs. Odd atoms get $(n+1)/2$ and $(n-1)/2$ in an alternating sequence, in order of appearance (controlled by the input in the atomic coordinates block).

files Reads localized-function information from a file and chooses automatically the functional to be used.

ON.MaxNumIter `1000` *(integer)*
Maximum number of iterations in the conjugate minimization of the electronic energy, in each

SCF cycle.

ON.Etol 10^{-8} (real)

Relative-energy tolerance in the conjugate minimization of the electronic energy. The minimization finishes if $2(E_n - E_{n-1})/(E_n + E_{n-1}) \leq \mathbf{ON.Etol}$.

ON.eta 0 eV (energy)

Fermi level parameter of Kim *et al.*. This should be in the energy gap, and tuned to obtain the correct number of electrons. If the calculation is spin polarised, then separate Fermi levels for each spin can be specified.

ON.eta.alpha 0 eV (energy)

Fermi level parameter of Kim *et al.* for alpha spin electrons. This should be in the energy gap, and tuned to obtain the correct number of electrons. Note that if the Fermi level is not specified individually for each spin then the same global eta will be used.

ON.eta.beta 0 eV (energy)

Fermi level parameter of Kim *et al.* for beta spin electrons. This should be in the energy gap, and tuned to obtain the correct number of electrons. Note that if the Fermi level is not specified individually for each spin then the same global eta will be used.

ON.RcLWF 9.5 Bohr (length)

Localization radius for the Localized Wave Functions (LWF's).

ON.ChemicalPotential false (logical)

Specifies whether to calculate an order- N estimate of the Chemical Potential, by the projection method (Goedecker and Teter, PRB **51**, 9455 (1995); Stephan, Drabold and Martin, PRB **58**, 13472 (1998)). This is done by expanding the Fermi function (or density matrix) at a given temperature, by means of Chebyshev polynomials, and imposing a real space truncation on the density matrix. To obtain a realistic estimate, the temperature should be small enough (typically, smaller than the energy gap), the localization range large enough (of the order of the one you would use for the Localized Wannier Functions), and the order of the polynomial expansion sufficiently large (how large depends on the temperature; typically, 50-100).

NOTE: this option does not work in parallel. An alternative is to obtain the approximate value of the chemical potential using an initial diagonalization.

ON.ChemicalPotential.Use false (logical)

Specifies whether to use the calculated estimate of the Chemical Potential, instead of the parameter **ON.eta** for the order- N energy functional minimization. This is useful if you do not know the position of the Fermi level, typically in the beginning of an order- N run.

NOTE: this overrides the value of **ON.eta** and **ON.ChemicalPotential**. Also, this option does not work in parallel. An alternative is to obtain the approximate value of the chemical potential using an initial diagonalization.

ON.ChemicalPotential.Rc 9.5 Bohr (length)

Defines the cutoff radius for the density matrix or Fermi operator in the calculation of the estimate of the Chemical Potential.

ON.ChemicalPotential.Temperature 0.05 Ry *(temperature/energy)*

Defines the temperature to be used in the Fermi function expansion in the calculation of the estimate of the Chemical Potential. To have an accurate results, this temperature should be smaller than the gap of the system.

ON.ChemicalPotential.Order 100 *(integer)*

Order of the Chebishev expansion to calculate the estimate of the Chemical Potential.

ON.LowerMemory false *(logical)*

If **true**, then a slightly reduced memory algorithm is used in the 3-point line search during the order N minimisation. Only affects parallel runs.

Output of localized wavefunctions At the end of each conjugate gradient minimization of the energy functional, the LWF's are stored on disk. These can be used as an input for the same system in a restart, or in case something goes wrong. The LWF's are stored in sparse form in file `SystemLabel.LWF`

It is important to keep very good care of this file, since the first minimizations can take MANY steps. Loosing them will mean performing the whole minimization again. It is also a good practice to save it periodically during the simulation, in case a mid-run restart is necessary.

ON.UseSaveLWF false *(logical)*

Instructs to read the localized wave functions stored in file `SystemLabel.LWF` by a previous run.

6.13 The ELSI solver family

The ELSI library provides a unified interface to a growing collection of solvers, including ELPA (diagonalization), PEXSI, OMM, density-matrix purification (using NTPoly), SIPS (based on spectrum slicing), EigenExa, and MAGMA, and ChaSE. It can be downloaded and installed freely from <http://elsi-interchange.org>.

The current interface in SIESTA is based on the “DM” mode in ELSI: sparse H and S matrices are passed to the library, and sparse DM and EDM matrices are returned. Internally, the library performs any extra operations needed. K-points and/or spin-polarization (in collinear form) are also supported, in fully parallel mode. This means that k-points/spin-polarized calculations will be carried out splitting the communicator for orbital distribution into $nk \times nspin$ copies, where nk is the number of k-points and $nspin$ the number of spin components. The total number of MPI processors must be a multiple of $nk \times nspin$.

See Sec. 2.3 for details on installing SIESTA with ELSI support.

6.13.1 Input parameters

The most important parameter is the solver to be used. Most other options are given default values by the program which are nearly always appropriate.

ELSI.Solver ELPA *(string)*

A specification of the solver wanted. Possible values are

- elpa
- omm
- pexsi
- ntpoly
- sips
- eigenexa
- magma

The EigenExa and MAGMA solvers are available after version 2.4.1 of the ELSI library, but they are optional, as are the PEXSI and SIPS solvers. Only the solvers actually supported by the ELSI library linked with SIESTA can be used.

Note that the native interface in SIESTA for an older version of the PEXSI solver is still available (`-DWITH_PEXSI=ON`).

There is also native support for the ELPA library (`-DWITH_ELPA=ON`), which can coexist with the ELSI interface if ELSI has been compiled with (the same) external ELPA library. The native OMM interface in SIESTA (enabled by default) has no coexistence problems with ELSI.

The compatibility options for libraries are handled by CMake, but some corner cases might remain.

ELSI.Broadening.Method `"fermi"` *(string)*

The scheme for level broadening. Valid options are “fermi”, “gauss”, “mp” (Methfessel-Paxton), “cold” (Marzari-Vanderbilt cold smearing).

ELSI.BuildDM.Method `"unset"` *(string)*

The scheme for building the DM/EDM matrices. It is best to leave it unspecified, except when running benchmarks or tests. Valid options are “unset” (no preference), “elpa-multiply-at-a” (not always available), “elpa-multiply” “rank-update” , “pdgemm” (not recommended, except for tests).

ELSI.Output.Level `0` *(integer)*

The level of detail provided in the output. Possible values are 0, 1, 2, and 3.

ELSI.Output.Json `1` *(integer)*

If not 0, a separate log file in JSON format will be written out.

Other fdf options are mapped in a direct way to the options described in the ELSI manual:

ELSI.Broadening.MPOrder `1` *(integer)*

ELSI.Ill-Condition.Check `0` *(integer)*

ELSI.Ill-Condition.Tolerance `10-5` *(real)*

ELSI.ELPA.Flavor `2` *(integer)*

Use the two-stage (conversion first to banded, then to tridiagonal form) of the ELPA solver by default.

ELSI.ELPA.NSinglePrecision `0` *(integer)*

Number of initial scf steps carried out in single precision (if supported by the externally compiled version of the ELPA library). Typically, most scf steps can be computed in single precision, with only a final one or two in double precision, to achieve the standard results. The use of this option increases performance significantly.

ELSI.ELPA.Autotune 0 *(integer)*

Use the auto-tune feature (if supported by the externally compiled version of the ELPA library). See the ELPA documentation for more details.

ELSI.ELPA.GPU 0 *(integer)*

Request use of GPU acceleration (if supported by the compiled version of the ELPA library).

ELSI.OMM.Flavor 0 *(integer)*

ELSI.OMM.ELPA.Steps 3 *(integer)*

ELSI.OMM.Tolerance 10^{-9} *(real)*

ELSI.PEXSI.Method 3 *(integer)*

Method for computing the poles: 1: Contour integral, 2: Minimax rational approximation, 3 (default): Adaptive Antoulas-Anderson (AAA). Please see the manual for a fuller discussion.

Support for setting this parameter was introduced in ELSI in v2.4.1. If your library does not have it, please define the preprocessor flag `-DSIESTA__ELSI_DOES_NOT_HAVE_PEXSI_METHOD` at compile time.

ELSI.PEXSI.TasksPerPole no default *(integer)*

See the discussion on the three-level PEXSI parallelism in the ELSI manual.

ELSI.PEXSI.TasksSymbolic 1 *(integer)*

ELSI.PEXSI.Number-of-Poles 20 *(integer)*

ELSI.PEXSI.Number-of-Mu-Points 2 *(integer)*

ELSI.PEXSI.Inertia-Tolerance 0.05 *(real)*

ELSI.PEXSI.Initial-Mu-Min -1.0 Ry *(real)*

Initial lower bound for μ at the beginning of the scf loop.

ELSI.PEXSI.Initial-Mu-Max 0.0 Ry *(real)*

Initial upper bound for μ at the beginning of the scf loop.

ELSI.NTPoly.Method 2 *(integer)*

ELSI.NTPoly.Filter 10^{-9} *(real)*

ELSI.NTPoly.Tolerance 10^{-6} *(real)*

ELSI.SIPS.Slices no default *(integer)*

See the discussion in the ELSI manual.

ELSI.SIPS.ELPA.Steps 2 *(integer)*

ELSI.EigenExa.Method 2 *(integer)*

Selects between the tridiagonalization (1) and pentadiagonalization (2) methods. The latter is usually faster and more scalable.

ELSI.MAGMA.Solver-Method 1 *(integer)*

Selects between the 1-stage (1) and 2-stage (2) solvers in the MAGMA library. Please see the documentation of the MAGMA library for the details about compilation and support for accelerator devices.

6.14 The CheSS solver

The CheSS solver uses an expansion based on Chebyshev polynomials to calculate the density matrix, thereby exploiting the sparsity of the overlap and Hamiltonian matrices. It works best for systems exhibiting a finite HOMO-LUMO gap and a small spectral width.

CheSS exhibits a two level parallelization using MPI and OpenMP and can scale to many thousands of cores. It can be downloaded and installed freely from <https://launchpad.net/chess> (but note that this version might not be compatible with the latest versions of the BigDFT lower-level libraries on which it depends).

Note that this is still an experimental feature. Feedback is welcome.

6.14.1 Input parameters

Usually CheSS only requires little user input, as the default values for the input parameters work in general quite well. Moreover CheSS has the capability to determine certain optimal values on its own. The only input parameters which usually require some human action are the values of the buffers required for the matrix multiplications to calculate the Chebyshev polynomials.

CheSS.Buffer.Kernel 4.0 Bohr *(length)*

Buffer for the density kernel within the CheSS calculation.

CheSS.Buffer.Mult 6.0 Bohr *(length)*

Buffer for the matrix vector multiplication within the CheSS calculation.

CheSS.Fscale 10^{-1} Ry *(energy)*

Initial guess for the error function decay length (will be adjusted automatically).

CheSS.FscaleLowerbound 10^{-2} Ry *(energy)*

Lower bound for the error function decay length.

CheSS.FscaleUpperbound 10^{-1} Ry *(energy)*

Upper bound for the error function decay length.

CheSS.evlowH -2.0 Ry *(energy)*

Initial guess for the lower bound of the eigenvalue spectrum of the Hamiltonian matrix, will be adjusted automatically if chosen improperly.

CheSS.evhighH 2.0 Ry *(energy)*

Initial guess for the upper bound of the eigenvalue spectrum of the Hamiltonian matrix, will be adjusted automatically if chosen improperly.

CheSS.evlowS 0.5 *(real)*

Initial guess for the lower bound of the eigenvalue spectrum of the overlap matrix, will be adjusted automatically if chosen improperly.

CheSS.evhighS 1.5 *(real)*

Initial guess for the upper bound of the eigenvalue spectrum of the overlap matrix, will be adjusted automatically if chosen improperly.

6.15 The PEXSI solver (native interface)

The PEXSI solver is based on the combination of the pole expansion of the Fermi-Dirac function and the computation of only a selected (sparse) subset of the elements of the matrices $(H - z_l S)^{-1}$ at each pole z_l .

This solver can efficiently use the sparsity pattern of the Hamiltonian and overlap matrices generated in SIESTA, and for large systems has a much lower computational complexity than that associated with the matrix diagonalization procedure. It is also highly scalable.

This section applies to the native interface to the PEXSI library. There is another PEXSI interface offered through the ELSI library of solvers (see Section 6.13).

The PEXSI technique can be used in this version of SIESTA to evaluate the electron density, free energy, atomic forces, density of states and local density of states without computing any eigenvalue or eigenvector of the Kohn-Sham Hamiltonian. It can achieve an accuracy fully comparable to that obtained from a matrix diagonalization procedure for general systems, including metallic systems at low temperature.

The current implementation of the PEXSI solver in SIESTA makes use of a full fine-grained-level interface to the PEXSI library (<http://pexsi.org>), and can deal with (collinear) spin-polarization, but it is still restricted to Γ -point calculations. The PEXSI interface offered by ELSI offers some more options, although not currently the density-of-states calculation.

The following is a brief description of the input-file parameters relevant to the workings of the (native interface) PEXSI solver. For more background, including a discussion of the conditions under which this solver is competitive, the user is referred to the paper Lin et al.^[10], and references therein.

The technology involved in the PEXSI solver can also be used to compute densities of states and “local densities of states”. These features are documented in this section and also linked to in the relevant general sections.

6.15.1 Pole handling

Note that the temperature for the Fermi-Dirac distribution which is pole-expanded is taken directly from the **ElectronicTemperature** parameter (see Sec. 6.12.3).

PEXSI.NumPoles 40 *(integer)*

Effective number of poles used to expand the Fermi-Dirac function.

When using the pole-generation method used in this legacy interface (contour integral), the allowed values for NumPoles are: 10, 20, 30, ..., 110, and 120. Typically 60 to 100 poles are needed to get an accuracy comparable to diagonalization.

PEXSI.deltaE 3 Ry *(energy)*

In principle **PEXSI.deltaE** should be $E_{\max} - \mu$, where $E_{\max}$ is the largest eigenvalue for (H, S) , and μ is the chemical potential. However, due to the fast decay of the Fermi-Dirac function, **PEXSI.deltaE** can often be chosen to be much lower. In practice we set the default to be 3 Ryd. This number should be set to be larger if the difference between $\text{Tr}[H \cdot \text{DM}]$ and $\text{Tr}[S \cdot \text{EDM}]$ (displayed in the output if **PEXSI.Verbosity** is at least 2) does not decrease with the increase of the number of poles.

PEXSI.Gap 0 Ry *(energy)*

Spectral gap. This can be set to be 0 in most cases.

6.15.2 Parallel environment and control options

MPI.Nprocs.SIESTA $\langle \text{total processors} \rangle$ *(integer)*

Specifies the number of MPI processes to be used in those parts of the program (such as Hamiltonian setup and computation of forces) which are outside of the PEXSI solver itself. This is needed in large-scale calculations, for which the number of processors that can be used by the PEXSI solver is much higher than those needed by other parts of the code.

Note that when the PEXSI solver is not used, this parameter will simply reduce the number of processors actually used by all parts of the program, leaving the rest idle for the whole calculation. This will adversely affect the computing budget, so take care not to use this option in that case.

PEXSI.NP-per-pole 4 *(integer)*

Number of MPI processes used to perform the PEXSI computations in one pole. If the total number of MPI processes is smaller than this number times the number of poles (times the spin multiplicity), the PEXSI library will compute appropriate groups of poles in sequence. The minimum time to solution is achieved by increasing this parameter as much as it is reasonable for parallel efficiency, and using enough MPI processes to allow complete parallelization over poles. On the other hand, the minimum computational cost (in the sense of computing budget) is obtained by using the minimum value of this parameter which is compatible with the memory footprint. The additional parallelization over poles will be irrelevant for cost, but it will obviously affect the time to solution.

Internally, SIESTA computes the processor grid parameters **nproc** and **nproc1** for the PEXSI library, with **nproc** $\geq$ **nproc1**, and as similar as possible. So it is best to choose **PEXSI.NP-per-pole** as the product of two similar numbers.

NOTE: The total number of MPI processes must be divisible by **PEXSI.NP-per-pole**. In case of spin-polarized calculations, the total number of MPI processes must be divisible by **PEXSI.NP-per-pole** times 2.

PEXSI.Ordering 1 *(integer)*

For large matrices, symbolic factorization should be performed in parallel to reduce the wall clock time. This can be done using ParMETIS/PT-Scotch by setting **PEXSI.Ordering** to

0. However, we have been experiencing some instability problem of the symbolic factorization phase when ParMETIS/PT-Scotch is used. In such case, for relatively small matrices one can either use the sequential METIS (**PEXSI.Ordering** = 1) or set **PEXSI.NP-symbfact** to 1.

PEXSI.NP-symbfact 1 *(integer)*

Number of MPI processes used to perform the symbolic factorizations needed in the PEXSI procedure. A default value should be given to reduce the instability problem. From experience so far setting this to be 1 is most stable, but going beyond 64 does not usually improve much.

PEXSI.Verbosity 1 *(integer)*

It determines the amount of information logged by the solver in different places. A value of zero gives minimal information.

- In the files logPEXSI[0-9]+, the verbosity level is interpreted by the PEXSI library itself. In the latest version, when PEXSI is compiled in RELEASE mode, only logPEXSI0 is given in the output. This is because we have observed that simultaneous output for all processors can have very significant cost for a large number of processors (>10000).
- In the SIESTA output file, a verbosity level of 1 and above will print lines (prefixed by &o) indicating the various heuristics used at each scf step. A verbosity level of 2 and above will print extra information.

The design of the output logging is still in flux.

6.15.3 Electron tolerance and the PEXSI solver

PEXSI.num-electron-tolerance 10^{-4} *(real)*

Tolerance in the number of electrons for the PEXSI solver. At each iteration of the solver, the number of electrons is computed as the trace of the density matrix times the overlap matrix, and compared with the total number of electrons in the system. This tolerance can be fixed, or dynamically determined as a function of the degree of convergence of the self-consistent-field loop.

PEXSI.num-electron-tolerance-lower-bound 10^{-2} *(real)*

See **PEXSI.num-electron-tolerance-upper-bound**.

PEXSI.num-electron-tolerance-upper-bound 0.5 *(real)*

The upper and lower bounds for the electron tolerance are used to dynamically change the tolerance in the PEXSI solver, following the simple algorithm:

```
tolerance = Max(lower_bound, Min(dDmax, upper_bound))
```

The first scf step uses the upper bound of the tolerance range, and subsequent steps use progressively lower values, in correspondence with the convergence-monitoring variable **dDmax**.

NOTE: This simple update schedule tends to work quite well. There is an experimental algorithm, documented only in the code itself, which allows a finer degree of control of the tolerance update.

PEXSI.mu-max-iter 10 *(integer)*

Maximum number of iterations of the PEXSI solver. Note that in this implementation there is no fallback procedure if the solver fails to converge in this number of iterations to the prescribed

tolerance. In this case, the resulting density matrix might still be re-normalized, and the calculation able to continue, if the tolerance for non normalized DMs is not set too tight. For example,

```
# (true_no_electrons/no_electrons) - 1.0
DM.NormalizationTolerance 1.0e-3
```

will allow a 0.1% error in the number of electrons. For obvious reasons, this feature, which is also useful in connection with the dynamic tolerance update, should not be abused.

If the parameters of the PEXSI solver are adjusted correctly (including a judicious use of inertia-counting to refine the μ bracket), we should expect that the maximum number of solver iterations needed is around 3

PEXSI.mu -0.6 Ry *(energy)*

The starting guess for the chemical potential for the PEXSI solver. Note that this value does not affect the initial μ bracket for the inertia-count refinement, which is controlled by **PEXSI.mu-min** and **PEXSI.mu-max**. After an inertia-count phase, μ will be reset, and further iterations inherit this estimate, so this parameter is only relevant if there is no inertia-counting phase.

PEXSI.mu-pexsi-safeguard 0.05 Ry *(energy)*

NOTE: This feature has been deactivated for now. The condition for starting a new phase of inertia-counting is that the Newton estimation falls outside the current bracket. The bracket is expanded accordingly.

The PEXSI solver uses Newton's method to update the estimate of μ . If the attempted change in μ is larger than **PEXSI.mu-pexsi-safeguard**, the solver cycle is stopped and a fresh phase of inertia-counting is started.

6.15.4 Inertia-counting

PEXSI.Inertia-Counts 3 *(integer)*

In a given scf step, the PEXSI procedure can optionally employ a μ bracket-refinement procedure based on inertia-counting. Typically, this is used only in the first few scf steps, and this parameter determines how many. If positive, inertia-counting will be performed for exactly that number of scf steps. If negative, inertia-counting will be performed for at least that number of scf steps, and then for as long as the scf cycle is not yet deemed to be near convergence (as determined by the **PEXSI.safe-dDmax-no-inertia** parameter).

NOTE: Since it is cheaper to perform an inertia-count phase than to execute one iteration of the solver, it pays to call the solver only when the μ bracket is sufficiently refined.

PEXSI.mu-min -1 Ry *(energy)*

The lower bound of the initial range for μ used in the inertia-count refinement. In runs with multiple geometry iterations, it is used only for the very first scf iteration at the first geometry step. Further iterations inherit possibly refined values of this parameter.

PEXSI.mu-max 0 Ry *(energy)*

The upper bound of the initial range for μ used in the inertia-count refinement. In runs with multiple geometry iterations, it is used only for the very first scf iteration at the first geometry step. Further iterations inherit possibly refined values of this parameter.

PEXSI.safe-dDmax-no-inertia 0.05 *(real)*

During the scf cycle, the variable conventionally called **dDmax** monitors how far the cycle is from convergence. If **PEXSI.Inertia-Counts** is negative, an inertia-counting phase will be performed in a given scf step for as long as **dDmax** is greater than **PEXSI.safe-dDmax-no-inertia**.

NOTE: Even though **dDmax** represents historically how far from convergence the density-matrix is, the same mechanism applies to other forms of mixing in which other magnitudes are monitored for convergence (Hamiltonian, charge density...).

PEXSI.lateral-expansion-inertia 3 eV *(energy)*

If the correct μ is outside the bracket provided to the inertia-counting phase, the bracket is expanded in the appropriate direction(s) by this amount.

PEXSI.Inertia-mu-tolerance 0.05 Ry *(energy)*

One of the criteria for early termination of the inertia-counting phase. The value of the estimated μ (basically the center of the resulting brackets) is monitored, and the cycle stopped if its change from one iteration to the next is below this parameter.

PEXSI.Inertia-max-iter 5 *(integer)*

Maximum number of inertia-count iterations per cycle.

PEXSI.Inertia-min-num-shifts 10 *(integer)*

Minimum number of sampling points for inertia counts.

PEXSI.Inertia-energy-width-tolerance $\langle \text{PEXSI.Inertia-mu-tolerance} \rangle$ *(energy)*

One of the criteria for early termination of the inertia-counting phase. The cycle stops if the width of the resulting bracket is below this parameter.

6.15.5 Re-use of μ information accross iterations

This is an important issue, as the efficiency of the PEXSI procedure depends on how close a guess of μ we have at our disposal. There are two types of information re-use:

- Bracketing information used in the inertia-counting phase.
- The values of μ itself for the solver.

PEXSI.safe-width-ic-bracket 4 eV *(energy)*

By default, the μ bracket used for the inertia-counting phase in scf steps other than the first is taken as an interval of width **PEXSI.safe-width-ic-bracket** around the latest estimate of μ .

PEXSI.safe-dDmax-ef-inertia 0.1 *(real)*

The change in μ from one scf iteration to the next can be crudely estimated by assuming that the change in the band structure energy (estimated as $\text{Tr}\Delta H_{\text{DM}}$) is due to a rigid shift. When the scf cycle is near convergence, this $\Delta\mu$ can be used to estimate the new initial bracket for the inertia-counting phase, rigidly shifting the output bracket from the previous scf step. The cycle is assumed to be near convergence when the monitoring variable **dDmax** is smaller than **PEXSI.safe-dDmax-ef-inertia**.

NOTE: Even though **dDmax** represents historically how far from convergence the density-matrix is, the same mechanism applies to other forms of mixing in which other magnitudes are monitored for convergence (Hamiltonian, charge density...).

NOTE: This criterion will lead in general to tighter brackets than the previous one, but oscillations in H in the first few iterations might make it more dangerous. More information from real use cases is needed to refine the heuristics in this area.

PEXSI.safe-dDmax-ef-solver 0.05 *(real)*

When the scf cycle is near convergence, the $\Delta\mu$ estimated as above can be used to shift the initial guess for μ for the PEXSI solver. The cycle is assumed to be near convergence when the monitoring variable **dDmax** is smaller than **PEXSI.safe-dDmax-ef-solver**.

NOTE: Even though **dDmax** represents historically how far from convergence the density-matrix is, the same mechanism applies to other forms of mixing in which other magnitudes are monitored for convergence (Hamiltonian, charge density...).

PEXSI.safe-width-solver-bracket 4 eV *(energy)*

In all cases, a “safe” bracket around μ is provided even in direct calls to the PEXSI solver, in case a fallback to executing internally a cycle of inertia-counting is needed. The size of the bracket is given by **PEXSI.safe-width-solver-bracket**

6.15.6 Calculation of the density of states by inertia-counting

The cumulative or integrated density of states (INTDOS) can be easily obtained by inertia-counting, which involves a factorization of $H - \sigma S$ for varying σ (see SIESTA-PEXSI paper). Apart from the DOS-specific options below, the “ordering”, “symbolic factorization”, and “pole group size” (re-interpreted as the number of MPI processes dealing with a given σ) options are honored.

The current version of the code generates a file with the energy-INTDOS information, **PEXSI_INTDOS**, which can be later processed to generate the DOS by direct numerical differentiation, or a SIESTA-style **SystemLabel.EIG** file (using the **Util/PEXSI/intdos2eig** program).

PEXSI.DOS false *(logical)*

Whether to compute the DOS (actually, the INTDOS — see above) using the PEXSI technology.

PEXSI.DOS.Emin -1 Ry *(energy)*

Lower bound of energy window to compute the DOS in.

See **PEXSI.DOS.Ef.Reference**.

PEXSI.DOS.Emax 1 Ry *(energy)*

Upper bound of energy window to compute the DOS in.

See **PEXSI.DOS.Ef.Reference**.

PEXSI.DOS.Ef.Reference true *(logical)*

If this flag is true, the bounds of the energy window (**PEXSI.DOS.Emin** and **PEXSI.DOS.Emax**) are with respect to the Fermi level.

PEXSI.DOS.NPoints 200 *(integer)*

The number of points in the energy interval at which the DOS is computed. It is rounded up

to the nearest multiple of the number of available factorization groups, as the operations are perfectly parallel and there will be no extra cost involved.

6.15.7 Calculation of the LDOS by selected-inversion

The local-density-of-states (LDOS) around a given reference energy ε , representing the contribution to the charge density of the states with eigenvalues in the vicinity of ε , can be obtained formally by a “one-pole expansion” with suitable broadening (see SIESTA-PEXSI paper).

Apart from the LDOS-specific options below, the “ordering”, “verbosity”, and “symbolic factorization” options are honored.

The current version of the code generates a real-space grid file with extension `SystemLabel.LDOS`, and (if netCDF is compiled-in) a file `LDOS.grid.nc`.

NOTE: The LDOS computed with this procedure is not exactly the same as the vanilla SIESTA LDOS, which uses an explicit energy interval. Here the broadening acts around a single value of the energy.

PEXSI.LDOS `false` *(logical)*

Whether to compute the LDOS using the PEXSI technology.

NOTE: this flag is not compatible with **LocalDensityOfStates**.

PEXSI.LDOS.Energy `0 Ry` *(energy)*

The (absolute) energy at which to compute the LDOS.

PEXSI.LDOS.Broadening `0.01 Ry` *(energy)*

The broadening parameter for the LDOS.

PEXSI.LDOS.NP-per-pole `<PEXSI.NP-per-pole>` *(integer)*

The value of this parameter supersedes **PEXSI.NP-per-pole** for the calculation of the LDOS, which otherwise would keep idle all but **PEXSI.NP-per-pole** MPI processes, as it essentially consists of a “one-pole” procedure.

6.16 Band-structure analysis

This calculation of the band structure is performed optionally after the geometry loop finishes, and the output information written to the `SystemLabel.bands` file (see below for the format).

BandLinesScale `pi/a` *(string)*

Specifies the scale of the k vectors given in **BandLines** and **BandPoints** below. The options are:

pi/a k -vector coordinates are given in Cartesian coordinates, in units of π/a , where a is the lattice constant. NOTE: this results in a k -cell that is orthorhombic, so it might not be useful for systems in which the unit cell itself is non-orthorhombic.

ReciprocalLatticeVectors k vectors are given in reciprocal-lattice-vector coordinates

NOTE: you might need to define explicitly a `LatticeConstant` tag in your `fdf` file if you do not

already have one, and make it consistent with the scale of the k-points and any unit-cell vectors you might have already defined.

%block BandLines *<None>* *(block)*

Specifies the lines along which band energies are calculated (usually along high-symmetry directions). An example for an FCC lattice is:

```
%block BandLines
  1  1.000  1.000  1.000  L          # Begin at L
 20  0.000  0.000  0.000  \Gamma    # 20 points from L to gamma
 25  2.000  0.000  0.000  X          # 25 points from gamma to X
 30  2.000  2.000  2.000  \Gamma    # 30 points from X to gamma
%endblock BandLines
```

where the last column is an optional L^AT_EX label for use in the band plot. If only given points (not lines) are required, simply specify 1 in the first column of each line. The first column of the first line must be always 1.

NOTE: this block is not used if **BandPoints** is present.

%block BandPoints *<None>* *(block)*

Band energies are calculated for the list of arbitrary k points given in the block. Units defined by **BandLinesScale** as for **BandLines**. The generated **SystemLabel.bands** file will contain the k point coordinates (in a.u.) and the corresponding band energies (in eV). Example:

```
%block BandPoints
  0.000  0.000  0.000  # This is a comment. eg this is gamma
  1.000  0.000  0.000
  0.500  0.500  0.500
%endblock BandPoints
```

See also **BandLines**.

WriteKbands *false* *(logical)*

If **true**, it writes the coordinates of the $\vec{k}$ vectors defined for band plotting, to the main output file.

WriteBands *false* *(logical)*

If **true**, it writes the Hamiltonian eigenvalues corresponding to the $\vec{k}$ vectors defined for band plotting, in the main output file.

6.16.1 Format of the .bands file

```
FermiEnergy (all energies in eV) \\
kmin, kmax (along the k-lines path, i.e. range of k in the band plot) \\
Emin, Emax (range of all eigenvalues) \\
NumberOfBands, NumberOfSpins (1 or 2), NumberOfkPoints \\
k1, ((ek(iband,ispin,1),iband=1,NumberOfBands),ispin=1,NumberOfSpins) \\
k2, ek \\
. \\
. \\
```



```

. \\
klast, ek \\
NumberOfkLines \\
kAtBegOfLine1, kPointLabel \\
kAtEndOfLine1, kPointLabel \\
. \\
. \\
. \\
kAtEndOfLastLine, kPointLabel \\

```

The `gnubands` postprocessing utility program (found in the `Util/Bands` directory) reads the `SystemLabel.bands` for plotting. See the **BandLines** data descriptor above for more information.

6.16.2 Output of wavefunctions associated to bands

The user can optionally request that the wavefunctions corresponding to the computed bands be written to file. They are written to the `SystemLabel.bands.WFSX` file. The relevant options are:

WFS.Write.For.Bands `false` *(logical)*

Instructs the program to compute and write the wave functions associated to the bands specified (by a **BandLines** or a **BandPoints** block) to the file `SystemLabel.WFSX`.

The information in this file might be useful, among other things, to generate “fatbands” plots, in which both band eigenvalues and information about orbital projections is presented. See the `fat` program in the `Util/COOP` directory for details.

WFS.Band.Min `1` *(integer)*

Specifies the lowest band index of the wave-functions to be written to the file `SystemLabel.WFSX` for each k -point (all k -points in the band set are affected).

WFS.Band.Max `number of orbitals` *(integer)*

Specifies the highest band index of the wave-functions to be written to the file `SystemLabel.WFSX` for each k -point (all k -points in the band set are affected).

6.17 Output of selected wavefunctions

The user can optionally request that specific wavefunctions are written to file. These wavefunctions are re-computed after the geometry loop (if any) finishes, using the last (presumably converged) density matrix produced during the last self-consistent field loop (after a final mixing). They are written to the `SystemLabel.selected.WFSX` file.

Note that the complete set of wavefunctions obtained during the last iteration of the SCF loop will be written to `SystemLabel.fullBZ.WFSX` if the **COOP.Write** option is in effect.

Note that the complete set of wavefunctions obtained during the last iteration of the SCF loop will be written to a NetCDF file `WFS.nc` if the **Diag.UseNewDiagk** option is in effect.

WaveFuncKPointsScale `pi/a` *(string)*

Specifies the scale of the k vectors given in **WaveFuncKPoints** below. The options are:

pi/a k-vector coordinates are given in Cartesian coordinates, in units of π/a , where a is the lattice constant

ReciprocalLatticeVectors k vectors are given in reciprocal-lattice-vector coordinates

%block WaveFuncKPoints **(None)** *(block)*

Specifies the k -points at which the electronic wavefunction coefficients are written. An example for an FCC lattice is:

```
%block WaveFuncKPoints
0.000 0.000 0.000 from 1 to 10 # Gamma wavefuncs 1 to 10
2.000 0.000 0.000 1 3 5       # X wavefuncs 1,3 and 5
1.500 1.500 1.500             # K wavefuncs, all
%endblock WaveFuncKPoints
```

The index of a wavefunction is defined by its energy, so that the first one has lowest energy.

The user can also narrow the energy-range used with the **WFS.Energy.Min** and **WFS.Energy.Max** options (both take an energy (with units) as extra argument – see section 6.19.3). Care should be taken to make sure that the actual values of the options make sense.

The output of the wavefunctions is described in Section 6.17.

WriteWaveFunctions **false** *(logical)*

If **true**, it writes to the output file a list of the wavefunctions actually written to the **SystemLabel.selected.WFSX** file, which is always produced.

The unformatted WFSX file contains the information of the k -points for which wavefunctions coefficients are written, and the energies and coefficients of each wavefunction which was specified in the input file (see **WaveFuncKPoints** descriptor above). It also contains information on the atomic species and the orbitals for postprocessing purposes.

NOTE: The **SystemLabel.WFSX** file is in a more compact form than the old WFS, and the wavefunctions are output in single precision. The **Util/WFS/wfsx2wfs** program can be used to convert to the old format.

The **readwf** and **readwfsx** postprocessing utilities programs (found in the **Util/WFS** directory) read the **SystemLabel.WFS** or **SystemLabel.WFSX** files, respectively, and generate a readable file.

6.18 Density of states

6.18.1 Total density of states

There are several options to obtain the total density of states:

- The Hamiltonian eigenvalues for the SCF sampling $\vec{k}$ points can be dumped into **SystemLabel.EIG** in a format analogous to **SystemLabel.bands**, but without the **kmin**, **kmax**, **emin**, **emax** information, and without the abscissa. The **Eig2DOS** postprocessing utility can be then used to obtain the density of states. See the **WriteEigenvalues** descriptor.
- As a side-product of a partial-density-of-states calculation (see below)

- As one of the files produced by the `Util/COOP/mprop` during the off-line analysis of the electronic structure. This method allows the flexibility of specifying energy ranges and resolutions at will, without re-running SIESTA See Sec. 6.19.3.
- Using the inertia-counting routines in the PEXSI solver (see Sec. 6.15.6).

The k-point specification for the partial and local density of states calculations described in the following two sections may optionally be given by

DOS.kgrid.? kgrid.?

The generic DOS k-grid specification.

See Sec. 6.5 for details. If *any* of **DOS.kgrid.MonkhorstPack**, **DOS.kgrid.Cutoff** or **DOS.kgrid.File** is present, they will be used, otherwise fall back to the SCF k-point sampling (**kgrid.?**).

NOTE: **DOS.kgrid.?** options are the default values for **ProjectedDensityOfStates** and **LocalDensityOfStates**, but they do not affect the sampling used to generate the `SystemLabel.EIG` file. This feature might be implemented in a later version.

6.18.2 Partial (projected) density of states

There are two options to obtain the partial density of states

- Using the options below
- Using the `Util/COOP/mprop` program for the off-line analysis of the electronic structure in PDOS mode. This method allows the flexibility of specifying energy ranges, orbitals, and resolutions at will, without re-running SIESTA. See Sec. 6.19.3.

%block ProjectedDensityOfStates `<None>` *(block)*

Instructs to write the Total Density Of States (Total DOS) and the Projected Density Of States (PDOS) on the basis orbitals, between two given energies, in files `SystemLabel.DOS` and `SystemLabel.PDOS`, respectively. The block must be a single line with the energies of the range for PDOS projection, (relative to the program's zero, i.e. the same as the eigenvalues printed by the program), the peak width (an energy) for broadening the eigenvalues, the number of points in the energy window, and the energy units. An example is:

```
%block ProjectedDensityOfStates
-20.00 10.00 0.200 500 eV
%endblock ProjectedDensityOfStates
```

Optionally one may start the line with `EF` as this:

```
%block ProjectedDensityOfStates
EF -20.00 10.00 0.200 500 eV
%endblock ProjectedDensityOfStates
```

This specifies the energies with respect to the Fermi-level.

The broadening of the states is the Gaussian distribution with the peak width being w :

$$f(E) = \frac{1}{w\sqrt{\pi}} \exp \left[- \left(\frac{E - \epsilon}{w} \right)^2 \right], \quad (14)$$

where ϵ is the eigenvalue of the state. Note that the peak width is equivalent to $\sigma\sqrt{2} = w$, with σ being the standard deviation.

By default the projected density of states is generated for the same grid of points in reciprocal space as used for the SCF calculation. However, a separate set of K-points, usually on a finer grid, can be generated by using **PDOS.kgrid.?** Note that if a gamma point calculation is being used in the SCF part, especially as part of a geometry optimisation, and this is then to be run with a grid of K-points for the PDOS calculation it is more efficient to run the SCF phase first and then restart to perform the PDOS evaluation using the density matrix saved from the SCF phase.

NOTE: the two energies of the range must be ordered, with lowest first.

The total DOS is stored in a file called **SystemLabel.DOS**. The format of this file is:

```
Energy value, Total DOS (spin up), Total DOS (spin down)
```

The Projected Density Of States for all the orbitals in the unit cell is dumped sequentially into a file called **SystemLabel.PDOS**. This file is structured using spacing and xml tags. A machine-readable (but not very human readable) xml file **SystemLabel.PDOS.xml** is also produced. Both can be processed by the program in **Util/pdosxml**. The **SystemLabel.PDOS** file can be processed by utilities in **Util/Contrib/APostnikov**.

In all cases, the units for the DOS are (number of states/eV), and the Total DOS, $g(\epsilon)$, is normalized as follows:

$$\int_{-\infty}^{\infty} g(\epsilon) d\epsilon = \text{number of basis orbitals in unit cell} \quad (15)$$

PDOS.kgrid.? **<DOS.kgrid.?**

This is PDOS only specification for the k-points. I.e. if one wishes to use a specific k-point sampling. These options are equivalent to the **kgrid.Cutoff**, **kgrid.MonkhorstPack** and **kgrid.File** options. Refer to them for additional details.

If **PDOS.kgrid.?** does not exist, then **DOS.kgrid.?** is checked, and if that does not exist then **kgrid.?** options are used.

6.18.3 Local density of states

The LDOS is formally the DOS weighted by the amplitude of the corresponding wavefunctions at different points in space, and is then a function of energy and position. SIESTA can output the LDOS integrated over a range of energies. This information can be used to obtain simple STM images in the Tersoff-Hamann approximation (See **Util/STM/simple-stm**).

%block LocalDensityOfStates **<(None)** *(block)*

Instructs to write the LDOS, integrated between two given energies, at the mesh used by DHSCF, in file **SystemLabel.LDOS**. This file can be read by routine IORHO, which may be used by an application program in later versions. The block must be a single line with the energies of the range for LDOS integration (relative to the program's zero, i.e. the same as the eigenvalues printed by the program) and their units. An example is:

```
%block LocalDensityOfStates
-3.50 0.00 eV
%endblock LocalDensityOfStates
```

One may optionally write `EF` as the first word to specify that the energies are with respect to the Fermi level

```
%block LocalDensityOfStates
EF -3.50 0.00 eV
%endblock LocalDensityOfStates
```

would calculate the LDOS from -3.5 eV below the Fermi-level up to the Fermi-level.

One may use **LDOS.kgrid.?** to fine-tune the k-point sampling in the LDOS calculation.

NOTE: the two energies of the range must be ordered, with lowest first.

NOTE: this flag is not compatible with **PEXSI.LDOS**.

If netCDF support is compiled in, the file `LDOS.grid.nc` is produced.

LDOS.kgrid.? <DOS.kgrid.?

This is LDOS only specification for the k-points. I.e. if one wishes to use a specific k-point sampling. These options are equivalent to the **kgrid.Cutoff**, **kgrid.MonkhorstPack** and **kgrid.File** options. Refer to them for additional details.

If **LDOS.kgrid.?** does not exist, then **DOS.kgrid.?** is checked, if that does not exist then **kgrid.?** are used.

6.19 Options for chemical analysis

6.19.1 Charge populations

Charge populations are ways to investigate the chemical nature of the environment the atoms reside in.

In SIESTA a number of methods exists. Be warned, that they will generally not be equal. For instance, Mulliken has a basis-set dependence, while the others are likely more stable with respect to basis, but still not *correct*.

The real problem in this, is how to distribute an electron to atoms. Should it be split among them, evenly, how etc. This is no trivial task, and still newer methods are researched.

Please try and read the existing litterature for each of the methods before trusting its value for chemical insights, they all have limitations and biases.

For all atomic charge populations, there is a common output format as described below. Currently, Hirshfeld and Voronoi can only perform atomic charge populations, and hence they default to this output format. Mulliken can provide extended details (orbital-orbital charges). As such Mulliken will generally output the format as described below, but also additional formats.

The common atomic output format is exemplified here:

```
Mulliken Atomic Populations:
Atom #   delta [q] valence [e]         S         Sx         Sy         Sz Species
   1     0.093470   7.906530   3.089994   0.162048   0.000000   3.085742 fe_nc
   2    -0.186941   8.186941   1.355661   1.355661   0.000000   0.000000 fe_nc
   3     0.093471   7.906529   3.089994   0.162048  -0.000000  -3.085742 fe_nc
-----
Total                                1.679757   1.679757   0.000000  -0.000000
```

The fields are described as this:

Atom # The index of the atom.

delta [q] The net charge of the atom, with respect to the valence charge of the pseudopotential.

valence [e] The valence charge of the atom.

Adding the charge and valence (this and the former column) will be equal to the valence charge of the lone atom, as specified in the pseudopotential.

S The length of the magnetic moment vector.

Only shown for non-collinear calculations.

Sx The x component of the magnetic moment vector.

Only shown for non-collinear calculations.

Sy The y component of the magnetic moment vector.

Only shown for non-collinear calculations.

Sz The z component of the magnetic moment vector.

Shown for polarized or non-collinear calculations.

Species The species name of the atom, equal to that specified in the **AtomicCoordinatesAndAtomicSpecies**.

Total Finally, the total spin population is shown if there is a spin component in the calculation.

Charge.Mulliken none

(*str*)

deprecates: **WriteMullikenPop**

Specify when the Mulliken analysis will be calculated.

The Mulliken analysis uses the density-matrix (which by definition contains information integrated up to the Fermi level) and the overlap matrix.

NOTE: this population analysis depends on the basis set.

none Disable the calculation of the Mulliken population.

init At the start of the geometry step.

geometry At the end of every geometry step.

scf After every SCF step (can create a lot of output).

For **SCF.Mix density** this will use the just calculated density matrix after the diagonalization step.

Useful when dealing with SCF problems, This can cause very verbose output files.

scf-after-mix After mixing.

For **SCF.Mix density** this will use the just mixed density matrix.

Useful when dealing with SCF problems, This can cause very verbose output files.

end At the end of the calculation.

One can combine various levels of print-outs to get more details during the calculation.

For instance, to get the Mulliken charge at the end of every geometry step and during the SCF steps, then use the following option:

`Charge.Mulliken scf+geometry`

Mulliken charges are sometimes used to estimate the “net charge” on an atom, which is an ill-defined concept to begin with. In addition, this method gives results that depend on the basis set used. For alternative ways to estimate the atomic charges, see 6.19.1 on Voronoi and Hirshfeld charges.

Atom-based Mulliken overlaps are useful to estimate the level of chemical interaction among two atoms.

For a finer analysis of the chemical bonding, it is advised to employ the COOP/COHP curves (see 6.19.3).

Charge.Mulliken.Format `<WriteMullikenPop>` *(int)*

depends on: **Charge.Mulliken**

deprecates: **WriteMullikenPop**

It determines the level of Mulliken analysis performed. This uses the density-matrix (which by definition contains information integrated up to the Fermi level) and the overlap matrix.

NOTE: when **Charge.Mulliken** is not **none**, it defaults to **1**.

Values accepted are:

0 none

1 Prints the atomic and orbital charges.

2 In addition to the Mulliken charges, it prints the Mulliken overlap populations, grouped by atom.

3 In addition to the Mulliken charges, and the atom-grouped overlaps, it prints the overlaps orbital by orbital (this can be quite verbose).

The order of the orbitals in the population lists is defined by the order of atoms. For each atom, populations for PAO orbitals and double- ζ , triple- ζ , etc... derived from them are displayed first for all the angular momenta. Then, populations for perturbative polarization orbitals are written. Within a l -shell be aware that the order is not conventional, being y, z, x for p orbitals, and xy, yz, z^2, xz , and $x^2 - y^2$ for d orbitals.

Atom-based Mulliken overlaps are useful to estimate the level of chemical interaction among two atoms.

For a finer analysis of the chemical bonding, it is advised to employ the COOP/COHP curves (see 6.19.3).

Charge.Hirshfeld `none` *(str)*

deprecates: **Write.HirshfeldPop**

Specify when the Hirshfeld analysis will be calculated.

The Hirshfeld analysis uses the fractional of atomic charge for the lone atoms with respect to the environment charge located at the same point in space.

For a definition of the Hirshfeld charges, see Hirshfeld, Theo Chem Acta **44**, 129 (1977) and Fonseca et al, J. Comp. Chem. **25**, 189 (2003). Hirshfeld charges are more reliable than Mulliken charges, specially for large basis sets. Value (**dQatom**) is the total net charge of the atom: the variation from the neutral charge, in units of $|e|$: positive (negative) values indicate deficiency (excess) of electrons in the atom.

The Hirshfeld charge can be requested at these invocations:

none Disable the calculation of the Hirshfeld population.

geometry At the end of every geometry step.

scf After every SCF step (can create a lot of output).

This will use the *input* density matrix for the charge analysis.

Useful when dealing with SCF problems, This can cause very verbose output files.

end At the end of the calculation.

See the discussion for the common atomic output format for details on output.

Charge.Voronoi none

(*str*)

deprecates: **Write.VoronoiPop**

Specify when the Voronoi analysis will be calculated.

The Voronoi “net” atomic populations on each atom in the system. For a definition of the Voronoi charges, see Bickelhaupt et al, Organometallics **15**, 2923 (1996) and Fonseca et al, J. Comp. Chem. **25**, 189 (2003). Voronoi charges are more reliable than Mulliken charges, specially for large basis sets. Value (**dQatom**) is the total net charge of the atom: the variation from the neutral charge, in units of $|e|$: positive (negative) values indicate deficiency (excess) of electrons in the atom.

The Voronoi charge can be requested at these invocations:

none Disable the calculation of the Voronoi population.

geometry At the end of every geometry step.

scf After every SCF step (can create a lot of output).

This will use the *input* density matrix for the charge analysis.

Useful when dealing with SCF problems, This can cause very verbose output files.

end At the end of the calculation.

See the discussion for the common atomic output format for details on output.

6.19.2 Deprecated population flags

WriteMullikenPop 0

(*integer*)

deprecated by: **Charge.Mulliken**, **Charge.Mulliken.Format**

See **Charge.Mulliken** and **Charge.Mulliken.Format** for details.

MullikenInSCF false

(*logical*)

deprecated by: **Charge.Mulliken scf**

See **Charge.Mulliken** for details.

SpinInSCF true

(*logical*)

deprecated by: **Charge.Spin scf**

If true, the size and components of the (total) spin polarization will be printed at every SCF step. This is analogous to the **MullikenInSCF** feature. Enabled by default for calculations involving spin.

Write.HirshfeldPop false

(*logical*)

deprecated by: Charge.Hirshfeld

See **Charge.Hirshfeld** for details.

Write.VoronoiPop false

(logical)

deprecated by: Charge.Voronoi

See **Charge.Voronoi** for details.

PartialChargesAtEveryGeometry false

(logical)

deprecated by: Charge.Voronoi, Charge.Hirshfeld

The Hirshfeld and Voronoi populations are computed after self-consistency is achieved, for all the geometry steps.

PartialChargesAtEverySCFStep false

(logical)

deprecated by: Charge.Voronoi, Charge.Hirshfeld

The Hirshfeld and Voronoi populations are computed for every step of the self-consistency process.

Performance note: The default behavior (computing at the end of the program) involves an extra calculation of the charge density.

6.19.3 Crystal-Orbital overlap and hamilton populations (COOP/COHP)

These curves are quite useful to analyze the electronic structure to get insight about bonding characteristics. See the **Util/COOP** directory for more details. The **COOP.Write** option must be activated to get the information needed.

References:

- Original COOP reference: Hughbanks, T.; Hoffmann, R., J. Am. Chem. Soc., 1983, 105, 3528.
- Original COHP reference: Dronskowski, R.; Blöchl, P. E., J. Phys. Chem., 1993, 97, 8617.
- A tutorial introduction: Dronskowski, R. Computational Chemistry of Solid State Materials; Wiley-VCH: Weinheim, 2005.
- Online material maintained by R. Dronskowski's group: <http://www.cohp.de/>

COOP.Write false

(logical)

Instructs the program to generate **SystemLabel.fullBZ.WFSX** (packed wavefunction file) and **SystemLabel.HSX** (H, S and X_{ij} file), to be processed by **Util/COOP/mprop** to generate COOP/COHP curves, (projected) densities of states, etc.

The **.WFSX** file is in a more compact form than the usual **.WFS**, and the wavefunctions are output in single precision. The **Util/wfsx2wfs** program can be used to convert to the old format. The **HSX** file is in a more compact form than the usual **HS**, and the Hamiltonian, overlap matrix, and relative-positions array (which is always output, even for gamma-point only calculations) are in single precision.

The user can narrow the energy-range used (and save some file space) by using the **WFS.Energy.Min** and **WFS.Energy.Max** options (both take an energy (with units) as extra argument), and/or the **WFS.Band.Min** and **WFS.Band.Max** options. Care should be taken to make sure that the actual values of the options make sense.

Note that the band range options could also affect the output of wave-functions associated to bands (see section 6.16.2), and that the energy range options could also affect the output of user-selected wave-functions with the **WaveFuncKPoints** block (see section 6.17).

WFS.Energy.Min $-\infty$ *(energy)*

Specifies the lowest value of the energy (eigenvalue) of the wave-functions to be written to the file `SystemLabel.fullBZ.WFSX` for each k -point (all k -points in the BZ sampling are affected).

WFS.Energy.Max ∞ *(energy)*

Specifies the highest value of the energy (eigenvalue) of the wave-functions to be written to the file `SystemLabel.fullBZ.WFSX` for each k -point (all k -points in the BZ sampling are affected).

6.20 Optical properties

OpticalCalculation `false` *(logical)*

If specified, the imaginary part of the dielectric function will be calculated and stored in a file called `SystemLabel.EPSIMG`. The calculation is performed using the simplest approach based on the dipolar transition matrix elements between different eigenfunctions of the self-consistent Hamiltonian. For molecules the calculation is performed using the position operator matrix elements, while for solids the calculation is carried out in the momentum space formulation. Corrections due to the non-locality of the pseudopotentials are introduced in the usual way.

Optical.Energy.Minimum `0 Ry` *(energy)*

This specifies the minimum of the energy range in which the frequency spectrum will be calculated.

Optical.Energy.Maximum `10 Ry` *(energy)*

This specifies the maximum of the energy range in which the frequency spectrum will be calculated.

Optical.Broaden `0 Ry` *(energy)*

If this value is set then a Gaussian broadening will be applied to the frequency values.

Optical.Scissor `0 Ry` *(energy)*

Because of the tendency of DFT calculations to under estimate the band gap, a rigid shift of the unoccupied states, known as the scissor operator, can be added to correct the gap and thereby improve the calculated results. This shift is only applied to the optical calculation and nowhere else within the calculation.

Optical.NumberOfBands `all bands` *(integer)*

This option controls the number of bands that are included in the optical property calculation. Clearly this number must be larger than the number of occupied bands and less than or equal to the number of basis functions (which determines the number of unoccupied bands available). Note, while including all the bands may be the most accurate choice this will also be the most expensive!

%block Optical.Mesh `<None>` *(block)*

This block contains 3 numbers that determine the mesh size used for the integration across the

Brillouin zone. For example:

```
%block Optical.Mesh
5 5 5
%endblock Optical.Mesh
```

The three values represent the number of mesh points in the direction of each reciprocal lattice vector.

Optical.OffsetMesh `false` *(logical)*

If set to true, then the mesh is offset away from the gamma point for odd numbers of points.

Optical.PolarizationType `polycrystal` *(string)*

This option has three possible values that represent the type of polarization to be used in the calculation. The options are

polarized implies the application of an electric field in a given direction

unpolarized implies the propagation of light in a given direction

polycrystal In the case of the first two options a direction in space must be specified for the electric field or propagation using the **Optical.Vector** data block.

%block Optical.Vector `<None>` *(block)*

This block contains 3 numbers that specify the vector direction for either the electric field or light propagation, for a polarized or unpolarized calculation, respectively. A typical block might look like:

```
%block Optical.Vector
1.0 0.0 0.5
%endblock Optical.Vector
```

6.21 Macroscopic polarization

%block PolarizationGrids `<None>` *(block)*

If specified, the macroscopic polarization will be calculated using the geometric Berry phase approach (R.D. King-Smith, and D. Vanderbilt, PRB **47**, 1651 (1993)). In this method the electronic contribution to the macroscopic polarization, along a given direction, is calculated using a discretized version of the formula

$$P_{e,\parallel} = \frac{ifq_e}{8\pi^3} \int_A d\mathbf{k}_\perp \sum_{n=1}^M \int_0^{|\mathbf{G}_\parallel|} dk_\parallel \langle u_{\mathbf{k}n} | \frac{\delta}{\delta k_\parallel} | u_{\mathbf{k}n} \rangle \quad (16)$$

where f is the occupation (2 for a non-magnetic system), q_e the electron charge, M is the number of occupied bands (the system **must** be an insulator), and $u_{\mathbf{k}n}$ are the periodic Bloch functions. $\mathbf{G}_\parallel$ is the shortest reciprocal vector along the chosen direction.

As it can be seen in formula (16), to compute each component of the polarization we must perform a surface integration of the result of a 1-D integral in the selected direction. The grids for the calculation along the direction of each of the three lattice vectors are specified in the block **PolarizationGrids**.

```

%block PolarizationGrids
    10   3   4      yes
    2   20  2      no
    4    4  15
%endblock PolarizationGrids

```

All three grids must be specified, therefore a 3×3 matrix of integer numbers must be given: the first row specifies the grid that will be used to calculate the polarization along the direction of the first lattice vector, the second row will be used for the calculation along the the direction of the second lattice vector, and the third row for the third lattice vector. The numbers in the diagonal of the matrix specifie the number of points to be used in the one dimensional line integrals along the different directions. The other numbers specifie the mesh used in the surface integrals. The last column specifies if the bidimensional grids are going to be diplaced from the origin or not, as in the Monkhorst-Pack algorithm (PRB **13**, 5188 (1976)). This last column is optional. If the number of points in one of the grids is zero, the calculation will not be performed for this particular direction.

For example, in the given example, for the computation in the direction of the first lattice vector, 15 points will be used for the line integrals, while a 3×4 mesh will be used for the surface integration. This last grid will be displaced from the origin, so Γ will not be included in the bidimensional integral. For the directions of the second and third lattice vectors, the number of points will be 20 and 2×2 , and 15 and 4×4 , respectively.

It has to be stressed that the macroscopic polarization can only be meaningfully calculated using this approach for insulators. Therefore, the presence of an energy gap is necessary, and no band can cross the Fermi level. The program performs a simple check of this condition, just by counting the electrons in the unit cell (the number must be even for a non-magnetic system, and the total spin polarization must have an integer value for spin polarized systems), however is the responsability of the user to check that the system under study is actually an insulator (for both spin components if spin polarized).

The total macroscopic polarization, given in the output of the program, is the sum of the electronic contribution (calculated as the Berry phase of the valence bands), and the ionic contribution, which is simply defined as the sum of the atomic positions within the unit cell multiply by the ionic charges ($\sum_i^{N_a} Z_i \mathbf{r}_i$). In the case of the magnetic systems, the bulk polarization for each spin component has been defined as

$$\mathbf{P}^\sigma = \mathbf{P}_e^\sigma + \frac{1}{2} \sum_i^{N_a} Z_i \mathbf{r}_i \quad (17)$$

N_a is the number of atoms in the unit cell, and $\mathbf{r}_i$ and Z_i are the positions and charges of the ions.

It is also worth noting, that the macroscopic polarization given by formula (16) is only defined modulo a “quantum” of polarization (the bulk polarization per unit cell is only well defined modulo $f q_e \mathbf{R}$, being $\mathbf{R}$ an arbitrary lattice vector). However, the experimentally observable quantities are associated to changes in the polarization induced by changes on the atomic positions (dynamical charges), strains (piezoelectric tensor), etc... The calculation of those changes, between different configurations of the solid, will be well defined as long as they are smaller than the “quantum”, i.e. the perturbations are small enough to create small changes in the polarization.

BornCharge false

(logical)

If true, the Born effective charge tensor is calculated for each atom by finite differences, by calculating the change in electric polarization (see **PolarizationGrids**) induced by the small displacements generated for the force constants calculation (see **MD.TypeOfRun FC**):

$$Z_{i,\alpha,\beta}^* = \frac{\Omega_0}{e} \left. \frac{\partial P_\alpha}{\partial u_{i,\beta}} \right|_{q=0} \quad (18)$$

where e is the charge of an electron and Ω_0 is the unit cell volume.

To calculate the Born charges it is necessary to specify both the Born charge flag and the mesh used to calculate the polarization, for example:

```
%block PolarizationGrids
  7  3  3
  3  7  3
  3  3  7
%endblock PolarizationGrids
BornCharge True
```

The Born effective charge matrix is then written to the file **SystemLabel.BC**.

The method by which the polarization is calculated may introduce an arbitrary phase (polarization quantum), which in general is far larger than the change in polarization which results from the atomic displacement. It is removed during the calculation of the Born effective charge tensor.

The Born effective charges allow the calculation of LO-TO splittings and infrared activities. The version of the Vibra utility code in which these magnitudes are calculated is not yet distributed with SIESTA, but can be obtained from Tom Archer (archert@tcd.ie).

6.22 Wannier Functions. Interface with wannier90

wannier90 (<http://www.wannier.org>) is a code to generate maximally localized wannier functions according to the original Marzari and Vanderbilt recipe.

A wrapper interface between **siesta** and **wannier90** (version 3.1.0) has been implemented, so that **wannier90** can be called from **siesta** on-the-fly, as well as used as a post-processing tool.

It is strongly recommended to read the original papers on which this method is based and the documentation of **wannier90** code. Here we shall focus only on those internal SIESTA variables required to produce the files that will be processed by **wannier90**.

6.22.1 wannier90 as a postprocessing tool

This interface is analogous to that found in other programs. The user first runs **wannier90** in pre-processing mode to get a **.nnkp** file. Then SIESTA is run with the appropriate options to generate the files needed by a wannierization run with **wannier90**.

A complete list of examples and tests (including molecules, metals, semiconductors, insulators, magnetic systems, plotting of Fermi surfaces or interpolation of bands), can be downloaded from

<http://personales.unican.es/junquera/Wannier-examples.tar.gz>

NOTE: The Bloch functions produced by a first-principles code have arbitrary phases that depend on the number of processors used and other possibly non-reproducible details of the calculation. In

what follows it is essential to maintain consistency in the handling of the overlap and Bloch-function files produced and fed to **wannier90**.

Siesta2Wannier90.WriteMmn `false` (logical)

This flag determines whether the overlaps between the periodic part of the Bloch states at neighbour k-points are computed and dumped into a file in the format required by **wannier90**. These overlaps are defined in Eq. (27) in the paper by N. Marzari *et al.*, Review of Modern Physics **84**, 1419 (2012), or Eq. (1.7) of the Wannier90 User Guide, Version 2.0.1.

The k-points for which the overlaps will be computed are read from a **.nnkp** file produced by **wannier90**. It is strongly recommended for the user to read the corresponding user guide.

The overlap matrices are written in a file with extension **.mmn**.

Siesta2Wannier90.WriteAmn `false` (logical)

This flag determines whether the overlaps between Bloch states and trial localized orbitals are computed and dumped into a file in the format required by **wannier90**. These projections are defined in Eq. (16) in the paper by N. Marzari *et al.*, Review of Modern Physics **84**, 1419 (2012), or Eq. (1.8) of the Wannier90 User Guide, Version 2.0.1.

The localized trial functions to use are taken from the **.nnkp** file produced by **wannier90**. It is strongly recommended for the user to read the corresponding user guide.

The overlap matrices are written in a file with extension **.amn**.

Siesta2Wannier90.WriteEig `false` (logical)

Flag that determines whether the Kohn-Sham eigenvalues (in eV) at each point in the Monkhorst-Pack mesh required by **wannier90** are written to file. This file is mandatory in **wannier90** if any of **disentanglement**, **plot_bands**, **plot_fermi_surface** or **hr_plot** options are set to true in the **wannier90** input file.

The eigenvalues are written in a file with extension **.eigW**. This extension is chosen to avoid name clashes with SIESTA's standard eigenvalue file in case-insensitive filesystems.

Siesta2Wannier90.WriteUnk `false` (logical)

Produces **UNKXXXXX.Y** files which contain the periodic part of a Bloch function in the unit cell on a grid given by global **unk_nx**, **unk_ny**, **unk_nz** variables. The name of the output files is assumed to have the previous form, where the **XXXXXX** refer to the k-point index (from 00001 to the total number of k-points considered), and the **Y** refers to the spin component (1 or 2)

The periodic part of the Bloch functions is defined by

$$u_{n\vec{k}}(\vec{r}) = \sum_{\vec{R}, \mu} c_{n\mu}(\vec{k}) e^{i\vec{k} \cdot (\vec{r}_\mu + \vec{R} - \vec{r})} \phi_\mu(\vec{r} - \vec{r}_\mu - \vec{R}), \quad (19)$$

where $\phi_\mu(\vec{r} - \vec{r}_\mu - \vec{R})$ is a basis set atomic orbital centered on atom μ in the unit cell $\vec{R}$, and $c_{n\mu}(\vec{k})$ are the coefficients of the wave function. The latter must be identical to the ones used for wannierization in M_{mn} . (See the above comment about arbitrary phases.)

Siesta2Wannier90.UnkGrid1 `<mesh points along A>` (integer)

Number of points along the first lattice vector in the grid where the periodic part of the wave functions will be plotted.

Siesta2Wannier90.UnkGrid2 `<mesh points along B>` (integer)

Number of points along the second lattice vector in the grid where the periodic part of the wave functions will be plotted.

Siesta2Wannier90.UnkGrid3 *<mesh points along C>* *(integer)*

Number of points along the third lattice vector in the grid where the periodic part of the wave functions will be plotted.

Siesta2Wannier90.UnkGridBinary *true* *(logical)*

Flag that determines whether the periodic part of the wave function in the real space grid is written in binary format (default) or in ASCII format.

Siesta2Wannier90.NumberOfBands *occupied bands* *(integer)*

In spin unpolarized calculations, number of bands that will be initially considered by SIESTA to generate the information required by **wannier90**. Note that it should be at least as large as the index of the highest-lying band in the **wannier90** post-processing. For example, if the wannierization is going to involve bands 3 to 5, the SIESTA number of bands should be at least 5. Bands 1 and 2 should appear in a “excluded” list.

NOTE: you are highly encouraged to explicitly specify the number of bands.

Siesta2Wannier90.NumberOfBandsUp *<Siesta2Wannier90.NumberOfBands>*
(integer)

In spin-polarized calculations, number of bands with spin up that will be initially considered by SIESTA to generate the information required by **wannier90**.

Siesta2Wannier90.NumberOfBandsDown *<Siesta2Wannier90.NumberOfBands>*
(integer)

In spin-polarized calculations, number of bands with spin down that will be initially considered by SIESTA to generate the information required by **wannier90**.

6.22.2 **wannier90** called on-the-fly within **siesta**

A wrapper interface to **wannier90** can be compiled and called directly from **siesta**. This presents several advantages:

- No need to prepare two different input files.
- No need to run **wannier90** in pre-processing mode.
- We can use the basis set of **siesta** (numerical atomic orbitals) as the initial guess for the projections.
- Wannierization of different manifolds can be done in the same run of **siesta**.
- The unitary matrices connecting the Bloch and the Wannier representations are available within **siesta**.
- The coefficients of the Wannier functions in the basis of the atomic orbitals of the supercell in **siesta** are written in a file with **SystemLabel.WANNX** extension. Then, the Wannier functions can be plotted using **denchar**, following the same method as for the wave functions.

Further details of the compilation of SIESTA with this functionality can be found in the file `External/Wannier/README.md`. Note also the (slightly outdated) presentation https://personales.unican.es/junquera/JavierJunquera_files/Metodos/Wannier/Exercise-Wannier90-within-siesta.pdf

%block Wannier.Manifolds `<None>` *(block)*

Each line denotes the name of a manifold to be processed by Wannier90.

Options for each manifold is specified in the **Wannier.Manifold.<>**

%block Wannier.Manifold.<> `<None>` *(block)*

Each line represents a setting for the Wannier manifold to be processed.

bands Two integers specifying the initial and final band of the manifold to be wannierized.

NOTE: required input

trial-orbitals Indices of the orbitals that will be used as localized trial orbitals in the first step of the minimization of the spreading. The user has to specify the same number of atomic orbitals as the number of Wannier functions required. For the sake of readiness, the number of trial orbitals can be split in several lines, all of them starting with **trial-orbitals**. These indices can be found by inspection of the `SystemLabel1.ORB_IND` file. If there are negative integers in this line, then the projectors will be generated à-la-Wannier90, with the instructions given in the WannierProjectors block.

If there are negative numbers, it is not strictly required that they must appear after the list of positive indices.

NOTE: required input

spreading.nitt Number of iterations that **wannier90** will carry out to minimize the spreading.

If zero, then the procedure is the same as a Löwdin orthonormalization. In such a case, the resulting Wannier function will keep the symmetry of the trial projection function, but it will not be maximally localized.

wannier-plot Instructs **wannier90** to produce the files required to plot the Wannier functions (if **w90.in.siesta.compute.unk** is set to true). The integer refers to the size of the supercell for plotting the Wannier functions (see the variable **wannier.plot.supercell** in the **wannier90** User's Guide). This will produce files with the `.xsf` extension, that can be directly plotted with **xcrysden**.

fermi-surface-plot Is the file required to plot the Fermi surface computed? If true, this will produce files with the `.bxsf` extension, that can be directly plotted with **xcrysden**.

write-hr Is the file with the Hamiltonian in real space in a basis of Wannier functions written? If true, this will produce files with the `_hr.dat` extension.

write-tb Is the file with the tight-binding parameters in a basis of Wannier functions written? (this includes the lattice vectors, Hamiltonian in real space, and position operator in a basis of Wannier functions. If true, this will produce files with the `_tb.dat` extension.

write-unk Are the files that contain the periodic part of a Bloch function in the unit cell on a grid computed? If true, files like those described in **Siesta2Wannier90.WriteUnk** are written for this manifold, and then the corresponding `.xsf` files directly readable by **xcrysden** will be produced. The computation of the UNK files might be rather expensive. To plot

the shape of the Wannier functions, the expansion of the Wannier functions in the basis of Numerical Atomic Orbitals to produce .WFSX files, and the subsequent use of **denchar** is recommended.

If a disentanglement procedure is required two extra lines are mandatory:

window It refers to the bottom and top of the outer energy window for band disentanglement. The units for the energy are introduced as the last character string of the line.

window.frozen It refers to the bottom and top of the inner energy window for band disentanglement. The units for the energy are introduced as the last character string of the line. This is the energy window where some Bloch states are forced to be preserved identically in the projected manifold.

threshold Specification of the threshold for the real part of the coefficients of a Wannier in a basis of NAO that will be written in a **SystemLabel.WANNX** extension file. This file can be used to plot the Wannier functions using **denchar**, following the same method as for the wave functions. This threshold is particularized for a particular manifold.

An example of a manifold:

```
%block Wannier.Manifold.second
bands 21 23
trial-orbitals [24 25 27]
spreading.nitt 0
wannier_plot 3
fermi_surface_plot true
write_hr true
write_tb true
%endblock
```

%block Wannier.Projectors *<projection functions as in wannier90>* *(block)*

Information on the projection functions à-la-wannier90, used to construct the initial guesses for the unitary transformations.

These are used when some of the atomic orbitals in the **trial-orbitals** lines of the block **Wannier.Manifold.<>** are negative.

For instance, to specify the projectors for the bottom of the conduction band of bulk SrTiO₃, we can write a block **Wannier.Manifold.<>** as

```
%block Wannier.Manifold.example
# Indices of the initial and final band of the manifold
bands 21 23
# Number of bands for Wannier transformation
trial-orbitals -1 -2 -3
spreading.nitt 0 # Number of iterations for the minimization of \Omega
wannier-plot 3 # Plot the Wannier function
fermi-surface-plot # Plot the Fermi surface
write-hr # Write the Hamiltonian in the WF basis
write-tb # Write lattice vectors, Hamiltonian, and position operator in WF basis
%endblock
```

Then, the three projector functions that will be generated following the recipe of **wannier90** will be

```
%block Wannier.Projectors.example
```

```

0.5 0.5 0.5 2 2 1 0.00 0.00 1.00 1.00 0.00 0.00 1.00
0.5 0.5 0.5 2 3 1 0.00 0.00 1.00 1.00 0.00 0.00 1.00
0.5 0.5 0.5 2 5 1 0.00 0.00 1.00 1.00 0.00 0.00 1.00
%endblock

```

The different lines in this block are written following the **wannier90** format provided in the **.nnkp** file.

centre: three real numbers; projection function centre in crystallographic co-ordinates relative to the direct lattice vectors.

l m_r r : three integers; l and m_r specify the angular part $\Theta_{lm_r}(\theta, \varphi)$, and r specifies the radial part $R_r(r)$ of the projection function (see Tables 3.1, 3.2 and 3.3 of the **wannier90** User's Guide).

z-axis: three real numbers; default is 0.0 0.0 1.0; defines the axis from which the polar angle θ in spherical polar coordinates is measured.

x-axis: three real numbers; must be orthogonal to z-axis default is 1.0 0.0 0.0 or a vector perpendicular to z-axis if z-axis is given; defines the axis from with the azimuthal angle φ in spherical polar coordinates is measured.

zona: real number; the value of $\frac{Z}{a}$ associated with the radial part of the atomic orbital. Units are in reciprocal Angstrom.

Wannier.Manifolds.Threshold 10^{-6} (real)

Global specification of the threshold for the real part of the coefficients of a Wannier in a basis of NAO that will be written in a **SystemLabel.WANNX** extension file. This file can be used to plot the Wannier functions using **denchar**, following the same method as for the wave functions.

Individual manifolds can be controlled via **Wannier.Manifold.<>.threshold**

Wannier.Manifolds.Unk **false** (logical)

Global flag that determines whether the periodic part of the wave function in the real space grid will be computed (as using **Siesta2Wannier90.WriteUnk**), and whether the files **xsf** directly readable by **xcrysden** will be produced.

Individual manifolds can be controlled via **Wannier.Manifold.<>.write-unk**.

The computation of the UNK files might be rather expensive. To plot the shape of the Wannier functions, the expansion of the Wannier functions in the basis of Numerical Atomic Orbitals to produce **SystemLabel.WFSX** files, and the subsequent use of **denchar** is recommended.

Wannier.k **Γ -point** (list/block)

Dimension of the Monkhorst-Pack grid of k-points that will be used during the wannierization. The overlap matrices between periodic parts of the wavefunctions at neighbour k-points in this grid will be computed.

```

Wannier.k [4 4 4]
# Or equivalently
%block Wannier.k
4 4 4
%endblock

```

6.23 Systems with net charge or dipole, and electric fields

NetCharge **0** (real)

Specify the net charge of the system (in units of $|e|$). For charged systems, the energy converges very slowly versus cell size. For molecules or atoms, a Madelung correction term is applied to the energy to make it converge much faster with cell size (this is done only if the cell is SC, FCC or BCC). For other cells, or for periodic systems (chains, slabs or bulk), this energy correction term can not be applied, and the user is warned by the program. It is not advised to do charged systems other than atoms and molecules in SC, FCC or BCC cells, unless you know what you are doing.

Use: For example, the F^- ion would have **NetCharge -1**, and the Na^+ ion would have **NetCharge 1**. Fractional charges can also be used.

NOTE: Doing non-neutral charge calculations with **Slab.DipoleCorrection** is discouraged.

SimulateDoping `false` *(logical)*

This option instructs the program to add a background charge density to simulate doping. The new “doping” routine calculates the net charge of the system, and adds a compensating background charge that makes the system neutral. This background charge is constant at points of the mesh near the atoms, and zero at points far from the atoms. This simulates situations like doped slabs, where the extra electrons (holes) are compensated by opposite charges at the material (the ionized dopant impurities), but not at the vacuum. This serves to simulate properly doped systems in which there are large portions of vacuum, such as doped slabs.

See **Tests/sic-slab**.

%block ExternalElectricField `<None>` *(block)*

It specifies an external electric field for molecules, chains and slabs. The electric field should be orthogonal to “bulk directions”, like those parallel to a slab (bulk electric fields, like in dielectrics or ferroelectrics, are not allowed). If it is not, an error message is issued and the components of the field in bulk directions are suppressed automatically. The input is a vector in Cartesian coordinates, in the specified units. Example:

```
%block ExternalElectricField
    0.000  0.000  0.500  V/Ang
%endblock ExternalElectricField
```

Starting with version 4.0, applying an electric field perpendicular to a slab will by default enable the slab dipole correction, see **Slab.DipoleCorrection**. To reproduce older calculations, set this correction option explicitly to **false** in the input file.

When examining a variety of electric fields it may be highly advantageous to re-use the **SystemLabel.DM** from a previous calculation with an electric field close to the current one.

Slab.DipoleCorrection `?|true|false|charge|vacuum|none` *(string)*

depends on: **ExternalElectricField**

If not **false**, SIESTA calculates the electric field required to compensate the dipole of the system at every iteration of the self-consistent cycle.

The dipole correction only works for Fourier transformed Poisson solutions of the Hartree potential since that will introduce a compensating field in the vacuum region to counter any inherent dipole in the system. Do not use this option together with **NetCharge** (charged systems).

There are two ways of calculating the dipole of the system:

charge|true The dipole of the system is calculated via

$$\mathbf{D} = -e \int (\mathbf{r} - \mathbf{r}_0) \delta \rho(\mathbf{r}) \quad (20)$$

where $\mathbf{r}_0$ is the dipole origin, see **Slab.DipoleCorrection.Origin**, and $\delta \rho$ is valence pseudocharge density minus the atomic valence pseudocharge densities.

vacuum The electric field of the system is calculated via

$$\mathbf{E} \propto \left. \iint d\mathbf{r}_{\perp \mathbf{D}} V(\mathbf{r}) \right|_{\mathbf{r}_{\text{vacuum}}} \quad (21)$$

where $\mathbf{r}_{\text{vacuum}}$ is a point located in the vacuum region, see **Slab.DipoleCorrection.Vacuum**. Once the field is determined it is converted to an intrinsic system dipole.

This feature is mainly intended for **Geometry.Charge** calculations where **Slab.DipoleCorrection charge** may fail if the dipole center is determined incorrectly.

For regular systems both this and **charge** should yield approximately (down to numeric precision) the same dipole moments.

The dipole correction should exactly compensate the electric field at the vacuum level thus allowing one to treat asymmetric slabs (including systems with an adsorbate on one surface) and compute properties such as the work function of each of the surfaces.

NOTE: If the program is fed a starting density matrix from an uncorrected calculation (i.e., with an exaggerated dipole), the first iteration might use a compensating field that is too big, with the risk of taking the system out of the convergence basin. In that case, it is advisable to use the **SCF.Mix.First** option to request a mix of the input and output density matrices after that first iteration.

NOTE: **charge** and **vacuum** will for many systems yield the same result. If in doubt try both and see which one gives the best result.

See **Tests/sic-slab**, **Tests/h2o_2_dipol_gate**.

This will default to **true** if an external field is applied to a slab calculation, otherwise it will default to **false**.

%block Slab.DipoleCorrection.Origin **<None>** *(block)*

depends on: Slab.DipoleCorrection charge

Specify the origin of the dipole in the calculation of the dipole from the charge distribution.

Its format is

```
%block Slab.DipoleCorrection.Origin
    0.000 10.000 0.500 Ang
%endblock
```

If this block is not specified the origin of the dipole will be the average position of the atoms.

NOTE: this will only be read if **Slab.DipoleCorrection charge** is used. **NOTE:** this should only affect calculations with **Geometry.Charge** due to the non-trivial dipole origin, see e.g. **Tests/h2o_2_dipol_gate** and try and see if you can manually place the dipole origin to achieve similar results as the vacuum method.

%block Slab.DipoleCorrection.Vacuum *<None>* *(block)*

depends on: **Slab.DipoleCorrection vacuum**

Options for the vacuum field determination.

direction Mandatory input for chain and molecule calculations.

Specify along which direction we should determine the electric field/dipole.

For slabs this defaults to the non-bulk direction.

position Specify a point in the vacuum region.

Defaults to the vacuum region based on the atomic coordinates.

tolerance Tolerance for determining whether we are in a vacuum region. The premise of the electric field calculation in the vacuum region is that the derivative of the potential (**E**) is flat. When the electric field changes by more than this tolerance the region is not vacuum anymore and the point is disregarded.

Defaults to 10^{-4} eV/Ang/e.

Its format is

```
%block Slab.DipoleCorrection.Vacuum
# this is optional
# default position is the center of system + 0.5 lattice vector
# along 'direction'
position 0.000 10.000 0.500 Ang
# this is optional
# default is 1e-4 eV/Ang/e
tolerance 0.001 eV/Ang/e
# this is mandatory
direction 0.000 1.000 0.
%endblock
```

NOTE: this will only be read if **Slab.DipoleCorrection vacuum** is used.

%block Geometry.Hartree *<None>* *(block)*

Allows introduction of regions with changed Hartree potential. Introducing a potential can act as a repulsion (positive value) or attraction (negative value) region.

The regions are defined as geometrical objects and there are no limits to the number of defined geometries.

Details regarding this implementation may be found in Papior et al.^[13].

Currently 4 different kinds of geometries are allowed:

Infinite plane Define a geometry by an infinite plane which cuts the unit-cell.

This geometry is defined by a single point which is in the plane and a vector normal to the plane.

This geometry has 3 different settings:

delta An infinite plane with δ -height.

gauss An infinite plane with a Gaussian distributed height profile.

exp An infinite plane with an exponentially distributed height profile.

Bounded plane Define a geometric plane which is bounded, i.e. not infinite.

This geometry is defined by an origo of the bounded plane and two vectors which span the plane, both originating in the respective origo.

This geometry has 3 different settings:

delta A plane with δ -height.

gauss A plane with a Gaussian distributed height profile.

exp A plane with an exponentially distributed height profile.

Box This geometry is defined by an origo of the box and three vectors which span the box, all originating from the respective origo.

This geometry has 1 setting:

delta No decay-region outside the box.

Spheres This geometry is defined by a list of spheres and a common radii.

This geometry has 2 settings:

gauss All spheres have an gaussian distribution about their centre.

exp All spheres have an exponential decay.

Here is a list of all options combined in one block:

```
%block Geometry.Hartree
plane 1. eV          # The lifting potential on the geometry
delta
  1.0 1.0 1.0 Ang    # An intersection point, in the plane
  1.0 0.5 0.2        # The normal vector to the plane
plane -1. eV         # The lifting potential on the geometry
gauss 1. 2. Ang      # the std. and the cut-off length
  1.0 1.0 1.0 Ang    # An intersection point, in the plane
  1.0 0.5 0.2        # The normal vector to the plane
plane 1. eV          # The lifting potential on the geometry
exp 1. 2. Ang        # the half-length and the cut-off length
  1.0 1.0 1.0 Ang    # An intersection point, in the plane
  1.0 0.5 0.2        # The normal vector to the plane
square 1. eV         # The lifting potential on the geometry
delta
  1.0 1.0 1.0 Ang    # The starting point of the square
  2.0 0.5 0.2 Ang    # The first spanning vector
  0.0 2.5 0.2 Ang    # The second spanning vector
square 1. eV         # The lifting potential on the geometry
gauss 1. 2. Ang      # the std. and the cut-off length
  1.0 1.0 1.0 Ang    # The starting point of the square
  2.0 0.5 0.2 Ang    # The first spanning vector
  0.0 2.5 0.2 Ang    # The second spanning vector
square 1. eV         # The lifting potential on the geometry
exp 1. 2. Ang        # the half-length and the cut-off length
  1.0 1.0 1.0 Ang    # The starting point of the square
  2.0 0.5 0.2 Ang    # The first spanning vector
  0.0 2.5 0.2 Ang    # The second spanning vector
box 1. eV            # The lifting potential on the geometry
delta
  1.0 1.0 1.0 Ang    # Origo of the box
```

```

2.0 0.5 0.2 Ang # The first spanning vector
0.0 2.5 0.2 Ang # The second spanning vector
0.0 0.5 3.2 Ang # The third spanning vector
coords 1. eV # The lifting potential on the geometry
gauss 2. 4. Ang # First is std. deviation, second is cut-off radii
2 spheres # How many spheres in the following lines
0.0 4. 2. Ang # The centre coordinate of 1. sphere
1.3 4. 2. Ang # The centre coordinate of 2. sphere
coords 1. eV # The lifting potential on the geometry
exp 2. 4. Ang # First is half-length, second is cut-off radii
2 spheres # How many spheres in the following lines
0.0 4. 2. Ang # The centre coordinate of 1. sphere
1.3 4. 2. Ang # The centre coordinate of 2. sphere
%endblock Geometry.Hartree

```

%block Geometry.Charge *<None>* *(block)*

This is similar to the **Geometry.Hartree** block. However, instead of specifying a potential, one defines the total charge that is spread on the geometry.

To see how the input should be formatted, see **Geometry.Hartree** and remove the unit-specification. Note that the input value is number of electrons (similar to **NetCharge**, however this method ensures charge-neutrality).

Details regarding this implementation may be found in Papior et al.^[13].

6.23.1 Bulk current

SIESTA enables a crude way of calculating a bulk current. The basic principle may be understood from basic condensed matter physics by filling all right-moving states up to $E_F + V/2$ and emptying all left-moving states down to $E_F - V/2$ (for a positively defined V).

When using this method the resulting eigenvalue spectrum in **SystemLabel.EIG** contains the shifted eigenvalues corresponding to whether they are left/right movers.

The occupation function for left/right movers uses that provided in **OccupationFunction**.

BulkBias.Voltage 0.eV *(energy)*
depends on: **BulkBias.Direction**, **Diag.ParallelOverK**, **TimeReversalSymmetryForKpoints**

The applied bias shift in the band-structure. All right-moving states will be shifted halve this value down in energy (more filled), while all left-moving states will be shifted halve this value up in energy (less filled).

Since states are filled differently close to the Fermi level it is imperative that the **k**-point sampling is very high to discretize the integration around the Fermi level sufficiently.

NOTE: this requires **Diag.ParallelOverK** to be set to **true**, and **TimeReversalSymmetryForKpoints** to be set to **false**.

%block BulkBias.Direction *<None>* *(block)*
depends on: **BulkBias.Tolerance**

The direction in which the electrons are moving. All electrons having velocities with a positive projection onto this direction are considered “right-movers” while all having a negative

projection are considered “left-movers”:

$$p = \mathbf{v} \cdot \hat{\mathbf{V}}, \quad (22)$$

where $\mathbf{v}$ is the band velocity and $\hat{\mathbf{V}}$ is the bias unit vector.

An example of a direction pointing along the diagonal xy direction. Internally the direction will be normalized.

```
%block BulkBias.Direction
1. 1. 0.
%endblock
```

BulkBias.Tolerance 10^{-15} (real)

The tolerance used for determining whether the velocity projection is positive or negative. States with projections below this tolerance value will not be shifted.

This value may be regarded as the velocity in atomic units and thus having a larger value will only shift eigenstates with higher velocities projected onto the potential-direction. The current value corresponds roughly to a velocity of $1 \cdot 10^{-11}$ Ang/ps.

BulkBias.Current **true** (logical)

Calculate and print out the bulk-bias current during each SCF and also correct the total energy with respect to the applied bias. The calculated current is given by the expression:

$$p_{\mathbf{k},i} = \mathbf{v}_{\mathbf{k},i} \cdot \hat{\mathbf{V}} \quad (23)$$

$$I(V) = \frac{2e}{\Omega} \sum_i \int d\mathbf{k} p_{\mathbf{k},i} \Theta(p_{\mathbf{k},i}) [n_F(\epsilon_{\mathbf{k},i} - V/2) - n_F(\epsilon_{\mathbf{k},i} + V/2)], \quad (24)$$

where $\mathbf{v}_{\mathbf{k},i}$ is the velocity of the i th eigenstate at $\mathbf{k}$, $\hat{\mathbf{V}}$ is the velocity unit vector describing the direction of the field. $\Theta(x)$ is the heaviside step function. Finally Ω is the Brillouin zone volume which depends on the dimensionality of the system:

1D Ω has unit length, and the resulting current is in A,

2D Ω has unit area, and the resulting current is A/Å,

3D Ω has unit volume, and the resulting current is A/Å².

The factor 2 comes from spin degeneracy and is neglected in polarized and non-colinear calculations.

When this is **true** the free energy will be corrected with the following:

$$E_{bV} = -V/2(q^+ - q^-), \quad (25)$$

where $q^{+/-}$ refer to the charges positively/negatively along the applied bias. If this option is **false** SIESTA cannot calculate the energy correction and $E_{bV} = 0$. Users are encouraged to have this to **true** but may for parameter searches turn this off to speed up calculations.

For non-colinear calculations the spin-alignment of the current is also calculated.

NOTE: there is a slight performance penalty of calculating the current in the SCF. It requires the calculation of the velocities one more time, however, it should be a relatively small overhead.

6.24 Output of charge densities and potentials on the grid

SIESTA represents these magnitudes on the real-space grid. The following options control the generation of the appropriate files, which can be processed by the programs in the `Util/Grid` directory, and also by Andrei Postnikov's utilities in `Util/Contrib/APostnikov`. See also `Util/Denchar` for an alternative way to plot the charge density (and wavefunctions).

SaveRho `false` *(logical)*

Instructs to write the valence pseudocharge density at the mesh used by DHSCF, in file `SystemLabel.RHO`.

NOTE: file `.RHO` is only written, not read, by siesta. This file can be read by routine `IORHO`, which may be used by other application programs.

If netCDF support is compiled in, the file `Rho.grid.nc` is produced.

SaveDeltaRho `false` *(logical)*

Instructs to write $\delta\rho(\vec{r}) = \rho(\vec{r}) - \rho_{atm}(\vec{r})$, i.e., the valence pseudocharge density minus the sum of atomic valence pseudocharge densities. It is done for the mesh points used by DHSCF and it comes in file `SystemLabel.DRHO`. This file can be read by routine `IORHO`, which may be used by an application program in later versions.

NOTE: file `.DRHO` is only written, not read, by siesta.

If netCDF support is compiled in, the file `DeltaRho.grid.nc` is produced.

SaveRhoXC `false` *(logical)*

Instructs to write the valence pseudocharge density at the mesh, including the nonlocal core corrections used to calculate the exchange-correlation energy, in file `SystemLabel.RHOXC`.

Use: File `.RHOXC` is only written, not read, by siesta.

If netCDF support is compiled in, the file `RhoXC.grid.nc` is produced.

SaveElectrostaticPotential `false` *(logical)*

Instructs to write the total electrostatic potential, defined as the sum of the hartree potential plus the local pseudopotential, at the mesh used by DHSCF, in file `SystemLabel.VH`. This file can be read by routine `IORHO`, which may be used by an application program in later versions.

Use: File `.VH` is only written, not read, by siesta.

If netCDF support is compiled in, the file `ElectrostaticPotential.grid.nc` is produced.

SaveNeutralAtomPotential `false` *(logical)*

Instructs to write the neutral-atom potential, defined as the sum of the hartree potential of a "pseudo atomic valence charge" plus the local pseudopotential, at the mesh used by DHSCF, in file `SystemLabel.VNA`. It is written at the start of the self-consistency cycle, as this potential does not change.

Use: File `.VNA` is only written, not read, by siesta.

If netCDF support is compiled in, the file `Vna.grid.nc` is produced.

SaveTotalPotential `false` *(logical)*

Instructs to write the valence total effective local potential (local pseudopotential + Hartree + V_{xc}), at the mesh used by DHSCF, in file `SystemLabel.VT`. This file can be read by routine `IORHO`, which may be used by an application program in later versions.

Use: File `.VT` is only written, not read, by siesta.

If netCDF support is compiled in, the file `TotalPotential.grid.nc` is produced.

NOTE: a side effect; the vacuum level, defined as the effective potential at grid points with zero density, is printed in the standard output whenever such points exist (molecules, slabs) and either **SaveElectrostaticPotential** or **SaveTotalPotential** are **true**. In a symmetric (nonpolar) slab, the work function can be computed as the difference between the vacuum level and the Fermi energy.

SaveIonicCharge `false` *(logical)*

Instructs to write the soft diffuse ionic charge at the mesh used by DHSCF, in file `SystemLabel.IOCH`. This file can be read by routine IORHO, which may be used by an application program in later versions. Remember that, within the SIESTA sign convention, the electron charge density is positive and the ionic charge density is negative.

Use: File `.IOCH` is only written, not read, by siesta.

If netCDF support is compiled in, the file `Chlocal.grid.nc` is produced.

SaveTotalCharge `false` *(logical)*

Instructs to write the total charge density (ionic+electronic) at the mesh used by DHSCF, in file `SystemLabel.TOCH`. This file can be read by routine IORHO, which may be used by an application program in later versions. Remember that, within the SIESTA sign convention, the electron charge density is positive and the ionic charge density is negative.

Use: File `.TOCH` is only written, not read, by siesta.

If netCDF support is compiled in, the file `TotalCharge.grid.nc` is produced.

SaveGridFunc.Format `binary` *(string)*

Format of the (requested) output files `SystemLabel.RHO`, `SystemLabel.DRHO`, `SystemLabel.RHOXC`, `SystemLabel.VH`, `SystemLabel.VNA`, `SystemLabel.VT`, `SystemLabel.IOCH`, and `SystemLabel.TOCH`. The options are

- `ascii` : ASCII text format
- `binary` : unformatted (machine dependent)

NOTE: ASCII files require much more space than binary and NetCDF files. Consider using the tools in `Util/Grid` to translate between formats.

SaveBaderCharge `false` *(logical)*

Instructs the program to save the charge density for further post-processing by a Bader-analysis program. This “Bader charge” is the sum of the electronic valence charge density and a set of “model core charges” placed at the atomic sites. For a given atom, the model core charge is a generalized Gaussian, but confined to a radius of 1.0 Bohr (by default), and integrating to the total core charge ($Z - Z_{\text{val}}$). These core charges are needed to provide local maxima for the charge density at the atomic sites, which are not guaranteed in a pseudopotential calculation. For hydrogen, an artificial core of 1 electron is added, with a confinement radius of 0.6 Bohr by default. The Bader charge is projected on the grid points of the mesh used by DHSCF, and saved in file `SystemLabel.BADER`. This file can be post-processed by the program `Util/grid2cube` to convert it to the “cube” format, accepted by several Bader-analysis programs (for example, see <http://theory.cm.utexas.edu/bader/>). Due to the need to represent a localized core

charge, it is advisable to use a moderately high Mesh!Cutoff when invoking this option (300-500 Ry). The size of the “basin of attraction” around each atom in the Bader analysis should be monitored to check that the model core charge is contained in it.

The radii for the model core charges can be specified in the input fdf file. For example:

```
bader-core-radius-standard 1.3 Bohr
bader-core-radius-hydrogen 0.4 Bohr
```

The suggested way to run the Bader analysis with the Univ. of Texas code is to use both the RHO and BADER files (both in “cube” format), with the BADER file providing the “reference” and the RHO file the actual significant valence charge data which is important in bonding. (See the notes for pseudopotential codes in the above web page.) For example, for the h2o-pop example:

```
bader h2o-pop.RHO.cube -ref h2o-pop.BADER.cube
```

If netCDF support is compiled in, the file `BaderCharge.grid.nc` is produced.

AnalyzeChargeDensityOnly `false` *(logical)*

If **true**, the program optionally generates charge density files and computes partial atomic charges (Hirshfeld, Voronoi, Bader) from the information in the input density matrix, and stops. This is useful to analyze the properties of the charge density without a diagonalization step, and with a user-selectable mesh cutoff. Note that the **DM.UseSaveDM** option should be active. Note also that if an initial density matrix (DM file) is used, it is not normalized. All the relevant fdf options for charge-density file production and partial charge calculation can be used with this option.

SaveInitialChargeDensity `false` *(logical)*

*deprecated by: **AnalyzeChargeDensityOnly***

If **true**, the program generates a `SystemLabel.RHOINIT` file (and a `RhoInit.grid.nc` file if netCDF support is compiled in) containing the charge density used to start the first self-consistency step, and it stops. Note that if an initial density matrix (DM file) is used, it is not normalized. This is useful to generate the charge density associated to “partial” DMs, as created by programs such as `dm_creator` and `dm_filter`.

(This option is to be deprecated in favor of **AnalyzeChargeDensityOnly**).

6.25 Auxiliary Force field

It is possible to supplement the DFT interactions with a limited set of force-field options, typically useful to simulate dispersion interactions. It is not yet possible to turn off DFT and base the dynamics only on the force field. The GULP program should be used for that.

%block MM.Potentials `<None>` *(block)*

This block allows the input of molecular mechanics potentials between species. The following potentials are currently implemented:

- C6, C8, C10 powers of the Tang-Toennies damped dispersion potential.
- A harmonic interaction.
- A dispersion potential of the Grimme type (similar to the C6 type but with a different

damping function). (See S. Grimme, J. Comput. Chem. Vol 27, 1787-1799 (2006)). See also **MM.Grimme.D** and **MM.Grimme.S6** below.

The format of the input is the two species numbers that are to interact, the potential name (C6, C8, C10, harm, or Grimme), followed by the potential parameters. For the damped dispersion potentials the first number is the coefficient and the second is the exponent of the damping term (i.e., a reciprocal length). A value of zero for the latter term implies no damping. For the harmonic potential the force constant is given first, followed by r0. For the Grimme potential C6 is given first, followed by the (corrected) sum of the van der Waals radii for the interacting species (a real length). Positive values of the C6, C8, and C10 coefficients imply attractive potentials.

```
%block MM.Potentials
  1 1 C6 32.0 2.0
  1 2 harm 3.0 1.4
  2 3 Grimme 6.0 3.2
%endblock MM.Potentials
```

To automatically create input for Grimme’s method, please see the utility: **Util/Grimme** which can read an fdf file and create the correct input for Grimme’s method.

MM.Cutoff 30 Bohr *(length)*
 Specifies the distance out to which molecular mechanics potential will act before being treated as going to zero.

MM.UnitsEnergy eV *(unit)*
 Specifies the units to be used for energy in the molecular mechanics potentials.

MM.UnitsDistance Ang *(unit)*
 Specifies the units to be used for distance in the molecular mechanics potentials.

MM.Grimme.D 20.0 *(real)*
 Specifies the scale factor d for the scaling function in the Grimme dispersion potential (see above).

MM.Grimme.S6 1.66 *(real)*
 Specifies the overall fitting factor s_6 for the Grimme dispersion potential (see above). This number depends on the quality of the basis set, the exchange-correlation functional, and the fitting set.

6.26 Grimme’s DFT-D3 dispersion model

The current implementation has the possibility of adding D3 corrections to DFT calculations (See Grimme, J. Chem. Phys. 132 (2010), 154104. DOI: 10.1063/1.3382344). The following options provide a great deal of fine-tuning within this model; see in the above reference for insight on the parameters Sn, rSn and alpha, which correspond to the following equations:

$$E_{D3} = E_{2body} + E_{3body}$$

$$E_{2body} = \sum_{A,B} \frac{s_6 C_6^{AB}}{(r_{AB})^6} f_6(r_{AB}) + \sum_{A,B} \frac{s_8 C_8^{AB}}{(r_{AB})^8} f_8(r_{AB}) ; f_n(r_{AB}) = \frac{1}{1+6 \left(\frac{r_{AB}}{S r_n R_0^{AB}} \right)^{-\alpha_n}}$$

The 3-body interaction is also calculated but there are no input parameters involved except for enabling or disabling it entirely. In this case, the value of α is always 16 and the value of S_r is 4/3.

$$E_{3body} = \sum_{A,B,C} f_3(r_{ABC}) E_{ABC}$$

$$E_{ABC} = \frac{1+3\cos(\theta_{ABC})\cos(\theta_{BCA})\cos(\theta_{ACB})}{(r_{AB}r_{BC}r_{AC})^3} C_9^{ABC} ; C_9^{ABC} = -\sqrt{C_6^{AB}C_6^{BC}C_6^{AC}}$$

DFTD3 false (logical)

If **true**, D3 corrections are enabled for the current calculation.

DFTD3.UseXCDefaults true (logical)

When doing D3 corrections, SIESTA may use default parameters for the D3 model which were already available for some functionals. At the moment this covers only PBE, PBESol, RevPBE, RPBE, LYP, BLYP, but more of them may be added in the future. With LIBXC, HS6 and PBE0 are also available.

DFTD3.2BodyCutOff 60.0bohr (length)

Cut-off distance for 2-body dispersion interactions. Interactions corresponding to atom pairs farther away than this distance are ignored.

DFTD3.3BodyCutOff 40.0bohr (length)

Cut-off distance for 3-body dispersion interactions. Interactions corresponding to atom pairs farther away than this distance are ignored.

DFTD3.CoordinationCutoff 10.0bohr (length)

Cut-off distance for coordination number calculation (i.e. first neighbours count). This is relevant for the correct calculation of the C6 and C8 factors.

DFTD3.BJdamping true (logical)

If **true**, uses the Becke-Johnson damping for D3 interaction. If not, uses the zero-damping variant.

DFTD3.s6 1.0 (real)

Sets the value for the s6 coefficient in the D3 model, with s6 being the factor that multiplies the C6 interaction terms.

DFTD3.rs6 1.0 (real)

Sets the value for the rs6, which is the prefactor present in the C6 damping function.

DFTD3.s8 1.0 (real)

Sets the value for the s8 coefficient in the D3 model, with s8 being the factor that multiplies the C8 interaction terms.

DFTD3.rs8 1.0 (real)

Sets the value for the rs8, which is the prefactor present in the C8 damping function. This is usually set to 1.0 and not changed.

DFTD3.alpha 14.0 *(real)*

Sets the value for the the exponent in the C6 damping function. The C8 damping function automatically takes the value of alpha + 2.

DFTD3.a1 0.4 *(real)*

Value of the a1 coefficient for Becke-Johnson damping.

DFTD3.a2 5.0 *(real)*

Value of the a2 coefficient for Becke-Johnson damping.

6.26.1 DFTD3 and periodicity

SIESTA will try to guess system periodicity from the existing coordinates and lattice vectors; however, detection might fail in non-orthogonal simulation cells, giving the following output:

```
Could not guess the right amount of periodicity directions for D3.  
Enabling periodicity on all directions.
```

You can enforce the desired periodicity using the **DFTD3.Periodic** list option:

DFTD3.Periodic [1 2 3] *(list)*

Indicates which lattice vectors are periodic. A value of [1 3], for example, will enforce the periodicity for Simple-D3 in the directions of the first and third lattice vectors.

6.26.2 A note on LIBXC functionals

SIESTA has now LIBXC functionality enabled via GRIDXC. However, not every single one of the possibilities provided by that library are present in the standard D3 model. Most of the one that are already present, are already the standard SIESTA GGA functionals. So in case you want to try something different, we recommend referring to the following webpage for already existing D3 parameters:

<https://www.chemie.uni-bonn.de/pctc/mulliken-center/software/dft-d3/dft-d3>

Don't forget to set **DFTD3.UseXCDefaults** to F when adding external parameters.

6.27 Parallel options

BlockSize <automatic> *(integer)*

The orbitals are distributed over the processors when running in parallel using a 1-D block-cyclic algorithm. **BlockSize** is the number of consecutive orbitals which are located on a given processor before moving to the next one. Large values of this parameter lead to poor load balancing, while small values can lead to inefficient execution. The performance of the parallel code can be optimised by varying this parameter until a suitable value is found.

ProcessorY <automatic> *(integer)*

The mesh points are divided in the Y and Z directions (more precisely, along the second and third lattice vectors) over the processors in a 2-D grid. **ProcessorY** specifies the dimension of

the processor grid in the Y-direction and must be a factor of the total number of processors. Ideally the processors should be divided so that the number of mesh points per processor along each axis is as similar as possible.

Defaults to a value set automatically by the program. There are two methods. The default is to set **ProcessorY** to a factor of the number of processors which takes into account the relative sizes of the second and third lattice vectors. An older method based only on searching for factors of the number of processors in the set {2,3,5} can be enabled by the following option.

FFT.ProcessorY.Traditional false *(logical)*

If **true**, the program sets the default value for the FFT ProcessorY variable by searching for factors of the total number of processors in the set {2,3,5}. Note that this default value can still be overridden by setting **ProcessorY** explicitly.

6.27.1 Parallel decompositions for O(N)

Apart from the default block-cyclic decomposition of the orbital data, O(N) calculations can use other schemes which should be more efficient: spatial decomposition (based on atom proximity), and domain decomposition (based on the most efficient abstract partition of the interaction graph of the Hamiltonian).

UseDomainDecomposition false *(logical)*

This option instructs the program to employ a graph-partitioning algorithm (using the METIS library (See www.cs.umn.edu/~metis) to find an efficient distribution of the orbital data over processors. To use this option (meaningful only in parallel) the program has to be compiled with the preprocessor option SIESTA__METIS (or the deprecated ON_DOMAIN_DECOMP) and the METIS library has to be linked in.

UseSpatialDecomposition false *(logical)*

When performing a parallel order N calculation, this option instructs the program to execute a spatial decomposition algorithm in which the system is divided into cells, which are then assigned, together with the orbitals centered in them, to the different processors. The size of the cells is, by default, equal to the maximum distance at which there is a non-zero matrix element in the Hamiltonian between two orbitals, or the radius of the Localized Wannier function - which ever is the larger. If this is the case, then an orbital will only interact with other orbitals in the same or neighbouring cells. However, by decreasing the cell size and searching over more cells it is possible to achieve better load balance in some cases. This is controlled by the variable **RcSpatial**.

NOTE: the distribution algorithm is quite fragile and a careful tuning of **RcSpatial** might be needed. This option is therefore not enabled by default.

RcSpatial <maximum orbital range> *(length)*

Controls the cell size during the spatial decomposition.

6.28 Efficiency options

DirectPhi false *(logical)*

The calculation of the matrix elements on the mesh requires the value of the orbitals on the

mesh points. This array represents one of the largest uses of memory within the code. If set to true this option allows the code to generate the orbital values when needed rather than storing the values. This obviously costs more computer time but will make it possible to run larger jobs where memory is the limiting factor.

This controls whether the values of the orbitals at the mesh points are stored or calculated on the fly.

Poisson.Method `FFTW` *(string)*

Method or backend used to calculate the Fourier transform in SIESTA in the Poisson equation. If SIESTA has not been compiled with FFTW support, it will use the internal implementation instead.

6.29 Memory, CPU-time, and Wall time accounting options

AllocReportLevel `0` *(integer)*

Sets the level of the allocation report, printed in file `SystemLabel.alloc`. However, not all the allocated arrays are included in the report (this will be corrected in future versions). The allowed values are:

- level 0 : no report at all (the default)
- level 1 : only total memory peak and where it occurred
- level 2 : detailed report printed only at normal program termination
- level 3 : detailed report printed at every new memory peak
- level 4 : print every individual (re)allocation or deallocation

NOTE: In MPI runs, only node-0 peak reports are produced.

AllocReportThreshold `0.` *(real)*

Sets the minimum size (in bytes) of the arrays whose memory use is individually printed in the detailed allocation reports (levels 2 and 3). It does not affect the reported memory sums and peaks, which always include all arrays.

TimerReportThreshold `0.` *(real)*

Sets the minimum fraction, of total CPU time, of the subroutines or code sections whose CPU time is individually printed in the detailed timer reports. To obtain the accounting of MPI communication times in parallel executions, you must compile with option `-DMPI_TIMING`. In serial execution, the CPU times are printed at the end of the output file. In parallel execution, they are reported in a separated file named `SystemLabel.times`.

UseTreeTimer `false` *(logical)*

Enable an experimental timer which is based on wall time on the master node and is aware of the tree-structure of the timed sections. At the end of the program, a report is generated in the output file, and a `time.json` file in JSON format is also written. This file can be used by third-party scripts to process timing data.

NOTE: , if used with the PEXSI solver (see Sec. 6.15) this defaults to **true**.

UseParallelTimer `true` *(logical)*

Determine whether timings are performed in parallel. This may introduce slight overhead.

NOTE: , if used with the PEXSI solver (see Sec. 6.15) this defaults to **false**.

TimingSplitScfSteps **false** *(logical)*

The timings for individual scf steps will be recorded separately.

NOTE: The 'tree' timer should be used to make meaningful use of this information. It is enabled by default if this variable is **true**.

MaxWalltime **Infinity** *(real time)*

Set an internal limit to the wall time allotted to the program's execution. Typically this is related to the external limit imposed by queuing systems. The code checks its wall time periodically and will abort if nearing the limit, with some slack left for clean-up operations (proper closing of files, emergency output...), as determined by **MaxWalltime.Slack**. See Sec. 18 for available units of time (**s**, **mins**, **hours**, **days**).

MaxWalltime.Slack **5 s** *(real time)*

The code checks its wall time T_{wall} periodically and will abort if $T_{\text{wall}} > T_{\text{max}} - T_{\text{slack}}$, so that some slack is left for any clean-up operations.

6.30 The catch-all option UseSaveData

This is a dangerous feature, and is deprecated, but retained for historical compatibility. Use the individual options instead.

UseSaveData **false** *(logical)*

Instructs to use as much information as possible stored from previous runs in files **SystemLabel.XV**, **SystemLabel.DM** and **SystemLabel.LWF**,

NOTE: if the files are not existing it will read the information from the **fdf** file.

6.31 Output of information for Denchar

The program **denchar** in **Util/Denchar** can generate charge-density and wavefunction information in real space.

Write.Denchar **false** *(logical)*

Instructs to write information needed by the utility program **DENCHAR** (by J. Junquera and P. Ordejón) to generate valence charge densities and/or wavefunctions in real space (see **Util/Denchar**). The information is written in files **SystemLabel.PLD** and **SystemLabel.DIM**.

To run **DENCHAR** you will need, apart from the **.PLD** and **.DIM** files, the Density-Matrix (**DM**) file and/or a wavefunction (**.WFSX**) file, and the **.ion** files containing the information about the basis orbitals.

6.32 NetCDF (CDF4) output file

NOTE: this requires SIESTA compiled with CDF4 support.

To unify and construct a simple output file for an entire SIESTA calculation a generic NetCDF file will be created if SIESTA is compiled with **ncdf** support, see Sec. 2.3 and the **ncdf** section.

Generally all output to NetCDF flags, **SaveElectrostaticPotential**, etc. apply to this file as well. One may control the output file with compressibility and parallel I/O, if needed.

CDF.Save `false` *(logical)*

Create the `SystemLabel.nc` file which is a NetCDF file.

This file will be created with a large set of *groups* which make separating the quantities easily. Also it will inherently denote the units for the stored quantities.

NOTE: this option is not available for MD/relaxations, only for force constant runs.

CDF.Compress `0` *(integer)*

Integer between 0 and 9. The former represents *no* compressing and the latter is the highest compressing.

The higher the number the more computation time is spent on compressing the data. A good compromise between speed and compression is 3.

NOTE: if one requests parallel I/O (**CDF.MPI**) this will automatically be set to 0. One cannot perform parallel IO and compress the data simultaneously.

NOTE: instead of using SIESTA for compression you may compress after execution by:

```
nccopy -d 3 -s noncompressed.nc compressed.nc
```

CDF.MPI `false` *(logical)*

Write `SystemLabel.nc` in parallel using MPI for increased performance. This has almost no memory overhead but may for very large number of processors saturate the file-system.

NOTE: this is an experimental flag.

CDF.Grid.Precision `single|double` *(string)*

At which precision should the real-space grid quantities be stored, such as the density, electrostatic potential etc.

7 STRUCTURAL RELAXATION, PHONONS, AND MOLECULAR DYNAMICS

This functionality is not SIESTA-specific, but is implemented to provide a more complete simulation package. The program has an outer geometry loop: it computes the electronic structure (and thus the forces and stresses) for a given geometry, updates the atomic positions (and maybe the cell vectors) accordingly and moves on to the next cycle. If there are molecular dynamics options missing you are highly recommend to look into **MD.TypeOfRun Lua** or **MD.TypeOfRun Master**.

Several options for MD and structural optimizations are implemented, selected by

MD.TypeOfRun `CG` *(string)*

CG Performs an atomic coordinates optimization by using the conjugate gradients method. If **MD.VariableCell** is enabled (see below), the optimization includes the cell vectors.

Broyden Performs an atomic coordinates optimization by using a modified Broyden method, which falls within the Quasi-Newton family of algorithms. If **MD.VariableCell** is enabled

(see below), the optimization includes the cell vectors.

FIRE Performs an atomic coordinates optimization by using the Fast Inertial Relaxation Engine (E. Bitzek et al, PRL 97, 170201, (2006)). If **MD.VariableCell** is enabled (see below), the optimization includes the cell vectors. FIRE avoids the need for linear search, thus making each individual iteration faster when compared to Quasi-Newton methods. However, it also needs more iterations to converge, so its efficiency is system-dependent.

Verlet Standard Velocity-Verlet algorithm for NVE molecular dynamics.

Nose Constant temperature (NVT) MD with using a Nosé thermostat.

ParrinelloRahman Constant pressure (NPE) MD, controlled by the Parrinello-Rahman method.

NoseParrinelloRahman Constant temperature and pressure (NPT) MD using both methods above, the Nosé thermostat and the Parrinello-Rahman method.

Anneal Constant temperature and/or pressure MD (see the variable **MD.AnnealOption** below), using a very simple velocity rescaling method (i.e. Berendsen thermostat and/or barostat^[3]). It can be used to quickly equilibrate a system to a desired temperature and pressure. However, atomic velocities resulting from this option are non-canonical and thus tend to produce physically-inaccurate results. Therefore, one must rely on the Nosé and/or ParrinelloRahman options for production MD runs after the equilibration is done.

FC Compute force constants matrix for phonon calculations.

Master|Forces Receive coordinates from, and return forces to, an external driver program, using MPI, Unix pipes, or Inet sockets for communication. The routines in module **fsiesta** allow the user's program to perform this communication transparently, as if SIESTA were a conventional force-field subroutine. See **Util/SiestaSubroutine/README** for details. WARNING: if this option is specified without a driver program sending data, siesta may hang without any notice.

See directory **Util/Scripting** for other driving options.

Lua Fully control the MD cycle and convergence path using an external Lua script.

With an external Lua script one may control nearly everything from a script. One can query *any* internal data-structures in SIESTA and, similarly, return *any* data thus overwriting the internals. A list of ideas which may be implemented in such a Lua script are:

- New geometry relaxation algorithms
- NEB calculations
- New MD routines
- Convergence tests of **Mesh.Cutoff** and **kgrid.MonkhorstPack**, or other parameters (currently basis set optimizations cannot be performed in the Lua script).

Sec. 10 for additional details (and a description of **flos** which implements some of the above mentioned items).

Using this option requires the compilation of SIESTA with the **flook** library. If SIESTA is not compiled as prescribed in Sec. 2.3 this option will make SIESTA die.

TDED New option to perform time-dependent electron dynamics simulations (TDED) within RT-TDDFT. For more details see Sec. 9.

The second run of SIESTA uses this option with the files `SystemLabel.TDWF` and `SystemLabel.TDXV` present in the working directory. In this option ions and electrons are assumed to move simultaneously. The occupied electronic states are time-evolved instead of the usual SCF calculations in each step. Choose this option even if you intend to do only-electron dynamics. If you want to do an electron dynamics-only calculation set **MD.FinalTimeStep** equal to 1. For optical response calculations switch off the external field during the second run. The **MD.LengthTimeStep**, unlike in the standard MD simulation, is defined by multiplication of **TDED.TimeStep** and **TDED.Nsteps**. In TDDFT calculations, the user defined **MD.LengthTimeStep** is ignored.

NOTE: if **Compat.Pre-v4-Dynamics** is **true** this will default to **Verlet**.

Note that some options specified in later variables (like quenching) modify the behavior of these MD options.

Appart from being able to act as a force subroutine for a driver program that uses module `fsiesta`, SIESTA is also prepared to communicate with the i-PI code (see <https://github.com/i-pi/i-pi>). To do this, SIESTA must be started after i-PI (it acts as a client of i-PI, communicating with it through Inet or Unix sockets), and the following lines must be present in the `.fdf` data file:

```
MD.TypeOfRun      Master      # equivalent to 'Forces'
Master.code        i-pi        # ( fsiesta | i-pi )
Master.interface   socket      # ( pipes | socket | mpi )
Master.address     localhost    # or driver's IP, e.g. 150.242.7.140
Master.port        10001       # 10000+siesta_process_order
Master.socketType  inet        # ( inet | unix )
```

7.1 Compatibility with pre-v4 versions

Starting in the summer of 2015, some changes were made to the behavior of the program regarding default dynamics options and choice of coordinates to work with during post-processing of the electronic structure. The changes are:

- The default dynamics option is “CG” instead of “Verlet”.
- The coordinates, if moved by the dynamics routines, are reset to their values at the previous step for the analysis of the electronic structure (band structure calculations, DOS, LDOS, etc).
- Some output files reflect the values of the “un-moved” coordinates.
- The default convergence criteria is now *both* density and Hamiltonian convergence, see **SCF.DM.Converge** and **SCF.H.Converge**.

To recover the previous behavior, the user can turn on the compatibility switch **Compat.Pre-v4-Dynamics**, which is off by default.

Note that complete compatibility cannot be perfectly guaranteed.

7.2 Structural relaxation

In this mode of operation, the program moves the atoms (and optionally the cell vectors) trying to minimize the forces (and stresses) on them.

These are the options common to all relaxation methods. If the `Zmatrix` input option is in effect (see Sec. 6.4.2) the `Zmatrix`-specific options take precedence. The 'MD' prefix is misleading but kept for historical reasons.

MD.VariableCell `false` *(logical)*

If **true**, the lattice is relaxed together with the atomic coordinates. It allows to target hydrostatic pressures or arbitrary stress tensors. See **MD.MaxStressTol**, **Target.Pressure**, **Target.Stress.Voigt**, **Constant.Volume**, and **MD.PreconditionVariableCell**.

NOTE: only compatible with **MD.TypeOfRun** `CG`, `Broyden` or `fire`.

Constant.Volume `false` *(logical)*

deprecates: **MD.ConstantVolume**

If **true**, the cell volume is kept constant in a variable-cell relaxation: only the cell shape and the atomic coordinates are allowed to change. Note that it does not make much sense to specify a target stress or pressure in this case, except for anisotropic (traceless) stresses. See **MD.VariableCell**, **Target.Stress.Voigt**.

NOTE: only compatible with **MD.TypeOfRun** `CG`, `Broyden` or `fire`.

MD.RelaxCellOnly `false` *(logical)*

If **true**, only the cell parameters are relaxed (by the Broyden or FIRE method, not `CG`). The atomic coordinates are re-scaled to the new cell, keeping the fractional coordinates constant. For **Zmatrix** calculations, the fractional position of the first atom in each molecule is kept fixed, and no attempt is made to rescale the bond distances or angles.

NOTE: only compatible with **MD.TypeOfRun** `Broyden` or `fire`.

MD.MaxForceTol `0.04 eV/Ang` *(force)*

Force tolerance in coordinate optimization. Run stops if the maximum atomic force is smaller than **MD.MaxForceTol** (see **MD.MaxStressTol** for variable cell).

MD.MaxStressTol `1 GPa` *(pressure)*

Stress tolerance in variable-cell `CG` optimization. Run stops if the maximum atomic force is smaller than **MD.MaxForceTol** and the maximum stress component is smaller than **MD.MaxStressTol**.

Special consideration is needed if used with Sankey-type basis sets, since the combination of orbital kinks at the cutoff radii and the finite-grid integration originate discontinuities in the stress components, whose magnitude depends on the cutoff radii (or energy shift) and the mesh cutoff. The tolerance has to be larger than the discontinuities to avoid endless optimizations if the target stress happens to be in a discontinuity.

MD.Steps `0` *(integer)*

deprecates: **MD.NumCGsteps**

Maximum number of steps in a minimization routine (the minimization will stop if tolerance is reached before; see **MD.MaxForceTol** below).

NOTE: The old flag **MD.NumCGsteps** will remain for historical reasons.

MD.MaxDispl 0.2 Bohr (length)
deprecates: **MD.MaxCGDispl**

Maximum atomic displacements in an optimization move.

In the Broyden optimization method, it is also possible to limit indirectly the *initial* atomic displacements using **MD.Broyden.Initial.Inverse.Jacobian**. For the **FIRE** method, the same result can be obtained by choosing a small time step.

Note that there are Zmatrix-specific options that override this option.

NOTE: The old flag **MD.MaxCGDispl** will remain for historical reasons.

MD.PreconditionVariableCell 5 Ang (length)

A length to multiply to the strain components in a variable-cell optimization. The strain components enter the minimization on the same footing as the coordinates. For good efficiency, this length should make the scale of energy variation with strain similar to the one due to atomic displacements. It is also used for the application of the **MD.MaxDispl** value to the strain components.

ZM.ForceTolLength 0.00155574 Ry/Bohr (force)

Parameter that controls the convergence with respect to forces on Z-matrix lengths

ZM.ForceTolAngle 0.00356549 Ry/rad (torque)

Parameter that controls the convergence with respect to forces on Z-matrix angles

ZM.MaxDisplLength 0.2 Bohr (length)

Parameter that controls the maximum change in a Z-matrix length during an optimisation step.

ZM.MaxDisplAngle 0.003 rad (angle)

Parameter that controls the maximum change in a Z-matrix angle during an optimisation step.

7.2.1 Conjugate-gradients optimization

This was historically the default geometry-optimization method, and all the above options were introduced specifically for it, hence their names. The following pertains only to this method:

MD.UseSaveCG false (logical)

Instructs to read the conjugate-gradient history information stored in file **SystemLabel.CG** by a previous run.

NOTE: to get actual continuation of interrupted CG runs, use together with **MD.UseSaveXV true** with the **.XV** file generated in the same run as the CG file. If the required file does not exist, a warning is printed but the program does not stop. Overrides **UseSaveData**.

NOTE: no such feature exists yet for a Broyden-based relaxation.

7.2.2 Broyden optimization

It uses the modified Broyden algorithm to build up the Jacobian matrix. (See D.D. Johnson, PRB 38, 12807 (1988)). (Note: This is not BFGS.)

MD.Broyden.History.Steps 5 *(integer)*

Number of relaxation steps during which the modified Broyden algorithm builds up the Jacobian matrix.

MD.Broyden.Cycle.On.Maxit true *(logical)*

Upon reaching the maximum number of history data sets which are kept for Jacobian estimation, throw away the oldest and shift the rest to make room for a new data set. The alternative is to re-start the Broyden minimization algorithm from a first step of a diagonal inverse Jacobian (which might be useful when the minimization is stuck).

MD.Broyden.Initial.Inverse.Jacobian 1 *(real)*

Initial inverse Jacobian for the optimization procedure. (The units are those implied by the internal SIESTA usage. The default value seems to work well for most systems.

7.2.3 FIRE relaxation

Implementation of the Fast Inertial Relaxation Engine (FIRE) method (E. Bitzek et al, PRL 97, 170201, (2006) in a manner compatible with the CG and Broyden modes of relaxation. (An older implementation activated by the **MD.FireQuench** variable is still available).

MD.FIRE.TimeStep $\langle$ MD.LengthTimeStep $\rangle$ *(time)*

The (fictitious) time-step for FIRE relaxation. This is the main user-variable when the option **FIRE** for **MD.TypeOfRun** is active.

NOTE: the default value is encouraged to be changed as the link to **MD.LengthTimeStep** is misleading.

There are other low-level options tunable by the user (see the routines `fire_optim` and `cell_fire_optim` for more details.

7.3 Target stress options

Useful for structural optimizations and constant-pressure molecular dynamics.

Target.Pressure 0 GPa *(pressure)*

deprecates: MD.TargetPressure

Target pressure for Parrinello-Rahman method, variable cell optimizations, and annealing options.

NOTE: this is only compatible with **MD.TypeOfRun** `ParrinelloRahman`, `NoseParrinelloRahman`, `CG`, `Broyden` or `FIRE` (variable cell), or `Anneal` (if **MD.AnnealOption** `Pressure` or `TemperatureandPressure`).

%block Target.Stress.Voigt -1 -1 -1 0 0 0 *(block)*

deprecates: MD.TargetStress

External or target stress tensor for variable cell optimizations. Stress components are given in a line, in the Voigt order `xx`, `yy`, `zz`, `yz`, `xz`, `xy`. In units of **Target.Pressure**, but with the opposite sign. For example, a uniaxial compressive stress of 2 GPa along the 100 direction would be given by

```

Target.Pressure  2. GPa
%block Target.Stress.Voigt
  -1.0  0.0  0.0  0.0  0.0  0.0
%endblock

```

Only used if **MD.TypeOfRun** is **CG**, **Broyden** or **FIRE** and **MD.VariableCell** is **true**.

```
%block MD.TargetStress  -1 -1 -1 0 0 0
```

(block)

deprecated by: Target.Stress.Voigt

Same as **Target.Stress.Voigt** but the order is same as older SIESTA version (prior to 4.1). Order is **xx**, **yy**, **zz**, **xy**, **xz**, **yz**.

```
MD.RemoveIntramolecularPressure  false
```

(logical)

If **true**, the contribution to the stress coming from the internal degrees of freedom of the molecules will be subtracted from the stress tensor used in variable-cell optimization or variable-cell molecular-dynamics. This is done in an approximate manner, using the virial form of the stress, and assuming that the “mean force” over the coordinates of the molecule represents the “inter-molecular” stress. The correction term was already computed in earlier versions of SIESTA and used to report the “molecule pressure”. The correction is now computed molecule-by-molecule if the Zmatrix format is used.

If the intra-molecular stress is removed, the corrected static and total stresses are printed in addition to the uncorrected items. The corrected Voigt form is also printed.

NOTE: versions prior to 4.1 (also 4.1-beta releases) printed the Voigt stress-tensor in this format: [x, y, z, xy, yz, xz]. In 4.1 and later SIESTA *only* show the correct Voigt representation: [x, y, z, yz, xz, xy].

7.4 Molecular dynamics

In this mode of operation, the program moves the atoms (and optionally the cell vectors) in response to the forces (and stresses), using the classical equations of motion.

Note that the **Zmatrix** input option (see Sec. 6.4.2) is not compatible with molecular dynamics. The initial geometry can be specified using the Zmatrix format, but the Zmatrix generalized coordinates will not be updated.

```
MD.InitialTimeStep  1
```

(integer)

Initial time step of the MD simulation. In the current version of SIESTA it must be 1.

Used only if **MD.TypeOfRun** is not **CG** or **Broyden**.

```
MD.FinalTimeStep  <MD.Steps>
```

(integer)

Final time step of the MD simulation.

```
MD.LengthTimeStep  1 fs
```

(time)

Length of the time step of the MD simulation.

```
MD.InitialTemperature  0 K
```

(temperature/energy)

Initial temperature for the MD run. The atoms are assigned random velocities drawn from the Maxwell-Boltzmann distribution with the corresponding temperature. The constraint of zero center of mass velocity is imposed.

NOTE: only used if **MD.TypeOfRun** Verlet, Nose, ParrinelloRahman, NoseParrinelloRahman or Anneal.

MD.TargetTemperature 0 K *(temperature/energy)*

Target temperature for Nose thermostat and annealing options.

NOTE: only used if **MD.TypeOfRun** Nose, NoseParrinelloRahman or Anneal if **MD.AnnealOption** is Temperature or TemperatureandPressure.

MD.NoseMass 100 Ry fs² *(moment of inertia)*

Generalized mass of Nose variable. This determines the time scale of the Nose variable dynamics, and the coupling of the thermal bath to the physical system.

Only used for Nose MD runs.

MD.ParrinelloRahmanMass 100 Ry fs² *(moment of inertia)*

Generalized mass of Parrinello-Rahman variable. This determines the time scale of the Parrinello-Rahman variable dynamics, and its coupling to the physical system.

Only used for Parrinello-Rahman MD runs.

MD.AnnealOption TemperatureAndPressure *(string)*

Type of annealing MD to perform. The target temperature or pressure are achieved by velocity and unit cell rescaling, in a given time determined by the variable **MD.TauRelax** below. These are based on the Berendsen thermostat and barostats, respectively^[3].

Temperature Reach a target temperature by velocity rescaling

Pressure Reach a target pressure by scaling of the unit cell size and shape

TemperatureandPressure Reach a target temperature and pressure by velocity rescaling and by scaling of the unit cell size and shape

Only applicable for **MD.TypeOfRun** Anneal.

MD.TauRelax 100 fs *(time)*

Relaxation time to reach target temperature and/or pressure in annealing MD. Note that this is a “relaxation time”, and as such it gives a rough estimate of the time needed to achieve the given targets. As a normal simulation also exhibits oscillations, the actual time needed to reach the *averaged* targets will be significantly longer.

When using the barostat, the actual time required to reach the target pressure will depend on the ratio between **MD.TauRelax** and **MD.BulkModulus**.

Only applicable for **MD.TypeOfRun** Anneal.

MD.BulkModulus 100 Ry/Bohr³ *(pressure)*

Estimate (may be rough) of the bulk modulus of the system. This is needed to set the rate of change of cell shape to reach target pressure in annealing MD.

Only applicable for **MD.TypeOfRun** Anneal, when **MD.AnnealOption** is Pressure or TemperatureandPressure

7.5 Output options for dynamics

Every time the atoms move, either during coordinate relaxation or molecular dynamics, their positions **predicted for next step** and **current** velocities are stored in file `SystemLabel.XV`. The shape of the unit cell and its associated 'velocity' (in Parrinello-Rahman dynamics) are also stored in this file.

WriteCoorInitial `true` *(logical)*

It determines whether the initial atomic coordinates of the simulation are dumped into the main output file. These coordinates correspond to the ones actually used in the first step (see the section on precedence issues in structural input) and are output in Cartesian coordinates in Bohr units.

It is not affected by the setting of **LongOutput**.

WriteCoorStep `false` *(logical)*

If **true**, it writes the atomic coordinates to standard output at every MD time step or relaxation step. The coordinates are always written in the `SystemLabel.XV` file, but overridden at every step. They can be also accumulated in the `.MD` or `SystemLabel.MDX` files depending on **WriteMDHistory**.

WriteForces `false` *(logical)*

If **true**, it writes the atomic forces to the output file at every MD time step or relaxation step. Note that the forces of the last step can be found in the file `SystemLabel.FA`. If constraints are used, the file `SystemLabel.FAC` is also written.

WriteMDHistory `false` *(logical)*

If **true**, SIESTA accumulates the molecular dynamics trajectory in the following files:

- `SystemLabel.MD` : atomic coordinates and velocities (and lattice vectors and their time derivatives, if the dynamics implies variable cell). The information is stored unformatted for postprocessing with utility programs to analyze the MD trajectory.
- `SystemLabel.MDE` : shorter description of the run, with energy, temperature, etc., per time step.

These files are accumulative even for different runs.

The trajectory of a molecular dynamics run (or a conjugate gradient minimization) can be accumulated in different files: `SystemLabel.MD`, `SystemLabel.MDE`, and `SystemLabel.ANI`. The first file keeps the whole trajectory information, meaning positions and velocities at every time step, including lattice vectors if the cell varies. NOTE that the positions (and maybe the cell vectors) stored at each time step are the **predicted** values for the next step. Care should be taken if joint position-velocity correlations need to be computed from this file. The second gives global information (energy, temperature, etc), and the third has the coordinates in a form suited for XMol animation. See the **WriteMDHistory** and **WriteMDXmol** data descriptors above for information. SIESTA always appends new information on these files, making them accumulative even for different runs.

The `iomd` subroutine can generate both an unformatted file `.MD` (default) or ASCII formatted files `.MDX` and `.MDC` containing the atomic and lattice trajectories, respectively. Edit the file to change the settings if desired.

Write.OrbitalIndex `true` *(logical)*

If **true** it causes the writing of an extra file named `SystemLabel.ORB_INDX` containing all orbitals used in the calculation.

Its formatting is clearly specified at the end of the file.

7.6 Restarting geometry optimizations and MD runs

Every time the atoms move, either during coordinate relaxation or molecular dynamics, their **positions predicted for next step** and **current velocities** are stored in file `SystemLabel.XV`, where `SystemLabel` is the value of that FDF descriptor (or 'siesta' by default). The shape of the unit cell and its associated 'velocity' (in Parrinello-Rahman dynamics) are also stored in this file. For MD runs of type Verlet, Parrinello-Rahman, Nose, Nose-Parrinello-Rahman, or Anneal, a file named `SystemLabel.VERLET_RESTART`, `SystemLabel.PR_RESTART`, `SystemLabel.NOSE_RESTART`, `SystemLabel.NPR_RESTART`, or `SystemLabel.ANNEAL_RESTART`, respectively, is created to hold the values of auxiliary variables needed for a completely seamless continuation.

If the restart file is not available, a simulation can still make use of the XV information, and "restart" by basically repeating the last-computed step (the positions are shifted backwards by using a single Euler-like step with the current velocities as derivatives). While this feature does not result in seamless continuations, it allows cross-restarts (those in which a simulation of one kind (e.g., Anneal) is followed by another (e.g., Nose)), and permits to re-use dynamical information from old runs.

This restart fix is not satisfactory from a fundamental point of view, so the MD subsystem in SIESTA will have to be redesigned eventually. In the meantime, users are reminded that the scripting hooks being steadily introduced (see `Util/Scripting`) might be used to create custom-made MD scripts.

7.7 Use of general constraints

Note: The Zmatrix format (see Sec. 6.4.2) provides an alternative constraint formulation which can be useful for system involving molecules.

%block Geometry.Constraints `(None)` *(block)*

Constrains certain atomic coordinates or cell parameters in a consistent method.

There are a high number of configurable parameters that may be used to control the relaxation of the coordinates.

NOTE: SIESTA prints out a small section of how the constraints are recognized.

atom|position Fix certain atomic coordinates.

This option takes a variable number of integers which each correspond to the atomic index (or input sequence) in **AtomicCoordinatesAndAtomicSpecies**.

atom is now the preferred input option while **position** still works for backwards compatibility.

One may also specify ranges of atoms according to:

atom *A* [*B* [*C* [...]]] A sequence of atomic indices which are constrained.

atom from *A* **to** *B* [**step** *s*] Here atoms *A* up to and including *B* are constrained. If **step** *<s>* is given, the range *A:B* will be taken in steps of *s*.

`atom from 3 to 10 step 2`

will constrain atoms 3, 5, 7 and 9.

atom from A plus/minus B [step s] Here atoms A up to and including $A + B - 1$ are constrained. If **step** $\langle s \rangle$ is given, the range $A:A + B - 1$ will be taken in steps of s .

atom [A , B -- C [step s], D] Equivalent to **from ... to** specification, however in a shorter variant. Note that the list may contain arbitrary number of ranges and/or individual indices.

atom [2, 3 -- 10 step 2, 6]

will constrain atoms 2, 3, 5, 7, 9 and 6.

atom [2, 3 -- 6, 8]

will constrain atoms 2, 3, 4, 5, 6 and 8.

atom all Constrain all atoms.

NOTE: these specifications are apt for *directional* constraints.

Z Equivalent to **atom** with all indices of the atoms that have atomic number equal to the specified number.

NOTE: these specifications are apt for *directional* constraints.

species-i Equivalent to **atom** with all indices of the atoms that have species according to the **ChemicalSpeciesLabel** and **AtomicCoordinatesAndAtomicSpecies**.

NOTE: these specifications are apt for *directional* constraints.

center One may retain the coordinate center of a range of atoms (say molecules or other groups of atoms).

Atomic indices may be specified according to **atom**.

NOTE: these specifications are apt for *directional* constraints.

rigid|molecule Move a selection of atoms together as though they were one atom.

The forces are summed and averaged to get a net-force on the entire molecule.

Atomic indices may be specified according to **atom**.

NOTE: these specifications are apt for *directional* constraints.

rigid-max|molecule-max Move a selection of atoms together as though they were one atom.

The maximum force acting on one of the atoms in the selection will be expanded to act on all atoms specified.

Atomic indices may be specified according to **atom**.

NOTE: these specifications are apt for *directional* constraints.

cell-angle Control whether the cell angles (α , β , γ) may be altered.

This takes either one or more of **alpha/beta/gamma** as argument.

alpha is the angle between the 2nd and 3rd cell vector.

beta is the angle between the 1st and 3rd cell vector.

gamma is the angle between the 1st and 2nd cell vector.

NOTE: currently only one angle can be constrained at a time and it forces only the spanning vectors to be relaxed.

cell-vector Control whether the cell vectors (A , B , C) may be altered.

This takes either one or more of **A/B/C** as argument.

Constraining the cell-vectors are only allowed if they only have a component along their respective Cartesian direction. I.e. **B** must only have a *y*-component.

stress Control which of the 6 stress components are constrained.

Numbers $1 \leq i \leq 6$ where 1 corresponds to the *XX* stress-component, 2 is *YY*, 3 is *ZZ*, 4 is *YZ/ZY*, 5 is *XZ/ZX* and 6 is *XY/YX*.

The text specifications are also allowed.

routine This calls the **constr** routine specified in the file: **constr.f**. Without having changed the corresponding source file, this does nothing. See details and comments in the source-file.

clear Remove constraints on selected atoms from all previously specified constraints.

This may be handy when specifying constraints via **Z** or **species-i**.

Atomic indices may be specified according to **atom**.

clear-prev Remove constraints on selected atoms from the *previous* specified constraint.

This may be handy when specifying constraints via **Z** or **species-i**.

Atomic indices may be specified according to **atom**.

NOTE: two consecutive **clear-prev** may be used in conjunction as though the atoms were specified on the same line.

It is instructive to give an example of the input options presented.

Consider a benzene molecule (C_6H_6) and we wish to relax all Hydrogen atoms (and no stress in *x* and *y* directions). This may be accomplished with this

```
%block Geometry.Constraints
  Z 6
  stress 1 2
%endblock
```

Or as in this example

```
%block AtomicCoordinatesAndAtomicSpecies
... .. 1 # C 1
... .. 2 # H 2
... .. 1 # C 3
... .. 2 # H 4
... .. 1 # C 5
... .. 2 # H 6
... .. 1 # C 7
... .. 2 # H 8
... .. 1 # C 9
... .. 2 # H 10
... .. 1 # C 11
... .. 2 # H 12
  stress XX YY
%endblock
%block Geometry.Constraints
  atom from 1 to 12 step 2
  stress XX YY
%endblock
%block Geometry.Constraints
```

```

    atom [1 -- 12 step 2]
    stress XX 2
%endblock
%block Geometry.Constraints
    atom all
    clear-prev [2 -- 12 step 2]
    stress 1 YY
%endblock

```

where the 3 last blocks all create the same result.

Finally, the *directional* constraint is an important and often useful feature. The directional constraints will subtract the force projected onto the direction specified. Hence an x directional constraint will remove the force component along the x direction $f_x \rightarrow 0$.

When relaxing complex structures it may be advantageous to first relax along a given direction (where you expect the stress to be the largest) and subsequently let it fully relax. Another example would be to relax the binding distance between a molecule and a surface, before relaxing the entire system by forcing the molecule and adsorption site to relax together. To use directional constraints one may provide an additional 3 *reals* after the **atom/rigid**. For instance in the previous example (benzene) one may first relax all Hydrogen atoms along the y and z Cartesian vector by constraining the x Cartesian vector

```

%block Geometry.Constraints
    Z 6 # constrain Carbon
    Z 1 1. 0. 0. # constrain Hydrogen along x Cartesian vector
%endblock

```

Note that you *must* append a “.” to denote it a real. The vector specified need not be normalized. Also, if you want it to be constrained along the x - y vector you may do

```

%block Geometry.Constraints
    Z 6
    Z 1 1. 1. 0.
%endblock

```

Therefore the directional constraint will remove the force components that projects onto the direction specified.

7.8 Phonon calculations

If **MD.TypeOfRun** is **FC**, SIESTA sets up a special outer geometry loop that displaces individual atoms along the coordinate directions to build the force-constant matrix.

The output (see below) can be analyzed to extract phonon frequencies and vectors with the VIBRA package in the Util/Vibra directory. For computing the Born effective charges together with the force constants, see **BornCharge**.

FC.Displacement 0.04 Bohr (length)
deprecates: MD.FCDispl

Displacement to use for the computation of the force constant matrix for phonon calculations.

FC.First 1 (integer)
deprecates: MD.FCFfirst

Index of first atom to displace for the computation of the force constant matrix for phonon calculations.

FC.Last $\langle$ **FC.First** $\rangle$ (integer)
deprecates: MD.FCLast

Index of last atom to displace for the computation of the force constant matrix for phonon calculations.

The force-constants matrix is written in file **SystemLabel.FC**. The format is the following: for the displacement of each atom in each direction, the forces on each of the other atoms is written (divided by the value of the displacement), in units of $\text{eV}/\text{\AA}^2$. Each line has the forces in the x , y and z direction for one of the atoms.

If constraints are used, the file **SystemLabel.FCC** is also written.

FC.Save.dHS **false** (logical)

For **MD.TypeOfRun FC**, if **true**, SIESTA produces a single netCDF file **.dHSdR.nc** with the derivatives of the Hamiltonian and overlap matrix for each displaced atom along each Cartesian direction. *The derivatives are only calculated in the unit cell, not the auxiliary supercell.*

FC.dHdR.Tolerance -1 Ry/Bohr (force)

Threshold controlling which elements of the Hamiltonian derivative should be stored in **.dHSdR.nc**. All matrix elements smaller than the threshold are discarded. If threshold is negative no elements are discarded.

FC.dSdR.Tolerance -1 1/Bohr (inverse length)

Threshold controlling which elements of the Hamiltonian derivative should be stored in **.dHSdR.nc**. All matrix elements smaller than the threshold are discarded. If threshold is negative no elements are discarded.

8 DFT+U

NOTE: This implementation works for both LDA and GGA, hence named DFT+U in the main text.

NOTE: Current implementation is based on the simplified rotationally invariant DFT+U formulation of Dudarev and collaborators [see, Dudarev *et al.*, Phys. Rev. B **57**, 1505 (1998)]. Although the input allows to define independent values of the U and J parameters for each atomic shell, in the actual calculation the two parameters are combined to produce an effective Coulomb repulsion $U_{\text{eff}} = U - J$. U_{eff} is the parameter actually used in the calculations for the time being.

For large or intermediate values of U_{eff} the convergence is sometimes difficult. A step-by-step increase of the value of U_{eff} can be advisable in such cases.

If DFT+U is used in combination with non-collinear or spin-orbit coupling, the Liechtenstein approach is implemented, where the U and the exchange J parameters are treated separately [see, A. I. Liechtenstein *et al.*, Phys. Rev. B **52**, R5467 (1995)]. The generalization for the spin-orbit or non-collinear cases follows the recipe given by E. Bousquet and N. Spaldin, Phys. Rev. B **82**, 220402(R) (2010). Currently, only the d -shell can be considered as the correlated shell

where the U and J are applied. The computation of the occupancies on the orbitals of the correlated shells is done following the same recipe as for the Dudarev approach. That means that the following entries related with the generation of the DFT+U projectors are still relevant. However, the input options **DFTU.FirstIteration**, **DFTU.ThresholdTol**, **DFTU.PopTol**, and **DFTU.PotentialShift** are irrelevant when DFT+U is used in combination with spin-orbit or non-collinear magnetism.

DFTU.ProjectorGenerationMethod 2 (integer)

Generation method of the DFT+U projectors. The DFT+U projectors are the localized functions used to calculate the local populations used in a Hubbard-like term that modifies the LDA Hamiltonian and energy. It is important to recall that DFT+U projectors should be quite localized functions. Otherwise the calculated populations lose their atomic character and physical meaning. Even more importantly, the interaction range can increase so much that jeopardizes the efficiency of the calculation.

Two methods are currently implemented:

- 1 Projectors are slightly-excited numerical atomic orbitals similar to those used as an automatic basis set by SIESTA. The radii of these orbitals are controlled using the parameter **DFTU.EnergyShift** and/or the data included in the block **DFTU.Proj** (quite similar to the data block **PAO.Basis** used to specify the basis set, see below).
- 2 Projectors are exact solutions of the pseudoatomic problem (and, in principle, are not strictly localized) which are cut using a Fermi function $1/\{1 + \exp[(r - r_c)\omega]\}$. The values of r_c and ω are controlled using the parameter **DFTU.CutoffNorm** and/or the data included in the block **DFTU.Proj**.

DFTU.EnergyShift 0.05 Ry (energy)

Energy increase used to define the localization radius of the DFT+U projectors (similar to the parameter **PAO.EnergyShift**).

NOTE: only used when **DFTU.ProjectorGenerationMethod** is 1.

DFTU.CutoffNorm 0.9 (real)

Parameter used to define the value of r_c used in the Fermi distribution to cut the DFT+U projectors generated according to generation method 2 (see above). **DFTU.CutoffNorm** is the norm of the original pseudoatomic orbital contained inside a sphere of radius equal to r_c .

NOTE: only used when **DFTU.ProjectorGenerationMethod** is 2.

%block DFTU.Proj <None> (block)

Data block used to specify the DFT+U projectors.

- If **DFTU.ProjectorGenerationMethod** is 1, the syntax is as follows:

```
%block DFTU.Proj      # Define DFT+U projectors
Fe      2              # Label, l_shells
n=3 2   E 50.0 2.5     # n (opt if not using semicore levels),l,Softconf(opt)
      5.00 0.35         # U(eV), J(eV) for this shell
      2.30              # rc (Bohr)
      0.95              # scaleFactor (opt)
      0                 # 1
      1.00 0.05         # U(eV), J(eV) for this shell
      0.00              # rc(Bohr) (if 0, automatic r_c from DFTU.EnergyShift)
```



```
%endblock DFTU.Proj
```

- If **DFTU.ProjectorGenerationMethod** is **2**, the syntax is as follows:

```
%block DFTU.Proj      # Define DFTU projectors
Fe      2              # Label, l_shells
n=3 2   E 50.0 2.5     # n (opt if not using semicore levels),l,Softconf(opt)
      5.00 0.35        # U(eV), J(eV) for this shell
      2.30 0.15        # rc (Bohr), \omega(Bohr) (Fermi cutoff function)
      0.95             # scaleFactor (opt)
      0                # 1
      1.00 0.05        # U(eV), J(eV) for this shell
      0.00 0.00        # rc(Bohr), \omega(Bohr) (if 0 r_c from DFTU.CutoffNorm
%endblock DFTU.Proj    # and \omega from default value)
```

Certain of the quantites have default values:

U	0.0 eV
J	0.0 eV
ω	0.05 Bohr
Scale factor	1.0

r_c depends on **DFTU.EnergyShift** or **DFTU.CutoffNorm** depending on the generation method.

DFTU.FirstIteration false *(logical)*

If **true**, local populations are calculated and Hubbard-like term is switch on in the first iteration. Useful if restarting a calculation reading a converged or an almost converged density matrix from file.

DFTU.ThresholdTol 0.01 *(real)*

Local populations only calculated and/or updated if the change in the density matrix elements (dDmax) is lower than **DFTU.ThresholdTol**.

DFTU.PopTol 0.001 *(real)*

Convergence criterium for the DFT+U local populations. In the current implementation the Hubbard-like term of the Hamiltonian is only updated (except for the last iteration) if the variations of the local populations are larger than this value.

DFTU.PotentialShift false *(logical)*

If set to **true**, the value given to the U parameter in the input file is interpreted as a local potential shift. Recording the change of the local populations as a function of this potential shift, we can calculate the appropriate value of U for the system under study following the methology proposed by Cococcioni and Gironcoli in Phys. Rev. B **71**, 035105 (2005).

9 RT-TDDFT

Now it is possible to perform Real-Time Time-Dependent Density Functional Theory (RT-TDDFT)-based calculations using the SIESTA method. This section includes a brief introduction to the TDDFT method and implementation, shows how to run the TDDFT-based calculations, and provides a reference guide to the additional input options.

9.1 Brief description

The basic features of the TDDFT have been implemented within the SIESTA code. Most of the details can be found in the paper Phys. Rev. B **66** 235416 (2002), by A. Tsolakidis, D. Sánchez-Portal and, Richard M. Martin. However, the practical implementation of the present version is very different from the initial version. The present implementation of the TDDFT has been programmed with the primary aim of calculating the optical response of clusters and solids, however, it has been successfully used to calculate the electronic stopping power of solids as well.

For the calculation of the optical response of the electronic systems a perturbation to the system is applied at time step 0, and the system is allowed to reach a self-consistent solution. Then, the perturbation is switched off for all subsequent time steps, and the electrons are allowed to evolve according to time-dependent Kohn-Sham equations. For the case of a cluster the perturbation is a finite (small) electric field. For the case of bulk material (not yet fully implemented) the initial perturbation is different but the main strategy is similar.

The present version of the RT-TDDFT implementation is also capable of performing a simultaneous dynamics of electrons and ions but this is limited to the cases in which forces on the ions are within ignorable limit.

The general working scheme is as following. First, the system is allowed to reach a self-consistent solution for some initial conditions (for example an initial ionic configuration or an applied external field). The occupied Kohn-Sham orbitals (KSOs) are then selected and stored in memory. The occupied KSOs are then made to evolve in time, and the Hamiltonian is recalculated for each time step.

9.2 Partial Occupations

This is a note of caution. This implementation of RT-TDDFT can not propagate partially occupied orbitals. While partial occupation of states is a common occurrence, they must be avoided. The issue of partially occupied states becomes, particularly, tricky when dealing with metals and k-point sampling at the same time. The code tries to detect partial occupations and stops during the first run but it is not guaranteed. Consequently, it can lead to additional or missing charge. Ultimately it is users' responsibility to make sure that the system has no partial occupations and missing or added charge. There are different ways to avoid partial occupations depending on the system and simulation parameters; for example changing spin-polarization and/or adding some k-point shift to k-points.

9.3 Input options for RT-TDDFT

A TDDFT calculation requires two runs of SIESTA. In the first run with appropriate flags it calculates the self-consistent initial state, i.e., only occupied initial KSOs stored in `SystemLabel.TDWF` file. The second run uses this file and the structure file `SystemLabel.TDXV` as input and evolves the occupied KSOs.

TDED.WF.Initialize `false` *(logical)*

If set to **true** in a standard self-consistent SIESTA calculation, it makes the program save the KSOs after reaching self-consistency. This constitutes the first run.

TDED.Nsteps 1 (integer)

Number of electronic time steps between each atomic movement. It can not be less than 1.

TDED.TimeStep 0.001 fs (time)

Length of time for each electronic step. The default value is only suggestive. Users must determine an appropriate value for the electronic time step.

TDED.Extrapolate false (logical)

An extrapolated Hamiltonian is applied to evolve KSOs for **TDED.Extrapolate.Substeps** number of substeps within a single electronic step without re-evaluating the Hamiltonian.

TDED.Extrapolate.Substeps 3 (integer)

Number of electronic substeps when an extrapolated Hamiltonian is applied to propagate the KSOs. Effective only when **TDED.Extrapolate** set to be true.

TDED.Inverse.Linear true (logical)

If **true** the inverse of matrix

$$\mathbf{S} + i\mathbf{H}(t)\frac{dt}{2} \quad (26)$$

is calculated by solving a system of linear equations which implicitly multiplies the inverted matrix to the right hand side matrix. The alternative is explicit inversion and multiplication. The two options may differ in performance.

TDED.WF.Save false (logical)

Option to save wavefunctions at the end of a simulation for a possible restart or analysis. Wavefunctions are saved in file `SystemLabel.TDWF`. A TDED restart requires `SystemLabel.TDWF`, `SystemLabel.TDXV`, and `SystemLabel.VERLET_RESTART` from the previous run. The first step of the restart is same as the last of the previous run.

TDED.Write.Etot true (logical)

If **true** the total energy for every time step is stored in the file `SystemLabel.TDETOT`.

TDED.Write.Dipole false (logical)

If **true** a file `SystemLabel.TDDIPOL` is created that can be further processed to calculate polarizability.

TDED.Write.Eig false (logical)

If **true** the quantities $\langle \phi(t) | H(t) | \phi(t) \rangle$ in every time step are calculated and stored in the file `SystemLabel.TDEIG`. This is not trivial, hence can increase computational time.

TDED.Saverho false (logical)

If **true** the instantaneous time-dependent density is saved to `<istep>.TDRho` after every **TDED.Nsaverho** number of steps.

TDED.Nsaverho 100 (integer)

Fixes the number of steps of ion-electron dynamics after which the instantaneous time-dependent density is saved. May require a lot of disk space.

10 External control of SIESTA

Since SIESTA 4.1 an additional method of controlling the convergence and MD of SIESTA is enabled through external scripting capability. The external control comes in two variants:

- Implicit control of MD through updating/changing parameters and optimizing forces. For instance one may use a **Verlet** MD method but additionally update the forces through some external force-field to amend limitations by the **Verlet** method for your particular case. In the implicit control the molecular dynamics is controlled by SIESTA.
- Explicit control of MD. In this mode the molecular dynamics *must* be controlled in the external Lua script and the convergence of the geometry should also be controlled via this script.

The implicit control is in use if **MD.TypeOfRun** is something other than **lua**, while if the option is **lua** the explicit control is in use.

For examples on the usage of the Lua scripting engine and the power you may find the library **flos**⁷, see <https://github.com/siesta-project/flos>. At the time of writing the **flos** library already implements new geometry/cell relaxation schemes and new force-constants algorithms. You are highly encouraged to use the new relaxation schemes as they may provide faster convergence of the relaxation.

Lua.Script *<none>*

(file)

Specify a Lua script file which may be used to control the internal variables in SIESTA. Such a script file must contain at least one function named **siesta_comm** with no arguments.

An example file could be this (note this is Lua code):

```
-- This function (siesta_comm) is REQUIRED
function siesta_comm()

    -- Define which variables we want to retrieve from SIESTA
    get_tbl = {"geom.xa", "E.total"}

    -- Signal to SIESTA which variables we want to explore
    siesta.receive(get_tbl)

    -- Now we have the required variables,
    -- convert to a simpler variable name (not nested tables)
    -- (note the returned quantities are in SIESTA units (Bohr, Ry)
    xa = siesta.geom.xa
    Etot = siesta.E.total

    -- If we know our energy is wrong by 0.001 Ry we may now
    -- change the total energy
    Etot = Etot - 0.001

    -- Return to SIESTA the total energy such that
    -- it internally has the "correct" energy.
```

⁷This library is implemented by Nick R. Papior to further enhance the inter-operability with SIESTA and external contributions.

```

    siesta.E.total = Etot
    ret_tbl = {"E.total"}

    siesta.send(ret_tbl)

end

```

Within this function there are certain *states* which defines different execution points in SIESTA:

Initialization This is right after SIESTA has read the options from the FDF file. Here you may query some of the FDF options (and even change them) for your particular problem.

NOTE: `siesta.state == siesta.INITIALIZE`.

Initialize-MD Right before the SCF step starts. This point is somewhat superfluous, but is necessary to communicate the actual meshcutoff used⁸.

NOTE: `siesta.state == siesta.INIT_MD`.

SCF Right after SIESTA has calculated the output density matrix, and just after SIESTA has performed mixing.

NOTE: `siesta.state == siesta.SCF_LOOP`.

Forces This stage is right after SIESTA has calculated the forces.

NOTE: `siesta.state == siesta.FORCES`.

Move This state will *only* be reached if **MD.TypeOfRun** is **lua**.

If one does not return updated atomic coordinates SIESTA will reuse the same geometry as just analyzed.

NOTE: `siesta.state == siesta.MOVE`.

After-move Right after determining the atomic coordinates for the next step. Therefore, this is the first thing that is done with the new atomic coordinates.

NOTE: `siesta.state == siesta.AFTER_MOVE`.

Analysis Just before SIESTA completes and exits.

NOTE: `siesta.state == siesta.ANALYSIS`.

Beginning with implementations of Lua scripts may be cumbersome. It is recommended to start by using **flos**, see <https://github.com/siesta-project/flos> which contains several examples on how to start implementing your own scripts. Currently **flos** implements a larger variety of relaxation schemes, for instance:

```

local flos = require "flos"
LBFGS = flos.LBFGS()
function siesta_comm()
    LBFGS:SIESTA(siesta)
end

```

which is the most minimal example of using the L-BFGS algorithm for geometry relaxation. Note that **flos** reads the parameters **MD.MaxDispl** and **MD.MaxForceTol** through SIESTA automatically.

NOTE: The number of available variables continues to grow and to find which quantities are accessible in Lua you may add this small code in your Lua script:

⁸Remember that the **Mesh.Cutoff** defined is the minimum cutoff used.

```
siesta.print_allowed()
```

which prints out a list of all accessible variables (note they are not sorted).

If there are any variables you require which are not in the list, please contact the developers.

If you want to stop SIESTA from Lua you can use the following:

```
siesta.Stop = true
siesta.send({"Stop"})
```

which will abort SIESTA.

Remark that since *anything* may be changed via Lua one may easily make SIESTA crash due to inconsistencies in the internal logic. This is because SIESTA does not check what has changed, it accepts everything *as is* and continues. Hence, one should be careful what is changed.

Lua.Debug false *(logical)*

Debug the Lua script mode by printing out (on stdout) information everytime SIESTA communicates with Lua.

Lua.Debug.MPI false *(logical)*

Debug all nodes (if in a parallel run).

Lua.Interactive false *(logical)*

Start an interactive Lua session at all the states in the program and ask for user-input. This is primarily intended for debugging purposes. The interactive session is executed just *before* the `siesta_comm` function call (if the script is used).

For serial runs `siesta.send` may be used. For parallel runs do *not* use `siesta.send` as the code is only executed on the first MPI node.

There are various commands that are caught if they are the only content on a line:

/debug Turn on/off debugging information.

/show Show the currently collected lines of code.

/clear Clears the currently collected lines of code.

;
; Run the currently collected lines of code and continue collecting lines.

/run Same as ;.

/cont Run the currently collected lines of code and continue SIESTA.

/stop Run the currently collected lines of code and stop all future interactive Lua sessions.

Currently this only works if **Lua.Script** is having a valid Lua file (note the file may be empty).

10.1 Examples of Lua programs

Please look in the `Tests/lua_*` folders where examples of basic Lua scripts are found. Below is a description of the `*` examples.

h2o Changes the mixing weight continuously in the SCF loop. This will effectively speed up convergence time if one can attain the best mixing weight per SCF-step.

si111 Change the mixing method based on certain convergence criteria. I.e. after a certain convergence one can switch to a more aggressive mixing method.

A combination of the above two examples may greatly improve convergence, however, creating a generic method to adaptively change the mixing parameters may be very difficult to implement. If you do create such a Lua script, please share it on the mailing list.

10.2 External MD/relaxation methods

Using the Lua interface allows a very easy interface for creating external MD and/or relaxation methods.

A public library (`flos`, <https://github.com/siesta-project/flos>) already implements a wider range of relaxation methods than intrinsically enabled in SIESTA. Secondly, by using external scripting mechanisms one can customize the routines to a much greater extend while simultaneously create custom constraints.

You are *highly* encouraged to try out the `flos` library (please note that `flook` is required, see installation instructions above).

11 TRANSIESTA

SIESTA includes the possibility of performing calculations of electronic transport properties using the TRANSIESTA method. This Section describes how to use these capabilities, and a reference guide to the relevant FDF options. We describe here only the additional options available for TRANSIESTA calculations, while the rest of the SIESTA functionalities and variables are described in the previous sections of this User's Guide.

An accompanying Python toolbox is available which will assist with TRANSIESTA calculations. Please use (and cite) SISL^[16].

11.1 Source code structure

In this implementation, the TRANSIESTA routines have been grouped in a set of modules whose file names begin with `m_ts` or `ts`.

11.2 Compilation

Prior to SIESTA 4.1 TRANSIESTA was a separate executable. Now TRANSIESTA is fully incorporated into SIESTA. *Only* compile SIESTA and the full functionality is present. Sec. 2 for details on compiling SIESTA.

11.3 Brief description

The TRANSIESTA method is a procedure to solve the electronic structure of an open system formed by a finite structure sandwiched between semi-infinite metallic leads. A finite bias can be

applied between leads, to drive a finite current. The method is described in detail in Brandbyge et al.^[5]; Papior et al.^[14]. In practical terms, calculations using TRANSIESTA involve the solution of the electronic density from the DFT Hamiltonian using Greens functions techniques, instead of the usual diagonalization procedure. Therefore, TRANSIESTA calculations involve a SIESTA run, in which a set of routines are invoked to solve the Greens functions and the charge density for the open system. These routines are packed in a set of modules, and we will refer to it as the 'TRANSIESTA module' in what follows.

TRANSIESTA was originally developed by Mads Brandbyge, José-Luis Mozos, Pablo Ordejón, Jeremy Taylor and Kurt Stokbro^[5]. It consisted, mainly, in setting up an interface between SIESTA and the (tight-binding) transport codes developed by M. Brandbyge and K. Stokbro. Initially everything was written in Fortran-77. As SIESTA started to be translated to Fortran-90, so were the TRANSIESTA parts of the code. This was accomplished by José-Luis Mozos, who also worked on the parallelization of TRANSIESTA. Subsequently Frederico D. Novaes extended TRANSIESTA to allow k -point sampling for transverse directions. Additional extensions was added by Nick R. Papior during 2012.

The current TRANSIESTA module has been completely rewritten by Nick R. Papior and encompass highly advanced inversion algorithms as well as allowing $N \geq 1$ electrode setups among many new features. Furthermore, the utility TBTRANS has also been fully re-coded (by Nick R. Papior) to be a generic tight-binding code capable of analyzing physics from the Greens function perspective in $N \geq 1$ setups^[14].

- Transport calculations involve *electrode* (EL) calculations, and subsequently the Scattering Region (SR) calculation. The *electrode* calculations are usual SIESTA calculations, but where files `SystemLabel.HSX/SystemLabel.TSHS` (the `SystemLabel.TSHS` is deprecated), and optionally `SystemLabel.TSDE`, are generated. These files contain the information necessary for calculation of the self-energies. If any electrodes have identical structures (see below) the same files can and should be used to describe those. In general, however, electrodes can be different and therefore two different `SystemLabel.HSX` files must be generated. The location of these electrode files must be specified in the FDF input file of the SR calculation, see **TS.Elec.<>.HS**.
- For the SR, TRANSIESTA starts with the usual SIESTA procedure, converging a Density Matrix (DM) with the usual Kohn-Sham scheme for periodic systems. It uses this solution as an initial input for the Greens function self consistent cycle. Effectively you will start a TRANSIESTA calculation from a fully periodic calculation. This is why the 0V calculation should be the only calculation where you start from SIESTA.

TRANSIESTA stores the SCF DM in a file named `SystemLabel.TSDE`. In a rerun of the same system (meaning the same **SystemLabel**), if the code finds a `SystemLabel.TSDE` file in the directory, it will take this DM as the initial input and this is then considered a continuation run. In this case it does not perform an initial SIESTA run. It must be clear that when starting a calculation from scratch, in the end one will find both files, `SystemLabel.DM` and `SystemLabel.TSDE`. The first one stores the SIESTA density matrix (periodic boundary conditions in all directions and no voltage), and the latter the TRANSIESTA solution.

- When performing several bias calculations, it is heavily advised to run different bias' in different directories. To drastically improve convergence (and throughput) one should copy the `SystemLabel.TSDE` from the closest, previously, calculated bias to the current bias.

- The `SystemLabel.TSDE` may be read equivalently as the `SystemLabel.DM`. Thus, it may be used by fx. `denchar` to analyze the non-equilibrium charge density. Alternatively one can use `SISL`^[16] to interpolate the DM and EDM to speed up convergence.
- As in the case of SIESTA calculations, what TRANSIESTA does is to obtain a converged DM, but for open boundary conditions and possibly a finite bias applied between electrodes. The corresponding Hamiltonian matrix (once self consistency is achieved) of the SR is also stored in a `SystemLabel.HSX` file. Subsequently, transport properties are obtained in a post-processing procedure using the TBTRANS code (located in the `Util/TS/TBtrans` directory). We note that the `SystemLabel.HSX` files contain all the needed structural information (atomic positions, matrix elements, ...), and so the input (fdf) flags for the geometry and basis have no influence of the subsequent TBTRANS calculations.
- When the non-equilibrium calculation uses different electrodes one should use so-called *buffer* atoms behind the electrodes to act as additional screening regions when calculating the initial guess (using SIESTA) for TRANSIESTA. Essentially they may be used to achieve a better “bulk-like” environment at the electrodes in the SR calculation.
- An important parameter is the lower bound of the energy contours. It is a good practice, to start with a SIESTA calculation for the SR and look at the eigenvalues of the system. The lower bound of the contours must be *well* below the lowest eigenvalue.
- Periodic boundary conditions are assumed in 2 cases.
 1. For $N_{\mathbf{e}} \neq 2$ all lattice vectors are periodic, users *must* manually define **TS.kgrid.MonkhorstPack**
 2. For $N_{\mathbf{e}} = 2$ TRANSIESTA will auto-detect if both electrodes are semi-infinite along the same lattice vector. If so, only 1 k point will be used along that lattice vector.
- The default algorithm for matrix inversion is the BTD method, before starting a TRANSIESTA calculation please run with the analyzation step **TS.Analyze** (note this is very fast and can be done on any desktop computer, regardless of system size).
- Importantly(!) the k -point sampling need typically be much higher in a TBTRANS calculation to achieve a converged transmission function.
- Energies from TRANSIESTA are *not* to be trusted since the open boundaries complicates the energy calculation. Therefore care needs to be taken when comparing energies between different calculations and/or different bias’.
- Always ensure that charges are preserved in the scattering region calculation. Doing the SCF an output like the following will be shown:

ts-q:	D	E1	C1	E2	C2	dQ
ts-q:	436.147	392.146	3.871	392.146	3.871	7.996E-3

Always ensure the last column (dQ) is a very small fraction of the total number of electrons. Ideally this should be 0. For 0 bias calculations this should be very small, typically less than 0.1 % of the total charge in the system. If this is not the case, it probably means that there is not enough screening towards the electrodes which can be solved by adding more electrode layers between the electrode and the scattering region. This layer thickness is *very* important to obtain a correct open boundary calculation.

- Do *not* perform TRANSIESTA calculations using semi-conducting electrodes. The basic premise of TRANSIESTA calculations is that the electrodes *behave like bulk* in the electrode regions of the SR. This means that the distance between the electrode and the perturbed must equal the screening length of the electrode.

This is problematic for semi-conducting systems since they intrinsically have a very long screening length.

In addition, the Fermi-level of semi-conductors are not well-defined since it may be placed anywhere in the band gap.

11.4 Electrodes

To calculate the electronic structure of a system under external bias, TRANSIESTA attaches the system to semi-infinite electrodes which extend to their respective semi-infinite directions. Examples of electrodes would include surfaces, nanowires, nanotubes or fully infinite regions. The electrode must be large enough (in the semi-infinite direction) so that orbitals within the unit cell only interact with a single nearest neighbor cell in the semi-infinite direction (the size of the unit cell can thus be derived from the range of support for the orbital basis functions). TRANSIESTA will stop if this is not enforced. The electrodes are generated by a separate TRANSIESTA run on a bulk system. This implies that the proper bulk properties are obtained by a sufficiently high k -point sampling. If in doubt, use 100 k -points along the semi-infinite direction. The results are saved in a file with extension `SystemLabel.HSX` which contains a description of the electrode unit cell, the position of the atoms within the unit cell, as well as the Hamiltonian and overlap matrices that describe the electronic structure of the lead. One can generate a variety of electrodes and the typical use of TRANSIESTA would involve reusing the same electrode for several setups. At runtime, the TRANSIESTA coordinates are checked against the electrode coordinates and the program stops if there is a mismatch to a certain precision (10^{-4} Bohr). Note that the atomic coordinates are compared relatively. Hence the *input* atomic coordinates of the electrode and the device need not be the same (see e.g. the tests in the `Tests` directory).

To run an electrode calculation one should do:

```
siesta --electrode RUN.fdf
```

or define these options in the electrode fdf files: **Save.HS** and **TS.DE.Save** to **true** (the above `-electrode` is a shorthand to forcefully define the two options).

11.4.1 Matching coordinates

Here are some rules required to successfully construct the appropriate coordinates of the scattering region. Contrary to versions prior to 4.1, the order of atoms is largely irrelevant. One may define all electrodes, then subsequently the device, or vice versa. Similarly, buffer atoms are not restricted to be the first/last atoms.

However, atoms in any given electrode *must* be consecutive in the device file. I.e. if an electrode input option is given by:

```
%block TS.Elec.<>
HS ../elec-<>/siesta.HSX
```

```

    bloch 1 3 1
    used-atoms 4
    electrode-position 10
    ...
%endblock

```

then the atoms from 10 to $10 + 4 * 3 - 1$ must coincide with the atoms of the calculation performed in the `../elec-<>/` subdirectory. The above options will be discussed in the following section.

When using the Bloch expansion (highly recommended if your system allows it) it is advised to follow the *tiling* method. However both of the below sequences are allowed.

Tile Here the atoms are copied and displaced by the full electrode. Generally this expansion should be preferred over the *repeat* expansion due to much faster execution.

```

iaD = 10 ! as per the above input option
do iC = 0 , nC - 1
do iB = 0 , nB - 1
do iA = 0 , nA - 1
  do iaE = 1 , na_u
    xyz_device(:, iaD) = xyz_elec(:, iaE) + &
      cell_elec(:, 1) * iA + &
      cell_elec(:, 2) * iB + &
      cell_elec(:, 3) * iC
    iaD = iaD + 1
  end do
end do
end do
end do

```

By using SISL^[16] one can achieve the tiling scheme by using the following command-line utility on an input `ELEC.fdf` structure with the minimal electrode:

```

sgeom -tx 1 -ty 3 -tz 1 ELEC.fdf DEVICE_ELEC.fdf

```

Repeat Here the atoms are copied individually. Generally this expansion should *not* be used since it is much slower than tiling.

```

iaD = 10 ! as per the above input option
do iaE = 1 , na_u
  do iC = 0 , nC - 1
  do iB = 0 , nB - 1
  do iA = 0 , nA - 1
    xyz_device(:, iaD) = xyz_elec(:, iaE) + &
      cell_elec(:, 1) * iA + &
      cell_elec(:, 2) * iB + &
      cell_elec(:, 3) * iC
    iaD = iaD + 1
  end do
end do
end do
end do

```

By using SISL^[16] one can achieve the repeating scheme by using the following command-line utility on an input `ELEC.fdf` structure with the minimal electrode:

```
sgeom -rz 1 -ry 3 -rx 1 ELEC.fdf DEVICE_ELEC.fdf
```

11.4.2 Principal layer interactions

It is *extremely* important that the electrodes only interact with one neighboring supercell due to the self-energy calculation^[17]. TRANSIESTA will print out a block as this (<> is the electrode name):

```
<> principal cell is perfect!
```

if the electrode is correctly setup and it only interacts with its neighboring supercell. In case the electrode is erroneously setup, something similar to the following will be shown in the output file.

```
<> principal cell is extending out with 96 elements:
Atom 1 connects with atom 3
Orbital 8 connects with orbital 26
Hamiltonian value: |H(8,6587)|@R=-2 = 0.651E-13 eV
Overlap           : |S(8,6587)|@R=-2 = 0.00
```

It is imperative that you have a *perfect* electrode as otherwise nonphysical results will occur. This means that you need to add more layers in your electrode calculation (and hence also in your scattering region). An example is an ABC stacking electrode. If the above error is shown one *has* to create an electrode with ABCABC stacking in order to retain periodicity.

By default TRANSIESTA will die if there are connections beyond the principal cell. One may control whether this is allowed or not by using **TS.Elecs.Neglect.Principal**.

11.5 Convergence of electrodes and scattering regions

For successful TRANSIESTA calculations it is imperative that the electrodes and scattering regions are well-converged. The basic principle is equivalent to the SIESTA convergence, see Sec. 6.9.

The steps should be something along the line of (only done at 0 V).

1. Converge electrodes and find optimal **Mesh.Cutoff**, **kgrid.MonkhorstPack** etc.

Electrode k points should be very high along the semi-infinite direction. The default is 100, but at least > 50 should easily be reachable.

2. Use the parameters from the electrodes and also converge the same parameters for the scattering region SCF.

This is an iterative process since the scattering region forces the electrodes to use equivalent k points (see **TS.Elec.<>.check-kgrid**).

Note that k points should be limited in the TRANSIESTA run, see **TS.kgrid.MonkhorstPack**.

One should always use the same parameters in both the electrode and scattering region calculations, except the number of k points for the electrode calculations along their respective semi-infinite directions.

3. Once TRANSIESTA is completed one should also converge the number of k points for TBTRANS. Note that k point sampling in TBTRANS should generally be much denser but *always* fulfill $N_k^{\text{TRANSIESTA}} \geq N_k^{\text{TBTRANS}}$

The converged parameters obtained at 0 V should be used for all subsequent bias calculations. Remember to copy the **SystemLabel.TSDE** from the closest, previously, calculated bias for restart and much faster convergence.

TRANSIESTA is also more difficult to converge during the SCF steps. This may be due to several interrelated problems:

- A too short screening distance between the scattering atoms and the electrode layers.
- In case buffer atoms (**TS.Atoms.Buffer**) are used with vacuum on the backside it may be that there are too few buffer atoms to accurately screen off the vacuum region for a sufficiently good initial guess. This effect is only true for 0 V calculations.
- The mixing parameters may need to be smaller than for SIESTA, see Sec. 6.9.2 and it is never guaranteed that it will converge. It is *always* a trial and error method, there are *no* omnipotent mixing parameters.
- Very high bias' may be extremely difficult to converge. Generally one can force bias convergence by doing smaller steps of bias. E.g. if problems arise at 0.5 V with an initial DM from a 0.25 V calculation, one could try and 0.3 V first.
- If a particular bias point is hard to converge, even by doing the previous step, it may be related to an eigenstate close to the chemical potentials of either electrode (e.g. a molecular eigenstate in the junction). In such cases one could try an even higher bias and see if this converges more smoothly.

11.6 NEGF equations

The options available for TRANSIESTA will impact how the calculation is performed. It is vital that the users carefully read this section and the options that refer to these.

The NEGF equation are primarily concerning the Green function:

$$\mathbf{G}(E) = [(E + i\eta)\mathbf{S} - \mathbf{H} - \sum_{\epsilon} \Sigma_{\epsilon}(E)]. \quad (27)$$

The electrode self-energy is calculated from the bulk electrode calculation

$$\Sigma_{\epsilon}(E) \leftarrow \{\mathbf{H}_{\epsilon}, \mathbf{S}_{\epsilon}\}. \quad (28)$$

TRANSIESTA has options to discern which Hamiltonian elements can be used in which parts of the calculation. Default is that the electrode matrices ($\mathbf{H}_{\epsilon}, \mathbf{S}_{\epsilon}$) are used whenever the electrode enters a

matrix. Lets show a partitioning of the Green function for a particular electrode ($z = E + i\eta$)

$$\mathbf{G}(z) = \begin{bmatrix} \mathbf{M}_{\epsilon,\epsilon} & \mathbf{M}_{\epsilon,D} & \dots \\ \mathbf{M}_{D,\epsilon} & \mathbf{M}_{D,D} & \\ \vdots & & \ddots \end{bmatrix}^{-1} = \begin{bmatrix} (z + \mu_{elec})\mathbf{S}_{\epsilon} - \mathbf{H}_{\epsilon} - \mathbf{\Sigma}_{\epsilon}(E) & z\mathbf{S}_{\epsilon,D} - \mathbf{H}_{\epsilon,D} & \dots \\ z\mathbf{S}_{D,\epsilon} - \mathbf{H}_{D,\epsilon} & z\mathbf{S}_D - \mathbf{H}_D & \\ \vdots & & \ddots \end{bmatrix}^{-1}. \quad (29)$$

The following options alter the above equation slightly:

- **TS.Elec.<>.Bulk**
- **TS.Elec.<>.Eta**
- **TS.Elec.<>.chemical-potential**
- **TS.Elec.<>.V-fraction** (experts only!)
- **TS.Elec.<>.delta-Ef** (experts only!)

11.7 TranSIESTA Options

The fdf options shown here are only to be used at the input file for the scattering region. When using TRANSIESTA for electrode calculations, only the usual SIESTA options are relevant. Note that since TRANSIESTA is a generic N_{ϵ} electrode NEGF code the input options are heavily changed compared to versions prior to 4.1.

11.7.1 Quick and dirty

Since 4.1, TRANSIESTA has been fully re-implemented. And so have *every* input fdf-flag. To accommodate an easy transition between previous input files and the new version format a small utility called **ts2ts**. It may be compiled in **Util/TS/ts2ts**. It is recommended that you use this tool if you are familiar with previous TRANSIESTA versions.

One may input options as in the old TRANSIESTA version and then run

```
ts2ts OLD.fdf > NEW.fdf
```

which translates all keys to the new, equivalent, input format. If you are familiar with the old-style flags this is highly recommendable while becoming comfortable with the new input format. Please note that some defaults have changed to more conservative values in the newer release.

If one does not know the old flags and wish to get a basic example of an input file, a script **Util/TS/tselecs.sh** exists that can create the basic input for N_{ϵ} electrodes. One may call it like:

```
tselecs.sh -2 > TWO_ELECTRODE.fdf
tselecs.sh -3 > THREE_ELECTRODE.fdf
tselecs.sh -4 > FOUR_ELECTRODE.fdf
...
```

where the first call creates an input fdf for 2 electrode setups, the second for a 3 electrode setup, and so on. See the help (-h) for the program for additional options.

Before endeavoring on large scale calculations you are advised to run an analyzation of the system at hand, you may run your system as

```
siesta -fdf TS.Analyze RUN.fdf > analyze.out
```

which will analyze the sparsity pattern and print out several different pivoting schemes. Please see **TS.Analyze** for more information.

11.7.2 General options

One have to set **SolutionMethod** to **transiesta** to enable TRANSIESTA.

TS.SolutionMethod `btd|mumps|full` *(string)*

Control the algorithm used for calculating the Green function. Generally the BTM method is the fastest and this option need not be changed.

BTM Use the block-tri-diagonal algorithm for matrix inversion.

This is generally the recommended method.

MUMPS Use sparse matrix inversion algorithm (MUMPS). This requires TRANSIESTA to be compiled with MUMPS.

full Use full matrix inversion algorithm (LAPACK). Generally only usable for debugging purposes.

TS.Voltage `0 eV` *(energy)*

Define the reference applied bias. For $N_{\text{e}} = 2$ electrode calculations this refers to the actual potential drop between the electrodes, while for $N_{\text{e}} \neq 2$ this is a reference bias. In the latter case it *must* be equivalent to the maximum difference between the chemical potential of any two electrodes.

NOTE: Specifying -V on the command-line overwrites the value in the fdf file.

%block TS.kgrid.MonkhorstPack `(kgrid.MonkhorstPack)` *(block)*

k points used for the TRANSIESTA calculation.

For $N_{\text{e}} \neq 2$ this should always be defined. Always take care to use only 1 k point along non-periodic lattice vectors. An electrode semi-infinite region is considered non-periodic since it is integrated out through the self-energies.

This defaults to **kgrid.MonkhorstPack**.

TS.Atoms.Buffer `(None)` *(block/list)*

Specify atoms that will be removed in the TRANSIESTA SCF. They are not considered in the calculation and may be used to improve the initial guess for the Hamiltonian.

An intended use for buffer atoms is to ensure a bulk behavior in the electrode regions when electrodes are different. As an example: a 2 electrode calculation with left consisting of Au atoms and the right consisting of Pt atoms. In such calculations one cannot create a periodic geometry along the transport direction. One needs to add vacuum between the Au and Pt

atoms that comprise the electrodes. However, this creates an artificial edge of the electrostatic environment for the electrodes since in SIESTA there is vacuum, whereas in TRANSIESTA the effective Hamiltonian sees a bulk environment. To ensure that SIESTA also exhibits a bulk environment on the electrodes we add *buffer* atoms towards the vacuum region to screen off the electrode region. These *buffer* atoms is thus a technicality that has no influence on the TRANSIESTA calculation but they are necessary to ensure the electrode bulk properties.

The above discussion is even more important when doing N_{e} -electrode calculations.

NOTE: all lines are additive for the buffer atoms and the input method is similar to that of **Geometry.Constraints** for the **atom** line(s).

```
%block TS.Atoms.Buffer
  atom [ 1 -- 5 ]
%endblock
# Or equivalently as a list
TS.Atoms.Buffer [1 -- 5]
```

will remove atoms [1–5] from the calculation.

TS.ElectronicTemperature $\langle \text{ElectronicTemperature} \rangle$ (energy)

Define the temperature used for the Fermi distributions for the chemical potentials. See **TS.ChemPot.<>.ElectronicTemperature**.

TS.SCF.DM.Tolerance $\langle \text{SCF.DM.Tolerance} \rangle$ (real)

depends on: **SCF.DM.Tolerance**, **SCF.DM.Converge**

The density matrix tolerance for the TRANSIESTA SCF cycle.

TS.SCF.H.Tolerance $\langle \text{SCF.H.Tolerance} \rangle$ (energy)

depends on: **SCF.H.Tolerance**, **SCF.H.Converge**

The Hamiltonian tolerance for the TRANSIESTA SCF cycle.

TS.SCF.dQ.Converge **true** (logical)

Whether TRANSIESTA should check whether the total charge is within a provided tolerance, see **TS.SCF.dQ.Tolerance**.

TS.SCF.dQ.Tolerance $Q(\text{device}) \cdot 10^{-3}$ (real)

depends on: **TS.SCF.dQ.Converge**

The charge tolerance during the SCF.

The charge is not stable in TRANSIESTA calculations and this flag ensures that one does not, by accident, do post-processing of files where the charge distribution is completely wrong.

A too high tolerance may heavily influence the electrostatics of the simulation.

NOTE: Please see **TS.dQ** for ways to reduce charge loss in equilibrium calculations.

TS.SCF.Initialize **diagon|transiesta** (string)

Control which initial guess should be used for TRANSIESTA. The general way is the **diagon** solution method (which is preferred), however, one can start a TRANSIESTA run immediately. If you start directly with TRANSIESTA please refer to these flags: **TS.Elecs.DM.Init** and **TS.Fermi.Initial**.

NOTE: Setting this to **transiesta** is highly experimental and convergence may be extremely poor.

TS.Fermi.Initial $\sum_i^{N_E} E_F^i / N_E$ (energy)

Manually set the initial Fermi level to a predefined value.

NOTE: this may also be used to change the Fermi level for calculations where you restart calculations. Using this feature is highly experimental.

TS.Weight.Method `orb-orb` | `[[un]correlated+]` | `[sum|tr]-atom-[atom|orb]` | `mean` (*string*)

Control how the NEGF weighting scheme is conducted. Generally one should only use the **orb-orb** while the others are present for more advanced usage. They refer to how the weighting coefficients of the different non-equilibrium contours are performed. In the following the weight are denoted in a two-electrode setup while they are generalized for multiple electrodes.

Define the normalised geometric mean as $\propto^{\parallel}$ via

$$w \propto^{\parallel} \langle \cdot^L \rangle \equiv \frac{\langle \cdot^L \rangle}{\langle \cdot^L \rangle + \langle \cdot^R \rangle}. \quad (30)$$

When applying a bias, TRANSIESTA will printout the following during the SCF cycle:

```
ts-err-D: ij( 447, 447), M = 1.8275, ew = -.257E-2, em = 0.258E-2. avg_em = 0.542E-06
ts-err-E: ij( 447, 447), M = -6.7845, ew = 0.438E-3, em = -.439E-3. avg_em = -.981E-07
ts-w-q:      qP1      qP2
ts-w-q:      219.150  216.997
ts-q:      D      E1      C1      E2      C2      dQ
ts-q:  436.147  392.146  3.871  392.146  3.871  7.996E-3
```

The extra output corresponds to fine details in the integration scheme.

ts-err-* are estimated error outputs from the different integrals, for the density matrix (D) and the energy density matrix (E), see Eq. (12) in^[14]. All values (except **avg_em**) are for the given orbital site

ij(A,B) refers to the matrix element between orbital A and B

M is the weighted matrix element value, $\sum_{\epsilon} w_{\epsilon} \rho^{\epsilon}$

ew is the maximum difference between $\sum_{\epsilon} w_{\epsilon} \rho^{\epsilon} - \rho^{\epsilon}$ for all ϵ .

em is the maximum difference between $\rho^{\epsilon'} - \rho^{\epsilon}$ for all combinations of ϵ and ϵ' .

avg_em is the averaged difference of **em** for all orbital sites.

ts-w-q is the Mulliken charge from the different integrals: $\text{Tr}[w_{\epsilon} \rho^{\epsilon} \mathbf{S}]$

orb-orb Weight each orbital-density matrix element individually.

tr-atom-atom Weight according to the trace of the atomic density matrix sub-blocks

$$w_{ij}^{\text{Tr}} \propto^{\parallel} \sqrt{\sum_{\epsilon \in \{i\}} (\Delta \rho_{\mu\mu}^L)^2 \sum_{\epsilon \in \{j\}} (\Delta \rho_{\mu\mu}^L)^2} \quad (31)$$

tr-atom-orb Weight according to the trace of the atomic density matrix sub-block times the weight of the orbital weight

$$w_{ij,\mu\nu}^{\text{Tr}} \propto^{\parallel} \sqrt{w_{ij}^{\text{Tr}} w_{ij,\mu\nu}} \quad (32)$$

sum-atom-atom Weight according to the total sum of the atomic density matrix sub-blocks

$$w_{ij,\mu\nu}^{\Sigma} \propto^{\parallel} \sqrt{\sum_{\epsilon \in \{i\}} (\Delta \rho_{\mu\nu}^L)^2 \sum_{\epsilon \in \{j\}} (\Delta \rho_{\mu\nu}^L)^2} \quad (33)$$

sum-atom-orb Weight according to the total sum of the atomic density matrix sub-block times the weight of the orbital weight

$$w_{ij,\mu\nu}^{\Sigma} \propto \sqrt{w_{ij}^{\Sigma} w_{ij,\mu\nu}} \quad (34)$$

mean A standard average.

Each of the methods (except **mean**) comes in a correlated and uncorrelated variant where Σ is either outside or inside the square, respectively.

TS.Weight.k.Method `correlated|uncorrelated` (string)

Control weighting *per k*-point or the full sum. I.e. if **uncorrelated** is used it will weight n_k times if there are n_k *k*-points in the Brillouin zone.

TS.Forces `true` (logical)

Control whether the forces are calculated. If *not* TRANSIESTA will use slightly less memory and the performance slightly increased, however the final forces shown are incorrect.

If this is **true** the file `SystemLabel.TSFA` (and possibly the `SystemLabel.TSFAC`) will be created. They contain forces for the atoms that are having updated density-matrix elements (**TS.Elec.<>.DM-update all**).

Generally one should not expect good forces close to the electrode/device interface since this typically has some electrostatic effects that are inherent to the TRANSIESTA method. Forces on atoms *far* from the electrode can safely be analyzed.

TS.dQ `none|buffer|fermi` (string)

Any excess/deficiency of charge can be re-adjusted after each TRANSIESTA cycle to reduce charge fluctuations in the cell.

NOTE: recommended to *only* use charge corrections for 0 V calculations.

The non-neutral charge in TRANSIESTA cycles is an expression of one of the following things:

1. An incorrect screening towards the electrodes. To check this, simply add more electrode layers towards the device at each electrode and see how the charge evolves. It should tend to zero.

The best way to check this is to follow these steps:

- (a) Perform a SIESTA-only calculation (the resulting DM should be used as the starting point for both following calculations)
- (b) Perform a TRANSIESTA calculation with the option **TS.Elecs.DM.Init diagon** (please note that the electrode option has precedence, so remove any entry from the **TS.Elec.<>** block)
- (c) Perform a TRANSIESTA calculation with the option **TS.Elec.<>.DM-init bulk** (please note that the electrode option has precedence, so remove any entry from the **TS.Elec.<>** block)

Now compare the final output and the initial charge distribution, e.g.:

```
>>> TS.Elecs.DM.Init diagon
transiesta: Charge distribution, target =    396.00000
Total charge                [Q] :    396.00000

>>> TS.Elecs.DM.Init bulk
```

```
transiesta: Charge distribution, target =    396.00000
Total charge           [Q] :    395.9995
```

The above shows that there is very little charge difference between the bulk electrode DM and the scattering region. This ensures that the charge distribution are similar and that your electrode is sufficiently screened.

Additionally one may compare the final output such as total energies, calculated DOS and ADOS (see TBTRANS). If the two calculations show different properties, one should carefully examine the system setup.

2. An incorrect reference energy level. In TRANSIESTA the Fermi level is calculated from the SIESTA SCF. However, the SIESTA Fermi level corresponds to a periodic calculation and *not* an open system calculation such as NEGF.

If the first step shows a good screening towards the electrode it is usually the reference energy level, then use **TS.dQ fermi**.

3. A combination of the above, this is the typical case.

none No charge corrections are introduced.

buffer Excess/missing electrons are placed in the buffer regions (buffer atoms are required to exist)

fermi Correct the charge filling by calculating a new reference energy level (referred to as the Fermi level).

We approximate the contribution to be constant around the Fermi level and find

$$dE_F = \frac{Q' - Q}{Q|_{E_F}}, \quad (35)$$

where Q' is the charge from a TRANSIESTA SCF step and $Q|_{E_F}$ is the equilibrium charge at the current Fermi level, Q is the supposed charge to reside in the calculation. Fermi correction utilizes Eq. (35) for the first correction and all subsequent corrections are based on a cubic spline interpolation to faster converge the “correct” Fermi level.

This method will create a file called **TS_FERMI**.

NOTE: correcting the reference energy level is a costly operation since the SCF cycle typically gets *corrupted* resulting in many more SCF cycles.

TS.dQ.Factor 0.8 (real)

Any positive value close to 1. 0 means no charge correction. 1 means total charge correction. This will reduce the fluctuations in the SCF and setting this to 1 may result in difficulties in converging.

TS.dQ.Fermi.Tolerance 0.01 (real)

The tolerance at which the charge correction will converge. Any excess/missing charge ($|Q' - Q| > \text{Tol}$) will result in a correction for the Fermi level.

TS.dQ.Fermi.Max 1.5 eV (energy)

The maximally allowed value that the Fermi level will change from a charge correction using the Fermi correction method. In case the Fermi level lies in between two bands a DOS of 0 at the Fermi level will make the Fermi change equal to ∞ . This is not physical and the user can thus truncate the correction.

NOTE: If you know the band-gap, setting this to 1/4 (or smaller) of the band gap seems like a better value than the rather arbitrarily default one.

TS.dQ.Fermi.Eta 1 meV (energy)

The η value that we extrapolate the charge at the poles to. Usually a smaller η value will mean larger changes in the Fermi level. If the charge convergence w.r.t. the Fermi level is fluctuating a lot one should increase this η value.

TS.HS.Save true (logical)

Must be **true** for saving the Hamiltonian (`SystemLabel.TSHS`). Can only be set if **SolutionMethod** is not **transiesta**.

The default is **false** for **SolutionMethod** different from **transiesta** and if **-electrode** has not been passed as a command line argument.

NOTE: The `SystemLabel.TSHS` file format is deprecated and may be removed in future SIESTA versions. The `SystemLabel.TS.HSX` file format (**Save.HS**) is feature complete, and should be used.

TS.DE.Save true (logical)

Must be **true** for saving the density and energy density matrix for continuation runs (`SystemLabel.TSDE`). Can only be set if **SolutionMethod** is not **transiesta**.

The default is **false** for **SolutionMethod** different from **transiesta** and if **-electrode** has not been passed as a command line argument.

TS.S.Save false (logical)

This is a flag mainly used for the Inelastica code to produce overlap matrices for Pulay corrections. This should only be used by advanced users.

TS.SIESTA.Only false (logical)

Stop TRANSIESTA right after the initial diagonalization run in SIESTA. Upon exit it will also create the `SystemLabel.TSDE` file which may be used for initialization runs later.

This may be used to start several calculations from the same initial density matrix, and it may also be used to rescale the Fermi level of electrodes. The rescaling is primarily used for semi-conductors where the Fermi levels of the device and electrodes may be misaligned.

TS.Analyze false (logical)

When using the BTD solution method (**TS.SolutionMethod**) this will analyze the Hamiltonian and printout an analysis of the sparsity pattern for optimal choice of the BTD partitioning algorithm.

This yields information regarding the **TS.BTD.Pivot** flag.

NOTE: we advice users to *always* run an analyzation step prior to actual calculation and select the *best* BTD format. This analyzing step is very fast and may be performed on small work-station computers, even on systems of $\gg 10,000$ orbitals.

To run the analyzing step you may do:

```
siesta -fdf TS.Analyze RUN.fdf > analyze.out
```

note that there is little gain on using MPI and it should complete within a few minutes, no matter the number of orbitals.

Choosing the best one may be difficult. Generally one should choose the pivoting scheme that uses the least amount of memory. However, one should also choose the method with the largest block-size being as small as possible. As an example:

```
TS.BTD.Pivot atom+GPS
...
  BTD partitions (7):
    [ 2984, 2776, 192, 192, 1639, 4050, 105 ]
  BTD matrix block size [max] / [average]: 4050 / 1705.429
  BTD matrix elements in % of full matrix: 47.88707 %

TS.BTD.Pivot atom+GGPS
...
  BTD partitions (6):
    [ 2880, 2916, 174, 174, 2884, 2910 ]
  BTD matrix block size [max] / [average]: 2916 / 1989.667
  BTD matrix elements in % of full matrix: 48.62867 %
```

Although the GPS method uses the least amount of memory, the GGPS will likely perform better as the largest block in GPS is 4050 vs. 2916 for the GGPS method.

TS.Analyze.Graphviz `false` *(logical)*
depends on: TS.Analyze

If performing the analysis, also create the connectivity graph and store it as `GRAPHVIZ_atom.gv` or `GRAPHVIZ_orbital.gv` to be post-processed in Graphviz⁹.

11.8 *k*-point sampling

The options for *k*-point sampling are identical to the SIESTA options, **kgrid.MonkhorstPack**, **kgrid.Cutoff** or **kgrid.File**.

One may however use specific TRANSIESTA *k*-points by using these options:

%block TS.kgrid.MonkhorstPack `<kgrid.MonkhorstPack>` *(block)*
 See **kgrid.MonkhorstPack** for details.

TS.kgrid.Cutoff `0. Bohr` *(length)*
 See **kgrid.Cutoff** for details.

TS.kgrid.File `none` *(string)*
 See **kgrid.File** for details.

11.8.1 Algorithm specific options

These options adhere to the specific solution methods available for TRANSIESTA. For instance the **TS.BTD.*** options adhere only when using **TS.SolutionMethod BTD**, similarly for options with MUMPS.

TS.BTD.Pivot `<first electrode>` *(string)*

⁹www.graphviz.org

Decide on the partitioning for the BTM matrix. One may denote either **atom+** or **orb+** as a prefix which does the analysis on the atomic sparsity pattern or the full orbital sparsity pattern, respectively. If neither are used it will default to **atom+**.

Please see **TS.Analyze**.

<elec-name>|CG-<elec-name> The partitioning will be a connectivity graph starting from the electrode denoted by the name. This name *must* be found in the **TS.Elects** block. One can append more than one electrode to simultaneously start from more than 1 electrode. This may be necessary for multi-terminal calculations.

rev-CM Use the reverse Cuthill-McKee for pivoting the matrix elements to reduce bandwidth. One may omit **rev-** to use the standard Cuthill-McKee algorithm (not recommended).

This pivoting scheme depends on the initial starting electrodes, append **+<elec-name>** to start the Cuthill-McKee algorithm from the specified electrode(s).

GPS Use the Gibbs-Poole-Stockmeyer algorithm for reducing the bandwidth.

GGPS Use the generalized Gibbs-Poole-Stockmeyer algorithm for reducing the bandwidth.

NOTE: this algorithm does not work on dis-connected graphs.

PCG Use the peripheral connectivity graph algorithm for reducing the bandwidth.

This pivoting scheme *may* depend on the initial starting electrode(s), append **+<elec-name>** to initialize the PCG algorithm from the specified electrode(s).

Examples are

```
TS.BTD.Pivot atom+GGPS
TS.BTD.Pivot GGPS
TS.BTD.Pivot orb+GGPS
TS.BTD.Pivot orb+PCG+Left
```

where the first two are equivalent. The 3rd and 4th are more heavy on analysis and will typically not improve the bandwidth reduction.

TS.BTD.Optimize speed|memory (string)

When selecting the smallest blocks for the BTM matrix there are certain criteria that may change the size of each block. For very memory consuming jobs one may choose the **memory**.

NOTE: often both methods provide *exactly* the same BTM matrix due to constraints on the matrix.

TS.BTD.Guess1.Min <empirically determined> (int)

depends on: **TS.BTD.Guess1.Max**

Constructing the blocks for the BTM starts by *guessing* the first block size. One could guess on all different block sizes, but to speed up the process one can define a smaller range of guesses by defining **TS.BTD.Guess1.Min** and **TS.BTD.Guess1.Max**.

The initial guessed block size will be between the two values.

By default this is 1/4 of the minimum bandwidth for a selected first set of orbitals.

NOTE: setting this to 1 may sometimes improve the final BTM matrix blocks.

TS.BTD.Guess1.Max <empirically determined> (int)

depends on: **TS.BTD.Guess1.Min**

See **TS.BTD.Guess1.Min**.

NOTE: for improved initialization performance setting Min/Max flags to the first block size for a given pivoting scheme will drastically reduce the search space and make initialization much faster.

TS.BTD.Spectral **propagation|column** (*string*)

How to compute the spectral function ($GTG^\dagger$).

For $N_{\mathfrak{e}} < 4$ this defaults to **propagation** which should be the fastest.

For $N_{\mathfrak{e}} \geq 4$ this defaults to **column**.

Check which has the best performance for your system if you endeavor on huge amounts of calculations for the same system.

TS.MUMPS.Ordering **<read MUMPS manual>** (*string*)

One may select from a number of different matrix orderings which are all described in the MUMPS manual.

The following list of orderings are available (without detailing their differences): **auto**, **AMD**, **AMF**, **SCOTCH**, **PORD**, **METIS**, **QAMD**.

TS.MUMPS.Memory **20** (*integer*)

Specify a factor for the memory consumption in MUMPS. See the **INFOG(9)** entry in the MUMPS manual. Generally if TRANSIESTA dies and **INFOG(9)=-9** one should increase this number.

TS.MUMPS.BlockingFactor **112** (*integer*)

Specify the number of internal block sizes. Larger numbers increases performance at the cost of memory.

NOTE: this option may heavily influence performance.

11.8.2 Poisson solution for fixed boundary conditions

TRANSIESTA requires fixed boundary conditions and forcing this is an intricate and important detail.

It is important that these options are exactly the same if one reuses the `SystemLabel.TSDE` files.

TS.Poisson **ramp|elec-box|<file>** (*string*)

Define how the correction of the Poisson equation is superimposed. The default is to apply the linear correction across the entire cell (if there are two semi-infinite aligned electrodes). Otherwise this defaults to the *box* solution which will introduce spurious effects at the electrode boundaries. In this case you are encouraged to supply a **file**.

If the input is a **file**, it should be a NetCDF file containing the grid information which acts as the boundary conditions for the SCF cycle. The grid information should conform to the grid size of the unit-cell in the simulation. **NOTE:** the file option is only applicable if compiled with CDF4 compliance.

ramp Apply the ramp for the full cell. This is the default for 2 electrodes.

<file> Specify an external file used as the boundary conditions for the applied bias. This is

encouraged to use for $N_{\text{e}} > 2$ electrode calculations but may also be used when an *a priori* potential profile is known.

The file should contain something similar to this output (`ncdump -h`):

```
netcdf <file> {
  dimensions:
    one = 1 ;
    a = 43 ;
    b = 451 ;
    c = 350 ;
  variables:
    double Vmin(one) ;
        Vmin:unit = "Ry" ;
    double Vmax(one) ;
        Vmax:unit = "Ry" ;
    double V(c, b, a) ;
        V:unit = "Ry" ;
}
```

Note that the units should be in Ry. `Vmax/Vmin` should contain the maximum/minimum fixed boundary conditions in the Poisson solution. This is used internally by TRANSIESTA to scale the potential to arbitrary V . This enables the Poisson solution to only be solved *once* independent on subsequent calculations. For chemical potential configurations where the Poisson solution is not linearly dependent one has to create separate files for each applied bias.

elec-box The default potential profile for $N_{\text{e}} > 2$, or when the electrodes are not aligned (in terms of their transport direction).

NOTE: usage of this Poisson solution is *highly* discouraged. Please see **TS.Poisson** <file>.

TS.Hartree.Fix [-+][ABC] (string)

Specify which plane to fix the Hartree potential at. For regular (2 electrode calculations with a single transport direction) this should not be set. For $N_{\text{e}} \neq 2$ electrode systems one *has* to specify a plane to fix. One can specify one or several planes to fix. Users are encouraged to fix the plane where the entire plane has the highest/lowest potential.

TS.Hartree.Fix.Frac 1. (real)

Fraction of the correction that is applied.

NOTE: this is an experimental feature!

TS.Hartree.Offset 0 eV (energy)

An offset in the Hartree potential to match the electrode potential.

This value may be useful in certain cases where the Hartree potentials are very different between the electrode and device region calculations.

This should not be changed between different bias calculations. It directly relates to the reference energy level (E_F).

11.8.3 Electrode description options

As TRANSIESTA supports N_{e} electrodes one needs to specify all electrodes in a generic input format.

Note that electrodes should be *metallic* so that the Fermi-level is well defined. Please see Sec. 11.3 for more details.

%block TS.Elecs `<None>` (block)

Each line denote an electrode which is queried in **TS.Elec.<>** for its setup.

%block TS.Elec.<> `<None>` (block)

Each line represents a setting for electrode `<>`. There are a few lines that *must* be present, **HS**, **semi-inf-dir**, **electrode-pos**, **chem-pot**. The remaining options are optional.

NOTE: Options prefixed with **tbt** are neglected in TRANSIESTA calculations. In TBTRANS calculations these flags has precedence over the other options and *must* be placed at the end of the block.

HS The Hamiltonian information from the initial electrode calculation. This file retains the geometrical information as well as the Hamiltonian, overlap matrix and the Fermi-level of the electrode. This is a file-path and the Hamiltonian file need not be in the same directory, i.e. the path can be relative.

NOTE: `SystemLabel.HSX`, `SystemLabel.TSHS` and `SystemLabel.nc` files are supported. See **Save.HS**, **TS.HS.Save** or **CDF.Save** are the relevant flags for this.

NOTE: Please note that TRANSIESTA expects a metallic electrode. Results can not be trusted for semi-conductors.

semi-inf-direction|semi-inf-dir|semi-inf The semi-infinite direction of the electrode with respect to the electrode unit-cell.

It may be one of `[-+][abc]`, `[-+]A[123]`, **ab**, **ac**, **bc** or **abc**. The latter four all refer to a real-space self-energy as described in^[15].

NOTE: this direction is *not* with respect to the scattering region unit cell. It is with respect to the electrode unit cell. TRANSIESTA will figure out the alignment of the electrode unit cell and the scattering region unit-cell.

chemical-potential|chem-pot|mu The chemical potential that is associated with this electrode. This is a string that should be present in the **TS.ChemPots** block.

electrode-position|elec-pos The index of the electrode in the scattering region. This may be given by either **elec-pos <idx>**, which refers to the first atomic index of the electrode residing at index `<idx>`. Else the electrode position may be given via **elec-pos end <idx>** where the last index of the electrode will be located at `<idx>`.

used-atoms *depends on:* **TS.Elec.<>.semi-inf-direction**

Number of atoms from the electrode calculation that is used in the scattering region as electrode. This may be useful when the periodicity of the electrodes forces extensive electrodes in the semi-infinite direction.

If the semi-infinite direction is *positive*, the first atoms will be retained. Contrary, if the semi-infinite direction is *negative*, the last atoms will be retained.

NOTE: do not set this if you use all atoms in the electrode.

Bulk Control whether the Hamiltonian of the electrode region in the scattering region is enforced *bulk* or whether the Hamiltonian is taken from the scattering region elements.

This defaults to **true**. If there are buffer atoms *behind* the electrode it may be advantageous

to set this to false to extend the electrode region, otherwise it is recommended to keep the default.

This option changes how $\mathbf{M}_{\epsilon,\epsilon}$, see Eq. (29), is setup.

For **true** $\{\mathbf{H}_\epsilon, \mathbf{S}_\epsilon\}$ are taken from the electrode file (**TS.Elec.<>.HS**).

For **false** $\{\mathbf{H}_\epsilon, \mathbf{S}_\epsilon\}$ are substituted by the device calculations electrode region. I.e. it is the self-consistent Hamiltonian.

DM-update

depends on: **TS.Elec.<>.Bulk**

String of values **none**, **cross-terms** or **all** which controls which part of the electrode density matrix elements that are updated. The density matrices that comprises an electrode and device-electrode region can be written as (omitting the central device region)

$$\rho = \begin{bmatrix} \rho_\epsilon & \rho_{\epsilon D} & 0 \\ \rho_{D\epsilon} & \ddots & \ddots \\ 0 & \ddots & \ddots \end{bmatrix} \quad (36)$$

This flag determines whether ρ_ϵ (**all**) or $\rho_{\epsilon D}$ (**cross-terms** and **all**) or neither (**none**) are updated in the SCF. The density matrices contains the charges and thus affects the Hamiltonian and Poisson solutions. Generally the default value will suffice and is recommended.

If **TS.Elec.<>.Bulk false** this is forced to **all** and cannot be changed.

If **TS.Elec.<>.Bulk true** this defaults to **cross-terms**, but may be changed.

NOTE: if this is **none** the forces on the atoms coupled to the electrode regions are *not* to be trusted. The value **none** should be avoided, if possible.

DM-init

depends on: **TS.Elec.DM.Init**, **TS.Elec.<>.Bulk**, **TS.Voltage**

String of values **bulk**, **diagon** (default) or **force-bulk** which controls whether the DM is initially overwritten by the DM from the bulk electrode calculation. This requires the DM file for the electrode to be present. Only **force-bulk** will have effect if $V \neq 0$. Otherwise this option only affects $V = 0$ calculations.

The density matrix elements in the electrodes of the scattering region may be forcefully set to the bulk values by reading in the DM of the corresponding electrode. If one uses **TS.Elec.<>.Bulk false** it may be dis-advantageous to set this to **bulk**. If the system is well setup (good screening towards electrodes), setting this to **bulk** may be advantageous.

This option may be used to check how good the electrodes are screened, see **TS.dQ fermi**.

out-of-core

depends on: **TS.Elec.<>.Gf**

If **true** (default) the GF files are created which contain the surface Green function. If **false** the surface Green function will be calculated when needed. Setting this to **false** will heavily degrade performance and it is highly discouraged!

Gf String with filename of the surface Green function data (**SystemLabel.TSGF***). This may be used to place a common surface Green function file in a top directory which may then be used in all calculations using the same electrode and the same contour. If doing many calculations with the same electrode and $\mathbf{k}$, E grids, then this can greatly improve throughput. It has a minor cost of disk-space. Note that the energy-grids are dependent on the applied bias.

Gf-Reuse

depends on: **TS.Elec.<>.Out-of-core**, **TS.Elec.<>.Gf**

Logical deciding whether the surface Green function file should be re-used or deleted. If this is **false** the surface Green function file is deleted and re-created upon start.

pre-expand

depends on: **TS.Elec.<>.out-of-core**

String denoting how the expansion of the surface Green function file will be performed. This only affects the Green function file if **Bloch** is larger than 1. By default the Green function file will contain the fully expanded surface Green function, but not Hamiltonian and overlap matrices (**Green**). One may reduce the file size by setting this to **Green** which only expands the surface Green function. Finally **none** may be passed to reduce the file size to the bare minimum. For performance reasons **all** is preferred.

If disk-space is a limited resource and the **SystemLabel.TSGF*** files are really big, try **none**.

Eta

depends on: **TS.Elec.Eta**

Control the imaginary energy (η) of the surface Green function for this electrode.

The imaginary part is *only* used in the non-equilibrium contours since the equilibrium are already lifted into the complex plane. Thus this η reflects the imaginary part in the $GTG^\dagger$ calculations. Ensure that all imaginary values are larger than 0 as otherwise TRANSIESTA may seg-fault.

NOTE: if this energy is negative the complex value associated with the non-equilibrium contour is used. This is particularly useful when providing a user-defined contour along the real axis.

See Sec. 11.6 for details. This options changes the η value in the calculated self-energy ($\Sigma(E + i\eta)$), while it does not change the η value used in the device region.

DE

depends on: **TS.Elec.<>.DM-init**

Density and energy density matrix file for the electrode. This may be used to initialize the density matrix elements in the electrode region by the bulk values. See **TS.Elec.<>.DM-init bulk**.

NOTE: this should only be performed on one TRANSIESTA calculation as then the scattering region **SystemLabel.TSDE** contains the electrode density matrix.

Bloch 3 integers should be present on this line which each denote the number of times bigger the scattering region electrode is compared to the electrode, in each lattice direction. Remark that these expansion coefficients are with regard to the electrode unit-cell. This is denoted “Bloch” because it is an expansion based on Bloch waves.

NOTE: Using symmetries such as periodicity will greatly increase performance.

Bloch-A/a1|B/a2|C/a3 Specific Bloch expansions in each of the electrode unit-cell direction. See **Bloch** for details.

Accuracy

depends on: **TS.Elec.Accuracy**

Control the convergence accuracy required for the self-energy calculation when using the Lopez-Sancho, Lopez-Sancho iterative scheme.

NOTE: advanced use *only*.

delta-Ef Specify an offset for the Fermi-level of the electrode. This will directly be added to the Fermi-level found in the electrode file.

Effectively this will transform the used chemical potential to

$$\mu'_{\text{used}} = \mu_{\text{used}} + \delta E_F. \quad (37)$$

NOTE: this option only makes sense for semi-conducting electrodes since it shifts the entire electronic structure. This is because the Fermi-level may be arbitrarily placed anywhere in

the band gap. It is the users responsibility to define a value which does not introduce a potential drop between the electrode and device region. Please do not use unless you really know what you are doing.

V-fraction Specify the fraction of the chemical potential shift in the electrode-device coupling region. This corresponds to altering Eq. (29) by:

$$\mathbf{H}_{\epsilon,D} \leftarrow \mathbf{H}_{\epsilon,D} + \mu_{\epsilon}V - \text{fraction}\mathbf{S}_{\epsilon,D} \quad (38)$$

in the coupling region. Consequently the value *must* be between 0 and 1.

NOTE: this option *only* makes sense for **TS.Elec.<>.DM-update none** since otherwise the electrostatic potential will be incorporated in the Hamiltonian.

Only expert users should play with this number.

check-kgrid For N_{ϵ} electrode calculations the **k** mesh will sometimes not be equivalent for the electrodes and the device region calculations. However, TRANSIESTA requires that the device and electrode **k** samplings are commensurate. This flag controls whether this check is enforced for a given electrode.

NOTE: only use if fully aware of the implications!

There are several flags which are globally controlling the variables for the electrodes (with **TS.Elec.<>** taking precedence).

TS.Elecs.Bulk true (logical)

This globally controls how the Hamiltonian is treated in all electrodes. See **TS.Elec.<>.Bulk**.

TS.Elecs.Eta 1 meV (energy)

Globally control the imaginary energy (η) used for the surface Green function calculation on the non-equilibrium contour. See **TS.Elec.<>.Eta** for extended details on the usage of this flag.

TS.Elecs.Accuracy 10^{-13} eV (energy)

Globally control the accuracy required for convergence of the self-energy. See **TS.Elec.<>.Accuracy**.

TS.Elecs.Neglect.Principal false (logical)

If this is **false** TRANSIESTA dies if there are connections beyond the principal cell.

NOTE: set this to **true** with care, non-physical results may arise. Use at your own risk!

TS.Elecs.Gf.Reuse true (logical)

Globally control whether the surface Green function files should be re-used (**true**) or re-created (**false**).

See **TS.Elec.<>.Gf-Reuse**.

TS.Elecs.Out-of-core true (logical)

Whether the electrodes will calculate the self energy at each SCF step. Using this will not require the surface Green function files but at the cost of heavily degraded performance.

See **TS.Elec.<>.Out-of-core**.

TS.Elecs.DM.Update cross-terms|all|none (string)

Globally controls which parts of the electrode density matrix gets updated.

See **TS.Elec.<>.DM-update**.

TS.Elecs.DM.Init *diagon|bulk|force-bulk* (*string*)

Specify how the density matrix elements in the electrode regions of the scattering region will be initialized when starting TRANSIESTA.

See **TS.Elec.<>.DM-init**.

TS.Elecs.Coord.EPS 0.001 Ang (*length*)

When using Bloch expansion of the self-energies one may experience difficulties in obtaining perfectly aligned electrode coordinates.

This parameter controls how strict the criteria for equivalent atomic coordinates is. If TRANSIESTA crashes due to mismatch between the electrode atomic coordinates and the scattering region calculation, one may increase this criteria. This should only be done if one is sure that the atomic coordinates are almost similar and that the difference in electronic structures of the two may be negligible.

11.8.4 Chemical potentials

For N_{ϵ} electrodes there will also be N_{μ} chemical potentials. They are defined via blocks similar to **TS.Elecs**.

%block TS.ChemPots *<None>* (*block*)

Each line denotes a new chemical potential which is defined in the **TS.ChemPot.<>** block.

%block TS.ChemPot.<> *<None>* (*block*)

Each line defines a setting for the chemical potential named *<>*.

chemical-shift|mu Define the chemical shift (an energy) for this chemical potential. One may specify the shift in terms of the applied bias using **V/<integer>** instead of explicitly typing the energy.

contour.eq A subblock which defines the integration curves for the equilibrium contour for this equilibrium chemical potential. One may supply as many different contours to create whatever shape of the contour

Its format is

```
contour.eq
begin
  <contour-name-1>
  <contour-name-2>
  ...
end
```

NOTE: If you do *not* specify **contour.eq** in the block one will automatically use the continued fraction method and you are encouraged to use 50 or more poles^[12].

ElectronicTemperature|Temp|kT Specify the electronic temperature (as an energy or in Kelvin). This defaults to **TS.ElectronicTemperature**.

One may specify this in units of **TS.ElectronicTemperature** by using the unit **kT**.

contour.eq.pole Define the number of poles used via an energy specification. TRANSIESTA will automatically convert the energy to the closest number of poles (rounding up).

NOTE: this has precedence over **TS.ChemPot.<>.contour.eq.pole.N** if it is specified *and* a positive energy. Set this to a negative energy to directly control the number of poles.

contour.eq.pole.N Define the number of poles via an integer.

NOTE: this will only take effect if **TS.ChemPot.<>.contour.eq.pole** is a negative energy.

NOTE: It is important to realize that the parametrization in 4.1 of the voltage into the chemical potentials enables one to have a *single* input file which is never required to be changed, even when changing the applied bias (if using the command line options for specifying the applied bias). This is different from 4.0 and prior versions since one had to manually change the **TS.biasContour.NumPoints** for each applied bias.

These options complicate the input sequence for regular 2 electrode which is unfortunate.

Using `tselecs.sh -only-mu` yields this output:

```
%block TS.ChemPots
  Left
  Right
%endblock
%block TS.ChemPot.Left
  mu V/2
  contour.eq
    begin
      C-Left
      T-Left
    end
%endblock
%block TS.ChemPot.Right
  mu -V/2
  contour.eq
    begin
      C-Right
      T-Right
    end
%endblock
```

Note that the default is a 2 electrode setup with chemical potentials associated directly with the electrode names “Left”/“Right”. Each chemical potential has two parts of the equilibrium contour named according to their name.

11.8.5 Complex contour integration options

Specifying the contour for N_{e} electrode systems is a bit extensive due to the possibility of more than 2 chemical potentials. Please use the `Util/TS/tselecs.sh` as a means to create default input blocks.

The contours are split in two segments. One, being the equilibrium contour of each of the different chemical potentials. The second for the non-equilibrium contour. The equilibrium contours are shifted according to their chemical potentials with respect to a reference energy. Note that for

TRANSIESTA the reference energy is named the Fermi-level, which is rather unfortunate (for non-equilibrium but not equilibrium). Fortunately the non-equilibrium contours are defined from different chemical potentials Fermi functions, and as such this contour is defined in the window of the minimum and maximum chemical potentials. Because the reference energy is the periodic Fermi level it is advised to retain the average chemical potentials equal to 0. Otherwise applying different bias will shift transmission curves calculated via TBTRANS relative to the average chemical potential.

In this section the equilibrium contours are defined, and in the next section the non-equilibrium contours are defined.

TS.Contours.Eq.Pole 1.5 eV (energy)

The imaginary part of the line integral crossing the chemical potential. Note that the actual number of poles may differ between different calculations where the electronic temperatures are different.

NOTE: if the energy specified is negative, **TS.Contours.Eq.Pole.N** takes effect.

TS.Contours.Eq.Pole.N 8 (integer)

Manually select the number poles for the equilibrium contour.

NOTE: this flag will only take effect if **TS.Contours.Eq.Pole** is a negative energy.

%block TS.Contour.<> <None> (block)

Specify a contour named <> with options within the block.

The names <> are taken from the **TS.ChemPot.<>.contour.eq** block in the chemical potentials.

The format of this block is made up of at least 4 lines, in the following order of appearance.

part Specify which part of the equilibrium contour this is:

circle The initial circular part of the contour

square The initial square part of the contour

line The straight line of the contour

tail The final part of the contour *must* be a tail which denotes the Fermi function tail.

from a to b Define the integration range on the energy axis. Thus *a* and *b* are energies.

The parameters may also be given values **prev/next** which is the equivalent of specifying the same energy as the previous contour it is connected to.

NOTE: that *b* may be supplied as **inf** for **tail** parts.

points/delta Define the number of integration points/energy separation. If specifying the number of points an integer should be supplied.

If specifying the separation between consecutive points an energy should be supplied.

method Specify the numerical method used to conduct the integration. Here a number of different numerical integration schemes are accessible

mid|mid-rule Use the mid-rule for integration.

simpson|simpson-mix Use the composite Simpson 3/8 rule (three point Newton-Cotes).

boole|boole-mix Use the composite Booles rule (five point Newton-Cotes).

G-legendre Gauss-Legendre quadrature.

NOTE: has **opt left**

NOTE: has **opt right**

tanh-sinh Tanh-Sinh quadrature.

NOTE: has **opt precision <>**

NOTE: has **opt left**

NOTE: has **opt right**

G-Fermi Gauss-Fermi quadrature (only on tails).

opt Specify additional options for the **method**. Only a selected subset of the methods have additional options.

These options complicate the input sequence for regular 2 electrode which is unfortunate. However, it allows highly customizable contours.

Using `tselecs.sh -only-c` yields this output:

```
TS.Contours.Eq.Pole 2.5 eV
%block TS.Contour.C-Left
  part circle
    from -40. eV + V/2 to -10 kT + V/2
    points 25
    method g-legendre
    opt right
%endblock
%block TS.Contour.T-Left
  part tail
    from prev to inf
    points 10
    method g-fermi
%endblock
%block TS.Contour.C-Right
  part circle
    from -40. eV -V/2 to -10 kT -V/2
    points 25
    method g-legendre
    opt right
%endblock
%block TS.Contour.T-Right
  part tail
    from prev to inf
    points 10
    method g-fermi
%endblock
```

These contour options refer to input options for the chemical potentials as shown in Sec. 11.8.4 (p. 181). Importantly one should note the shift of the contours corresponding to the chemical potential (the shift corresponds to difference from the reference energy used in TRANSIESTA).

11.8.6 Bias contour integration options

The bias contour is similarly defined as the equilibrium contours. Please use the `Util/TS/tselecs.sh` as a means to create default input blocks.

TS.Contours.nEq.Eta $\min[\eta_\epsilon]/10$ (energy)
depends on: **TS.Elecs.Eta**

The imaginary part (η) of the device states. While this may be set to 0 for most systems it defaults to the minimum η value for the electrodes ($\min[\eta_\epsilon]/10$). This ensures that the device broadening is always smaller than the electrodes while allowing broadening of localized states.

TS.Contours.nEq.Fermi.Cutoff $5 k_B T$ (energy)

The bias contour is limited by the Fermi function tails. Numerically it does not make sense to integrate to infinity. This energy defines where the bias integration window is turned into zero. Thus above $-|V|/2 - E$ or below $|V|/2 + E$ the DOS is defined as exactly zero.

%block TS.Contours.nEq `<None>` (block)

Each line defines a new contour on the non-equilibrium bias window. The contours defined *must* be defined in **TS.Contour.nEq.<>**.

These contours must all be **part line** or **part tail**.

%block TS.Contour.nEq.<> `<None>` (block)

This block is *exactly* equivalently defined as the **TS.Contour.<>**. See page 183.

The default options related to the non-equilibrium bias contour are defined as this:

```
%block TS.Contours.nEq
  neq
%endblock TS.Contours.nEq
%block TS.Contour.nEq.neq
  part line
  from -|V|/2 - 5 kT to |V|/2 + 5 kT
  delta 0.01 eV
  method mid-rule
%endblock TS.Contour.nEq.neq
```

If one chooses a different reference energy than 0, then the limits should change accordingly. Note that here **kT** refers to **TS.ElectronicTemperature**.

11.9 Output

TRANSIESTA generates several output files.

SystemLabel.DM : The SIESTA density matrix. SIESTA initially performs a calculation at zero bias assuming periodic boundary conditions in all directions, and no voltage, which is used as a starting point for the TRANSIESTA calculation.

SystemLabel.TSDE : The TRANSIESTA density matrix and energy density matrix. During a TRANSIESTA run, the **SystemLabel.DM** values are used for the density matrix in the buffer

(if used) and electrode regions. The coupling terms may or may not be updated in a TRANSIESTA run, see **TS.Elec.<>.DM-update**.

SystemLabel.TS.HSX : The Hamiltonian corresponding to **SystemLabel.TSDE**. This file also contains geometry information etc. needed by TRANSIESTA and TBTRANS.

SystemLabel.TSHS : The Hamiltonian corresponding to **SystemLabel.TSDE**. This file also contains geometry information etc. needed by TRANSIESTA and TBTRANS. Deprecated since the **SystemLabel.TS.HSX** file contains the same information.

SystemLabel.TS.KP : The k -points used in the TRANSIESTA calculation. See SIESTA **SystemLabel.KP** file for formatting information.

SystemLabel.TSFA : Forces only on atoms in the device region. See **TS.Forces** for details.

SystemLabel.TSCCEQ* : The equilibrium complex contour integration paths.

SystemLabel.TSCCNEQ* : The non-equilibrium complex contour integration paths for *correcting* the equilibrium contours.

SystemLabel.TSGF* : Self-energy files containing the used self-energies from the leads. These are very large files used in the SCF loop. Once completed one can safely delete these files. For heavily increased throughput these files may be re-used for the same electrode settings in various calculations.

11.10 Utilities for analysis: TBtrans

Please see the separate TBTRANS manual (tbtrans.pdf).

12 QM/MM with SIESTA

SIESTA is ready to work as a QM-engine for hybrid Quantum Mechanics/ Molecular Mechanics (QM/MM) simulations, using an external molecular mechanics software as a driver. For this purpose, you should run SIESTA independently, since communication is done via either pipes or sockets; which is to say, you must have both your MM code and SIESTA running simultaneously. You will need all of the regular components of a SIESTA calculation, but atomic positions for the QM region (and cell dimensions) will be overwritten with data coming from the MM driver.

NOTE: Please refer to the developer documentation on how to implement the communication from the MM side, and check the `qmmm_extern` test for a reference implementation.

It is also possible to run a stand-alone calculation using only SIESTA providing a set of point charges and their coordinates in the `fdf` files. After calculation, SIESTA will output the forces on the point charges in a `.FAPC` file. The following options refer to both the stand-alone calculations and when SIESTA is used as a QM engine for an external driver program.

QMMM.Enabled `false` *(logical)*

Enables the application of an external potential created by point charges near the QM system; i.e., enables QM/MM calculations.

QMMM.CoulombType 1 *(integer)*

When set to 1 (default), uses Ewald summations for the electrostatic interaction between QM and MM regions. Setting this variable to 2 switches this scheme to just real space distance cut-off. This is not recommended, and the option is kept only for testing purposes.

QMMM.CutOff 20 Bohr *(length)*

Cut-off distance for QM-MM interactions. This magnitude is only relevant in the real part of the Ewald summation, or in the entire cutoff scheme. It is highly advisable that this magnitude does not go above one half of the simulation box's shortest side.

QMMM.DensityCut 1.10^{-6} Bohr *(length)*

Tolerance when deciding if a given grid point is too close to a classical point charge. This is usually not worth changing.

QMMM.Ewald.Rcut 20 Bohr *(length)*

In Ewald summations, this value is used as a cut-off distance for how long the direct summation goes. Same considerations as with QMMM.CutOff apply here, and it is usually a good idea to keep of them in similar magnitudes if not the same.

This radius is also used to set the value of the Ewald alpha coefficient, and the reciprocal space summation cut-off.

%block PartialCharges *<None>* *(block)*

This block contains the positions (in Angstrom) and charge values (in e-) for classical point charges. Each line corresponds to a given charge, indicating first x, y and z coordinates, and then the charge value.

```
from iq = 1 to ncharges
  read: x(iq), y(iq), z(iq), q(iq)
```

It is important to note that this block will be ignored if SIESTA is used by an external MM driver.

The following options are experimental and enable some form of smoothing potential along the Z-axis when dealing with electrodes. Use these only if you know exactly what you're doing.

QMMM.SmoothElectrode false *(logical)*

Enables potential smoothing.

QMMM.SmoothElectrode.Electrode 0 Bohr *(length)*

Length of the electrodes.

QMMM.SmoothElectrode.Smooth 0 Bohr *(length)*

Length of the smoothing section.

12.1 Using external MM driver

The following options are only used if SIESTA is intended to run with an external MM driver. Most of these require knowing at least a bit of how the communication is handled by the MM program.

QMMM.Driver *unknown* (string)

Indicates to SIESTA the name of the MM driver used. This option has currently no effect on any part of the run, but may be used in the future.

QMMM.Driver.Type *socket* (string)

The type of communication used by the MM program, can be either "socket" or "pipe".

If the chosen communication type is "pipe", the following three options are also relevant:

QMMM.Driver.QMRegionFile *siesta.qmatoms* (string)

The name of the pipe/file containing the incoming QM coordinates and cell dimensions.

QMMM.Driver.MMChargeFile *siesta.mmatoms* (string)

The name of the pipe/file containing the incoming MM coordinates and their charge values.

QMMM.Driver.ForceOutFile *siesta.allforces* (string)

The name of the pipe/file containing the outgoing energy, forces and stress so that they are read by the MM program.

Otherwise, if the chosen communication type is "socket", the following three options are relevant instead:

QMMM.Driver.Address *localhost* (string)

The address of the socket used for communication. See also SocketType below in case the address is an IP address.

QMMM.Driver.Port *10002* (integer)

For sockets that go over IP, the number of the port used for communications.

QMMM.Driver.SocketType *inet* (string)

Can be either "inet" (for TCP/IP sockets) or "unix" for local sockets.

12.2 The in-house QM/MM driver

12.2.1 Running the driver

SIESTA comes with a simple Molecular Mechanics program to use as the main driver for QM/MM simulations. As an additional input for this program, one needs to provide an extra "amber.parm" parameter file which contains all of the AMBER forcefield parameters needed for the classical part. There are a few examples provided in /Util/QMMM-driver/Tests.

This driver can be used in two ways to communicate with siesta:

- Launching both the driver and SIESTA as different processes:

```
siesta_qmmm < driver_input.fdf > driver_out.out &
siesta_qmmm < siesta_input.fdf > siesta_out.out
```

Note that two separate input files are needed, one for the driver and one for SIESTA .

- Launching SIESTA as a subprocess from the driver, using the **LaunchSiesta** option. In this case, a separate SIESTA input file is not needed, and the driver will pass along the options present in its own fdf file.

Communications are done via pipes (see previous section), and if SIESTA is launched independently (first option above), the following block is necessary on the SIESTA input file:

```
QMMM.Driver.QMRegionFile {label}.siesta.coords
QMMM.Driver.MMChargeFile {label}.siesta.pc
QMMM.Driver.ForceOutFile {label}.siesta.forces
```

Replace {label} for the system label you are using in the QMMM driver.

12.2.2 The MM parameter file

By default, the MM parameter file is taken as an "amber.parm" file residing in the working directory. There are two different ways to specify a different path or filename:

- Via the SIESTA_MM_PARM_FILE environment variable. It should point to the specific file. (for example: SIESTA_MM_PARM_FILE=/home/myparm.parm)
- Via the **MM.ParmFile** option within the driver's fdf file.

12.2.3 QM/MM driver-specific options

The driver inherits from SIESTA most of the options related to system description and molecular dynamics, thus variables such as **NumberOfAtoms** will remain the same.

However, there are a few driver-specific options.

```
%block QM.AtomTypes <None> (block)
```

This block contains the MM identities for atoms in the QM region. This is only relevant for Lennard-Jones interactions. Atoms must be in the same order as in the **AtomicCoordinatesAndAtomicSpecies** block. For example, for a single water molecule in the QM region:

```
%block QM.AtomTypes
OW
HW
HW
%endblock QM.AtomTypes
```

The atom types (OW, HW) must coincide with the information available in the amber.parm file.

MM.ParmFile 'amber.parm' *(string)*

File containing the classical (MM) forcefield parameters.

NumberOfMMAtoms 0 *(integer)*

The number of classical MM atoms in the system (not counting link atoms).

%block MM.Atoms <None> *(block)*

This block contains names and coordinates for MM atoms. It follows a structure similar to that of a PDB file, thus making it easier to copy from an available PDB:

```
%block MM.Atoms
  ATOM      1  O   HOH      1      11.610  15.591  10.441
  ATOM      2  H1  HOH      1      10.681  15.570  10.811
  ATOM      3  H2  HOH      1      11.621  15.201   9.521
  ATOM      4  O   HOH      2      11.560  12.061   0.651
  ATOM      5  H1  HOH      2      11.911  11.890   1.571
  ATOM      6  H2  HOH      2      10.681  11.610   0.530
%endblock MM.Atoms
```

It begins with a single word, followed by the atom index, then the atom name, then the residue name and the residue index, and finally the X, Y, and Z coordinates. Both indices start from 1. **Residue and atom names must coincide with the information present in the amber.parm file.**

CoulombType 'ewald' *(string)*

Sets the type of description for coulomb interactions in MM-MM and QM-MM interactions. Can be either "ewald" or "cut-off".

MM.Cutoff 0.8, Ang *(length)*

For MM-MM interactions. When Ewald summations are used, this will also be the value for **QMMM.Ewald.Rcut**.

Block.Cutoff 100.0, Ang *(length)*

If given a value of 99.0 Ang or below, MM atoms that are further than a **Block.Cutoff** distance from the QM region will be frozen and prevented from moving during the molecular dynamics.

CenterQMSystem false *(logical)*

At every MD step, re-center the entire system on the geometrical center for the QM region.

LaunchSiesta false *(logical)*

Launch SIESTA as a subprocess for the QMMM driver. This avoids the need to run separate processes. By default, the driver will look for a SIESTA executable in the working directory, but a different location can be provided using the SIESTA environment variable. For example:

```
SIESTA=path/to/siesta/bin \
siesta_qmmm < driver_input.fdf > driver_out.out
```

SIESTA_PARALLEL_COMMAND can be combined with the SIESTA environment variable, see **MPICommand**.

NumberMPInodes 1 *(integer)*

When using **LaunchSiesta**, this option can be used to choose the amount of CPUs used for

SIESTA itself (the driver always runs in serial). When set to 1, **MPICommand** will be ignored.

This option can be overridden with the `SIESTA_PARALLEL_COMMAND` environment variable (see **MPICommand**).

MPICommand *'mpirun'* *(string)*

When using **LaunchSiesta**, this option can be used to choose a different program for parallelization, for example 'srun'.

This can be overridden with the `SIESTA_PARALLEL_COMMAND` environment variable. In that case, `SIESTA_PARALLEL_COMMAND` must contain all of the information needed for the MPI launcher (e.g., the number of CPUs). This can be used as follows:

```
SIESTA_PARALLEL_COMMAND="srun -n 8" \  
siesta_qmmm < driver_input.fdf > driver_out.out
```

`SIESTA_PARALLEL_COMMAND` can be combined with the `SIESTA` environment variable, see **LaunchSiesta**.

13 ANALYSIS TOOLS

There are a number of analysis tools and programs in the `Util` directory. Some of them have been directly or indirectly mentioned in this manual. Their documentation is the appropriate sub-directory of `Util`. See `Util/README`.

In addition to the shipped utilities SIESTA is also officially supported by `sisl`^[16] which is a Python library enabling many of the most commonly encountered things.

13.1 2D Lindhard function calculator

The Lindhard Function calculator computes the 2D Lindhard response ^[?] from SIESTA outputs. The outputs required are the `EIG` file (with the eigenvalues for each k-point) and the `KP` file (with the coordinates for each k-point). The original SIESTA calculation must be done with the **TimeReversalSymmetryForKpoints** option set to **false**. Other than that, the Lindhard-function specific options should be added to the same `fdf` file as the SIESTA calculation. Then, the utility can be used as:

```
lindhard < <fdf file> > <output file>
```

The `lindhard` utility accepts the following options:

Lindhard.Temperature 300 K *(temperature)*

The temperature for the Fermi distribution function used to calculate the Lindhard response.

Lindhard.FirstBand 0 *(integer)*

Index of the first band to be included in the calculation; i.e., the lowest energy eigenvalue.

Lindhard.LastBand 0 *(integer)*

Index of the last band to be included in the calculation; i.e., the highest energy eigenvalue.

- Lindhard.ngridx** 0 *(integer)*
 Number of k-grid points in the direction of the first reciprocal lattice vector. Must be an even number.
- Lindhard.ngridy** 0 *(integer)*
 Number of k-grid points in the direction of the second reciprocal lattice vector. Must be an even number.
- Lindhard.ngridz** 0 *(integer)*
 Number of k-grid points in the direction of the third reciprocal lattice vector. Must be an even number.
- Lindhard.nq1** 1 *(integer)*
 Step size in the first direction. In the direction of the first reciprocal lattice vector, calculate only every nq1 points. This only affects the granularity of the output, not the calculation itself.
- Lindhard.nq2** 1 *(integer)*
 Step size in the second direction. In the direction of the second reciprocal lattice vector, calculate only every nq2 points. This only affects the granularity of the output, not the calculation itself.

14 SCRIPTING

In the `Util/Scripting` directory we provide an experimental python scripting framework built on top of the “Atomic Simulation Environment” (see <https://wiki.fysik.dtu.dk/ase>) by the CAMD group at DTU, Denmark.

(NOTE: “ASE version 2”, not the new version 3, is needed)

There are objects implementing the “Siesta as server/subroutine” feature, and also hooks for file-oriented-communication usage. This interface is different from the SIESTA-specific functionality already contained in the ASE framework.

Users can create their own scripts to customize the “outer geometry loop” in SIESTA, or to perform various repetitive calculations in compact form.

Note that the interfaces in this framework are still evolving and are subject to change.

Suggestions for improvements can be sent to Alberto Garcia (albertog@icmab.es)

15 PROBLEM HANDLING

15.1 Error and warning messages

chkdim: ERROR: In routine dimension parameter = value. It must be ... And other similar messages.

Description: Some array dimensions which change infrequently, and do not lead to much memory use, are fixed to oversized values. This message means that one of this parameters is too small and needs to be increased. However, if this occurs and your system is not very

large, or unusual in some sense, you should suspect first of a mistake in the data file (incorrect atomic positions or cell dimensions, too large cutoff radii, etc).

Fix: Check again the data file. Look for previous warnings or suspicious values in the output. If you find nothing unusual, edit the specified routine and change the corresponding parameter.

16 REPORTING BUGS

Your assistance is essential to help improve the program. If you find any problem, or would like to offer a suggestion for improvement, please follow the instructions in the file `Docs/REPORTING_BUGS`.

Since SIESTA has moved to <https://gitlab.com/siesta-project/siesta> you are encouraged to follow the instructions by pressing “New Issue” and selecting “Bug” in the Description drop-down. Also please follow the debug build options, see Sec. 2.1.1

17 ACKNOWLEDGMENTS

We want to acknowledge the use of a small number of routines, written by other authors, in developing the siesta code. In most cases, these routines were acquired by now-forgotten routes, and the reported authorships are based on their headings. If you detect any incorrect or incomplete attribution, or suspect that other routines may be due to different authors, please let us know.

- The main nonpublic contribution, that we thank thoroughly, are modified versions of a number of routines, originally written by **A. R. Williams** around 1985, for the solution of the radial Schrödinger and Poisson equations in the APW code of Soler and Williams (PRB **42**, 9728 (1990)). Within SIESTA, they are kept in files `arw.f` and `periodic_table.f`, and they are used for the generation of the basis orbitals and the screened pseudopotentials.
- The exchange-correlation routines contained in SiestaXC were written by J.M.Soler in 1996 and 1997, in collaboration with **C. Balbás** and **J. L. Martins**. Routine `pzxc`, which implements the Perdew-Zunger LDA parametrization of `xc`, is based on routine `velect`, written by **S. Froyen**.
- The serial version of the multivariate fast fourier transform used to solve Poisson’s equation was written by **Clive Temperton**.
- Subroutine `iomd.f` for writing MD history in files was originally written by **J. Kohanoff**.

We want to thank very specially **O. F. Sankey**, **D. J. Niklewski** and **D. A. Drabold** for making the FIREBALL code available to P. Ordejón. Although we no longer use the routines in that code, it was essential in the initial development of the SIESTA project, which still uses many of the algorithms developed by them.

We thank **V. Heine** for his support and encouraging us in this project.

The SIESTA project is supported by the Spanish DGES through several contracts. We also acknowledge past support by the Fundación Ramón Areces.

18 APPENDIX: Physical unit names recognized by FDF

Since SIESTA 5.0 the units follow the CODATA 2018 values. This affects comparisons with prior versions of SIESTA due to small numeric differences.

To compare numerical values between SIESTA 5.0 and prior versions one have to recompile 5.0 with

```
cmake [.....] -DWITH_UNIT_CONVENTION=legacy
```

. Please *only* use this for comparisons and not for production runs.

fdf accepts nearly all conventional units used in physics and chemistry. If a unit is not accepted a list of accepted units for the requested dimension will be printed to standard out.

19 APPENDIX: XML Output

From version 2.0, SIESTA includes an option to write its output to an XML file. The XML it produces is in accordance with the CMLComp subset of version 2.2 of the Chemical Markup Language. Further information and resources can be found at <http://cmlcomp.org/> and tools for working with the XML file can be found in the `Util/CMLComp` directory.

The main motivation for standardised XML (CML) output is as a step towards standardising formats for uses like the following.

- To have SIESTA communicating with other software, either for postprocessing or as part of a larger workflow scheme. In such a scenario, the XML output of one SIESTA simulation may be easily parsed in order to direct further simulations. Detailed discussion of this is out of the scope of this manual.
- To generate webpages showing SIESTA output in a more accessible, graphically rich, fashion. This section will explain how to do this.

19.1 Controlling XML output

XML.Write `false` *(logical)*

Determine if the main XML file should be created for this run.

19.2 Converting XML to XHTML

The translation of the SIESTA XML output to a HTML-based webpage is done using XSLT technology. The stylesheets conform to XSLT-1.0 plus EXSLT extensions; an xslt processor capable of dealing with this is necessary. However, in order to make the system easy to use, a script called `ccViz` is provided in `Util/CMLComp` that works on most Unix or Mac OS X systems. It is run like so:

```
./ccViz SystemLabel.xml
```

A new file will be produced. Point your web-browser at `SystemLabel.xhtml` to view the output.

The generated webpages include support for viewing three-dimensional interactive images of the system. If you want to do this, you will either need `jMol` (<http://jmol.sourceforge.net>) installed or access to the internet. As this is a Java applet, you will also need a working Java Runtime Environment and browser plugin - installation instructions for these are outside the scope of this manual, though. However, the webpages are still useful and may be viewed without this plugin.

An online version of this tool is available from <http://cmlcomp.org/ccViz/>, as are updated versions of the `ccViz` script.

20 APPENDIX: Selection of precision for storage

Some of the real arrays used in SIESTA are by default single-precision, to save memory. This applies to the array that holds the values of the basis orbitals on the real-space grid, to the historical data sets in Broyden mixing, and to the arrays used in the $O(N)$ routines. Note that the grid functions (charge densities, potentials, etc) are (since mid January 2010) in double precision by default.

The following options and pre-processing symbols control the precision selection.

- Add `-DWITH_GRID_SP` to the CMake invocation to use single-precision for all the grid magnitudes, including the orbitals array and charge densities and potentials. This will cause some numerical differences and will have a negligible effect on memory consumption, since the orbitals array is the main user of memory on the grid, and it is single-precision by default. This setting will recover the default behavior of versions prior to 4.0.
- Use `-DFortran_FLAGS="-DGRID_DP"` to use double-precision for all the grid magnitudes, including the orbitals array. This will significantly increase the memory used for large problems, with negligible differences in accuracy.
- Use `-DFortran_FLAGS="-DBROYDEN_DP"` to use double-precision arrays for the data sets in the Broyden mixing for SCF convergence acceleration.
- Use `-DFortran_FLAGS="-DON_DP"` to use double-precision for all the arrays in the $O(N)$ routines.

21 APPENDIX: Data structures and reference counting

To implement some of the new features (e.g. charge mixing and DM extrapolation), SIESTA uses new flexible data structures. These are defined and handled through a combination and extension of ideas already in the Fortran community:

- Simple templating using the “include file” mechanism, as for example in the FLIBS project led by Arjen Markus (<http://flibs.sourceforge.net>).
- The classic reference-counting mechanism to avoid memory leaks, as implemented in the PyF95++ project (<http://blockit.sourceforge.net>).

Reference counting makes it much simpler to store data in container objects. For example, a circular stack is used in the charge-mixing module. A number of future enhancements depend on this paradigm.

References

- [1] T. Auckenthaler, V. Blum, H.-J. Bungartz, T. Huckle, R. Johanni, L. Krämer, B. Lang, H. Lederer, and P.R. Willems. Parallel solution of partial symmetric eigenvalue problems from electronic structure calculations. *Parallel Computing*, 37(12):783 – 794, 2011. ISSN 0167-8191. doi: <http://dx.doi.org/10.1016/j.parco.2011.05.002>. URL <http://www.sciencedirect.com/science/article/pii/S0167819111000494>. 6th International Workshop on Parallel Matrix Algorithms and Applications (PMAA’10).
- [2] Amartya S. Banerjee, Phanish Suryanarayana, and John E. Pask. Periodic Pulay method for robust and efficient convergence acceleration of self-consistent field iterations. *Chemical Physics Letters*, 647:31–35, mar 2016. ISSN 00092614. doi: 10.1016/j.cplett.2016.01.033. URL <http://linkinghub.elsevier.com/retrieve/pii/S0009261416000464>.
- [3] H. J. C. Berendsen, J. P. M. Postma, W. F. van Gunsteren, A. DiNola, and J. R. Haak. Molecular dynamics with coupling to an external bath. *The Journal of Chemical Physics*, 81(8):3684–3690, 10 1984. ISSN 0021-9606. doi: 10.1063/1.448118. URL <https://doi.org/10.1063/1.448118>.
- [4] D.R Bowler and M.J Gillan. An efficient and robust technique for achieving self consistency in electronic structure calculations. *Chemical Physics Letters*, 325(4):473–476, jul 2000. ISSN 00092614. doi: 10.1016/S0009-2614(00)00750-8. URL <http://linkinghub.elsevier.com/retrieve/pii/S0009261400007508>.
- [5] Mads Brandbyge, José-Luis Mozos, Pablo Ordejón, Jeremy Taylor, and Kurt Stokbro. Density-functional method for nonequilibrium electron transport. *Physical Review B*, 65(16):165401, mar 2002. ISSN 0163-1829. doi: 10.1103/PhysRevB.65.165401. URL <http://link.aps.org/doi/10.1103/PhysRevB.65.165401>.
- [6] R. Cuadrado, R. Robles, A. García, M. Pruneda, P. Ordejón, J. Ferrer, and Jorge I. Cerdá. Validity of the on-site spin-orbit coupling approximation. *Phys. Rev. B*, 104:195104, Nov 2021. doi: 10.1103/PhysRevB.104.195104. URL <https://link.aps.org/doi/10.1103/PhysRevB.104.195104>.
- [7] Alberto García, Matthieu J. Verstraete, Yann Pouillon, and Javier Junquera. The psml format and library for norm-conserving pseudopotential data curation and interoperability. *Comput. Phys. Commun.*, 227:51 – 71, 2018. ISSN 0010-4655. doi: 10.1016/j.cpc.2018.02.011. URL <http://www.sciencedirect.com/science/article/pii/S0010465518300390>.
- [8] Alberto García, Nick Papior, Arsalan Akhtar, Emilio Artacho, Volker Blum, Emanuele Bosoni, Pedro Brandimarte, Mads Brandbyge, J. I. Cerdá, Fabiano Corsetti, Ramón Cuadrado, Vladimir Dikan, Jaime Ferrer, Julian Gale, Pablo García-Fernández, V. M. García-Suárez, Sandra García, Georg Huhs, Sergio Illera, Richard Korytár, Peter Koval, Irina Lebedeva, Lin Lin, Pablo López-Tarifa, Sara G. Mayo, Stephan Mohr, Pablo Ordejón, Andrei Postnikov, Yann Pouillon, Miguel Pruneda, Roberto Robles, Daniel Sánchez-Portal, Jose M. Soler, Rafi Ullah, Victor Wen-zhe Yu, and Javier Junquera. Siesta: Recent developments and applications. *The Journal of Chemical Physics*, 152(20):204108, 2020. doi: 10.1063/5.0005077. URL <https://doi.org/10.1063/5.0005077>.

- [9] G. Kresse and J. Furthmüller. Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set. *Computational Materials Science*, 6(1):15–50, jul 1996. ISSN 09270256. doi: 10.1016/0927-0256(96)00008-0. URL <http://linkinghub.elsevier.com/retrieve/pii/0927025696000080>.
- [10] Lin Lin, Alberto García, Georg Huhs, and Chao Yang. SIESTA-PEXSI: massively parallel method for efficient and accurate ab initio materials simulation without matrix diagonalization. *Journal of Physics: Condensed Matter*, 26(30):305503, jul 2014. ISSN 0953-8984. doi: 10.1088/0953-8984/26/30/305503. URL <http://stacks.iop.org/0953-8984/26/i=30/a=305503?key=crossref.dd07c5e621546c5e67b1052b8800daca>.
- [11] A Marek, V Blum, R Johanni, V Havu, B Lang, T Auckenthaler, A Heinecke, H-J Bungartz, and H Lederer. The elpa library: scalable parallel eigenvalue solutions for electronic structure theory and computational science. *Journal of Physics: Condensed Matter*, 26(21):213201, 2014. URL <http://stacks.iop.org/0953-8984/26/i=21/a=213201>.
- [12] Taisuke Ozaki, Kengo Nishio, and Hiori Kino. Efficient implementation of the nonequilibrium Green function method for electronic transport calculations. *Physical Review B*, 81(3):035116, jan 2010. ISSN 1098-0121. doi: 10.1103/PhysRevB.81.035116. URL <http://link.aps.org/doi/10.1103/PhysRevB.81.035116>.
- [13] Nick Papior, Tue Gunst, Daniele Stradi, and Mads Brandbyge. Manipulating the voltage drop in graphene nanojunctions using a gate potential. *Phys. Chem. Chem. Phys.*, 18(2):1025–1031, 2016. ISSN 1463-9076. doi: 10.1039/C5CP04613K. URL <http://xlink.rsc.org/?DOI=C5CP04613K>.
- [14] Nick Papior, Nicolás Lorente, Thomas Frederiksen, Alberto García, and Mads Brandbyge. Improvements on non-equilibrium and transport Green function techniques: The next-generation TranSiesta. *Computer Physics Communications*, 212:8–24, mar 2017. ISSN 00104655. doi: 10.1016/j.cpc.2016.09.022. URL <https://doi.org/10.1016/j.cpc.2016.09.022>.
- [15] Nick Papior, Gaetano Calogero, Susanne Leitherer, and Mads Brandbyge. Removing all periodic boundary conditions: Efficient nonequilibrium Green’s function calculations. *Physical Review B*, 100(19):195417, nov 2019. ISSN 2469-9950. doi: 10.1103/PhysRevB.100.195417. URL <https://link.aps.org/doi/10.1103/PhysRevB.100.195417>.
- [16] Nick R. Papior. sisl, 2020. URL <https://doi.org/10.5281/zenodo.597181>.
- [17] M P Lopez Sancho, J M Lopez Sancho, and J. Rubio. Highly convergent schemes for the calculation of bulk and surface Green functions. *Journal of Physics F: Metal Physics*, 15(4): 851–858, apr 1985. ISSN 0305-4608. doi: 10.1088/0305-4608/15/4/009. URL <http://stacks.iop.org/0305-4608/15/i=4/a=009?key=crossref.8c77f34b0366ff84eaf622609268f5a2>.
- [18] José M. Soler and Eduardo Anglada. Optimal fourier filtering of a function that is strictly confined within a sphere. *Computer Physics Communications*, 180(7):1134 – 1136, 2009. ISSN 0010-4655. doi: <https://doi.org/10.1016/j.cpc.2009.01.017>. URL <http://www.sciencedirect.com/science/article/pii/S0010465509000332>.

Index

- animation, 52
- antiferromagnetic initial DM, 73
- Backward compatibility, 68, 140
- band structure, 104
- basis, 41
 - basis set superposition error (BSSE), 40
 - Bessel functions, 40
 - confinement radius expansion, 34
 - default soft confinement, 35
 - default soft confinement potential, 35
 - default soft confinement radius, 35
 - filteret basis set, 39
 - filtering, 40
 - fix split-valence table, 33
 - Gen-basis standalone program, 41
 - Gen-basis standalone program, 41
 - ghost atoms, 40
 - minimal, 31
 - new split-valence code, 32
 - PAO, 30, 31, 37
 - per-shell split norm, 39
 - point at infinity, 43
 - polarization, 31, 40
 - polarization orbitals, 34
 - reparametrization of pseudopotential, 43
 - soft confinement potential, 39
 - split valence, 32
 - split valence for H, 32
 - User basis, 41
 - User basis (NetCDF format), 41
- Berry phase, 115
- Bessel functions, 40
- %block, 19
- Born effective charges, 116
- Broyden mixing, 196
- Broyden optimization, 143
- bug reports, 193
- bulk polarization, 115
- cell relaxation, 141
- CERIUS2, 52
- Charge confinement, 30, 39
- Charge of the system, 122, 127
- Chebyshev Polynomials, 92
- Chemical Potential, 92, 93
- CheSS, 16
- CheSS solver, 96
- CML, 195
- compile
 - cmake
 - DSIESTA_WITH_CHESS, 16
 - DSIESTA_WITH_DFTD3, 16
 - DSIESTA_WITH_ELPA, 15
 - DSIESTA_WITH_ELSI, 16
 - DSIESTA_WITH_MPI, 12, 15
 - DSIESTA_WITH_NCDF, 15
 - DSIESTA_WITH_NETCDF, 15
 - DSIESTA_WITH_OPENMP, 13
 - DSIESTA_WITH_PEXSI, 15
 - issues, 17
 - libraries, 14
 - MPI, 12
 - OpenMP, 13
 - pre-processor
 - DCDF, 15
 - DCDF4, 15
 - DMPI, 13
 - DMPI_TIMING, 136
 - DNCDF, 15
 - DNCDF_4, 137
 - DSIESTA__DIAG_2STAGE, 86
 - DSIESTA__METIS, 15
 - DSIESTA__MRRR, 85, 87
 - DSIESTA__MUMPS, 15
 - DSIESTA__UNITS_ORIGINAL, 194
- Conjugate-gradient history information, 142
- constant-volume cell relaxation, 141
- constraints in relaxations, 147
- COOP/COHP curves, 113
 - Folding in Gamma-point calculations, 83
 - Folding in Gamma-point calculations, 82
- cutoff radius, 38
- Data Structures, 197
- denchar, 137
- density of states, 88, 106
- DFT-D3, 16
- Dielectric function,optical absorption, 114

- diffuse orbitals, 30
- Doping, 122, 127
- double- ζ , 31
- egg-box effect, 79–81
- ElG2DOS, 88, 106
- ELPA, 15
- ELSI, 16
- ELSI solvers, 93
- exchange-correlation
 - AM05, 56
 - BH, 57
 - BLYP, 56
 - C09, 57
 - CA, 55
 - cellXC, 58
 - DRSLL, 56
 - GGA, 56
 - KBM, 57
 - LDA, 55
 - LMKLL, 56
 - LSD, 55
 - PBE, 56
 - PBEGcGxHEG, 56
 - PBEGcGxLO, 56
 - PBEJsJrHEG, 56
 - PBEJsJrLO, 56
 - PBEsol, 56
 - PW91, 56
 - PW92, 56
 - PZ, 55
 - revPBE, 56
 - RPBE, 56
 - vdW, 56
 - vdW-DF, 56
 - vdW-DF1, 56
 - vdW-DF2, 56
 - VV, 57
 - WC, 56
- External library
 - BLAS, 14
 - CheSS, 16
 - dft-d3, 16
 - ELPA, 15
 - ELSI, 16
 - fdict, 15
 - flook, 16, 139
 - LAPACK, 14
 - libfdf, 14
 - libGridXC, 14
 - libPSML, 14
 - libXC, 14
 - Metis, 15
 - MPI, 12
 - MUMPS, 15, 167
 - ncdf, 15
 - NetCDF, 15
 - OpenMP, 13
 - PEXSI, 15
 - ScaLAPACK, 15
 - xmlf90, 14
- fatbands, 105
- FDF, 19
- ferromagnetic initial DM, 73
- finite-range pseudo-atomic orbitals, 30
- fixed spin state, 58
- flook, 16, 139
- Force Constants Matrix, 139, 150, 151
- fractional program, 23
- Gate, 125
 - bounded plane, 125
 - box, 126
 - infinite plane, 125
 - spheres, 126
- Gaussians, 31
- Gen-basis, 27
- Gen-basis, 41
- ghost atoms, 22
- ghost atoms, 40
- gnubands, 105
- grid, 78, 79
- Grid precision, 196
- Ground-state atomic configuration, 31
- Hirshfeld population analysis, 111, 113
- input file, 19
- interatomic distances, 53
- isotopes, 24
- JMoL, 51
- JSON timing report, 136
- Kleinman-Bylander projectors, 35

- from PSML file, 35
- LibXC library, 55, 57
- Localized Wave Functions, 92, 93
- Lower order N memory, 93
- LSD, 58
- Macroscopic Polarization, 115
- mesh, 78, 79
- Metis, 15
- minimal basis, 31
- mixps program, 23
- MOLDEN, 51, 52
- Mulliken population analysis, 21, 110, 112
- multiple- ζ , 30, 32
- MUMPS, 15, 167
- NetCDF format, 15, 41
- output
 - $\delta\rho(\vec{r})$, 129
 - atomic coordinates
 - in a dynamics step, 21, 146
 - initial, 146
 - Bader charge, 130
 - band $\vec{k}$ points, 21, 104
 - band structure, 104
 - basis, 41
 - charge density, 129–131
 - charge density and/or wfs for DENCHAR
 - code, 137
 - customization, 21
 - dedicated files, 21
 - density matrix, 74–76
 - density matrix history, 76
 - eigenvalues, 21, 88, 106
 - electrostatic potential, 129
 - forces, 21, 146
 - grid $\vec{k}$ points, 21, 55
 - Hamiltonian, 76
 - Hamiltonian & overlap, 81, 82
 - Hamiltonian history, 76
 - Hirshfeld analysis, 111, 113
 - HSX file, 82
 - Information for COOP/COHP curves, 113
 - ionic charge, 130
 - kpoint mesh for Wannierization, 122
 - local density of states, 108
 - long, 21
 - main output file, 20
 - molecular dynamics
 - Force Constants Matrix, 150
 - history, 146
 - Mulliken analysis, 21, 110, 112
 - overlap matrix, 76
 - overlap matrix history, 76
 - projected density of states, 107
 - total charge, 130
 - total potential, 129
 - TS.HSX file, 82
 - Voronoi analysis, 112, 113
 - wave functions, 21, 106
- output of wave functions for bands, 105
- perturbative polarization, 31
- perturbative polarization, 40
- PEXSI, 15
- PEXSI solver, 97
- polarization orbitals, 30
- Population analysis
 - Hirshfeld, 111
 - Mulliken, 110
 - Voronoi, 112
- Precision selection, 196
- ProcessorY, 135
- pseudopotential
 - ATOM code, 25
 - files, 24
 - generation, 24
 - oncvpsp code, 25
 - PSML format, 25
- PSML, 35
 - from SIESTA's vnl-operator, 25
 - from oncvpsp code, 25
- PSML format, 25
- reading saved data, 137
 - all, 137
 - CG, 142
 - charge density, 74
 - deformation charge density, 74
 - density matrix, 72
 - localized wave functions (order- N), 93
 - XV, 52
 - ZM, 53
- readwf, 106

- readwfsx, 106
- Reference counting, 197
- relaxation of cell parameters only, 141
- removal of intramolecular pressure, 144
- Restart of O(N) calculations, 93
- rippling, 79–81
- RT-TDDFT, 153
- scale factor, 39
- SCF, 62
 - compat-pre4-dm-h, 68
 - Doping, 122, 127
 - mixing, 62, 69
 - Broyden, 64
 - Charge, 63, 69, 70
 - Density, 63
 - Density matrix convergence, 76
 - end of cycle, 69
 - energy convergence, 77
 - energy density matrix convergence, 77
 - Hamiltonian, 63
 - Hamiltonian convergence, 77
 - harris energy convergence, 77
 - Linear, 64
 - Pulay, 64
 - Potential, 125
 - Recomputing H, 69
- SCF convergence criteria, 76
- Scripting, 139
- Sies2arc, 52
- SIES2ARC, 52
- SIESTA, 9
- single- ζ , 31
- Slab dipole correction, 124
- Slabs with net charge, 123
- species, 22
- spin, 58
 - initialization, 72, 73
- split valence, 31
- structure input precedence issues, 53
- synthetic atoms, 23
- TBTRANS, 186
- TDDFT, 153
- Tests, 17, 162
 - lua, 16
- TRANSIESTA, 10
- transiesta
 - electrode
 - principal layer, 164
 - valence configuration (alternate), 23
 - Variational character of E_KS, 61
 - VCA, 23
 - VIBRA, 150
 - Voronoi population analysis, 112, 113
 - Wannier Functions, 122
 - XML, 195
 - XMOL, 51, 52

List of SIESTA files

<istep>.TDRho, 155
BaderCharge.grid.nc, 131
Chlocal.grid.nc, 130
constr.f, 149
DeltaRho.grid.nc, 129
DeltaRho.IN.grid.nc, 74
DM-NNNN.nc, 76
DM.nc, 75
DM_MIXED, 82
DM_MIXED.blocked, 75
DM_OUT, 82
DM_OUT.blocked, 75
DMHS-NNNN.nc, 76
DMHS.nc, 76
ElectrostaticPotential.grid.nc, 129
External/ELSI, 16
External/Wannier/README.md, 120
fdf-XXXXX.log, 20
fdf.log, 20–22
GRAPHVIZ_atom.gv, 173
GRAPHVIZ_orbital.gv, 173
H_DMGEN, 75, 82
H_MIXED, 75, 82
INSTALL.md, 16
LDOS.grid.nc, 103, 109
m_new_dm.F, 83
NEXT_ITER.UCELL.ZMATRIX, 51
OUT.UCELL.ZMATRIX, 51
PEXSI_INTDOS, 102
Rho.grid.nc, 129
Rho.IN.grid.nc, 74
RhoInit.grid.nc, 131
RhoXC.grid.nc, 129
Src/m_new_dm.F, 71
SystemLabel.alloc, 136
SystemLabel.amn, 118
SystemLabel.ANI, 52
SystemLabel.arc, 52
SystemLabel.ATOM-N.HSX, 76
SystemLabel.ATOM.gv, 32
SystemLabel.BADER, 130
SystemLabel.bands, 103–105
SystemLabel.bands.WFSX, 105
SystemLabel.BASIS_ENTHALPY, 42
SystemLabel.BC, 117
SystemLabel.BONDS, 53
SystemLabel.BONDS_FINAL, 53
SystemLabel.CG, 142
SystemLabel.dHSdR.nc, 151
SystemLabel.DIM, 137
SystemLabel.DM, 58, 62, 72, 75, 123, 137, 160, 161, 185
SystemLabel.DMF, 72
SystemLabel.DOS, 107, 108
SystemLabel.DRHO, 129, 130
SystemLabel.EIG, 88, 102, 106, 107, 127
SystemLabel.eigW, 118
SystemLabel.EPSIMG, 114
SystemLabel.FA, 146
SystemLabel.FAC, 146
SystemLabel.FC, 151
SystemLabel.FCC, 151
SystemLabel.fullBZ.WFSX, 88, 113, 114
SystemLabel.grid.nc, 74
SystemLabel.HS, 82
SystemLabel.HSX, 76, 82, 113, 160–162, 177
SystemLabel.IOCH, 130
SystemLabel.KP, 54, 55, 186
SystemLabel.LDOS, 103, 108
SystemLabel.LWF, 93, 137
SystemLabel.MD, 51, 146
SystemLabel.MDC, 146
SystemLabel.MDE, 146
SystemLabel.MDX, 51, 146
SystemLabel.mmn, 118
SystemLabel.N.HSX, 76
SystemLabel.N.TSHS, 76
SystemLabel.nc, 138, 177

SystemLabel.nnkp, 118
 SystemLabel.ORB.gv, 32
 SystemLabel.ORB_INDX, 120, 147
 SystemLabel.PDOS, 107, 108
 SystemLabel.PDOS.xml, 108
 SystemLabel.PLD, 137
 SystemLabel.RHO, 129, 130
 SystemLabel.RHOINIT, 131
 SystemLabel.RHOXC, 129, 130
 SystemLabel.selected.WFSX, 105, 106
 SystemLabel.STRUCT_IN, 51, 52
 SystemLabel.STRUCT_NEXT_ITER, 51
 SystemLabel.STRUCT_OUT, 51
 SystemLabel.TDDIPOL, 155
 SystemLabel.TDEIG, 155
 SystemLabel.TDETOT, 155
 SystemLabel.TDWF, 140, 154, 155
 SystemLabel.TDXV, 140, 154, 155
 SystemLabel.times, 136
 SystemLabel.TOCH, 130
 SystemLabel.TS.HSX, 19, 82, 172, 186
 SystemLabel.TS.KP, 186
 SystemLabel.TSCCEQ*, 186
 SystemLabel.TSCCNEQ*, 186
 SystemLabel.TSDE, 19, 160, 161, 165, 172, 175,
 179, 185, 186
 SystemLabel.TSFA, 170, 186
 SystemLabel.TSFAC, 170
 SystemLabel.TSGF*, 178, 179, 186
 SystemLabel.TSHS, 19, 76, 160, 172, 177, 186
 SystemLabel.VERLET_RESTART, 155
 SystemLabel.VH, 129, 130
 SystemLabel.VNA, 129, 130
 SystemLabel.VT, 129, 130
 SystemLabel.WANNX, 119, 121, 122
 SystemLabel.WFS, 106, 113
 SystemLabel.WFSX, 105, 106, 113, 122, 137
 SystemLabel.xtl, 52
 SystemLabel.XV, 51–53, 137, 142, 146
 SystemLabel.xyz, 51
 SystemLabel.ZM, 53

 time.json, 136
 TotalCharge.grid.nc, 130
 TotalPotential.grid.nc, 130
 TS_FERMI, 171

 UNKXXXXX.Y, 118

 Vna.grid.nc, 129

 WFS.nc, 84, 88, 105

 xsf, 122

List of fdf flags

AllocReportLevel, 136
AllocReportThreshold, 136
AnalyzeChargeDensityOnly, 131
AtomCoorFormatOut, 46, 51, 52
AtomicCoordinatesAndAtomicSpecies, 22, 45, 47, 73, 110, 147, 148, 189
AtomicCoordinatesFormat, 46, 47, 51, 52
 Ang, 46
 Bohr, 46
 Fractional, 46
 LatticeConstant, 46
 NotScaledCartesianAng, 46
 NotScaledCartesianBohr, 46
 ScaledByLatticeVectors, 46
 ScaledCartesian, 46
AtomicCoordinatesOrigin, 46, 52
 COM, 47
 COP, 47
 MIN, 47
AtomicMass, 24
AtomSetupOnly, 41

BandLines, 87, 103–105
BandLinesScale, 103, 104
BandPoints, 87, 103–105
BasisPressure, 42
BasisPressure.Specs, 42
Block.Cutoff, 190
BlockSize, 84, 85, 90, 91, 134
BornCharge, 116, 150
BulkBias
 Current, 128
 Direction, 127
 Tolerance, 127, 128
 Voltage, 127

CDF
 Compress, 138
 Grid.Precision, 138
 MPI, 138
 Save, 138, 177
CenterQMSystem, 190
ChangeKgridInMD, 54, 55
Charge
 Hirshfeld, 111, 113
 end, 112
 geometry, 112
 none, 112
 scf, 112
Mulliken, 110–112
 end, 110
 geometry, 110
 init, 110
 none, 110
 scf, 110, 112
 scf-after-mix, 110
Mulliken.Format, 111, 112
Spin
 scf, 112
Voronoi, 112, 113
 end, 112
 geometry, 112
 none, 112
 scf, 112
ChemicalSpeciesLabel, 22–24, 38, 41, 42, 45, 52, 53, 148
CheSS
 Buffer
 Kernel, 96
 Mult, 96
 evhighH, 96
 evhighS, 97
 evlowH, 96
 evlowS, 97
 Fscale, 96
 FscaleLowerbound, 96
 FscaleUpperbound, 96
Command line options
 -L, 18
 -V, 19, 167
 -elec, 19
 -electrode, 19
 -fdf, 19
 -h, 18
 -o, 18
 -out, 18
 -v, 18
Compat
 Pre-v4-DM-H, 68, 69

- Pre-v4-Dynamics, 140
- Constant
 - Volume, 141
- COOP.Write, 82, 88, 105, 113
- CoulombType, 190
- Debug
 - DIIS, 70
- DFTD3, 132, 133
 - 2BodyCutOff, 133
 - 3BodyCutOff, 133
 - a1, 134
 - a2, 134
 - alpha, 134
 - BJdamping, 133
 - CoordinationCutoff, 133
 - Periodic, 134
 - rs6, 133
 - rs8, 133
 - s6, 133
 - s8, 133
 - UseXCDefaults, 133, 134
- DFTU
 - CutoffNorm, 152, 153
 - EnergyShift, 152, 153
 - FirstIteration, 152, 153
 - PopTol, 152, 153
 - PotentialShift, 152, 153
 - Proj, 152
 - ProjectorGenerationMethod, 152, 153
 - ThresholdTol, 152, 153
- Diag
 - AbsTol, 87
 - Algorithm, 84, 85, 87, 88
 - Divide-and-Conquer, 85
 - Divide-and-Conquer-2stage, 85
 - ELPA-1stage, 85
 - ELPA-2stage, 86
 - Expert, 85
 - Expert-2stage, 85
 - MRRR, 85
 - MRRR-2stage, 85
 - NoExpert, 85
 - NoExpert-2stage, 85
 - QR, 85
 - BlockSize, 84, 85
 - DivideAndConquer, 85–87
 - ELPA, 85–87
 - GPU, 86
 - Memory, 87
 - MRRR, 85–87
 - NoExpert, 85, 86, 88
 - OrFac, 87
 - ParallelOverK, 84, 86, 87, 127
 - ProcessorY, 84
 - UpperLower, 87
 - Use2D, 84, 85
 - UseNewDiagk, 105
 - WFS.Cache, 84
 - cdf, 84, 88
 - none, 84
- Diag.BlockSize, 86
- DirectPhi, 135
- DM
 - AllowExtrapolation, 74
 - AllowReuse, 74
 - FormattedFiles, 72
 - FormattedInput, 72
 - FormattedOutput, 72
 - History.Depth, 74
 - Init, 72
 - atomic, 72
 - RandomStates, 72
 - Init.RandomStates, 73
 - Init.Unfold, 72
 - InitSpin, 73
 - AF, 72, 73
 - KickMixingWeight, *see* SF.Mixer.Kick.Weight66
 - MixingWeight, 65, *see* SF.Mixer.Weight65, 69
 - UseSaveDM, 62, 72
- DM.EnergyTolerance, 77
- DM.InitSpin, 59
- DM.MixSCF1, 63, *see* SF.Mix.First63
- DM.Normalization.Tolerance, 77
- DM.NumberBroyden, 65, *see* SF.Mixer.History65, 66
- DM.NumberKick, *see* SF.Mixer.Kick66
- DM.NumberPulay, 65, *see* SF.Mixer.History65, 66
- DM.Require.Harris.Convergence, 77
- DM.RequireEnergyConvergence, 77
- DM.Tolerance, 76, 77

- DM.UseSaveDM, 91, 131
- DOS.kgrid.?, 107–109
- EggboxRemove, 80, 81
- EggboxScale, 80, 81
- ElectronicTemperature, 60, 88, 89, 97, 168
- ELSI
 - Broadening
 - Method, 94
 - MPOrder, 94
 - BuildDM
 - Method, 94
 - EigenExa
 - Method, 96
 - ELPA
 - Autotune, 95
 - Flavor, 94
 - GPU, 95
 - NSinglePrecision, 94
 - Ill-Condition
 - Check, 94
 - Tolerance, 94
 - MAGMA
 - Solver-Method, 96
 - NTPoly
 - Filter, 95
 - Method, 95
 - Tolerance, 95
 - OMM
 - Flavor, 95
 - Tolerance, 95
 - Output
 - Json, 94
 - Level, 94
 - PEXSI
 - Inertia-Tolerance, 95
 - Initial-Mu-Max, 95
 - Initial-Mu-Min, 95
 - Method, 95
 - Number-of-Mu-Points, 95
 - Number-of-Poles, 95
 - TasksPerPole, 95
 - TasksSymbolic, 95
 - SIPS
 - Slices, 95
 - Solver, 93
- ExternalElectricField, 123

- FC
 - dHdR.Tolerance, 151
 - Displacement, 150
 - dSdR.Tolerance, 151
 - First, 150, 151
 - Last, 151
 - Save.dHS, 151
- FFT
 - ProcessorY
 - Traditional, 135
- FilterCutoff, 39–41
- FilterTol, 40, 41
- ForceAuxCell, 82
- Geometry
 - Charge, 124, 127
 - Constraints, 147, 168
 - Hartree, 125, 127
- Grid.CellSampling, 79, 80
- Harris
 - Functional, 62
- KB.New.Reference.Orbitals, 37
- kgrid
 - Cutoff, 54, 108, 109, 173
 - File, 54, 108, 109, 173
 - MonkhorstPack, 46, 53, 54, 108, 109, 164, 167, 173
- kgrid.MonkhorstPack, 139
- LatticeConstant, 45, 59
- LatticeParameters, 45
- LatticeVectors, 45, 46, 54
- LaunchSiesta, 189–191
- LDAU
 - CutoffNorm, 152
 - EnergyShift, 152
 - FirstIteration, 153
 - PopTol, 153
 - PotentialShift, 153
 - Proj, 152
 - ProjectorGenerationMethod, 152
 - ThresholdTol, 153
- LDOS.kgrid.?, 109
- Lindhard
 - FirstBand, 191
 - LastBand, 191

- ngridx, 192
- ngridy, 192
- ngridz, 192
- nq1, 192
- nq2, 192
- Temperature, 191
- LocalDensityOfStates, 103, 107, 108
- LongOutput, 21, 55, 146
- Lua
 - Debug, 158
 - Debug.MPI, 158
 - Interactive, 158
 - Script, 156, 158
- MaxBondDistance, 53
- MaxSCFIterations, 62
- MaxWalltime, 137
 - Slack, 137
- MD
 - FCDispl, 150
 - FCFirst, 150
 - FCLast, 151
 - TypeOfRun
 - FC, 76
 - UseSaveXV, 52, 53
 - UseSaveZM, 53
- MD.AnnealOption, 139, 143, 145
- MD.Broyden
 - Cycle.On.Maxit, 143
 - History.Steps, 143
 - Initial.Inverse.Jacobian, 143
- MD.Broyden.Initial.Inverse.Jacobian, 142
- MD.BulkModulus, 145
- MD.ConstantVolume, 141
- MD.FinalTimeStep, 140, 144
- MD.FIRE.TimeStep, 143
- MD.InitialTemperature, 144
- MD.InitialTimeStep, 144
- MD.LengthTimeStep, 140, 143, 144
- MD.MaxCGDispl, 142
- MD.MaxDispl, 142, 157
- MD.MaxForceTol, 141, 157
- MD.MaxStressTol, 141
- MD.NoseMass, 145
- MD.NumCGsteps, 141, 142
- MD.ParrinelloRahmanMass, 145
- MD.PreconditionVariableCell, 141, 142
- MD.RelaxCellOnly, 141
- MD.RemoveIntramolecularPressure, 144
- MD.Steps, 141, 144
- MD.TargetPressure, 143
- MD.TargetStress, 143, 144
- MD.TargetTemperature, 145
- MD.TauRelax, 145
- MD.TypeOfRun, 53, 138, 143–145, 150, 156, 157
 - Anneal, 139, 145
 - Broyden, 138, 141
 - CG, 138, 141
 - FC, 82, 117, 139, 151
 - FIRE, 139
 - Forces, 139
 - Lua, 138, 139
 - Master, 138, 139
 - Nose, 139
 - NoseParrinelloRahman, 139
 - ParrinelloRahman, 139
 - TDED, 139
 - Verlet, 139
- MD.UseSaveCG, 142
- MD.UseSaveXV, 142
- MD.VariableCell, 81, 138, 139, 141, 144
- Mesh
 - Cutoff, 40, 41, 60, 78, 79, 139, 157, 164
 - Sizes, 78
 - SubDivisions, 78, 79
- MinSCFIterations, 62
- MM, 131
 - Cutoff, 132
 - Grimme.D, 132
 - Grimme.S6, 132
 - Potentials, 131
 - UnitsDistance, 132
 - UnitsEnergy, 132
- MM.Atoms, 190
- MM.Cutoff, 190
- MM.ParmFile, 189, 190
- MPI
 - Nprocs.SIESTA, 98
- MPICommand, 190, 191
- MullikenInSCF, 112
- MullikenInScf, 60
- NeglNonOverlapInt, 81
- NetCharge, 122, 123, 127

- New
 - A.Parameter, 43
 - B.Parameter, 43
- NonCollinearSpin, 58
- NumberMPINodes, 190
- NumberOfAtoms, 22, 46, 47, 189
- NumberOfEigenStates, 84–87
- NumberOfMMAAtoms, 190
- NumberOfSpecies, 22
- OccupationFunction, 88, 89, 127
- OccupationMPOOrder, 89
- OMM
 - BlockSize, 90, 91
 - Diagon, 90
 - DiagonFirstStep, 90
 - Eigenvalues, 91
 - LongOutput, 91
 - Precon, 90
 - PreconFirstStep, 90
 - ReadCoeffs, 91
 - RelTol, 91
 - TPreconScale, 90
 - Use2D, 90, 91
 - UseCholesky, 89, 90
 - UseSparse, 90
 - WriteCoeffs, 91
- ON
 - Etol, 91
- ON.ChemicalPotential, 92
- ON.ChemicalPotential.Order, 93
- ON.ChemicalPotential.Rc, 92
- ON.ChemicalPotential.Temperature, 93
- ON.ChemicalPotential.Use, 92
- ON.eta, 89, 92
- ON.eta.alpha, 92
- ON.eta.beta, 92
- ON.Etol, 92
- ON.functional, 91
- ON.LowerMemory, 93
- ON.MaxNumIter, 91
- ON.RcLWF, 92
- ON.UseSaveLWF, 93
- Optical.Broaden, 114
- Optical.Energy.Maximum, 114
- Optical.Energy.Minimum, 114
- Optical.Mesh, 114
- Optical.NumberOfBands, 114
- Optical.OffsetMesh, 115
- Optical.PolarizationType, 115
- Optical.Scissor, 114
- Optical.Vector, 115
- OpticalCalculation, 114
- PAO
 - Basis, 22, 23, 26, 28, 30–32, 34, 35, 37, 40, 152
 - BasisSize, 31, 32, 37
 - DZ, 31
 - DZP, 31
 - minimal, 31
 - SZ, 31
 - SZP, 31
 - BasisSizes, 31
 - BasisType, 28, 30, 31, 33, 37, 38
 - filteret, 31
 - nodes, 31
 - nonodes, 31
 - split, 31
 - splitgauss, 31
 - ContractionCutoff, 33
 - EnergyCutoff, 33
 - EnergyPolCutoff, 33
 - EnergyShift, 31, 32, 37, 38, 40, 42, 152
 - FixSplitTable, 33
 - true, 33
 - OldStylePolOrbs, 40
 - Polarization
 - NonPerturbative, 31, 34
 - NonPerturbative.Fallback, 26, 35
 - Rc-Expansion-Factor, 34
 - Scheme, 26, 31, 34
 - rc.unbound.state, 44
 - SoftDefault, 30, 35, 37
 - SoftInnerRadius, 35
 - SoftPotential, 35
 - SplitNorm, 31–33, 37, 38
 - SplitNormH, 32, 37
 - SplitTailNorm, 32
 - true, 26, 32
 - SplitValence
 - Legacy, 33
- PartialCharges, 187
- PartialChargesAtEveryGeometry, 113

- PartialChargesAtEverySCFStep, 113
- PDOS.kgrid.?, 108
- PEXSI
 - deltaE, 98
 - DOS, 102
 - Ef.Reference, 102
 - E_{max}, 102
 - E_{min}, 102
 - NPoints, 102
 - Gap, 98
 - Inertia-Counts, 100, 101
 - Inertia-energy-width-tolerance, 101
 - Inertia-max-iter, 101
 - Inertia-min-num-shifts, 101
 - Inertia-mu-tolerance, 101
 - lateral-expansion-inertia, 101
 - LDOS, 103, 109
 - Broadening, 103
 - Energy, 103
 - NP-per-pole, 103
 - mu, 100
 - mu-max, 100
 - mu-max-iter, 99
 - mu-min, 100
 - mu-pexsi-safeguard, 100
 - NP-per-pole, 98, 103
 - NP-symbfact, 99
 - num-electron-tolerance, 99
 - num-electron-tolerance-lower-bound, 99
 - num-electron-tolerance-upper-bound, 99
 - NumPoles, 97
 - Ordering, 98, 99
 - safe-dDmax-ef-inertia, 101
 - safe-dDmax-ef-solver, 102
 - safe-dDmax-no-inertia, 100, 101
 - safe-width-ic-bracket, 101
 - safe-width-solver-bracket, 102
 - Verbosity, 98, 99
- Poisson
 - Method, 136
- PolarizationGrids, 115, 117
- ProcessorY, 134, 135
- ProjectedDensityOfStates, 107
- projection functions as in wannier90, 121
- PS
 - l_{max}, 35, 36
- PS.KBprojectors, 36
- PSML
 - KB.projectors, 25, 35
 - Vlocal, 25, 35
- QM.AtomTypes, 189
- QMMM.CoulombType, 187
- QMMM.CutOff, 187
- QMMM.DensityCut, 187
- QMMM.Driver, 188
 - QMMM.Driver.Address, 188
 - QMMM.Driver.ForceOutFile, 188
 - QMMM.Driver.MMChargeFile, 188
 - QMMM.Driver.Port, 188
 - QMMM.Driver.QMRegionFile, 188
 - QMMM.Driver.SocketType, 188
 - QMMM.Driver.Type, 188
- QMMM.Enabled, 186
- QMMM.Ewald.Rcut, 187, 190
- QMMM.SmoothElectrode, 187
 - QMMM.SmoothElectrode.Electrode, 187
 - QMMM.SmoothElectrode.Smooth, 187
- RcSpatial, 135
- Reparametrize.Pseudos, 43
- Restricted.Radial.Grid, 43
- Rmax.Radial.Grid, 43
- S.Only, 75
- Save
 - HS, 19, 82, 162, 172, 177
 - false, 82
 - true, 19, 82
- SaveBaderCharge, 130
- SaveDeltaRho, 129
- SaveElectrostaticPotential, 129, 130, 138
- SaveGridFunc.Format, 130
- SaveInitialChargeDensity, 131
- SaveIonicCharge, 130
- SaveNeutralAtomPotential, 129
- SaveRho, 129
- SaveRhoXC, 129
- SaveTotalCharge, 130
- SaveTotalPotential, 129, 130
- SCF
 - Mixing, 82
 - MonitorForces, 62
 - MustConverge, 62
 - RecomputeHAfterSCF, 69

- RecomputeHAfterScf, 69
- Want.Variational.EKS, 61
- Write.Extra, 81
- SCF.DebugRhoGMixing, 70
- SCF.DM
 - Converge, 76, 78, 140, 168
 - Tolerance, 76, 77, 168
- SCF.EDM
 - Converge, 77
 - Tolerance, 77
- SCF.FreeE
 - Converge, 77
 - Tolerance, 77
- SCF.H
 - Converge, 77, 78, 140, 168
 - Tolerance, 60, 77, 168
- SCF.Harris
 - Converge, 77, 78
 - Tolerance, 77, 78
- SCF.Kerker.q0sq, 70
- SCF.Mix, 60, 62, 69
 - AfterConvergence, 61, 69, 75
 - charge, 63
 - density, 63, 110
 - First, 62, 63, 69, 124
 - First.Force, 63
 - Hamiltonian, 63
 - Spin, 63
- SCF.MixCharge
 - SCF1, 70
- SCF.Mixer
 - History, 65, 67
 - Kick, 66
 - Kick.Weight, 66
 - Linear.After, 66
 - Linear.After.Weight, 66
 - Method, 64–67
 - Restart, 66, 67
 - Restart.Save, 66, 67
 - Variant, 64, 65, 67
 - Weight, 65–67
- SCF.Mixer.<>, 66, 67
 - history, 67
 - iterations, 67
 - method, 67
 - next, 67
 - next.conv, 67
 - next.p, 67
 - restart, 67
 - restart.p, 67
 - restart.save, 67
 - variant, 67
 - weight, 67
 - weight.linear, 65, 67
- SCF.Mixers, 66
- SCF.Read.Charge.NetCDF, 74
- SCF.Read.Deformation.Charge.NetCDF, 74
- SCF.RhoG.DIIS.Depth, 70
- SCF.RhoG.Metric.Preconditioner.Cutoff, 70
- SCF.RhoGMixingCutoff, 70
- Siesta2Wannier90.NumberOfBands, 119
- Siesta2Wannier90.NumberOfBandsDown, 119
- Siesta2Wannier90.NumberOfBandsUp, 119
- Siesta2Wannier90.UnkGrid1, 118
- Siesta2Wannier90.UnkGrid2, 118
- Siesta2Wannier90.UnkGrid3, 119
- Siesta2Wannier90.UnkGridBinary, 119
- Siesta2Wannier90.WriteAmn, 118
- Siesta2Wannier90.WriteEig, 118
- Siesta2Wannier90.WriteMmn, 118
- Siesta2Wannier90.WriteUnk, 118, 120
- SimulateDoping, 123
- SingleExcitation, 59
- Slab.DipoleCorrection, 123
 - charge, 123, 124
 - Origin, 124
 - Vacuum, 124, 125
 - vacuum, 124, 125
- SOC.Split.SR.SO, 60
- SolutionMethod, 16, 54, 82, 83, 88, 90, 167, 172
- Spin, 55, 58, 59, 61, 72, 73, 89
 - Fix, 58, 89
 - non-colinear, 58
 - non-polarized, 58
 - OrbitStrength, 60
 - polarized, 58
 - spin-orbit, 58
 - Spiral, 55, 58, 59
 - Spiral.Scale, 59
 - Total, 58, 89
- SpinInSCF, 112
- SpinOrbit, 58
- SpinPolarized, 58
- SuperCell, 45, 46, 54

- SyntheticAtoms, 22, 23
- SystemLabel, 18, 22, 51, 160
- SystemName, 22
- Target
 - Pressure, 141, 143
 - Stress.Voigt, 141, 143, 144
- Target.Stress.Voigt, 141
- TDED
 - Extrapolate, 155
 - Extrapolate.Substeps, 155
 - Inverse.Linear, 155
 - Nsaverho, 155
 - Nsteps, 140, 155
 - Saverho, 155
 - TimeStep, 140, 155
 - WF.Initialize, 154
 - WF.Save, 155
 - Write.Dipole, 155
 - Write.Eig, 155
 - Write.Etot, 155
- TimeReversalSymmetryForKpoints, 55, 127, 191
- TimerReportThreshold, 136
- TimingSplitScfSteps, 137
- trial-orbitals, 121
- TS
 - Analyze, 161, 167, 172–174
 - Analyze.Graphviz, 173
 - Atoms.Buffer, 165, 167
 - BTD
 - Guess1.Max, 174
 - Guess1.Min, 174, 175
 - Optimize, 174
 - Pivot, 172, 173
 - Spectral, 175
 - ChemPot.<>, 181
 - chemical-shift, 181
 - contour.eq, 181, 183
 - contour.eq.pole, 181, 182
 - contour.eq.pole.N, 182
 - ElectronicTemperature, 168, 181
 - kT, 181
 - mu, 181
 - Temp, 181
- ChemPots, 177, 181
- Contour.<>, 183, 185
 - delta, 183
 - from, 183
 - method, 183
 - opt, 184
 - part, 183
 - points, 183
- Contour.nEq.<>, 185
- Contours
 - Eq.Pole, 183
 - Eq.Pole.N, 183
- Contours.nEq, 185
 - Eta, 185
 - Fermi.Cutoff, 185
- DE.Save, 19, 162, 172
 - true, 19, 172
- dQ, 168, 170
 - Factor, 171
 - fermi, 170, 171, 178
 - Fermi.Eta, 172
 - Fermi.Max, 171
 - Fermi.Tolerance, 171
- Elec.<>, 170, 177, 180
 - Accuracy, 179, 180
 - Bloch, 163, 179
 - Bulk, 166, 177, 178, 180
 - check-kgrid, 164, 180
 - chemical-potential, 166, 177
 - DE, 179
 - delta-Ef, 166, 179
 - DM-init, 178, 179, 181
 - DM-update, 178, 181, 186
 - electrode-position, 177
 - Eta, 166, 179, 180
 - Gf, 178
 - Gf-Reuse, 178, 180
 - HS, 160, 177, 178
 - Out-of-core, 178, 180
 - out-of-core, 179
 - pre-expand, 179
 - semi-inf-direction, 177
 - used-atoms, 177
 - V-fraction, 166, 180
- Elecs, 174, 177, 181
 - Accuracy, 179, 180
 - Bulk, 180
 - Coord.EPS, 181
 - DM.Init, 168, 178, 181
 - DM.Update, 180

- Eta, 179, 180, 185
- Gf.Reuse, 180
- Neglect.Principal, 164, 180
- Out-of-core, 180
- ElectronicTemperature, 168, 181, 185
- Fermi.Initial, 168, 169
- Forces, 170, 186
- Hartree.Fix, 176
 - Frac, 176
- Hartree.Offset, 176
- HS.Save, 19, 76, 172
 - true, 19, 172
- kgrid
 - MonkhorstPack, 161, 164, 167
- MUMPS
 - BlockingFactor, 175
 - Memory, 175
 - Ordering, 175
- Poisson, 175
 - <file>, 175, 176
 - elec-box, 176
 - ramp, 175
- S.Save, 172
- SCF
 - DM.Tolerance, 168
 - dQ.Converge, 168
 - dQ.Tolerance, 168
 - H.Tolerance, 168
- SCF.Initialize, 168
- SIESTA.Only, 172
- SolutionMethod, 167, 172
 - BTD, 167, 173
 - full, 167
 - MUMPS, 167
- Voltage, 19, 167, 178
- Weight.k.Method, 170
- Weight.Method, 169
 - mean, 170
 - orb-orb, 169
 - sum-atom-atom, 169
 - sum-atom-orb, 170
 - tr-atom-atom, 169
 - tr-atom-orb, 169
- TS.HS
 - Save, 177
- TS.kgrid
 - Cutoff, 173
- File, 173
 - MonkhorstPack, 173
- Use.Blocked.WriteMat, 74, 75
- UseDomainDecomposition, 135
- UseNewDiagk, 84
- UseParallelTimer, 136
- User
 - Basis, 41
 - Basis.NetCDF, 41
- User.Basis, 27, 44
- User.Basis.Netcdf, 44
- UseSaveData, 52, 53, 137, 142
- UseSpatialDecomposition, 135
- UseStructFile, 51–53
- UseTreeTimer, 136
- w90.in.siesta.compute.unk, 120
- Wannier
 - k, 122
 - Manifold.<>, 120, 121
 - bands, 120
 - fermi-surface-plot, 120
 - spreading.nitt, 120
 - threshold, 122
 - trial-orbitals, 120
 - wannier-plot, 120
 - window, 121
 - window.frozen, 121
 - window.threshold, 121
 - write-hr, 120
 - write-tb, 120
 - write-unk, 120, 122
- Manifolds, 120
 - Threshold, 122
 - Unk, 122
- Projectors, 121
- wannier.plot.supercell, 120
- WarningMinimumAtomicDistance, 53
- WaveFuncKPoints, 87, 105, 106, 114
- WaveFuncKPointsScale, 105
- WFS.Band.Max, 105, 113
- WFS.Band.Min, 105, 113
- WFS.Energy.Max, 106, 113, 114
- WFS.Energy.Min, 106, 113, 114
- WFS.Write.For.Bands, 105
- Write
 - Denchar, 137

- DM, 75
- DM.end.of.cycle, 75
- DM.History.NetCDF, 75, 76
- DM.NetCDF, 75, 76
- DMHS.History.NetCDF, 76, 82
- DMHS.NetCDF, 76, 82
- Graphviz, 32
- H, 75
- H.end.of.cycle, 75
- HirshfeldPop, 111, 112
- HS.History, 76
- TSHS.History, 76
- VoronoiPop, 112, 113
- Write.OrbitalIndex, 147
- WriteBands, 104
- WriteCoorCerius, 52
- WriteCoorInitial, 146
- WriteCoorStep, 21, 52, 146
- WriteCoorXmol, 51
- WriteEigenvalues, 21, 88, 106
- WriteForces, 21, 146
- WriteIonPlotFiles, 41
- WriteKbands, 21, 104
- WriteKpoints, 21, 55
- WriteMDHistory, 51, 146
- WriteMDXmol, 52, 146
- WriteMullikenPop, 21, 110–112
- WriteOrbMom, 60
- WriteWaveFunctions, 21, 106
- XC
 - Authors, 55
 - Functional, 55
 - Mix, 57
 - Use.BSC.CellXC, 58
- XC.mix, 55
- XML
 - Write, 195
- ZM
 - UnitsAngle, 50
 - UnitsLength, 50
- ZM.ForceTolAngle, 142
- ZM.ForceTolLength, 142
- ZM.MaxDisplAngle, 142
- ZM.MaxDisplLength, 142
- Zmatrix, 45, 47, 141, 144